

3-Party Deal Making Platform Software Design Document

Platform Architecture Team

June 2026

Table of Contents

3-Party Deal Making Platform — Software Design Document

Version: 1.0

Date: June 2026

Status: Draft for Review

Classification: Confidential

Executive Summary

The **3-Party Deal Making Platform** is a domain-agnostic, cloud-native marketplace that enables triadic value creation among three distinct party types: **Suppliers** (who hold underutilized resources), **Consumers** (who desire specific outputs), and **Enhancers** (who provide enabling expertise or services). The platform's core value proposition is the “**any domain, any initiator, win-win-win**” principle — any party type can initiate a deal in any industry vertical, and every deal is mathematically validated to ensure all three parties achieve positive net value.

Purpose

This document provides the comprehensive software design specification for the 3-Party Deal Making Platform, integrating domain modeling, workflow architecture, system architecture, API design, user experience design, and economic mechanisms into a single, cohesive blueprint. It serves as the authoritative reference for engineering teams, product managers, and stakeholders involved in building and evolving the platform.

The 3-Party Model at a Glance

The platform breaks away from traditional two-sided marketplaces by formalizing a three-sided structure:

- **Supplier** holds an underutilized resource or asset (e.g., idle farmland, vacant building, unused vehicle, idle machinery) and seeks to derive value from it.
- **Consumer** desires a specific output, product, or service (e.g., crop produce, rental space, transportation, processed goods).
- **Enhancer** bridges the gap by providing enabling input, expertise, or services (e.g., agrodealer furnishing seeds/fertilizer, contractor performing renovation, mechanic providing maintenance).

Any party can initiate a deal. Each party may be an individual, organization, or a **PartyGroup** — a collective of people/entities pooling resources or needs. The platform validates that every deal satisfies the **Win-Win-Win Principle**: all three parties must receive net positive value.

Key Features

Feature	Description
Multi-Party Matching	Algorithmic matching of compatible Suppliers, Consumers, and Enhancers across any domain
Win-Win-Win Validation	Mathematical/model-based validation that all parties benefit (see Chapter 8)
Flexible Negotiation	Multi-round, asynchronous negotiation with proposal versioning and threaded term dimensions
Escrow-Based Settlement	Milestone-tracked execution with conditional payment releases
Trust & Reputation	Multi-dimensional trust scoring, rating, review, and reputation system
Domain Adaptability	Configurable deal templates, resource types, and value models via a hierarchical category system
Group Support	PartyGroup entity enabling cooperatives, partnerships, and corporate alliances to participate as a single Party
Dispute Resolution	Three-tier resolution: direct negotiation, platform mediation, binding arbitration

Architecture Approach

The platform follows a **microservices architecture** with 16 core and supporting services, communicating via a combination of synchronous REST/gRPC APIs and asynchronous event-driven patterns via Apache Kafka. The architecture embraces:

- **Domain-Driven Design (DDD)** with clear aggregate boundaries and ubiquitous language
- **Event sourcing** for deal lifecycle ensuring full audit trail
- **CQRS** for read/write separation with near-realtime projections
- **Saga patterns** for distributed transaction coordination
- **API-first** design with versioned REST APIs, gRPC inter-service communication, GraphQL queries, and WebSocket real-time updates

Intended Outcomes

- Enable cross-domain deal creation with minimal platform customization
- Reduce deal negotiation friction through algorithmic matching and fairness validation
- Build trust among parties via transparent reputation scoring and escrow protection
- Scale to support millions of parties and hundreds of thousands of concurrent deals
- Achieve platform sustainability through a dynamic, trust-adjusted fee model

Table of Contents

- [Executive Summary](#)

- 1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions & Acronyms
 - 1.4 References
- 2. System Overview
 - 2.1 The 3-Party Model (Supplier-Consumer-Enhancer)
 - 2.2 Deal Lifecycle at a Glance
 - 2.3 Key Capabilities
 - 2.4 Stakeholders
- 3. Domain Model
 - 3.1 Overview & Domain Narrative
 - 3.2 Ubiquitous Language
 - 3.3 Core Entities & Attributes
 - 3.4 Relationships & Cardinalities
 - 3.5 Value Flow Model
 - 3.6 ER Diagram (Mermaid)
 - 3.7 Aggregate Boundaries (DDD)
 - 3.8 Domain Invariants & Business Rules
 - 3.9 Lifecycle State Machines
 - 3.10 Index & Query Model
- 4. Workflow & State Machine Design
 - 4.1 Deal Lifecycle State Machine
 - 4.2 Initiation Workflows
 - 4.3 Negotiation Workflow
 - 4.4 Matching & Discovery Workflow
 - 4.5 Execution & Settlement Workflow
 - 4.6 Swimlane Diagrams
- 5. System Architecture
 - 5.1 Executive Summary
 - 5.2 System Overview
 - 5.3 Service Decomposition (Microservices)
 - 5.4 Communication Patterns
 - 5.5 Data Architecture
 - 5.6 Integration Architecture
 - 5.7 Scalability & Resilience
 - 5.8 Security Architecture
 - 5.9 Deployment Architecture
 - 5.10 Architecture Diagrams
 - 5.11 Cross-Cutting Concerns
- 6. API Design
 - 6.1 API Standards & Conventions
 - 6.2 Party & Profile APIs
 - 6.3 Deal Management APIs

- 6.4 Resource, Need & Enhancement APIs
 - 6.5 Negotiation APIs
 - 6.6 Agreement APIs
 - 6.7 Execution & Milestone APIs
 - 6.8 Payment & Settlement APIs
 - 6.9 Matching & Discovery APIs
 - 6.10 Trust & Reputation APIs
 - 6.11 Notification APIs
 - 6.12 Event Schema (Async APIs)
 - 6.13 Webhook Contracts
 - 6.14 GraphQL Schema
 - 6.15 API Dependency Flows
 - 7. User Experience Design
 - 7.1 User Onboarding Flows
 - 7.2 Deal Initiation Flows
 - 7.3 Discovery & Matching UX
 - 7.4 Negotiation UX
 - 7.5 Dashboard Designs
 - 7.6 Deal Execution Tracking
 - 7.7 Notification Design
 - 7.8 Trust & Safety UX
 - 7.9 Mobile Considerations
 - 7.10 Wireframe Descriptions
 - 8. Value Distribution & Economic Mechanisms
 - 8.1 Win-Win-Win Validation Framework
 - 8.2 Value Distribution Models
 - 8.3 Pricing & Fee Mechanism
 - 8.4 Trust & Reputation System
 - 8.5 Incentive Alignment
 - 8.6 Risk Management
 - 8.7 Economic Sustainability
 - 8.8 Smart Contract / Automated Execution
 - 9. Cross-Cutting Concerns
 - 10. Implementation Roadmap
 - Appendix A: Glossary
 - Appendix B: Decision Log
-

1. Introduction

1.1 Purpose

This Software Design Document (SDD) specifies the complete architecture, design, and implementation blueprint for the 3-Party Deal Making Platform. It consolidates six specialist-authored design documents into a single unified reference covering:

- **Domain Model** — Entities, relationships, value flow, and aggregate boundaries

- **Workflow Design** — Deal lifecycle state machine, initiation flows, negotiation, matching, execution
- **System Architecture** — Microservices decomposition, communication patterns, data architecture, security
- **API Design** — REST APIs, events, webhooks, GraphQL schema
- **User Experience Design** — User flows, wireframes, dashboards, notification design
- **Value Distribution & Economic Mechanisms** — Value models, trust system, pricing, risk management

The intended audience includes software architects, backend/frontend engineers, DevOps engineers, product managers, QA engineers, and technical stakeholders.

1.2 Scope

This document covers the full design of the platform's core capabilities:

In Scope: - Party management (individuals, organizations, groups) - Deal lifecycle from creation through completion - Algorithmic matching and discovery - Multi-round negotiation with versioning - Win-Win-Win validation engine - Escrow-based payment and settlement - Trust scoring and reputation system - Review and rating mechanisms - Dispute resolution (3-tier system) - Multi-channel notification system - Category/taxonomy system for domain adaptability - Full microservices architecture with deployment topology - Complete API specification (REST, gRPC, GraphQL, Events, Webhooks) - User experience flows and interface specifications - Economic model: value distribution, pricing, risk management

Out of Scope: - Specific frontend framework implementation details - Detailed infrastructure cost estimates - Regulatory compliance certification processes - Physical hardware procurement

1.3 Definitions & Acronyms

Term/Acronym	Definition
3-Party Deal	A transaction involving Supplier, Consumer, and Enhancer roles around a specific value exchange
ABAC	Attribute-Based Access Control
CQRS	Command Query Responsibility Segregation
DDD	Domain-Driven Design
Enhancer	Party that adds value to enable the Supplier-Consumer transaction
ER	Entity-Relationship
ES	Event Sourcing
HPA	Horizontal Pod Autoscaler
JWT	JSON Web Token
KYC	Know Your Customer (identity verification)
MFA	Multi-Factor Authentication
MVP	Minimum Viable Product

Term/Acronym	Definition
OIDC	OpenID Connect
Party	An individual, organization, or group participating in the platform
PartyGroup	A composite entity allowing multiple people/organizations to act as a single Party
PVE	Present Value Equivalent
RBAC	Role-Based Access Control
REST	Representational State Transfer
Saga	A pattern for managing distributed transactions
SDD	Software Design Document (this document)
Supplier	Party offering resources/assets
Consumer	Party expressing needs/requirements
TDV	Total Deal Value
Trust Score	A composite reputation metric derived from reviews, completion history, and platform behavior
Win-Win-Win	The core platform principle that all three parties must derive positive net value
ZOPA	Zone of Possible Agreement — the overlap between party acceptable ranges

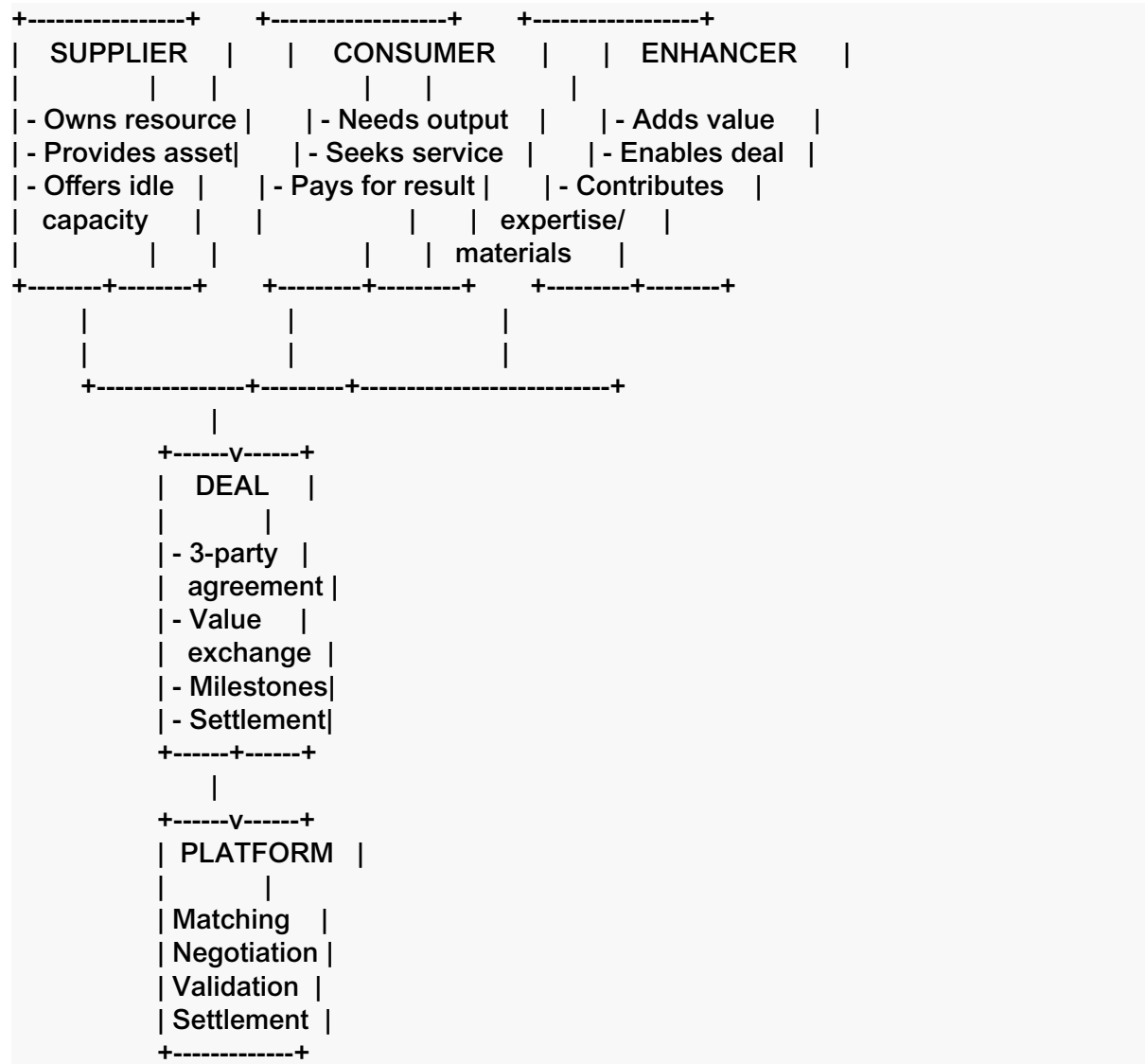
1.4 References

#	Document	Description
R1	This document, Chapter 3	Domain Model — entities, relationships, value flow
R2	This document, Chapter 4	Workflow & State Machine Design
R3	This document, Chapter 5	System Architecture
R4	This document, Chapter 6	API Design
R5	This document, Chapter 7	User Experience Design
R6	This document, Chapter 8	Value Distribution & Economic Mechanisms
R7	RFC 7807	Problem Details for HTTP APIs
R8	OpenAPI 3.1 Specification	API documentation standard
R9	JSON:API Specification	JSON formatting standard for APIs
R10	GDPR	General Data Protection Regulation (EU 2016/679)

2. System Overview

2.1 The 3-Party Model (Supplier-Consumer-Enhancer)

The platform operates on a triadic economic model where three distinct roles collaborate to create value that would be difficult or impossible to achieve in bilateral transactions.



Key insight: Unlike traditional two-sided marketplaces where the platform merely connects supply and demand, the 3-Party Deal Making Platform actively orchestrates a value-creation triangle. The Enhancer role is the critical differentiator — without the Enhancer’s input, many Supplier-Consumer pairings would be economically infeasible or operationally impractical.

Examples across domains:

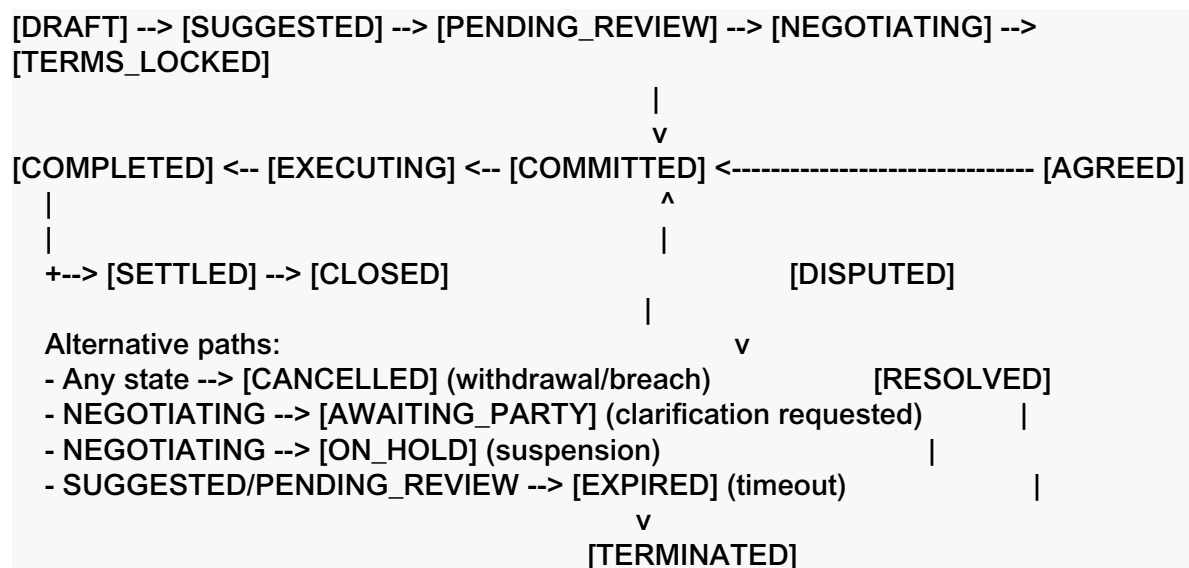
Domain	Supplier	Enhancer	Consumer
Agriculture	Farmer with idle land	Agrodealer (seeds, fertilizer, expertise)	Grocery chain needing produce
Real Estate	Building owner	Renovation contractor	Retailer needing

Domain	Supplier	Enhancer	Consumer
Transportation	Vehicle fleet owner	Maintenance mechanic	storefront Logistics company needing delivery capacity
Manufacturing	Raw material supplier	Quality certifier / processor	Finished goods manufacturer
Technology	Data owner	ML engineer / analyst	Company needing insights

Any party can initiate: A Supplier with idle land can find a Consumer needing crops and an Enhancer to bridge the gap. A Consumer needing rental space can find a Supplier with a vacant building and an Enhancer to renovate it. An Enhancer with processing expertise can proactively identify Supplier-Consumer pairs that would benefit from their service.

2.2 Deal Lifecycle at a Glance

The deal lifecycle is the central orchestration mechanism of the platform. It defines the states a deal progresses through from initial idea to final settlement.



The full 13-state deal lifecycle is specified in Chapter 4 (Workflow & State Machine Design). For a detailed state machine diagram with all transitions, timeouts, and validation gates, see Section 4.1.

2.3 Key Capabilities

Capability	Description	Related Chapters
Multi-Party Matching	Algorithmic compatibility scoring across 7 dimensions (resource-need alignment, value alignment, trust score, geographic fit, temporal alignment, historical success, risk	Chapter 3 (3.5, 3.14), Chapter 4 (4.4), Chapter 8 (8.2)

Capability	Description	Related Chapters
Win-Win-Win Validation	profile) Mathematical validation ensuring all three parties achieve positive net value; computes fairness score across absolute gain, proportional fairness, market benchmark, and opportunity cost dimensions	Chapter 3 (3.8), Chapter 8 (8.1)
Flexible Negotiation	Multi-threaded negotiation on value distribution, deliverables, timeline, roles, risk allocation, and termination; supports sequential, parallel, and hybrid paths	Chapter 4 (4.3), Chapter 7 (7.4)
Milestone Tracking	Progress tracking with conditional payment releases; 5 verification methods (photo evidence, document review, third-party inspection, IoT sensor data, counter-party acceptance)	Chapter 4 (4.5), Chapter 7 (7.6)
Trust & Reputation	Multi-dimensional trust scoring (0-100) across transaction history, reviews, profile completeness, verification, response rate, dispute history, longevity, and community contribution	Chapter 3 (3.21), Chapter 8 (8.4)
Domain Adaptability	Hierarchical category system allowing deals to be classified by industry, geography, resource type, and output type; pluggable domain-specific logic	Chapter 3 (3.24)
Escrow & Settlement	Consumer-funded escrow with milestone-based releases; supports 5 payment models (escrow	Chapter 4 (4.5.3), Chapter 8 (8.6)

Capability	Description	Related Chapters
	default, staged direct, performance bond, revenue share, hybrid)	
Dispute Resolution	Three-tier system: direct negotiation (7 days), platform mediation (10 days), binding arbitration (decision in 7 days)	Chapter 4 (4.5.5), Chapter 8 (8.6.4)
Group Support	PartyGroup entity with configurable governance models (any-one, majority, all, threshold, weighted) and internal voting workflows	Chapter 3 (3.5, 3.6), Chapter 4 (3.7)

2.4 Stakeholders

Stakeholder	Role	Interests
Suppliers	Resource/asset owners	Monetize idle resources, find reliable consumers and enhancers
Consumers	Output/service seekers	Access quality outputs at fair prices with supply assurance
Enhancers	Service/input providers	Find deal opportunities, build reputation, recurring revenue
Platform Operator	System owner	Facilitate deals, collect fees, grow marketplace liquidity
Platform Admins	Operations team	Moderate content, resolve disputes, manage platform config
Regulators	Government bodies	Ensure compliance, audit trails, fair dealing
Integration Partners	Third-party developers	Build on platform APIs, extend functionality

Document Information

Property	Value
Version	1.0.0
Status	Draft
Domain	Multi-Party Deal Facilitation Platform
Pattern	Domain-Driven Design (DDD)

3. Overview & Domain Narrative

The platform facilitates **triadic deals** among three distinct party types:

- **Supplier:** Holds an underutilized resource or asset (e.g., idle farmland, vacant building, unused vehicle, idle machinery) and seeks to derive value from it.
- **Consumer:** Desires a specific output, product, or service (e.g., crop produce, rental space, transportation, processed goods).
- **Enhancer:** Bridges the gap by providing enabling input, expertise, or services (e.g., agrodealer furnishing seeds/fertilizer, contractor performing renovation, mechanic providing maintenance).

Any party can initiate a deal. Each party may be an individual, organization, or a **PartyGroup** (a collective of people/entities pooling resources or needs). The platform validates that every deal satisfies the **Win-Win-Win Principle**: all three parties must receive net positive value.

The platform operates across **any domain**, using a flexible categorization system that allows deals to be classified by industry, geography, resource type, and output type.

3. Ubiquitous Language

Term	Definition
Deal	A potential or actualized 3-party arrangement connecting a Supplier, Consumer, and Enhancer around a specific value exchange.
Party	An individual, organization, or group participating in the platform in one or more roles.
Role	The function a party plays within a deal: Supplier, Consumer, or Enhancer.
Resource	An asset, property, or capability offered by a Supplier.
Need	A requirement, demand, or desired output specified by a Consumer.
Enhancement	An input, service, or value-add provided by an Enhancer to make the deal feasible.
Match	A platform-generated suggestion linking three compatible parties.
Term	A negotiable condition within a deal (price, timeline, quality standard, etc.).
Agreement	A mutually accepted set of terms formalizing a deal into a binding contract.
Value Distribution	The allocation of economic value among the three parties and the platform.
Trust Score	A composite reputation metric derived from reviews, completion history, and platform behavior.
Party Group	A composite entity allowing multiple people/organizations to act as a single Party.
Win-Win-Win Validation	The platform's verification that every deal delivers positive net value to all three parties.

3. Core Entities & Attributes

3.1 Party (Abstract Aggregate Root)

The central identity entity representing any participant on the platform.

Attribute	Type	Constraints	Description
partyId	UUID	PK, immutable	Unique platform identifier.
partyType	ENUM	INDIVIDUAL, ORGANIZATION, PARTY_GROUP	Classification of the party structure.
displayName	String	3–120 chars, required	Public-facing name or brand.
email	String	Unique, validated	Primary contact email.
phone	String	Optional, E.164 format	Primary contact phone.
taxId	String	Optional, encrypted at rest	Tax/VAT identifier for invoicing.
verificationStatus	ENUM	UNVERIFIED, PENDING, VERIFIED, REJECTED	KYC/Business verification state.
primaryDomain	UUID	FK → Category	Default domain of operation.
location	GeoPoint	lat/long + resolved address	Primary operational location.
serviceRadiusKm	Decimal	Nullable, ≥ 0	Geographic service reach.
trustScore	Decimal	0.00 – 5.00, derived	Current composite trust score.
totalDealsCompleted	Integer	≥ 0, derived	Count of successfully completed deals.
totalDealsInitiated	Integer	≥ 0, derived	Count of deals this party initiated.
isActive	Boolean	Default true	Soft-delete / suspension flag.
createdAt	Timestamp	Auto-set	Registration timestamp.
updatedAt	Timestamp	Auto-update	Last modification timestamp.

Inheritance Strategy: Single-Table Inheritance (STI) with **partyType** discriminator, or Mapped Superclass pattern for domain-specific extensions.

3.2 Supplier (Party Subtype)

A Party offering resources/assets.

Attribute	Type	Constraints	Description
supplierId	UUID	PK, 1:1 with Party	Subtype identifier.
resourceTypes	UUID[]	FK → Category[]	Categories of resources typically supplied.
typicalCapacity	String	Nullable	Usual volume/capacity description.
availabilitySchedule	JSON	Nullable	When resources are available.
preferredCompensation	ENUM[]	FIXED_FEE, REVENUE_SHARE, HYBRID, IN_KIND	How supplier prefers to be rewarded.
insuranceVerified	Boolean	Default false	Whether liability insurance is verified.

3.3 Consumer (Party Subtype)

A Party expressing needs/requirements.

Attribute	Type	Constraints	Description
consumerId	UUID	PK, 1:1 with Party	Subtype identifier.
needCategories	UUID[]	FK → Category[]	Categories of needs typically expressed.
typicalVolume	String	Nullable	Usual order volume description.
preferredQualityStandard	String	Nullable	Quality expectations.
budgetRangeMin	Decimal	≥ 0, nullable	Minimum budget.
budgetRangeMax	Decimal	≥ budgetRangeMin, nullable	Maximum budget.
preferredPaymentTerms	ENUM[]	UPFRONT, MILESTONE, ON_DELIVERY, DEFERRED	Payment timing preferences.

3.4 Enhancer (Party Subtype)

A Party providing enabling inputs or services.

Attribute	Type	Constraints	Description
enhancerId	UUID	PK, 1:1 with Party	Subtype identifier.
enhancementTypes	UUID[]	FK → Category[]	Types of enhancements offered.
skills	String[]	Optional	Specific skills or certifications.
certifications	JSON	Optional	Verified credentials.
hourlyRate	Decimal	≥ 0, nullable	Standard hourly rate if

Attribute	Type	Constraints	Description
fixedRate	Decimal	≥ 0 , nullable	applicable. Standard fixed fee if applicable.
equipmentOwned	String[]	Optional	Equipment/tools available.
availability	JSON	Nullable	Schedule of availability.
typicalEngagementDuration	String	Nullable	Typical time to complete enhancement.

3.5 PartyGroup

Enables multiple individuals/entities to act as a single Party. A Party of **partyType = PARTY_GROUP** has associated PartyGroup metadata.

Attribute	Type	Constraints	Description
groupId	UUID	PK	Unique group identifier.
partyId	UUID	FK → Party, unique	Links to the parent Party record.
groupName	String	3–200 chars	Name of the collective.
groupType	ENUM	COOPERATIVE, PARTNERSHIP, INFORMAL_POOL, CORPORATE_ALLIANCE	Structural form.
formationDate	Date	Optional	When the group was formed.
governanceModel	String	Optional	Decision-making description.
memberCount	Integer	≥ 1 , derived	Current number of members.
isOpenToNewMembers	Boolean	Default false	Whether new members can join.
minimumMemberApproval	Integer	≥ 1 , $\leq$ memberCount	Votes needed for deal approval.
sharedBankAccount	Boolean	Default false	Whether finances are pooled.

3.6 GroupMembership

Links individual Parties to a PartyGroup.

Attribute	Type	Constraints	Description
membershipId	UUID	PK	Unique membership identifier.
groupId	UUID	FK → PartyGroup	The group being joined.

Attribute	Type	Constraints	Description
memberPartyId	UUID	FK → Party	The individual member.
roleInGroup	ENUM	LEADER, ADMIN, MEMBER, OBSERVER	Authority level within the group.
joinedAt	Timestamp	Auto-set	Membership start date.
contributionPercentage	Decimal	0 – 100	Share of contribution/reward.
isActive	Boolean	Default true	Whether membership is current.

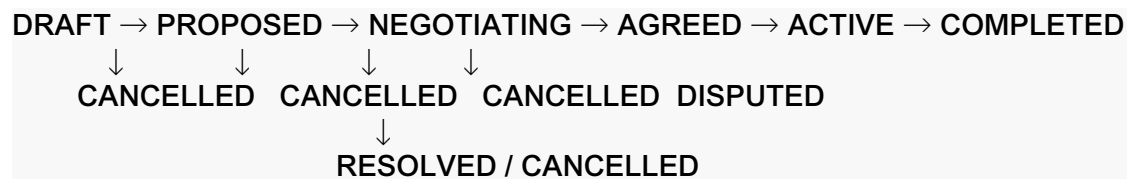
3.7 Deal (Aggregate Root)

The central entity representing a 3-party deal from inception through completion.

Attribute	Type	Constraints	Description
dealId	UUID	PK	Unique deal identifier.
initiatorPartyId	UUID	FK → Party, non-null	Who created/suggested the deal.
initiatorRole	ENUM	SUPPLIER, CONSUMER, ENHANCER	Role of the initiator.
dealStatus	ENUM	See state machine below	Current lifecycle status.
dealTitle	String	10–300 chars	Human-readable deal title.
dealDescription	Text	Optional	Detailed description.
domainCategoryId	UUID	FK → Category, non-null	Primary domain/industry.
expectedStartDate	Date	Optional	Anticipated commencement.
expectedEndDate	Date	≥ expectedStartDate	Anticipated completion.
actualStartDate	Date	Nullable	When the deal actually started.
actualEndDate	Date	Nullable	When the deal actually completed.
location	GeoPoint	Nullable	Deal location/region.
totalDealValue	Decimal	≥ 0, nullable	Total economic value of the deal.
currency	String	ISO 4217, default USD	Currency for all monetary values.
platformFeePercentage	Decimal	0 – 100, default platform rate	Platform commission rate.
platformFeeAmount	Decimal	≥ 0, derived	Calculated platform fee.
winWinWinValidated	Boolean	Default false	Whether the deal passes validation.

Attribute	Type	Constraints	Description
validationCheckedAt	Timestamp	Nullable	When validation was performed.
isPublic	Boolean	Default true	Visibility to non-participants.
createdAt	Timestamp	Auto-set	Creation timestamp.
updatedAt	Timestamp	Auto-update	Last modification timestamp.

Deal Status State Machine:



3.8 DealParticipation

Links a Party to a Deal with a specific role. Every Deal **MUST** have exactly 3 DealParticipation records (one per role).

Attribute	Type	Constraints	Description
participationId	UUID	PK	Unique participation identifier.
dealId	UUID	FK → Deal	The deal being joined.
partyId	UUID	FK → Party	The participating party.
role	ENUM	SUPPLIER, CONSUMER, ENHANCER	Role in this specific deal.
participationStatus	ENUM	INVITED, PENDING, ACCEPTED, DECLINED, WITHDRAWN	Current participation state.
invitedAt	Timestamp	Nullable	When the invitation was sent.
respondedAt	Timestamp	Nullable	When the party accepted/declined.
valueSharePercentage	Decimal	0 – 100	Share of total deal value.
valueShareAmount	Decimal	≥ 0, derived	Monetary value of share.
isInitiator	Boolean	Default false	Whether this party initiated the deal.

3.9 Resource (Supplier Offering)

Describes what the Supplier brings to the deal.

Attribute	Type	Constraints	Description
resourceId	UUID	PK	Unique resource identifier.
dealId	UUID	FK → Deal	Parent deal.
supplierPartyId	UUID	FK → Party	The supplying party.
resourceTypeId	UUID	FK → Category	Category classification.
resourceName	String	3–200 chars	Name of the resource.
description	Text	Optional	Detailed description.
quantity	Decimal	≥ 0	Amount/quantity available.
quantityUnit	String	Required	Unit of measure (acres, sqm, hours, etc.).
condition	ENUM	NEW, GOOD, FAIR, POOR, VARIABLE	Condition of the resource.
location	GeoPoint	Nullable	Where the resource is located.
availabilityStart	Date	Optional	When resource becomes available.
availabilityEnd	Date	Optional	When resource is no longer available.
documentUrls	String[]	Optional	Photos, deeds, certificates.
verifiedByPlatform	Boolean	Default false	Whether platform has verified the resource.

3.10 Need (Consumer Requirement)

Describes what the Consumer wants from the deal.

Attribute	Type	Constraints	Description
needId	UUID	PK	Unique need identifier.
dealId	UUID	FK → Deal	Parent deal.
consumerPartyId	UUID	FK → Party	The consuming party.
needCategoryId	UUID	FK → Category	Category classification.
needDescription	String	10–1000 chars	What the consumer wants.
requiredQuantity	Decimal	≥ 0	Volume/quantity needed.
quantityUnit	String	Required	Unit of measure.
qualityRequirements	Text	Optional	Quality specifications.
requiredByDate	Date	Optional	Deadline for delivery.
maxBudget	Decimal	≥ 0, nullable	Maximum willing to pay.
budgetCurrency	String	ISO 4217	Budget currency.
acceptableVariants	Text	Optional	Substitutes or alternatives

Attribute	Type	Constraints	Description
priority	ENUM	LOW, MEDIUM, HIGH, URGENT	accepted. Urgency level.

3.11 Enhancement (Enhancer Contribution)

Describes what the Enhancer provides to enable the deal.

Attribute	Type	Constraints	Description
enhancementId	UUID	PK	Unique enhancement identifier.
dealId	UUID	FK → Deal	Parent deal.
enhancerPartyId	UUID	FK → Party	The enhancing party.
enhancementTypeId	UUID	FK → Category	Category classification.
enhancementName	String	3–200 chars	Name of the input/service.
description	Text	Optional	Detailed description.
inputQuantity	Decimal	≥ 0	Amount of input provided.
quantityUnit	String	Required	Unit of measure.
serviceDurationHours	Decimal	≥ 0, nullable	Time commitment.
estimatedCompletionDays	Integer	≥ 1, nullable	Time to complete the enhancement.
deliverables	Text	Optional	Specific deliverables expected.
prerequisites	Text	Optional	What the enhancer needs from others.
isComplete	Boolean	Default false	Whether the enhancement is delivered.
completedAt	Timestamp	Nullable	Completion timestamp.

3.12 Term (Negotiable Condition)

Individual clauses negotiated among the parties.

Attribute	Type	Constraints	Description
termId	UUID	PK	Unique term identifier.
dealId	UUID	FK → Deal	Parent deal.
proposedByPartyId	UUID	FK → Party	Who suggested this term.
termType	ENUM	PRICE, TIMELINE, QUALITY, PAYMENT_SCHEDULE, LIABILITY,	Category of term.

Attribute	Type	Constraints	Description
		EXCLUSIVITY, TERMINATION, CUSTOM	
termName	String	3–200 chars	Short name/title.
description	Text	Required	Full term text.
negotiationStatus	ENUM	PROPOSED, UNDER_REVIEW, ACCEPTED, REJECTED, COUNTERED	Current negotiation state.
parentTermId	UUID	FK → Term, nullable	For counter-proposals (self-ref).
version	Integer	≥ 1, default 1	Term revision number.
proposedAt	Timestamp	Auto-set	When the term was proposed.
resolvedAt	Timestamp	Nullable	When the term was finalized.
isMandatory	Boolean	Default false	Whether the term is required by platform.
resolution	String	Nullable	Notes on how term was resolved.

3.13 ValueDistribution

Defines how the total deal value is allocated among all stakeholders.

Attribute	Type	Constraints	Description
distributionId	UUID	PK	Unique distribution identifier.
dealId	UUID	FK → Deal, unique	1:1 with Deal.
totalValue	Decimal	≥ 0	Total deal value being distributed.
currency	String	ISO 4217	Currency.
supplierSharePercentage	Decimal	0 – 100	Supplier's percentage.
supplierShareAmount	Decimal	≥ 0, derived	Supplier's monetary share.
consumerCostPercentage	Decimal	0 – 100	Consumer's contribution percentage.
consumerCostAmount	Decimal	≥ 0, derived	Consumer's monetary contribution.
enhancerSharePercentage	Decimal	0 – 100	Enhancer's percentage.
enhancerShareAmount	Decimal	≥ 0, derived	Enhancer's monetary

Attribute	Type	Constraints	Description
unt			share.
platformFeePercent	Decimal	0 – 100	Platform commission percentage.
platformFeeAmount	Decimal	≥ 0, derived	Platform fee.
distributionModel	ENUM	FIXED_PRICE, REVENUE_SHARE, COST_PLUS, HYBRID	How value is calculated.
paymentSchedule	JSON	Required	Milestones and payment triggers.
winWinWinScore	Decimal	0 – 100	Calculated fairness score.
createdAt	Timestamp	Auto-set	Creation timestamp.
updatedAt	Timestamp	Auto-update	Last update timestamp.

3.14 Match (Platform Suggestion)

A platform-generated compatibility suggestion linking parties to a potential deal.

Attribute	Type	Constraints	Description
matchId	UUID	PK	Unique match identifier.
matchStatus	ENUM	PENDING, ACCEPTED_BY_ALL, REJECTED, EXPIRED, CONVERTED_TO_DEAL	Lifecycle status.
supplierPartyId	UUID	FK → Party	Suggested supplier.
consumerPartyId	UUID	FK → Party	Suggested consumer.
enhancerPartyId	UUID	FK → Party	Suggested enhancer.
matchScore	Decimal	0.00 – 1.00	Algorithmic compatibility score.
matchReason	Text	Optional	Human-readable explanation of the match.
resourceCategoryId	UUID	FK → Category	Matched resource category.
needCategoryId	UUID	FK → Category	Matched need category.
enhancementCategoryId	UUID	FK → Category	Matched enhancement category.
suggestedDealValue	Decimal	≥ 0, nullable	Estimated deal value.
generatedBy	ENUM	ALGORITHM, PLATFORM_ADMIN, USER_REFERRAL	Source of the match.
expiresAt	Timestamp	Nullable	Match expiration time.
convertedDealId	UUID	FK → Deal, nullable	Deal created from this

Attribute	Type	Constraints	Description
createdAt	Timestamp	Auto-set	match. Generation timestamp.

3.15 MatchResponse

Individual party response to a match suggestion.

Attribute	Type	Constraints	Description
responseId	UUID	PK	Unique response identifier.
matchId	UUID	FK → Match	Parent match.
partyId	UUID	FK → Party	Responding party.
response	ENUM	PENDING, ACCEPTED, DECLINED, COUNTER_PROPOSED	Response value.
responseNotes	Text	Optional	Free-text feedback.
respondedAt	Timestamp	Nullable	When response was given.

3.16 Agreement (Aggregate Root)

The formalized, legally-binding contract that emerges from a negotiated deal.

Attribute	Type	Constraints	Description
agreementId	UUID	PK	Unique agreement identifier.
dealId	UUID	FK → Deal, unique	Source deal (1:1).
agreementStatus	ENUM	DRAFT, PENDING_SIGNATURES, FULLY_SIGNED, ACTIVE, COMPLETED, TERMINATED, DISPUTED	Lifecycle status.
agreementText	Text	Required	Full legal text of the agreement.
governingLaw	String	Required	Jurisdiction for disputes.
disputeResolution	ENUM	MEDIATION, ARBITRATION, LITIGATION	Dispute resolution mechanism.
effectiveDate	Date	Nullable	When agreement takes effect.
terminationDate	Date	Nullable	When agreement ends.
autoRenew	Boolean	Default false	Whether agreement renews automatically.

Attribute	Type	Constraints	Description
version	Integer	≥ 1	Agreement revision number.
digitalSignatureUrl	String	Optional	URL to signed document.
createdAt	Timestamp	Auto-set	Creation timestamp.
executedAt	Timestamp	Nullable	When all parties signed.

3.17 Signature

Individual party signature on an agreement.

Attribute	Type	Constraints	Description
signatureId	UUID	PK	Unique signature identifier.
agreementId	UUID	FK → Agreement	Parent agreement.
partyId	UUID	FK → Party	Signing party.
signedByUserId	UUID	Required	Specific user who signed.
signatureType	ENUM	DIGITAL, E_SIGN, BIOMETRIC, MANUAL_UPLOAD	Method of signature.
signatureData	String	Required	Signature token/hash/reference.
ipAddress	String	Optional	IP at time of signing.
signedAt	Timestamp	Auto-set	Signature timestamp.

3.18 Transaction (Event/Financial)

Records financial flows and milestone events within a deal.

Attribute	Type	Constraints	Description
transactionId	UUID	PK	Unique transaction identifier.
dealId	UUID	FK → Deal	Parent deal.
agreementId	UUID	FK → Agreement, nullable	Related agreement if post-agreement.
transactionType	ENUM	PAYMENT, ESCROW_DEPOSIT, ESCROW_RELEASE, MILESTONE, FEE, REFUND, ADJUSTMENT, DISPUTE_HOLD	Type of transaction.
fromPartyId	UUID	FK → Party, nullable	Payer/source.

Attribute	Type	Constraints	Description
toPartyId	UUID	FK → Party, nullable	Payee/destination.
amount	Decimal	≥ 0	Transaction amount.
currency	String	ISO 4217	Currency.
description	String	Optional	Transaction description.
status	ENUM	PENDING, COMPLETED, FAILED, REVERSED, HELD	Transaction status.
paymentMethod	ENUM	BANK_TRANSFER, CARD, PLATFORM_WALLET, ESCROW, CRYPTO, IN_KIND	Payment method.
externalReference	String	Optional	External payment gateway reference.
executedAt	Timestamp	Nullable	When transaction completed.
createdAt	Timestamp	Auto-set	Creation timestamp.

3.19 Milestone

Trackable deliverables within an active deal.

Attribute	Type	Constraints	Description
milestoneId	UUID	PK	Unique milestone identifier.
dealId	UUID	FK → Deal	Parent deal.
milestoneName	String	3–200 chars	Name of the milestone.
description	Text	Optional	Detailed description.
assignedToPartyId	UUID	FK → Party	Responsible party.
dueDate	Date	Optional	Target completion date.
completionCriteria	Text	Required	What constitutes completion.
milestoneStatus	ENUM	PENDING, IN_PROGRESS, COMPLETED, OVERDUE, BLOCKED	Current status.
completionPercentage	Decimal	0 – 100	Progress indicator.
paymentTriggerAmount	Decimal	≥ 0, nullable	Amount released on completion.
completedAt	Timestamp	Nullable	Actual completion time.
verifiedByPartyId	UUID	FK → Party, nullable	Who verified completion.

Attribute	Type	Constraints	Description
displayOrder	Integer	≥ 1	Sequence order.

3.20 Review (Rating & Feedback)

Post-deal reputation feedback.

Attribute	Type	Constraints	Description
reviewId	UUID	PK	Unique review identifier.
dealId	UUID	FK → Deal	Related deal.
reviewerPartyId	UUID	FK → Party	Who wrote the review.
reviewedPartyId	UUID	FK → Party	Who is being reviewed.
reviewedRole	ENUM	SUPPLIER, CONSUMER, ENHANCER	Role of reviewed party.
overallRating	Integer	1 – 5	Overall star rating.
communicationRating	Integer	1 – 5, nullable	Communication quality.
reliabilityRating	Integer	1 – 5, nullable	Reliability rating.
qualityRating	Integer	1 – 5, nullable	Quality of output/service.
timelinessRating	Integer	1 – 5, nullable	Timeliness rating.
reviewText	Text	Optional	Free-text review.
isVerified	Boolean	Default false	Whether reviewer completed the deal.
isPublic	Boolean	Default true	Visibility.
platformResponse	Text	Optional	Platform moderation note.
createdAt	Timestamp	Auto-set	Creation timestamp.

3.21 TrustScore (Aggregate Root)

Computed reputation metrics for each party.

Attribute	Type	Constraints	Description
trustScoreId	UUID	PK	Unique identifier.
partyId	UUID	FK → Party, unique	One score per party.
overallScore	Decimal	0.00 – 5.00	Weighted composite score.
asSupplierScore	Decimal	0.00 – 5.00, nullable	Score when acting as supplier.
asConsumerScore	Decimal	0.00 – 5.00, nullable	Score when acting as consumer.
asEnhancerScore	Decimal	0.00 – 5.00, nullable	Score when acting as enhancer.

Attribute	Type	Constraints	Description
dealsCompletedCount	Integer	≥ 0	Successful deals.
dealsCancelledCount	Integer	≥ 0	Cancelled deals.
dealsDisputedCount	Integer	≥ 0	Disputed deals.
averageResponseHours	Decimal	≥ 0 , nullable	Average time to respond.
profileCompleteness	Decimal	0 – 100	Profile completion percentage.
verificationLevel	Integer	0 – 5	Progressive verification tier.
longevityDays	Integer	≥ 0	Days since registration.
calculationFormula	JSON	Required	Weights used in calculation.
lastCalculatedAt	Timestamp	Auto-set	Last recalculation time.
nextCalculationAt	Timestamp	Required	Scheduled next recalculation.

3.22 Message (Communication)

Platform messaging between parties.

Attribute	Type	Constraints	Description
messageId	UUID	PK	Unique message identifier.
dealId	UUID	FK → Deal, nullable	Related deal if contextual.
matchId	UUID	FK → Match, nullable	Related match if pre-deal.
senderPartyId	UUID	FK → Party	Message sender.
recipientPartyId	UUID	FK → Party	Message recipient.
messageType	ENUM	TEXT, FILE, SYSTEM, OFFER, COUNTER_OFFER	Message category.
subject	String	Optional	Message subject line.
content	Text	Required	Message body.
attachmentUrls	String[]	Optional	Attached files.
isRead	Boolean	Default false	Read status.
readAt	Timestamp	Nullable	When message was read.
replyToMessageId	UUID	FK → Message, nullable	Threading reference.
isFlagged	Boolean	Default false	Moderation flag.
createdAt	Timestamp	Auto-set	Send timestamp.

Attribute	Type	Constraints	Description
	P		

3.23 Notification

System-generated alerts to parties.

Attribute	Type	Constraints	Description
notificationId	UUID	PK	Unique notification identifier.
partyId	UUID	FK → Party	Recipient party.
notificationType	ENUM	MATCH_SUGGESTION, DEAL_INVITE, TERM_UPDATE, PAYMENT_DUE, MILESTONE_DUE, REVIEW_REQUEST, DISPUTE_ALERT, TRUST_UPDATE, SYSTEM	Alert category.
title	String	Required	Notification headline.
body	Text	Required	Notification content.
relatedEntityType	ENUM	DEAL, MATCH, AGREEMENT, TRANSACTION, MILESTONE, REVIEW, MESSAGE	Context entity type.
relatedEntityId	UUID	Nullable	Context entity identifier.
channel	ENUM[]	IN_APP, EMAIL, SMS, PUSH	Delivery channels.
isRead	Boolean	Default false	Read status.
isActioned	Boolean	Default false	Whether user acted on notification.
actionUrl	String	Optional	Deep link to relevant page.
createdAt	Timestamp	Auto-set	Creation timestamp.
expiresAt	Timestamp	Nullable	Notification expiry.

3.24 Category (Domain Classification)

Hierarchical classification system for resources, needs, enhancements, and deals.

Attribute	Type	Constraints	Description
categoryId	UUID	PK	Unique category identifier.

Attribute	Type	Constraints	Description
parentCategoryId	UUID	FK → Category, nullable	Self-referencing hierarchy.
categoryName	String	2–100 chars, unique	Human-readable name.
categoryCode	String	Unique, slug	URL-safe identifier.
description	Text	Optional	Category description.
categoryType	ENUM	DOMAIN, RESOURCE_TYPE, NEED_TYPE, ENHANCEMENT_TYPE, LOCATION, CUSTOM	Classification purpose.
iconUrl	String	Optional	Visual icon.
metadataSchema	JSON	Optional	Expected attributes for this category.
isActive	Boolean	Default true	Whether category is available.
displayOrder	Integer	≥ 1	Sort order within parent.
dealCount	Integer	≥ 0, derived	Number of deals in this category.

3.25 Dispute

Handles conflicts arising during or after a deal.

Attribute	Type	Constraints	Description
disputeId	UUID	PK	Unique dispute identifier.
dealId	UUID	FK → Deal	Related deal.
agreementId	UUID	FK → Agreement, nullable	Related agreement.
raisedByPartyId	UUID	FK → Party	Complainant.
againstPartyId	UUID	FK → Party	Respondent.
disputeType	ENUM	NON_PAYMENT, NON_DELIVERY, QUALITY_ISSUE, BREACH_OF_TERMS, COMMUNICATION, OTHER	Nature of dispute.
description	Text	Required	Detailed complaint.
evidenceUrls	String[]	Optional	Supporting documents.
disputeStatus	ENUM	OPEN, UNDER_REVIEW, MEDIATION, RESOLVED, ESCALATED, CLOSED	Current status.
resolution	Text	Nullable	Resolution description.

Attribute	Type	Constraints	Description
resolutionType	ENUM	REFUND, PARTIAL_REFUND, REPLACEMENT, NO_ACTION, COMPENSATION, TERMINATION	How resolved.
resolvedBy	UUID	FK → Party, nullable	Mediator/admin who resolved.
compensationAmount	Decimal	≥ 0, nullable	Any monetary resolution.
createdAt	Timestamp	Auto-set	Filing timestamp.
resolvedAt	Timestamp	Nullable	Resolution timestamp.

3.26 PlatformWallet

Internal ledger for platform-managed payments.

Attribute	Type	Constraints	Description
walletId	UUID	PK	Unique wallet identifier.
partyId	UUID	FK → Party, unique	Owner of the wallet.
balance	Decimal	≥ 0	Current available balance.
escrowBalance	Decimal	≥ 0	Funds held in escrow.
pendingBalance	Decimal	≥ 0	Funds in pending transactions.
totalDeposited	Decimal	≥ 0, derived	Lifetime deposits.
totalWithdrawn	Decimal	≥ 0, derived	Lifetime withdrawals.
currency	String	ISO 4217	Wallet currency.
isActive	Boolean	Default true	Wallet status.
createdAt	Timestamp	Auto-set	Creation timestamp.
updatedAt	Timestamp	Auto-update	Last update timestamp.

3. Relationships & Cardinalities

3.1 Party-Centric Relationships

Relationship	Cardinality	Description
Party → Supplier	1:0..1	A Party may optionally be a

Relationship	Cardinality	Description
		Supplier.
Party → Consumer	1:0..1	A Party may optionally be a Consumer.
Party → Enhancer	1:0..1	A Party may optionally be an Enhancer.
Party → PartyGroup	1:0..1	A Party of type PARTY_GROUP has one PartyGroup detail.
PartyGroup → GroupMembership	1:N	A group has multiple memberships.
GroupMembership → Party	N:1	Each membership links to one member Party.
Party → TrustScore	1:1	Every Party has exactly one TrustScore.
Party → PlatformWallet	1:1	Every Party has exactly one wallet.
Party → Deal (as initiator)	1:N	A Party can initiate many Deals.
Party → DealParticipation	1:N	A Party can participate in many Deals.

3.2 Deal-Centric Relationships

Relationship	Cardinality	Description
Deal → DealParticipation	1:3	Every Deal has exactly 3 participations (Supplier, Consumer, Enhancer).
Deal → Resource	1:0..N	A Deal involves one or more Resources from the Supplier.
Deal → Need	1:0..N	A Deal addresses one or more Needs from the Consumer.
Deal → Enhancement	1:0..N	A Deal involves one or more Enhancements from the Enhancer.
Deal → Term	1:N	A Deal has many negotiated Terms.
Deal → ValueDistribution	1:1	Every Deal has exactly one ValueDistribution.
Deal → Agreement	1:0..1	A Deal may have one Agreement (0 until AGREED status).
Deal → Transaction	1:N	A Deal generates many Transactions.

Relationship	Cardinality	Description
Deal → Milestone	1:N	A Deal has many Milestones.
Deal → Message	1:N	A Deal has many contextual Messages.
Deal → Review	1:N	A Deal has Reviews from participating parties.
Deal → Dispute	1:0..N	A Deal may have zero or more Disputes.
Deal → Match	0..1:1	A Deal may originate from exactly one Match.

3.3 Match-Centric Relationships

Relationship	Cardinality	Description
Match → MatchResponse	1:3	Every Match has exactly 3 responses (one per party).
Match → Party (Supplier)	N:1	Many Matches can suggest the same Supplier.
Match → Party (Consumer)	N:1	Many Matches can suggest the same Consumer.
Match → Party (Enhancer)	N:1	Many Matches can suggest the same Enhancer.

3.4 Agreement-Centric Relationships

Relationship	Cardinality	Description
Agreement → Signature	1:3	Every Agreement has exactly 3 Signatures.
Agreement → Transaction	1:N	An Agreement generates financial Transactions.

3.5 Category Relationships

Relationship	Cardinality	Description
Category → Category (parent)	N:1	Self-referencing hierarchy (tree structure).
Category → Deal	1:N	A Category classifies many Deals.
Category → Resource	1:N	A Category classifies many Resources.
Category → Need	1:N	A Category classifies many Needs.
Category → Enhancement	1:N	A Category classifies many Enhancements.

3.6 Communication Relationships

Relationship	Cardinality	Description
Party → Message (sent)	1:N	A Party sends many Messages.
Party → Message (received)	1:N	A Party receives many Messages.
Party → Notification	1:N	A Party receives many Notifications.
Message → Message (reply)	1:N	Self-referencing thread structure.

3.7 Review Relationships

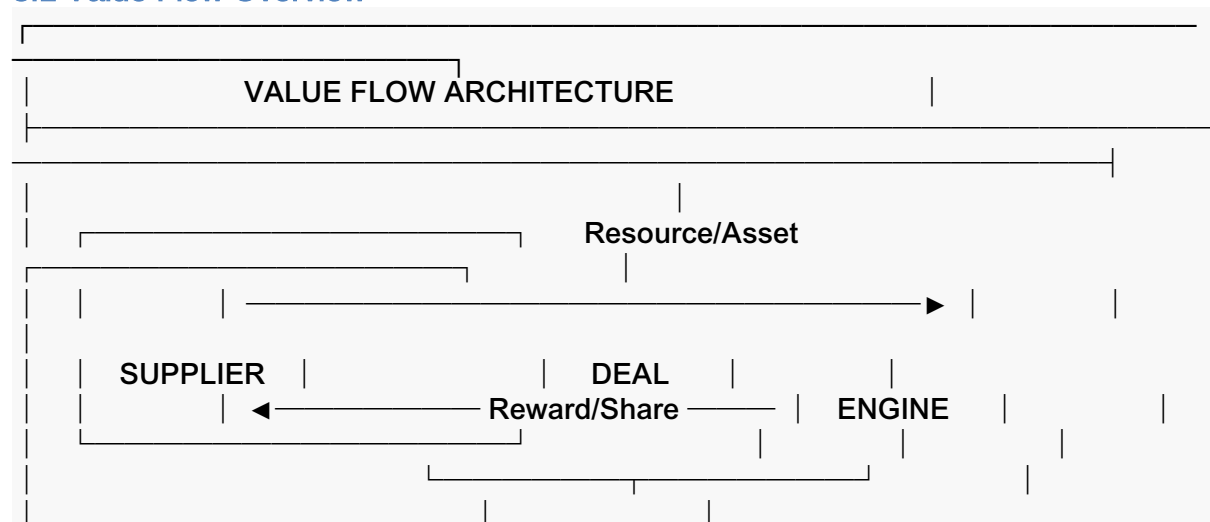
Relationship	Cardinality	Description
Party → Review (given)	1:N	A Party writes many Reviews.
Party → Review (received)	1:N	A Party receives many Reviews.
Deal → Review	1:N	A Deal has many Reviews (up to 6: each party reviews 2 others).

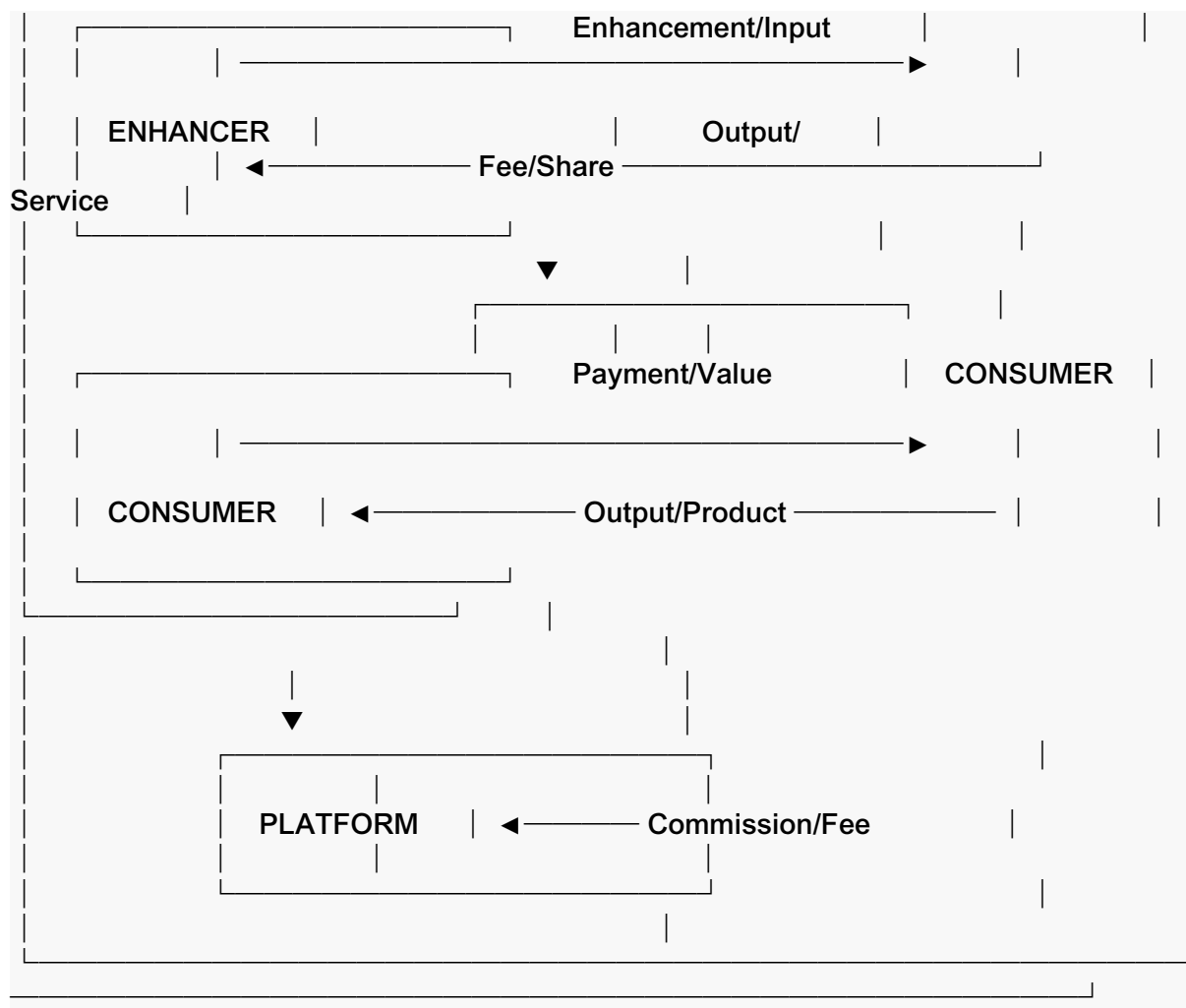
3.8 Financial Relationships

Relationship	Cardinality	Description
Party → Transaction (as sender)	1:N	A Party initiates many Transactions.
Party → Transaction (as receiver)	1:N	A Party receives many Transactions.
PlatformWallet → Transaction	1:N	A Wallet records many Transactions.

3. Value Flow Model

3.1 Value Flow Overview





3.2 Detailed Value Flow by Party

Supplier Value Flow

INPUTS:	OUTPUTS:
Resource/Asset (land, building, vehicle, machinery)	Reward Share (from ValueDistribution) (can be: fixed fee, revenue share, in-kind benefit, hybrid)
Availability (time, access)	Enhanced asset value (resource may be improved by enhancement)
Maintenance/upkeep responsibility	Potential ongoing revenue stream
Risk bearing (asset depreciation)	TrustScore increase on completion

Consumer Value Flow

INPUTS:	OUTPUTS:
Payment/Value (to platform escrow)	Desired Output (crop produce, rental space, transportation, processed goods)
Clear requirement specification	Quality-assured deliverable
Timely payment	Convenience (single deal, 3-party orchestration handled by platform)
Feedback/review	Potential long-term supply relationship

Enhancer Value Flow

INPUTS:	OUTPUTS:
└── Expertise/Skills	└── Fee/Share (from ValueDistribution)
└── Materials/Inputs (seeds, fertilizer, tools, equipment)	└── (can be: fixed fee, hourly rate, revenue share, milestone payments)
└── Labor/Time	└── Portfolio expansion (new domains, new relationships)
└── Risk bearing (input quality)	└── TrustScore as Enhancer
└── Quality guarantee	└── Potential recurring engagement

Platform Value Flow

INPUTS:	OUTPUTS:
└── Facilitation technology	└── Commission/Fee (% of deal value)
└── Matching algorithm	└── Data insights (market trends, pricing benchmarks)
└── Escrow/payment processing	└── Network effects (more parties, more deals)
└── Dispute resolution	└── Platform sustainability
└── Trust/reputation system	
└── Legal framework (terms, agreements)	

3.3 Value Distribution Mechanics

The **ValueDistribution** entity enforces the **Win-Win-Win Principle** through the following calculation:

Total Deal Value = Consumer Payment

Distributions:

- └── Supplier Share = $\text{totalValue} \times \text{supplierSharePercent}$
- └── Enhancer Share = $\text{totalValue} \times \text{enhancerSharePercent}$
- └── Platform Fee = $\text{totalValue} \times \text{platformFeePercent}$
- └── Consumer Surplus = $\text{totalValue} - (\text{supplier} + \text{enhancer} + \text{platform})$
(value received - value paid)

Validation Rules:

1. $\text{supplierSharePercent} + \text{enhancerSharePercent} + \text{platformFeePercent} = 100\%$
2. Each party's net value must be > 0 (positive for all three)
3. Consumer surplus $\geq$ minimum acceptable threshold (platform-defined)
4. The winWinWinScore must be $\geq$ platform minimum (e.g., 60/100)

3.4 Payment Flow Sequence

1. CONSUMER deposits funds → Platform ESCROW
2. ENHANCER delivers enhancement → Milestone verified
 - └── Partial release: enhancerShare released from escrow
3. SUPPLIER provides resource access → Milestone verified
 - └── Partial release: supplierShare released from escrow
4. CONSUMER receives output → Confirms satisfaction
 - └── Final release: remaining shares distributed
5. PLATFORM FEE automatically deducted at each release
6. If DISPUTE → Funds frozen → Resolution → Reallocation

3. ER Diagram (Mermaid)

title: 3-Party Deal Platform — Entity Relationship Diagram

erDiagram

%% =====

%% CORE PARTY AGGREGATE

%% =====

PARTY {

uuid partyId PK

enum partyType "INDIVIDUAL|ORGANIZATION|PARTY_GROUP"

string displayName

string email UK

string phone

string taxId

enum verificationStatus

uuid primaryDomainId FK

jsonb location

decimal serviceRadiusKm

decimal trustScore

int totalDealsCompleted

int totalDealsInitiated

boolean isActive

timestamp createdAt

timestamp updatedAt

}

SUPPLIER {

uuid supplierId PK

uuid partyId FK,UK

uuid[] resourceTypeIds

string typicalCapacity

jsonb availabilitySchedule

enum[] preferredCompensation

boolean insuranceVerified

}

CONSUMER {

uuid consumerId PK

uuid partyId FK,UK

uuid[] needCategoryIds

string typicalVolume

string preferredQualityStandard

decimal budgetRangeMin

decimal budgetRangeMax

enum[] preferredPaymentTerms

}

ENHANCER {

uuid enhancerId PK

uuid partyId FK,UK

uuid[] enhancementTypeIds

string[] skills

jsonb certifications

decimal hourlyRate

```
    decimal fixedRate
    string[] equipmentOwned
    jsonb availability
    string typicalEngagementDuration
}
```

```
PARTYGROUP {
    uuid groupId PK
    uuid partyId FK,UK
    string groupName
    enum groupType
    date formationDate
    string governanceModel
    int memberCount
    boolean isOpenToNewMembers
    int minimumMemberApproval
    boolean sharedBankAccount
}
```

```
GROUPMEMBERSHIP {
    uuid membershipId PK
    uuid groupId FK
    uuid memberPartyId FK
    enum roleInGroup
    timestamp joinedAt
    decimal contributionPercentage
    boolean isActive
}
```

%% =====

%% TRUST & REPUTATION

%% =====

```
TRUSTSCORE {
    uuid trustScoreId PK
    uuid partyId FK,UK
    decimal overallScore "0-5"
    decimal asSupplierScore "0-5"
    decimal asConsumerScore "0-5"
    decimal asEnhancerScore "0-5"
    int dealsCompletedCount
    int dealsCancelledCount
    int dealsDisputedCount
    decimal averageResponseHours
    decimal profileCompleteness
    int verificationLevel "0-5"
    int longevityDays
    jsonb calculationFormula
    timestamp lastCalculatedAt
    timestamp nextCalculationAt
}
```

%% =====

%% CATEGORY SYSTEM

%% =====

CATEGORY {

uuid categoryId PK
uuid parentCategoryId FK
string categoryName UK
string categoryCode UK
text description
enum categoryType
string iconUrl
jsonb metadataSchema
boolean isActive
int displayOrder
int dealCount

}

%% =====

%% DEAL AGGREGATE

%% =====

DEAL {

uuid dealId PK
uuid initiatorPartyId FK
enum initiatorRole
enum dealStatus
string dealTitle
text dealDescription
uuid domainCategoryId FK
date expectedStartDate
date expectedEndDate
date actualStartDate
date actualEndDate
jsonb location
decimal totalDealValue
string currency
decimal platformFeePercentage
decimal platformFeeAmount
boolean winWinWinValidated
timestamp validationCheckedAt
boolean isPublic
timestamp createdAt
timestamp updatedAt

}

DEALPARTICIPATION {

uuid participationId PK
uuid dealId FK
uuid partyId FK
enum role "SUPPLIER|CONSUMER|ENHANCER"
enum participationStatus
timestamp invitedAt
timestamp respondedAt
decimal valueSharePercentage

```
    decimal valueShareAmount
    boolean isInitiator
}
```

```
RESOURCE {
    uuid resourceId PK
    uuid dealId FK
    uuid supplierPartyId FK
    uuid resourceTypeId FK
    string resourceName
    text description
    decimal quantity
    string quantityUnit
    enum condition
    jsonb location
    date availabilityStart
    date availabilityEnd
    string[] documentUrls
    boolean verifiedByPlatform
}
```

```
NEED {
    uuid needId PK
    uuid dealId FK
    uuid consumerPartyId FK
    uuid needCategoryId FK
    string needDescription
    decimal requiredQuantity
    string quantityUnit
    text qualityRequirements
    date requiredByDate
    decimal maxBudget
    string budgetCurrency
    text acceptableVariants
    enum priority
}
```

```
ENHANCEMENT {
    uuid enhancementId PK
    uuid dealId FK
    uuid enhancerPartyId FK
    uuid enhancementTypeId FK
    string enhancementName
    text description
    decimal inputQuantity
    string quantityUnit
    decimal serviceDurationHours
    int estimatedCompletionDays
    text deliverables
    text prerequisites
    boolean isComplete
    timestamp completedAt
}
```

```
}
```

```
TERM {
```

```
  uuid termId PK
  uuid dealId FK
  uuid proposedByPartyId FK
  enum termType
  string termName
  text description
  enum negotiationStatus
  uuid parentTermId FK
  int version
  timestamp proposedAt
  timestamp resolvedAt
  boolean isMandatory
  string resolution
```

```
}
```

```
VALUEDISTRIBUTION {
```

```
  uuid distributionId PK
  uuid dealId FK,UK
  decimal totalValue
  string currency
  decimal supplierSharePercent
  decimal supplierShareAmount
  decimal consumerCostPercent
  decimal consumerCostAmount
  decimal enhancerSharePercent
  decimal enhancerShareAmount
  decimal platformFeePercent
  decimal platformFeeAmount
  enum distributionModel
  jsonb paymentSchedule
  decimal winWinWinScore
  timestamp createdAt
  timestamp updatedAt
```

```
}
```

```
MILESTONE {
```

```
  uuid milestoneId PK
  uuid dealId FK
  string milestoneName
  text description
  uuid assignedToPartyId FK
  date dueDate
  text completionCriteria
  enum milestoneStatus
  decimal completionPercentage
  decimal paymentTriggerAmount
  timestamp completedAt
  uuid verifiedByPartyId FK
  int displayOrder
```

```

}

%% =====
%% MATCH AGGREGATE
%% =====
MATCH {
    uuid matchId PK
    enum matchStatus
    uuid supplierPartyId FK
    uuid consumerPartyId FK
    uuid enhancerPartyId FK
    decimal matchScore "0-1"
    text matchReason
    uuid resourceCategoryId FK
    uuid needCategoryId FK
    uuid enhancementCategoryId FK
    decimal suggestedDealValue
    enum generatedBy
    timestamp expiresAt
    uuid convertedDealId FK
    timestamp createdAt
}

MATCHRESPONSE {
    uuid responseId PK
    uuid matchId FK
    uuid partyId FK
    enum response
    text responseNotes
    timestamp respondedAt
}

%% =====
%% AGREEMENT AGGREGATE
%% =====
AGREEMENT {
    uuid agreementId PK
    uuid dealId FK,UK
    enum agreementStatus
    text agreementText
    string governingLaw
    enum disputeResolution
    date effectiveDate
    date terminationDate
    boolean autoRenew
    int version
    string digitalSignatureUrl
    timestamp createdAt
    timestamp executedAt
}

SIGNATURE {

```

```

    uuid signatureId PK
    uuid agreementId FK
    uuid partyId FK
    uuid signedByUserId
    enum signatureType
    string signatureData
    string ipAddress
    timestamp signedAt
}

%% =====
%% FINANCIAL ENTITIES
%% =====

TRANSACTION {
    uuid transactionId PK
    uuid dealId FK
    uuid agreementId FK
    enum transactionType
    uuid fromPartyId FK
    uuid toPartyId FK
    decimal amount
    string currency
    string description
    enum status
    enum paymentMethod
    string externalReference
    timestamp executedAt
    timestamp createdAt
}

PLATFORMWALLET {
    uuid walletId PK
    uuid partyId FK,UK
    decimal balance
    decimal escrowBalance
    decimal pendingBalance
    decimal totalDeposited
    decimal totalWithdrawn
    string currency
    boolean isActive
    timestamp createdAt
    timestamp updatedAt
}

%% =====
%% REVIEW & REPUTATION
%% =====

REVIEW {
    uuid reviewId PK
    uuid dealId FK
    uuid reviewerPartyId FK
    uuid reviewedPartyId FK

```



```

enum reviewedRole
int overallRating "1-5"
int communicationRating
int reliabilityRating
int qualityRating
int timelinessRating
text reviewText
boolean isVerified
boolean isPublic
text platformResponse
timestamp createdAt
}

%% =====
%% DISPUTE
%% =====
DISPUTE {
    uuid disputelId PK
    uuid dealId FK
    uuid agreementId FK
    uuid raisedByPartyId FK
    uuid againstPartyId FK
    enum disputeType
    text description
    string[] evidenceUrls
    enum disputeStatus
    text resolution
    enum resolutionType
    uuid resolvedBy FK
    decimal compensationAmount
    timestamp createdAt
    timestamp resolvedAt
}

%% =====
%% COMMUNICATION
%% =====
MESSAGE {
    uuid messageId PK
    uuid dealId FK
    uuid matchId FK
    uuid senderPartyId FK
    uuid recipientPartyId FK
    enum messageType
    string subject
    text content
    string[] attachmentUrls
    boolean isRead
    timestamp readAt
    uuid replyToMessageId FK
    boolean isFlagged
    timestamp createdAt
}

```

```
}
```

```
NOTIFICATION {  
    uuid notificationId PK  
    uuid partyId FK  
    enum notificationType  
    string title  
    text body  
    enum relatedEntityType  
    uuid relatedEntityId  
    enum[] channel  
    boolean isRead  
    boolean isActioned  
    string actionUrl  
    timestamp createdAt  
    timestamp expiresAt  
}
```

```
%% =====  
%% RELATIONSHIP DEFINITIONS  
%% =====
```

```
%% Party hierarchy and specializations  
PARTY ||--o| SUPPLIER : "can be"  
PARTY ||--o| CONSUMER : "can be"  
PARTY ||--o| ENHANCER : "can be"  
PARTY ||--o| PARTYGROUP : "can be"  
PARTYGROUP ||--|{ GROUPMEMBERSHIP : "has members"  
GROUPMEMBERSHIP }o--|| PARTY : "member is"
```

```
%% Party trust  
PARTY ||--|| TRUSTSCORE : "has"
```

```
%% Party wallet  
PARTY ||--|| PLATFORMWALLET : "owns"
```

```
%% Party to Deal participation  
PARTY ||--|{ DEAL : "initiates"  
PARTY ||--|{ DEALPARTICIPATION : "participates in"  
DEAL ||--|{ DEALPARTICIPATION : "has"
```

```
%% Deal contents  
DEAL ||--|{ RESOURCE : "contains"  
DEAL ||--|{ NEED : "addresses"  
DEAL ||--|{ ENHANCEMENT : "requires"  
DEAL ||--|{ TERM : "negotiates"  
DEAL ||--|| VALUEDISTRIBUTION : "defines"  
DEAL ||--|{ MILESTONE : "tracks"
```

```
%% Deal lifecycle  
DEAL ||--o| AGREEMENT : "formalized as"  
DEAL ||--o| MATCH : "originated from"
```

```

DEAL ||--|{ TRANSACTION : "generates"
DEAL ||--|{ REVIEW : "receives"
DEAL ||--|{ DISPUTE : "may have"
DEAL ||--|{ MESSAGE : "discussed in"

%% Category classifications
CATEGORY ||--o| CATEGORY : "parent of"
CATEGORY ||--|{ DEAL : "classifies"
CATEGORY ||--|{ RESOURCE : "classifies"
CATEGORY ||--|{ NEED : "classifies"
CATEGORY ||--|{ ENHANCEMENT : "classifies"
CATEGORY ||--|{ MATCH : "categorizes"

%% Agreement
AGREEMENT ||--|{ SIGNATURE : "signed by"
AGREEMENT ||--|{ TRANSACTION : "triggers"

%% Match
MATCH ||--|{ MATCHRESPONSE : "receives"
MATCH ||--o| DEAL : "converts to"

%% Communications
PARTY ||--|{ MESSAGE : "sends"
PARTY ||--|{ NOTIFICATION : "receives"
MESSAGE ||--o| MESSAGE : "replies to"

%% Reviews (reviewer -> reviewed)
PARTY ||--|{ REVIEW : "writes"
PARTY ||--|{ REVIEW : "is reviewed in"

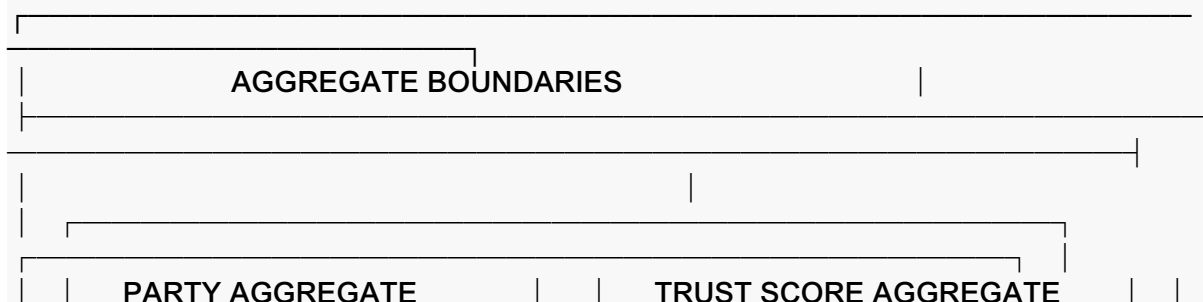
%% Financial flows
PARTY ||--|{ TRANSACTION : "sends"
PARTY ||--|{ TRANSACTION : "receives"
PLATFORMWALLET ||--|{ TRANSACTION : "records"

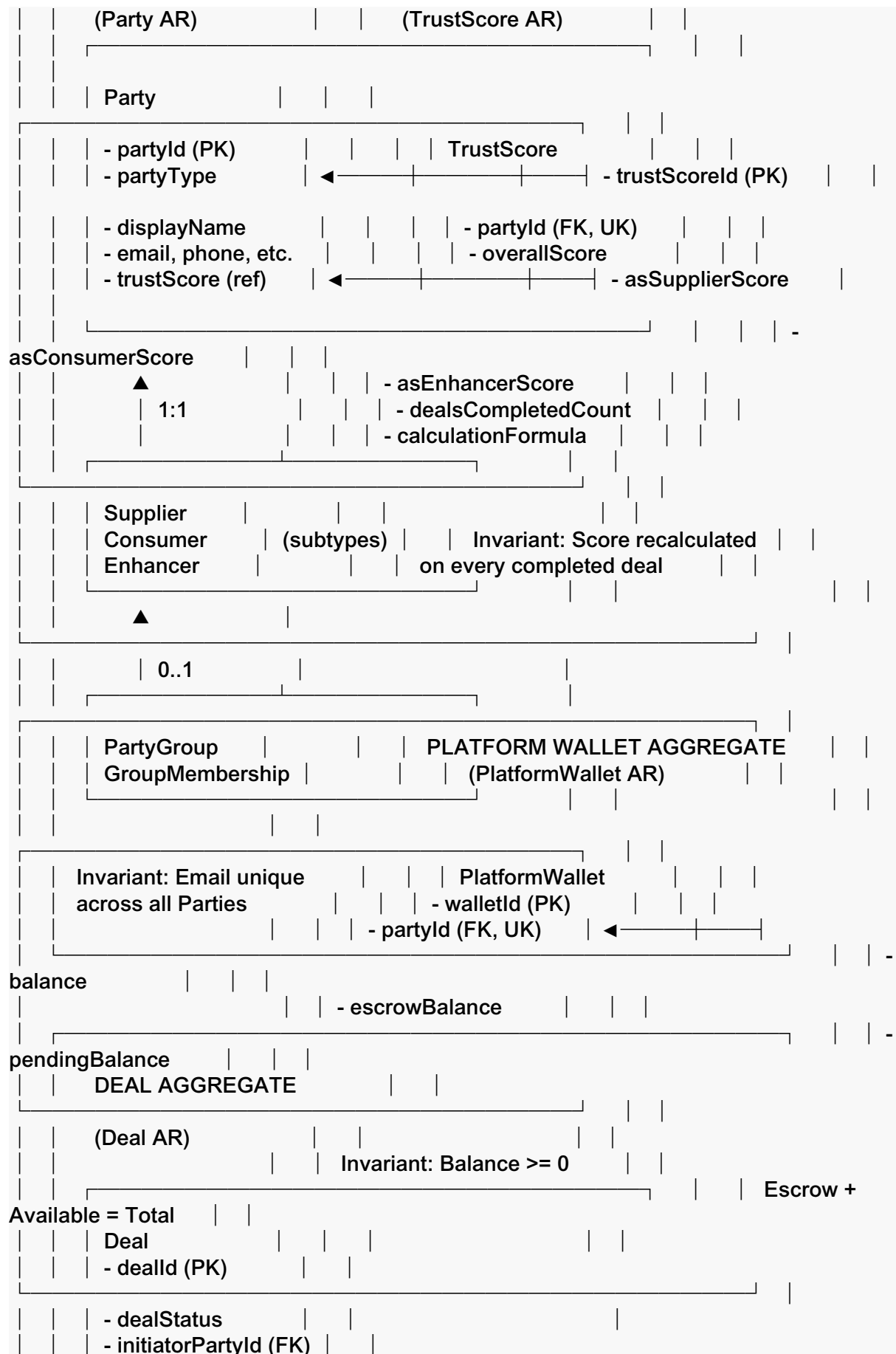
%% Disputes
PARTY ||--|{ DISPUTE : "raises"
PARTY ||--|{ DISPUTE : "defends"

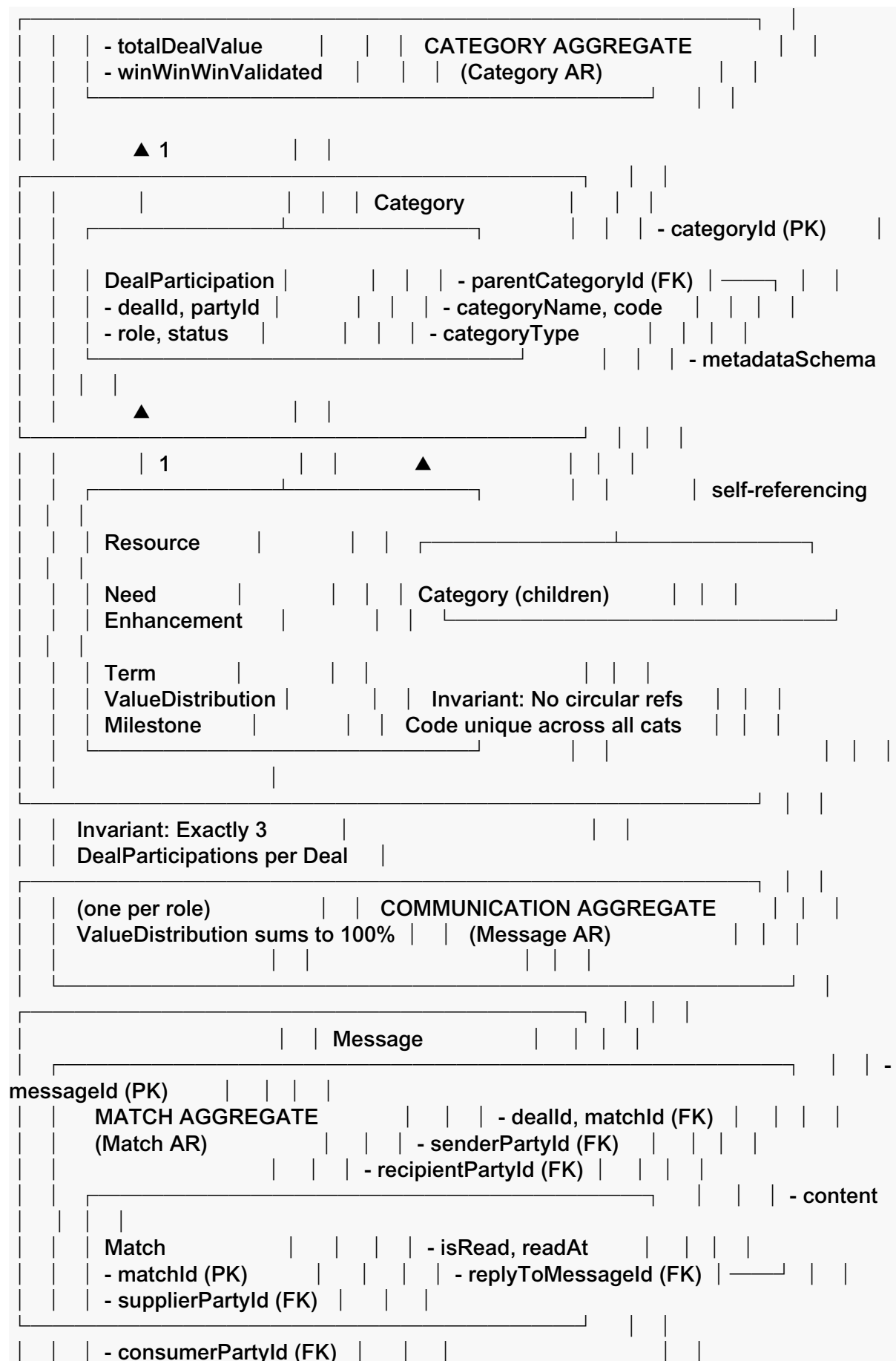
```

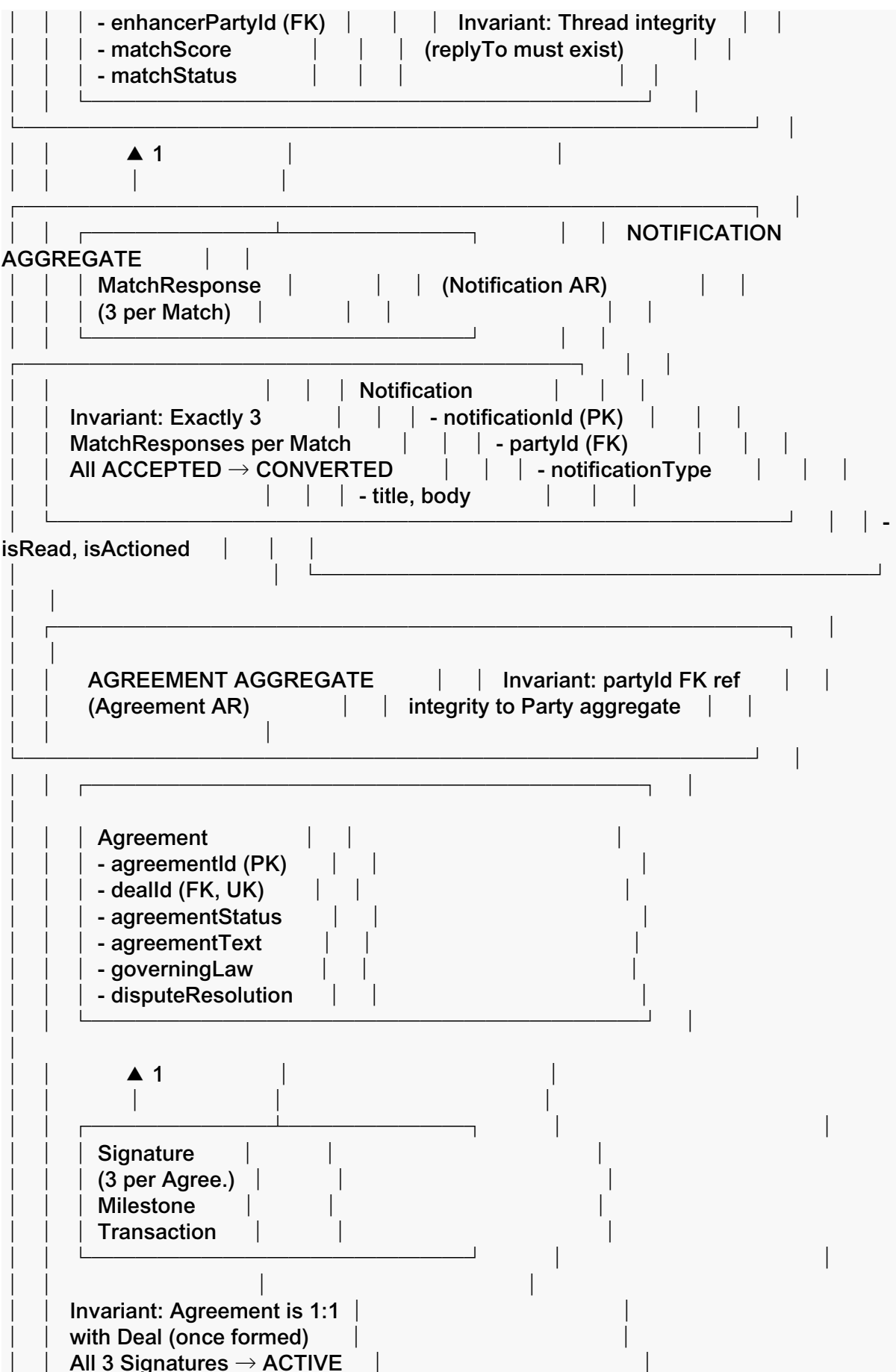
3. Aggregate Boundaries (DDD)

3.1 Aggregate Boundary Diagram









REVIEW AGGREGATE (Review AR)

Review

- reviewId (PK)
- dealId (FK)
- reviewerPartyId (FK)
- reviewedPartyId (FK)
- overallRating
- multi-dimension

Invariant: One review per
(deal, reviewer, reviewed)
combo. Must be verified deal
participant.

DISPUTE AGGREGATE (Dispute AR)

Dispute

- disputId (PK)
- dealId (FK)
- raisedByPartyId (FK)
- againstPartyId (FK)
- disputeStatus
- resolution

Invariant: Dispute can only
be raised by deal participants

3.2 Aggregate Root Summary Table

#	Aggregate Root	Key Entities	Transactional Invariants
1	Party	Party, Supplier, Consumer, Enhancer, PartyGroup, GroupMembership	Email uniqueness; PartyType consistency; GroupMembership quorum rules
2	Deal	Deal, DealParticipation, Resource, Need, Enhancement, Term, ValueDistribution, Milestone	Exactly 3 participations (one per role); ValueDistribution sums to 100%; Win-Win-Win validation passes before AGREEMENT
3	Match	Match, MatchResponse	Exactly 3 responses; All ACCEPTED → status CONVERTED; Expiry atomic handling
4	Agreement	Agreement, Signature	1:1 with Deal; All 3 signatures → status ACTIVE; Version immutable after execution
5	TrustScore	TrustScore	Eventually consistent; Recalculation on deal completion/dispute
6	PlatformWallet	PlatformWallet	Balance ≥ 0 always; Escrow + Available = Total; Double-entry for all transactions
7	Review	Review	One review per (deal, reviewer, reviewed); Only verified participants can review
8	Dispute	Dispute	Only deal participants; Status transitions audited; Resolution → automatic adjustment
9	Message	Message	Thread integrity (replyTo exists); Read receipts eventually consistent
10	Notification	Notification	Delivery tracking; Expiry cleanup; Idempotency by (partyId, type, entityRef)
11	Category	Category	Self-reference tree integrity (no cycles); Code uniqueness; Hierarchical consistency

3.3 Cross-Aggregate Rules

All cross-aggregate references use **foreign key identifiers only** (not object references) to maintain aggregate isolation:

From Aggregate	Reference	To Aggregate	Nature
Deal	initiatorPartyId	Party	FK

From Aggregate	Reference	To Aggregate	Nature
			reference
DealParticipation	partyId	Party	FK reference
DealParticipation	dealId	Deal	FK within same aggregate
Match	supplierPartyId, consumerPartyId, enhancerPartyId	Party	FK references
Match	convertedDealId	Deal	Nullable FK reference
Agreement	dealId	Deal	FK reference (1:1)
Review	reviewerPartyId, reviewedPartyId	Party	FK references
Review	dealId	Deal	FK reference
Transaction	fromPartyId, toPartyId	Party	FK references
Transaction	dealId	Deal	FK reference
Message	senderPartyId, recipientPartyId	Party	FK references
Message	dealId	Deal	Nullable FK reference
Notification	partyId	Party	FK reference
Dispute	raisedByPartyId, againstPartyId	Party	FK references
Dispute	dealId	Deal	FK reference
TrustScore	partyId	Party	FK reference (1:1)
PlatformWallet	partyId	Party	FK reference (1:1)

3. Domain Invariants & Business Rules

3.1 Core Business Rules

ID	Rule	Severity	Enforcement Point
BR-00	Every Deal	HARD	Deal aggregate factory method

ID	Rule	Severity	Enforcement Point
1	MUST have exactly 3 DealParticipations: one SUPPLIER, one CONSUMER, one ENHANCER.		
BR-002	ValueDistribution percentages MUST sum to exactly 100%.	HARD	ValueDistribution entity validation
BR-003	The Win-Win score MUST be $\geq$ platform minimum before a Deal can transition to AGREED.	HARD	Deal state transition guard
BR-004	All three parties MUST accept their MatchResponse before a Match converts to a Deal.	HARD	Match aggregate domain service
BR-005	An Agreement requires all 3 Signatures before it becomes ACTIVE.	HARD	Agreement state machine
BR-006	Transactions from PlatformW	HARD	Wallet entity invariant

ID	Rule	Severity	Enforcement Point
	allet CANNOT result in negative balance.		
BR-00 7	Only verified deal participants can leave Reviews.	HARD	Review factory validation
BR-00 8	A Party CANNOT review themselves.	HARD	Review entity validation
BR-00 9	Disputes can only be raised by parties involved in the Deal.	HARD	Dispute factory validation
BR-01 0	Group deal approval requires minimumM emberAppr oval votes from PartyGroup members.	HARD	PartyGroup domain service
BR-01 1	Platform fee percentage MUST be within platform- configured bounds (e.g., 1%- 20%).	HARD	ValueDistribution validation
BR-01 2	expectedEn dDate MUST be $\geq$ expectedSta rtDate for all Deals.	HARD	Deal entity validation
BR-01	A Deal	HARD	Deal state machine guard

ID	Rule	Severity	Enforcement Point
3	CANNOT transition from COMPLETED back to ACTIVE.		
BR-01 4	TrustScore recalculation MUST be scheduled within 24 hours of deal completion.	SOFT	Domain event handler
BR-01 5	Messages within a Deal context are only visible to Deal participants	HARD	Message query service
BR-01 6	Category code MUST be URL-safe and unique across the entire category tree.	HARD	Category entity validation

3.2 Win-Win-Win Validation Algorithm

```
def validate_win_win_win(deal, value_distribution):
```

```
    """
```

```
    Validates that a deal is positive for all three parties.
```

```
    Returns: (is_valid, score, reasons)
```

```
    """
```

```
    scores = {}
```

```
    reasons = []
```

```
    # Supplier value check
```

```
    supplier_value = value_distribution.supplier_share_amount
```

```
    supplier_cost = deal.resource.opportunity_cost # estimated
```

```
    supplier_net = supplier_value - supplier_cost
```

```
    scores['supplier'] = max(0, min(100, (supplier_net / max(supplier_cost, 1)) * 100))
```

```
    if supplier_net <= 0:
```

```
        reasons.append("Supplier net value is non-positive")
```

```

# Consumer value check
consumer_payment = value_distribution.consumer_cost_amount
consumer_received_value = deal.need.estimated_fulfillment_value
consumer_net = consumer_received_value - consumer_payment
scores['consumer'] = max(0, min(100, (consumer_net / max(consumer_payment, 1)) *
100))
if consumer_net <= 0:
    reasons.append("Consumer net value is non-positive")

# Enhancer value check
enhancer_value = value_distribution.enhancer_share_amount
enhancer_cost = deal.enhancement.estimated_input_cost
enhancer_net = enhancer_value - enhancer_cost
scores['enhancer'] = max(0, min(100, (enhancer_net / max(enhancer_cost, 1)) * 100))
if enhancer_net <= 0:
    reasons.append("Enhancer net value is non-positive")

# Composite score (geometric mean ensures all must be positive)
import math
composite = math.pow(scores['supplier'] * scores['consumer'] * scores['enhancer'], 1/3)

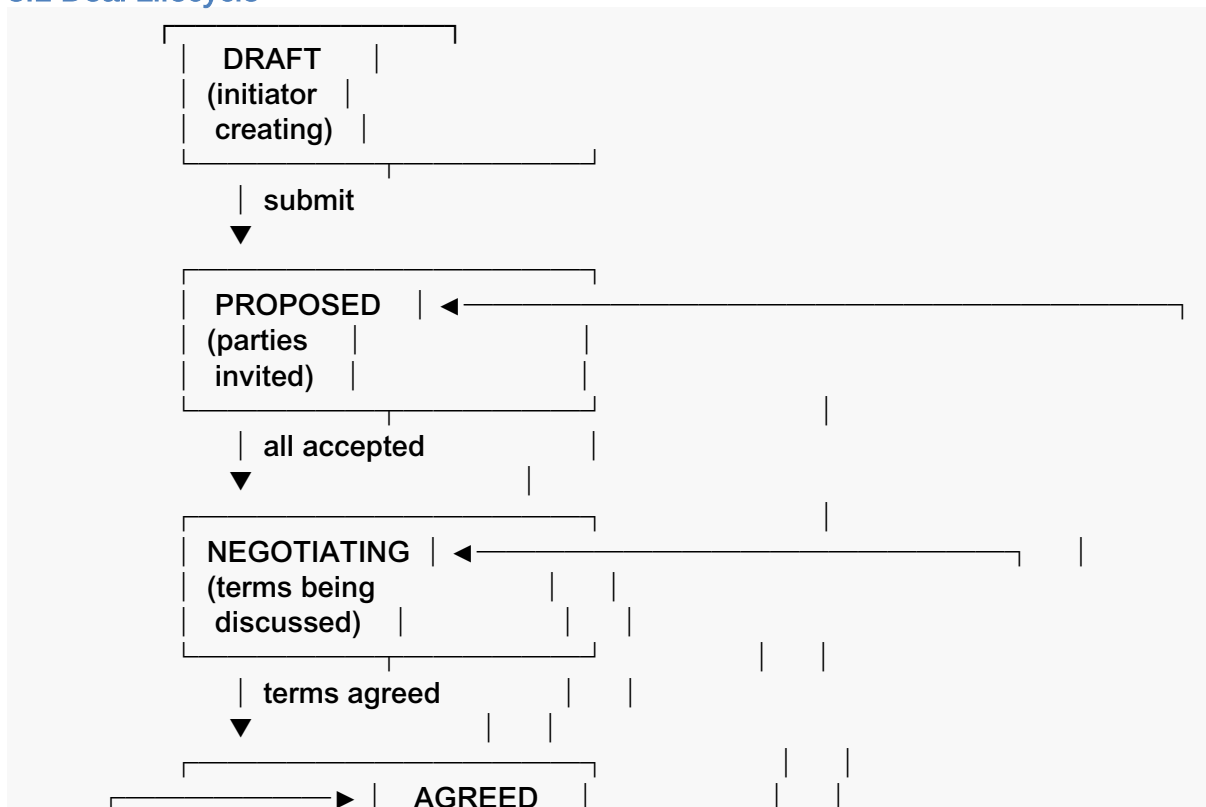
is_valid = all(v > 0 for v in [supplier_net, consumer_net, enhancer_net])
is_valid = is_valid and composite >= PLATFORM_MIN_WIN_SCORE # e.g., 60

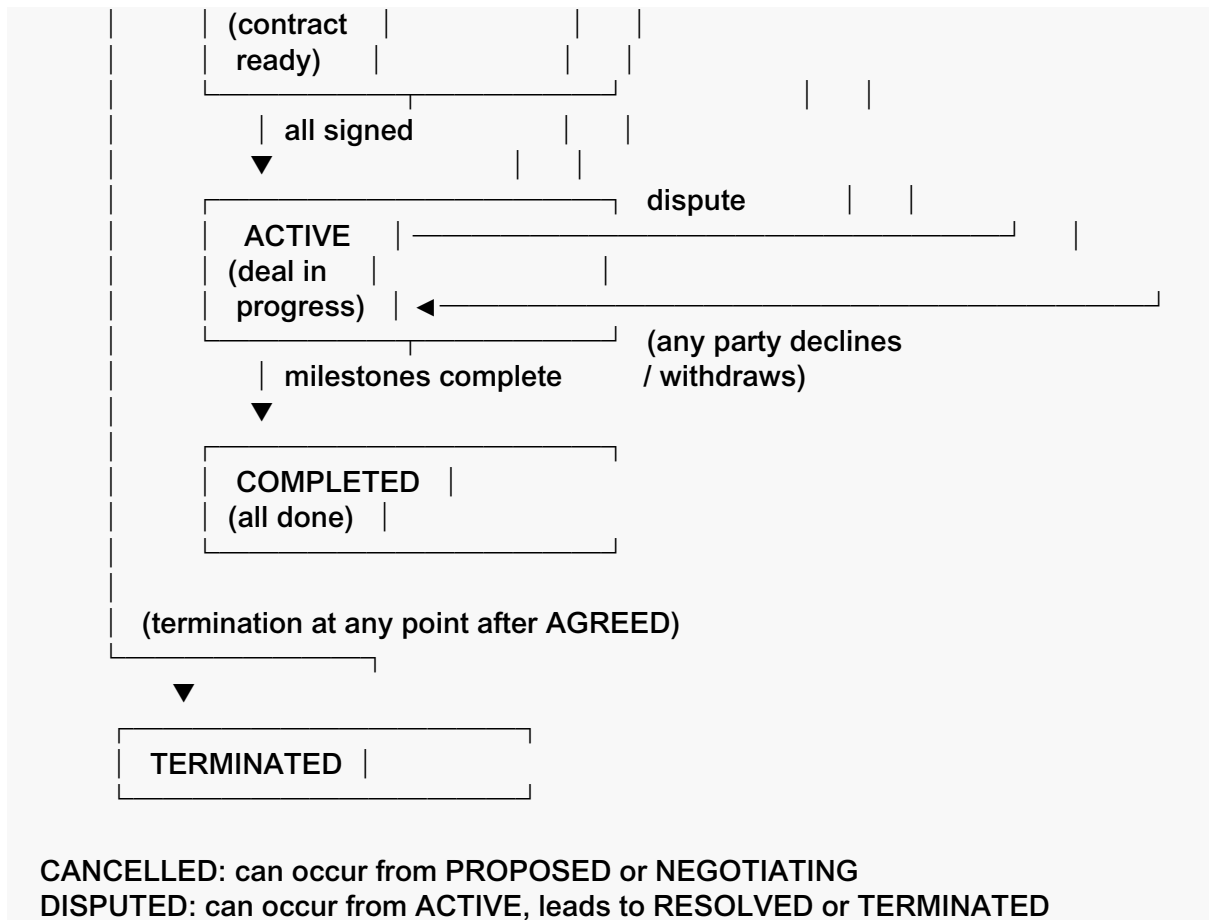
return is_valid, composite, reasons

```

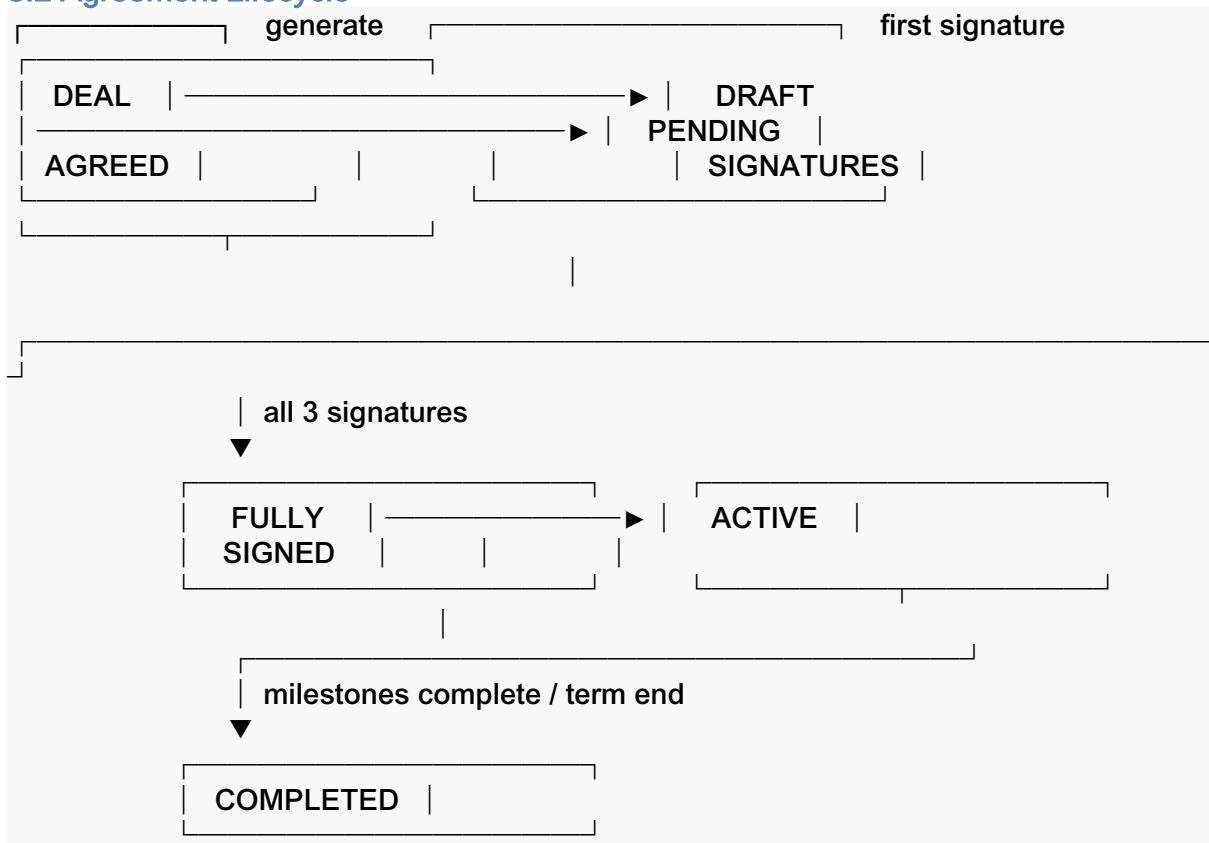
3. Lifecycle State Machines

3.1 Deal Lifecycle





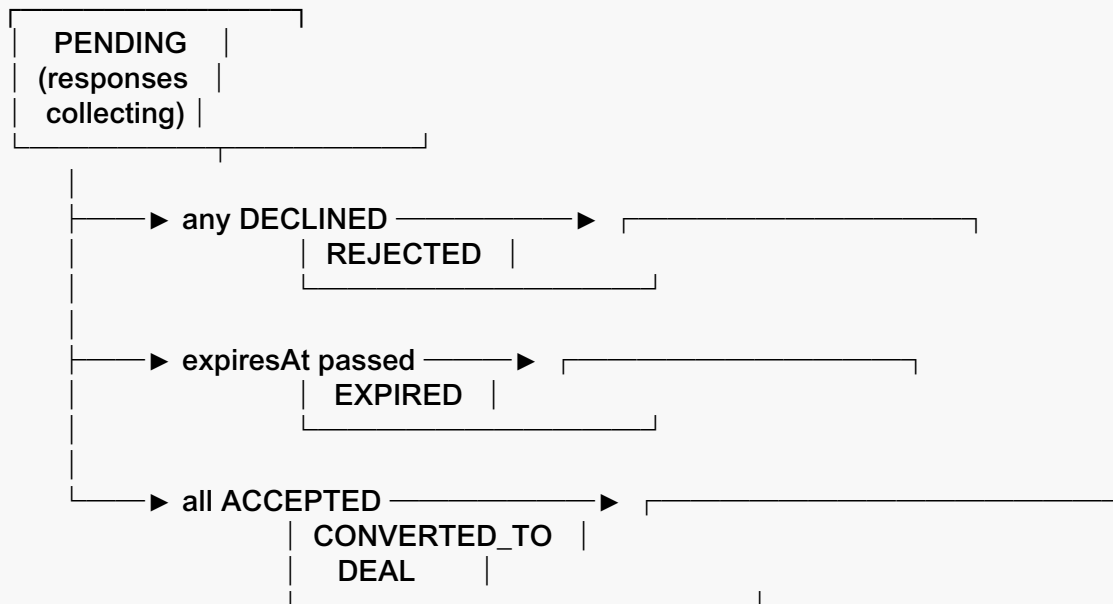
3.2 Agreement Lifecycle



DISPUTED: can occur from ACTIVE

TERMINATED: can occur from PENDING_SIGNATURES or ACTIVE

3.3 Match Lifecycle



3. Index & Query Model

3.1 Primary Query Patterns

Query	Entities	Filter Criteria	Sort
Find deals for party	Deal + DealParticipation	partyId = ? AND participationStatus = ACCEPTED	updatedAt DESC
Search available deals	Deal + Category	dealStatus = PROPOSED AND domainCategoryId = ? AND location within ?	createdAt DESC
Match suggestions for party	Match	supplierPartyId = ? OR consumerPartyId = ? OR enhancerPartyId = ?	matchScore DESC
Party reputation	TrustScore + Review	partyId = ?	createdAt DESC
Deal timeline	Transaction + Milestone	dealId = ?	createdAt ASC
Category tree	Category	parentCategoryId = ? AND isActive = true	displayOrder ASC
Party messages	Message	(senderPartyId = ? OR recipientPartyId = ?) AND dealId = ?	createdAt DESC
Pending notifications	Notification	partyId = ? AND isRead = false AND expiresAt > now()	createdAt DESC
Wallet history	Transaction +	fromPartyId = ? OR toPartyId = ?	createdAt

Query	Entities	Filter Criteria	Sort
	PlatformWallet		DESC
Dispute resolution queue	Dispute + Deal	disputeStatus = OPEN	createdAt ASC

3.2 Recommended Indexes

-- Party lookups

```
CREATE INDEX idx_party_email ON Party(email);
CREATE INDEX idx_party_location ON Party USING GIST(location);
CREATE INDEX idx_party_type ON Party(partyType, isActive);
```

-- Deal queries

```
CREATE INDEX idx_deal_status ON Deal(dealStatus, isPublic);
CREATE INDEX idx_deal_category ON Deal(domainCategoryId, dealStatus);
CREATE INDEX idx_deal_location ON Deal USING GIST(location);
CREATE INDEX idx_deal_initiator ON Deal(initiatorPartyId, createdAt DESC);
```

-- Participation

```
CREATE INDEX idx_participation_party ON DealParticipation(partyId, participationStatus);
CREATE INDEX idx_participation_deal ON DealParticipation(dealId, role);
```

-- Match scoring

```
CREATE INDEX idx_match_party_supplier ON Match(supplierPartyId, matchStatus);
CREATE INDEX idx_match_party_consumer ON Match(consumerPartyId, matchStatus);
CREATE INDEX idx_match_party_enhancer ON Match(enhancerPartyId, matchStatus);
CREATE INDEX idx_match_score ON Match(matchScore DESC) WHERE matchStatus = 'PENDING';
```

-- Reviews

```
CREATE INDEX idx_review_reviewed ON Review(reviewedPartyId, createdAt DESC);
CREATE INDEX idx_review_deal ON Review(dealId, reviewerPartyId);
```

-- Messages

```
CREATE INDEX idx_message_sender ON Message(senderPartyId, createdAt DESC);
CREATE INDEX idx_message_recipient ON Message(recipientPartyId, isRead, createdAt DESC);
CREATE INDEX idx_message_deal ON Message(dealId, createdAt DESC);
```

-- Transactions

```
CREATE INDEX idx_transaction_deal ON Transaction(dealId, createdAt DESC);
CREATE INDEX idx_transaction_party ON Transaction(fromPartyId, toPartyId, createdAt DESC);
```

-- Categories

```
CREATE INDEX idx_category_parent ON Category(parentCategoryId, categoryType, isActive);
```

-- Notifications

```
CREATE INDEX idx_notification_party ON Notification(partyId, isRead, createdAt DESC);
```

Appendix A: Glossary

Term	Definition
Aggregate	A cluster of associated objects treated as a single unit for data changes.
Aggregate Root	The single entity within an aggregate that serves as the entry point and enforces invariants.
Domain Invariant	A business rule that must always be true, enforced within an aggregate boundary.
Entity	A domain object with a unique identity that persists over time.
Value Object	An immutable object without identity, defined solely by its attributes (e.g., GeoPoint, Money).
Win-Win-Win	The core platform principle that all three parties must derive positive net value.

Appendix B: Revision History

Version	Date	Author	Changes
1.0.0	Current	DDD Architect	Initial comprehensive domain model

Document Information

- **Version:** 1.0
- **Status:** Draft
- **Scope:** Complete workflow architecture for Supplier-Consumer-Enhancer deal platform

4. Deal Lifecycle State Machine

4.1 State Definitions

State ID	State Name	Description	Entry Criteria	Auto-Timeout
S0	DRAFT	Deal being composed by initiator, not yet visible to others	Created by any party	7 days → auto-delete
S1	SUGGESTED	Deal proposed to at least one other party, awaiting response	Initiator submits draft	14 days → EXPIRED
S2	PENDING_REVIEW	All parties invited, awaiting initial review/acknowledgment	All parties invited	7 days → reminder, 14 days → EXPIRED

State ID	State Name	Description	Entry Criteria	Auto-Timeout
S3	NEGOTIATING	Active negotiation on terms, value distribution, milestones	All parties acknowledged	30 days → ON_HOLD
S4	AWAITING_PARTY	Paused waiting for specific party action/input	Any party triggers	7 days → reminder, 14 days → ON_HOLD
S5	TERMS_LOCKED	All terms agreed, pending final commitment signatures	All parties accept terms	7 days → reminder, 14 days → CANCELLED
S6	COMMITTED	All parties committed, deal is binding, pre-execution prep	All parties sign commitment	3 days → EXECUTING
S7	EXECUTING	Deal actively being performed, milestones tracked	Auto from COMMITTED	Per milestone schedule
S8	ON_HOLD	Deal suspended by platform or party, pending resolution	Timeout or party trigger	30 days → CANCELLED
S9	COMPLETED	All milestones met, value transferred, deal closed	All milestones verified	N/A (terminal)
S10	DISPUTED	Active dispute being resolved	Any party raises dispute	14 days → escalation
S11	CANCELLED	Deal terminated without completion	Various triggers	N/A (terminal)
S12	EXPIRED	Deal timed out without sufficient party engagement	Timeout expiry	N/A (terminal)

[illegible]

	SUGGESTED	-	all_invited	-	-	-	-	-	-	-	-	-
reject	timeout											
	PENDING_REVIEW	-	-	acknowledge	-	-	-	-	-	-	-	-
-	withdraw	timeout										
	NEGOTIATING	-	-	pause	lock_terms	-	-	hold	-			
raise	withdraw	-										
	AWAITING_PARTY	-	-	resume	-	-	-	hold	-			
-	withdraw	timeout										
	TERMS_LOCKED	-	-	renegotiate	-	-	commit	-	-	-	-	-
-	withdraw	timeout										
	COMMITTED	-	-	-	-	-	begin	-	-	-	-	-
withdraw	-											
	EXECUTING	-	-	pause	-	-	-	hold	complete			
raise	terminate	-										
	ON_HOLD	-	-	-	-	-	resume	-	-	-	-	-
cancel	-											
	COMPLETED	-	-	-	-	-	-	-	-	-	-	-
-												
	DISPUTED	-	-	-	-	-	resume	-	resolve	-		
cancel	-											
	CANCELLED	-	-	-	-	-	-	-	-	-	-	-
-												
	EXPIRED	-	-	-	-	-	-	-	-	-	-	-
-												

4.3 Complete State Machine (Mermaid)

stateDiagram-v2

[*] --> DRAFT : Any party creates deal

DRAFT --> SUGGESTED : Initiator submits proposal

DRAFT --> CANCELLED : Initiator abandons

DRAFT --> EXPIRED : 7-day timeout

SUGGESTED --> PENDING_REVIEW : All parties invited

SUGGESTED --> CANCELLED : Counter-party rejects

SUGGESTED --> EXPIRED : 14-day timeout

PENDING_REVIEW --> NEGOTIATING : All parties acknowledge

PENDING_REVIEW --> CANCELLED : Initiator withdraws

PENDING_REVIEW --> EXPIRED : 14-day timeout

NEGOTIATING --> TERMS_LOCKED : All parties accept terms

NEGOTIATING --> AWAITING_PARTY : Party requests clarification/info

NEGOTIATING --> ON_HOLD : Party requests hold / Platform suspends

NEGOTIATING --> CANCELLED : Any party withdraws

NEGOTIATING --> EXPIRED : 30-day timeout

AWAITING_PARTY --> NEGOTIATING : Requested party responds

AWAITING_PARTY --> ON_HOLD : 14-day no-response

AWAITING_PARTY --> CANCELLED : Any party withdraws

TERMS_LOCKED --> COMMITTED : All parties sign commitment

TERMS_LOCKED --> NEGOTIATING : Any party requests renegotiation

TERMS_LOCKED --> CANCELLED : 14-day timeout / Any party withdraws

COMMITTED --> EXECUTING : Auto-transition after 3-day prep

COMMITTED --> CANCELLED : Unanimous withdrawal within 3 days

EXECUTING --> COMPLETED : All milestones verified

EXECUTING --> DISPUTED : Any party raises dispute

EXECUTING --> AWAITING_PARTY : Party delays milestone

EXECUTING --> ON_HOLD : Platform/Party suspends

EXECUTING --> CANCELLED : Mutual agreement / Breach termination

ON_HOLD --> NEGOTIATING : Hold reason resolved

ON_HOLD --> EXECUTING : Hold reason resolved

ON_HOLD --> CANCELLED : 30-day hold timeout

DISPUTED --> EXECUTING : Dispute resolved, resume

DISPUTED --> COMPLETED : Dispute resolved, partial completion

DISPUTED --> CANCELLED : Dispute resolved, terminate

DISPUTED --> ON_HOLD : Awaiting external arbitration

COMPLETED --> [*]

CANCELLED --> [*]

EXPIRED --> [*]

4.4 Transition Details with Roles & Timeouts

S0 → S1: DRAFT → SUGGESTED

Attribute	Value
Trigger	Initiator clicks “Submit Proposal”
Role Required	Initiator (Supplier / Consumer / Enhancer)
Validation	All mandatory fields filled; at least 2 other parties specified; win-win validation passed
Timeout	7 days in DRAFT → auto-delete with notification
Notifications	Platform confirms submission; invited parties receive proposal

S1 → S2: SUGGESTED → PENDING_REVIEW

Attribute	Value
Trigger	Platform confirms all invited parties have accounts
Role Required	System (auto)
Validation	All party accounts exist and are verified; no duplicate active deals between same parties on same asset
Timeout	14 days in SUGGESTED without

Attribute	Value
Notifications	all parties invited → EXPIRED All parties: “New deal requires your review”

S2 → S3: PENDING_REVIEW → NEGOTIATING

Attribute	Value
Trigger	All parties click “Acknowledge & Enter Negotiation”
Role Required	All parties (Supplier, Consumer, Enhancer)
Validation	Each party has reviewed initial proposal; party identity verified
Timeout	7 days → reminder; 14 days → EXPIRED
Notifications	All parties: “Negotiation begins”; Platform assigns deal coordinator

S3 → S5: NEGOTIATING → TERMS_LOCKED

Attribute	Value
Trigger	All parties click “Accept Terms” on current proposal version
Role Required	All parties
Validation	Current proposal version has no pending counter-proposals; all required signatures present; escrow terms defined
Timeout	30 days in NEGOTIATING → ON_HOLD with warning
Notifications	All parties: “Terms locked - awaiting commitment”; Platform: “Deal [ID] ready for commitment”

S5 → S6: TERMS_LOCKED → COMMITTED

Attribute	Value
Trigger	All parties digitally sign commitment agreement
Role Required	All parties (with internal approval if multi-entity)
Validation	Digital signatures from all authorized representatives; escrow funded per agreement
Timeout	7 days → reminder; 14 days → CANCELLED

Attribute	Value
Notifications	All parties: “Deal committed - execution begins in 3 days”; Escrow agent notified

S6 → S7: COMMITTED → EXECUTING

Attribute	Value
Trigger	3-day preparation period expires OR all parties click “Begin Early”
Role Required	System (auto) or Unanimous
Validation	Escrow accounts funded; milestone tracking enabled; communication channels opened
Timeout	Auto after 3 days
Notifications	All parties: “Execution phase started”; Milestone calendar shared

S7 → S9: EXECUTING → COMPLETED

Attribute	Value
Trigger	All milestones marked complete AND verified by counter-party
Role Required	All parties + Platform verification
Validation	All milestone deliverables uploaded/confirmed; payments released from escrow; ratings/reviews submitted
Timeout	Per milestone schedule
Notifications	All parties: “Deal completed successfully”; Platform: “Deal [ID] closed”

S7 → S10: EXECUTING → DISPUTED

Attribute	Value
Trigger	Any party clicks “Raise Dispute” with evidence
Role Required	Any party
Validation	Dispute category selected; evidence uploaded; good-faith deposit paid (2% of deal value)
Timeout	Immediate transition
Notifications	All parties: “Dispute raised - case [ID] opened”; Platform mediator assigned

4.5 Timeout & Escalation Rules

State	Timeout	Escalation	Action
DRAFT	7 days	None	Auto-delete, notify initiator
SUGGESTED	14 days	Day 7: Reminder	→ EXPIRED, notify all
PENDING_REVIEW	14 days	Day 7: Reminder	→ EXPIRED, notify all
NEGOTIATING	30 days	Day 21: Warning	→ ON_HOLD, mediator assigned
AWAITING_PARTY	14 days	Day 7: Reminder	→ ON_HOLD, notify awaiting party
TERMS_LOCKED	14 days	Day 7: Reminder	→ CANCELLED, escrow refunded
COMMITTED	3 days	None	Auto → EXECUTING
EXECUTING	Per milestone	Day before: Alert	→ AWAITING_PARTY → DISPUTED
ON_HOLD	30 days	Day 14: Warning	→ CANCELLED if unresolved
DISPUTED	14 days	Day 7: Mediator	→ External arbitration if unresolved

4.6 Win-Win Validation Gate

Every state transition toward commitment must pass the **Win-Win-Win Validation Gate**:

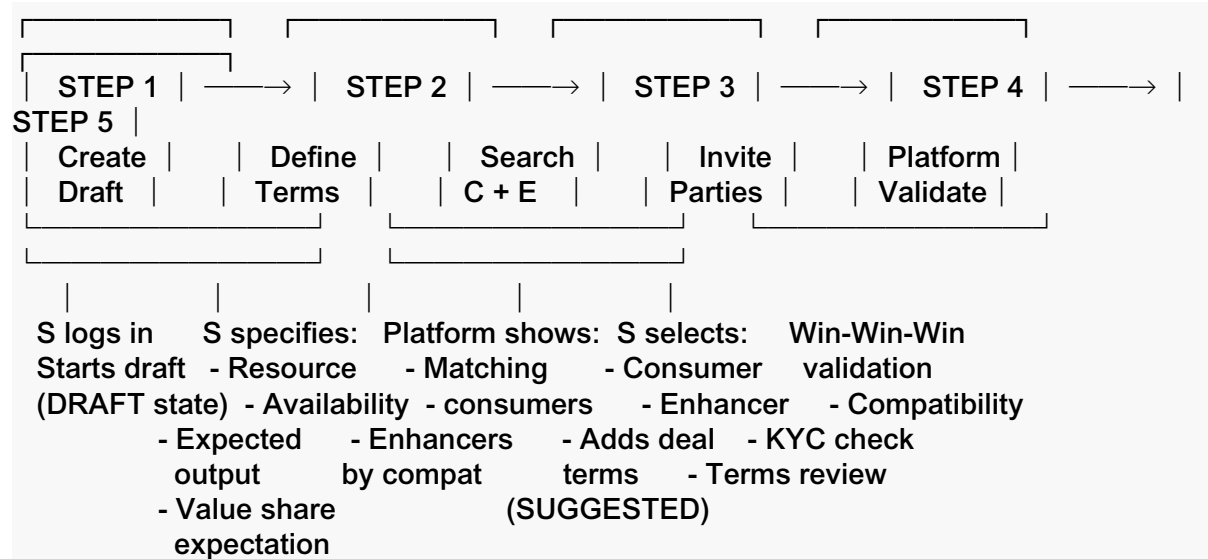
VALIDATION_CRITERIA:

- Economic Value Check
 - Each party's net benefit > 0 (after costs/fees)
 - No party's cost exceeds 80% of their stated budget/capacity
 - Total deal value ≥ platform minimum threshold
- Balance Check
 - No single party captures > 70% of total surplus
 - Gini coefficient of value distribution ≤ 0.5
 - Each party's ROI ≥ platform floor (5% minimum)
- Feasibility Check
 - Supplier resource available in required timeframe
 - Consumer need matches supplier output (type, quantity, quality)
 - Enhancer capability matches required value-add
- Risk Check
 - No party has > 3 active deals in EXECUTING state
 - No previous CANCELLED/DISPUTED pattern (3-strike rule)
- Compliance Check
 - All parties KYC/AML verified
 - Deal category not on restricted list
 - Regulatory requirements met for jurisdiction

4. Initiation Workflows

4.1 Supplier-Initiated Deal Flow

Scenario: Supplier (S) has idle arable farm. Finds Consumer (C) needing crop produce. Identifies Enhancer (E) - agrodealer with seeds/fertilizer.



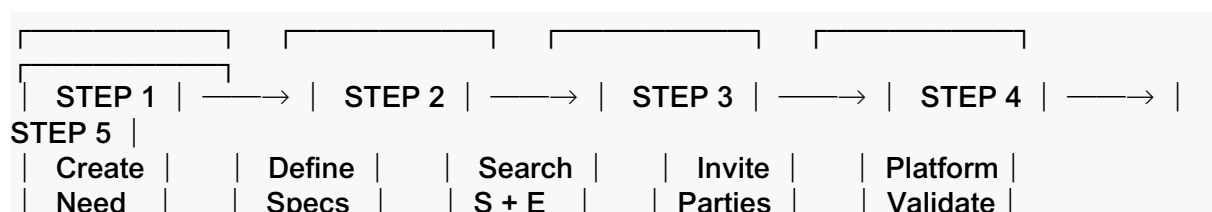
Detailed Steps:

Step	Action	Actor	System Actions	Decision Points
1.1	Supplier logs in, clicks "Create Deal"	Supplier	Create deal record in DRAFT state; Assign deal ID	Is supplier verified? → No → Redirect KYC
1.2	Supplier selects "I have a resource"	Supplier	Present resource template (farm, building, vehicle, machinery, custom)	Resource type determines required fields
1.3	Supplier describes resource	Supplier	Validate required fields; Auto-save draft	Missing required? → Highlight errors
1.4	Supplier specifies ideal Consumer profile	Supplier	Build compatibility query	Consumer profile completeness check
1.5	Supplier specifies ideal Enhancer profile	Supplier	Build compatibility query	Enhancer profile completeness check
1.6	Supplier	Supplier	Run matching algorithm	No matches? → Suggest

Step	Action	Actor	System Actions	Decision Points
	clicks “Find Matches”		(see Section 4)	broadening criteria
1.7	Platform returns ranked Consumer matches	System	Display compatibility scores, profiles, history	Supplier filters/sorts results
1.8	Platform returns ranked Enhancer matches	System	Display compatibility scores, profiles, history	Supplier filters/sorts results
1.9	Supplier selects Consumer (C) and Enhancer (E)	Supplier	Pre-populate deal with party references	Selected parties blocked for competing deals?
1.10	Supplier proposes value distribution	Supplier	Validate: each party $\geq$ 5% of total value	Distribution passes win-win gate?
1.11	Supplier proposes milestones	Supplier	Validate: logical sequence, realistic timing	Milestone gap analysis
1.12	Supplier reviews and submits	Supplier	Run full Win-Win-Win validation; Create SUGGESTED state	Validation fails? → Return with errors
1.13	Platform sends invitations	System	Notify C and E; Create pending party records	Track invitation status
1.14	C and E review invitations	Consumer, Enhancer	Present deal summary; Request acknowledgment	Decline? → Notify S, offer rematch

4.2 Consumer-Initiated Deal Flow

Scenario: Consumer (C) needs rental space for pop-up store. Finds Supplier (S) with vacant building. Identifies Enhancer (E) - contractor for renovation.



C logs in Creates need draft	C specifies: - Product/ - service need - Quantity - Quality requirements - Budget - Timeline	Platform shows: - Matching suppliers - Enhancers by compat	C selects: - Supplier - Enhancer - Adds deal terms - Terms review	Win-Win-Win validation - Compatibility - KYC check

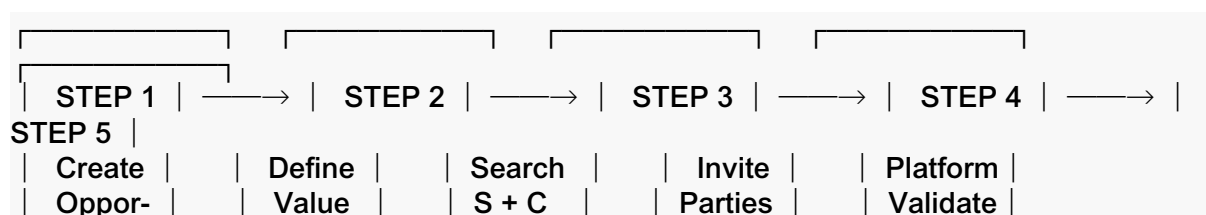
Detailed Steps:

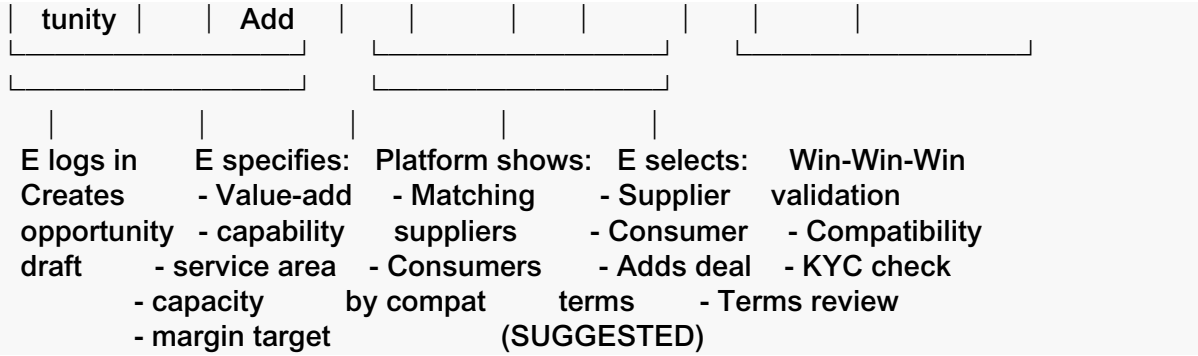
Step	Action	Actor	System Actions	Decision Points
2.1	Consumer logs in, clicks “Post Need”	Consumer	Create need record in DRAFT state	Is consumer verified? → No → Redirect KYC
2.2	Consumer selects “I need a product/service”	Consumer	Present need template (crops, space, transport, processing, custom)	Need type determines required fields
2.3	Consumer describes need in detail	Consumer	Validate required fields; Auto-save draft	Missing required? → Highlight errors
2.4	Consumer specifies budget range	Consumer	Validate: budget ≥ platform minimum	Budget realistic for need type?
2.5	Consumer specifies timeline	Consumer	Validate: timeline ≥ minimum for category	Timeline feasible?
2.6	Consumer specifies ideal Supplier profile	Consumer	Build compatibility query	Supplier profile completeness check
2.7	Consumer specifies ideal Enhancer profile	Consumer	Build compatibility query	Enhancer profile completeness check
2.8	Consumer clicks “Find	Consumer	Run matching algorithm	No matches? → Suggest broadening criteria

Step	Action	Actor	System Actions	Decision Points
	Matches”			
2.9	Platform returns ranked Supplier matches	System	Display compatibility scores, profiles, resource details	Consumer filters/sorts results
2.10	Platform returns ranked Enhancer matches	System	Display compatibility scores, profiles, history	Consumer filters/sorts results
2.11	Consumer selects Supplier (S) and Enhancer (E)	Consumer	Pre-populate deal with party references	Selected parties available?
2.12	Consumer proposes value distribution	Consumer	Validate: each party $\geq$ 5% of total value; C budget sufficient	Distribution passes win-win gate?
2.13	Consumer proposes payment schedule	Consumer	Validate: staged payments, escrow compatible	Payment schedule reasonable?
2.14	Consumer reviews and submits	Consumer	Run full Win-Win-Win validation; Create SUGGESTED state	Validation fails? → Return with errors
2.15	Platform sends invitations	System	Notify S and E; Create pending party records	Track invitation status
2.16	S and E review and acknowledge	Supplier, Enhancer	Present deal summary; Request acknowledgment	Decline? → Notify C, offer rematch

4.3 Enhancer-Initiated Deal Flow

Scenario: Enhancer (E) - mechanic specializing in fleet maintenance - sees opportunity. Finds Supplier (S) with idle vehicles and Consumer (C) needing transportation service.





Detailed Steps:

Step	Action	Actor	System Actions	Decision Points
3.1	Enhancer logs in, clicks “Create Opportunity”	Enhancer	Create opportunity record in DRAFT state	Is enhancer verified? → No → Redirect KYC
3.2	Enhancer selects “I can connect resources to needs”	Enhancer	Present opportunity template	Opportunity type determines fields
3.3	Enhancer describes value-add capability	Enhancer	Validate required fields; Auto-save draft	Capability specificity check
3.4	Enhancer specifies service area/scope	Enhancer	Geolocation tagging; Service radius	Area has sufficient deal density?
3.5	Enhancer specifies target margin	Enhancer	Validate: margin within platform norms	Margin realistic for capability?
3.6	Enhancer searches for available Supplier resources	Enhancer	Run supplier matching query	No suppliers? → Expand radius
3.7	Enhancer searches for Consumer needs	Enhancer	Run consumer matching query	No consumers? → Expand radius

Step	Action	Actor	System Actions	Decision Points
3.8	Enhancer proposes Supplier-Consumer pairing	Enhancer	Check: S resource matches C need	Compatibility score $\geq$ threshold?
3.9	Enhancer defines their value-add role	Enhancer	Pre-populate deal with all three roles	Role clearly defined?
3.10	Enhancer proposes value distribution	Enhancer	Validate: each party $\geq$ 5% of total value	Distribution passes win-win gate?
3.11	Enhancer proposes service terms	Enhancer	Validate: deliverables, timeline, quality metrics	Terms specific and measurable?
3.12	Enhancer reviews and submits	Enhancer	Run full Win-Win-Win validation; Create SUGGESTED state	Validation fails? $\rightarrow$ Return with errors
3.13	Platform sends dual invitations	System	Notify both S and C simultaneously; Create pending records	Track dual acceptance
3.14	S and C independently review	Supplier, Consumer	Present deal summary; Highlight E's value-add	Either declines? $\rightarrow$ Notify E, offer rematch

4.4 Initiation Decision Matrix

Decision Point	Supplier-Initiated	Consumer-Initiated	Enhancer-Initiated
Who sets price?	Supplier (anchor)	Consumer (budget)	Enhancer (market)
Who defines output?	Supplier	Consumer	Enhancer brokers
Who finds Enhancer?	Supplier suggests	Consumer suggests	Enhancer IS self
Value distribution	S-heavy default	C-heavy default	Balanced default
Matching priority	Consumer need	Supplier resource	Both simultaneously
Deal complexity	Medium	Low-Medium	High
Time to first match	Fast	Fast	Slower
Typical domain	Agriculture, RE	Retail, Service	Logistics, B2B

4. Negotiation Workflow

4.1 Negotiation Architecture Overview

The negotiation phase (S3: NEGOTIATING) is the most complex interaction in the deal lifecycle. It supports parallel multi-threaded negotiations on different term dimensions, with the ability to lock individual dimensions before finalizing all terms.

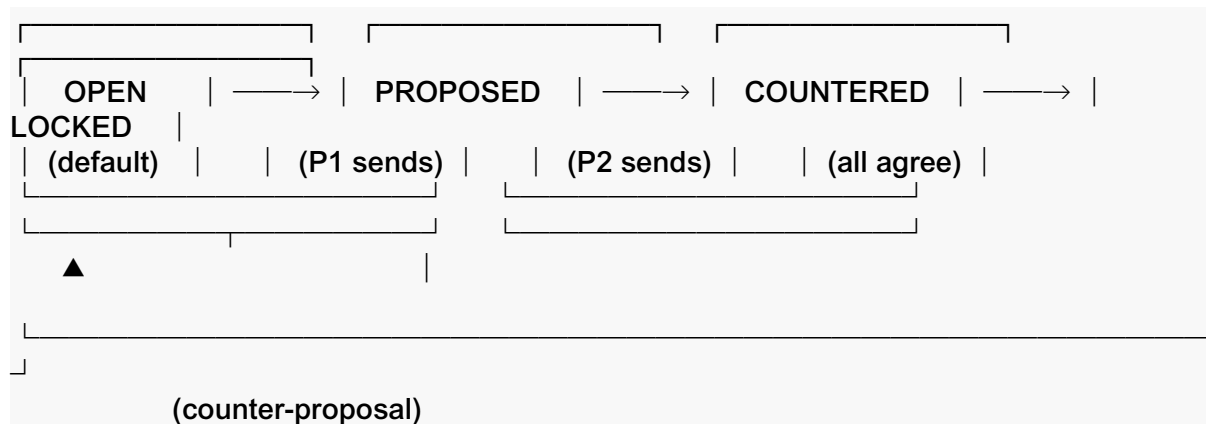
Core Principles: 1. **Threaded Negotiation:** Each term dimension can be negotiated independently 2. **Version Control:** Every proposal is versioned; full audit trail maintained 3. **Counter-Party Protection:** No party can unilaterally change locked terms 4. **Time-Bound:** All negotiations have explicit deadlines 5. **Win-Win Enforcement:** Platform validates no party is disadvantaged

4.2 Negotiable Term Dimensions

Dimension	Negotiable Elements
VALUE_DISTRIBUTION	Revenue/cost shares, percentages, absolute amounts Payment schedule, escrow terms, penalty clauses
DELIVERABLES	Output specifications, quality standards, quantities Acceptance criteria, inspection procedures
TIMELINE	Start date, milestone dates, completion deadline Extension conditions, early completion bonuses
ROLES_RESPONSIBILITIES	Specific duties per party, reporting lines Decision-making authority, sub-contracting rights
RISK_ALLOCATION	Liability limits, insurance requirements Force majeure, indemnification, guarantee terms
TERMINATION	Exit clauses, notice periods, termination for cause Asset return, IP ownership, post-termination duties

4.3 Negotiation State Sub-Machine

Within NEGOTIATING, each term dimension has its own sub-state:



4.4 Step-by-Step Negotiation Flow

Phase A: Opening Positions

Step	Action	Actor	System	Notes
N1	Initiator's opening proposal auto-loaded	Initiator	Display initial proposal from deal creation	All dimensions in OPEN state
N2	Counter-parties review opening proposal	Counter-parties	Show side-by-side: proposal vs. market benchmarks	Benchmark data from comparable deals
N3	Parties decide: accept / counter / request info	All	Create decision record; Route accordingly	Parallel decision collection

Phase B: Iterative Negotiation

Step	Action	Actor	System	Notes
N4	Party selects dimension to negotiate	Any party	Highlight negotiable dimensions; Show current status per dimension	Only OPEN dimensions selectable
N5	Party submits proposal on dimension	Proposer	Version the proposal; Timestamp; Notify other parties	Each dimension max 5 rounds
N6	Other parties review proposal	Reviewers	Show delta from current position; Run win-win impact analysis	Impact analysis displayed
N7	Each reviewer responds: accept / counter / hold	Reviewers	Aggregate responses; Update dimension state	All must respond within 72h
N8	If counter → go to N5 (next round)	Counterer	Increment round counter; Alert if	Max 5 rounds per dimension

Step	Action	Actor	System	Notes
N9	If all accept → dimension LOCKED	System	approaching limit Mark dimension LOCKED; Notify all parties	Locked dimensions visible in summary

Phase C: Information Request Handling

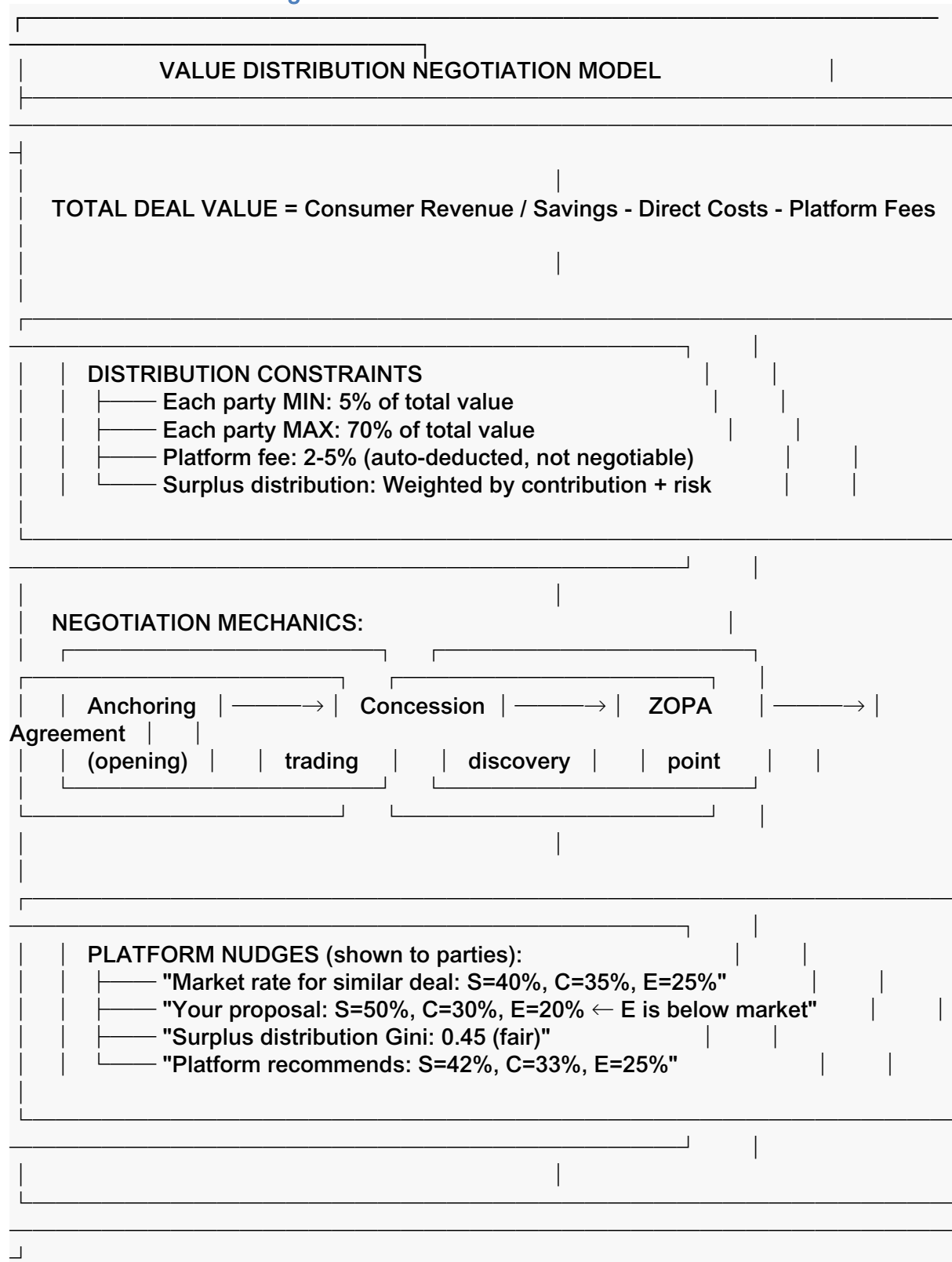
Step	Action	Actor	System	Notes
N10	Party requests clarification on dimension	Requester	Create info-request ticket; Assign to target party	Request tied to specific dimension
N11	Target party receives request	Target	Display request with context; Set 48h response deadline	Auto-reminder at 24h
N12	Target party responds with info/clarification	Target	Post response; Notify requester; Return to negotiation	Response added to deal record
N13	If no response in 48h → AWAITING_PARTY state	System	Escalate to deal coordinator; Start hold timer	See Section 1.4

Phase D: Term Locking

Step	Action	Actor	System	Notes
N14	When all dimensions LOCKED → TERMS_LOCK ready	System	Compile final term sheet; Generate diff from opening positions	Full audit trail included
N15	Platform runs final Win-Win-Win validation	System	Validate economic, balance, feasibility checks	Any failure → return to negotiation
N16	Platform generates commitment agreement	System	Populate template with all locked terms; Create signature blocks	Legal template per jurisdiction
N17	All parties review final agreement	All parties	Display agreement; Show key terms summary; Require explicit “I agree”	72h review period
N18	All parties	All parties	Capture signature;	→

Step	Action	Actor	System	Notes
	digitally sign		Timestamp; Verify authorization	TERMS_LOC KED state

4.5 Value Distribution Negotiation Framework



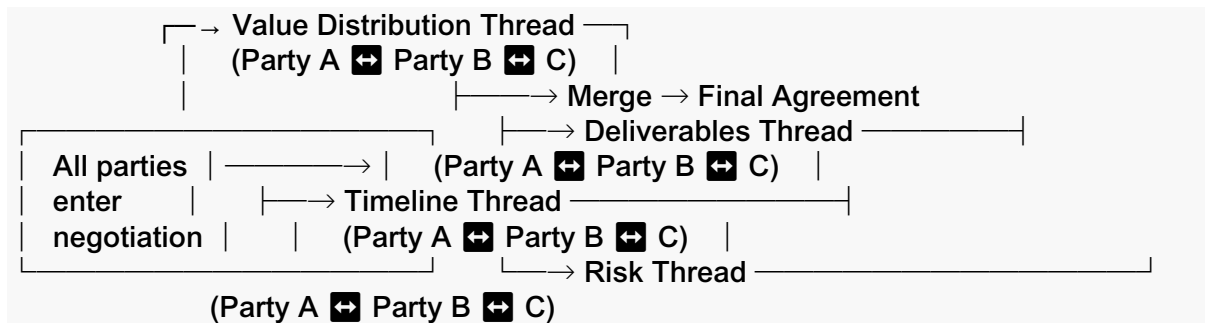
4.6 Parallel vs Sequential Negotiation Paths

Sequential Path (Default for simple deals):

Party A proposes → Party B responds → Party C responds → Revise → ... → Agreement

- Used when: Deal value < \$10K, 2-3 dimensions, parties prefer simplicity
- Timeout: 72h per round, max 5 rounds

Parallel Path (Default for complex deals):



- Used when: Deal value ≥ \$10K, 4+ dimensions, multiple entities per party
- Timeout: 72h per thread per round, max 5 rounds per thread
- Merge: When all threads LOCKED or when initiator calls for merge

Hybrid Path (Adaptive):

Start Sequential → If round count > 3 on any dimension → Switch to Parallel
OR party count per role > 1

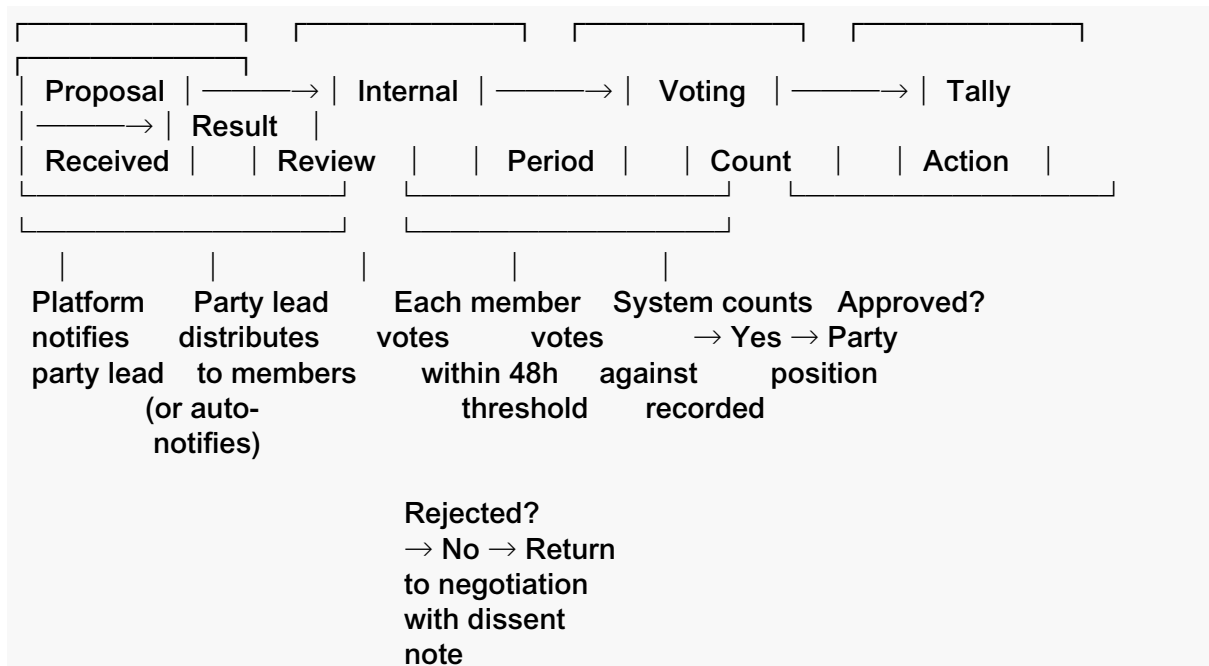
4.7 Multi-Entity Approval Workflow

When any party consists of multiple entities (e.g., a Consumer group with 3 co-investors):

MULTI-ENTITY APPROVAL PATTERNS	
Pattern	Rules
ANY_ONE	Any single authorized member can approve (suitable for sole proprietorships)
MAJORITY	>50% of members must approve (suitable for partnerships, small groups)
ALL	100% of members must approve (suitable for joint ventures, critical deals)
THRESHOLD	Custom threshold (e.g., 2 of 3, 75%)

	(configurable per party per deal)	
WEIGHTED	Approval weighted by ownership/equity share (suitable for companies with shareholders)	

Multi-Entity Voting Flow:



4.8 Negotiation Timeout Rules

Event	Timeout	Action
Proposal submitted	72h for response	Auto-reminder at 48h; Escalate at 72h
Information request	48h for response	Auto-reminder at 24h; AWAITING_PARTY at 48h
Round limit reached (5)	N/A	Platform suggests mediation or ON_HOLD
No activity on dimension	7 days	Auto-ping all parties; Flag at 14 days
Full negotiation	30 days total	ON_HOLD at 30 days; CANCELLED at 45 days

4. Matching & Discovery Workflow

4.1 Discovery Architecture

The platform employs a **multi-channel discovery system** combining algorithmic matching, network exploration, and proactive opportunity identification.

DISCOVERY CHANNELS	
Channel	Description
Algorithmic Match	Compatibility scoring based on profile data
Network Discovery	2nd/3rd degree connections, deal history
Opportunity Feed	Proactive suggestions based on intent signals
Browse & Search	Self-directed exploration of marketplace
Enhancer Brokerage	Enhancer-initiated matchmaking (Section 2.3)
Deal Cloning	Template-based deal replication in new contexts

4.2 Compatibility Scoring Algorithm

Overall Compatibility Score (0-100):

$\text{COMPATIBILITY_SCORE} = w_1 \cdot \text{Resource_Need_Match} + w_2 \cdot \text{Value_Alignment} + w_3 \cdot \text{Trust_Score} + w_4 \cdot \text{Geographic_Fit} + w_5 \cdot \text{Temporal_Alignment} + w_6 \cdot \text{Historical_Success} + w_7 \cdot \text{Risk_Profile_Match}$

Where: $w_1=0.25$, $w_2=0.20$, $w_3=0.15$, $w_4=0.10$, $w_5=0.10$, $w_6=0.10$, $w_7=0.10$

Dimension 1: RESOURCE-NEED MATCH ($w_1 = 0.25$)

- Type compatibility: Exact=1.0, Related=0.7, Convertible=0.4, Mismatch=0
- Quantity match: $|\text{supply} - \text{demand}| / \max(\text{supply}, \text{demand}) \rightarrow \text{score} = 1 - \text{ratio}$
- Quality alignment: Meets spec=1.0, Close=0.8, Requires upgrade=0.5
- Capacity utilization: Optimal zone (60-90%) = 1.0, else degrades

Dimension 2: VALUE ALIGNMENT ($w_2 = 0.20$)

- Price overlap: ZOPA exists=1.0, Nearby=0.7, No overlap=0
- Value distribution compatibility: $\text{Gini} < 0.3 = 1.0$, $< 0.5 = 0.7$, $> 0.5 = 0.3$
- Payment terms alignment: Matching=1.0, Flexible=0.8, Conflicting=0.3

Dimension 3: TRUST SCORE ($w_3 = 0.15$)

- Platform tenure: Normalized years on platform (0-1)
- Completion rate: Deals completed / Deals started
- Rating average: 5-star scale normalized
- Verification level: Basic=0.5, Verified=0.8, Premium=1.0
- Dispute history: $1 - (\text{disputes} / \text{total deals})$, floor at 0.3

Dimension 4: GEOGRAPHIC FIT ($w_4 = 0.10$)

- Distance: $< 10\text{km} = 1.0$, $< 50\text{km} = 0.9$, $< 200\text{km} = 0.7$, $< 1000\text{km} = 0.5$, $> 1000\text{km} = 0.3$
- Logistics feasibility: Road/rail/sea/air connectivity score
- Regulatory compatibility: Same jurisdiction=1.0, Compatible=0.7, Complex=0.3

Dimension 5: TEMPORAL ALIGNMENT ($w_5 = 0.10$)

- Availability overlap: Jaccard index of available windows

- Urgency match: Both urgent=1.0, One urgent=0.6, Neither urgent=0.8
- Seasonal fit: In-season=1.0, Off-season=0.5, Wrong season=0.1

Dimension 6: HISTORICAL SUCCESS ($w_6 = 0.10$)

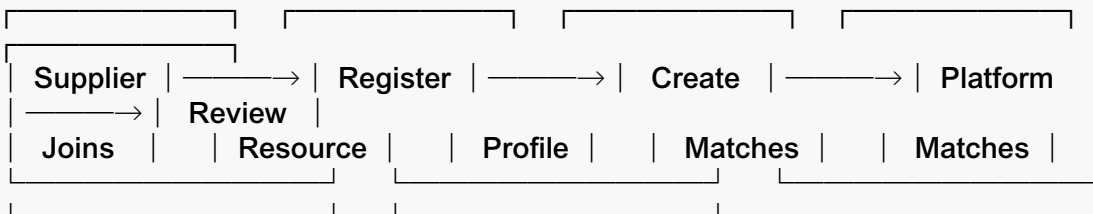
- Prior deals together: Each successful deal +0.2 (max 1.0)
- Similar deal success: Completed similar deal type=0.8
- Mutual connections: Shared successful partners=+0.1 each

Dimension 7: RISK PROFILE MATCH ($w_7 = 0.10$)

- Deal size comfort: Within party's typical range=1.0, Stretch=0.6
- Risk tolerance: Matching profiles=1.0, Complementary=0.7, Clashing=0.3
- Financial health: All parties solvent=1.0, Flagged=0.5

4.3 Discovery Flow per Party Type

Supplier Discovery Flow

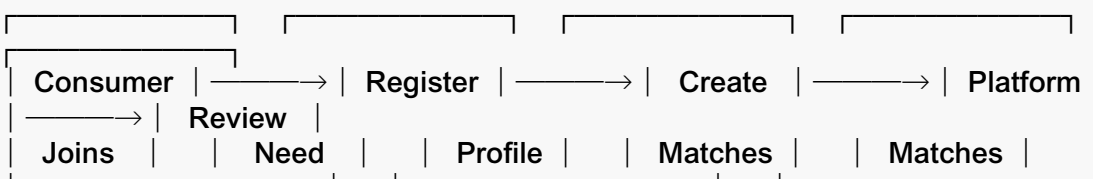


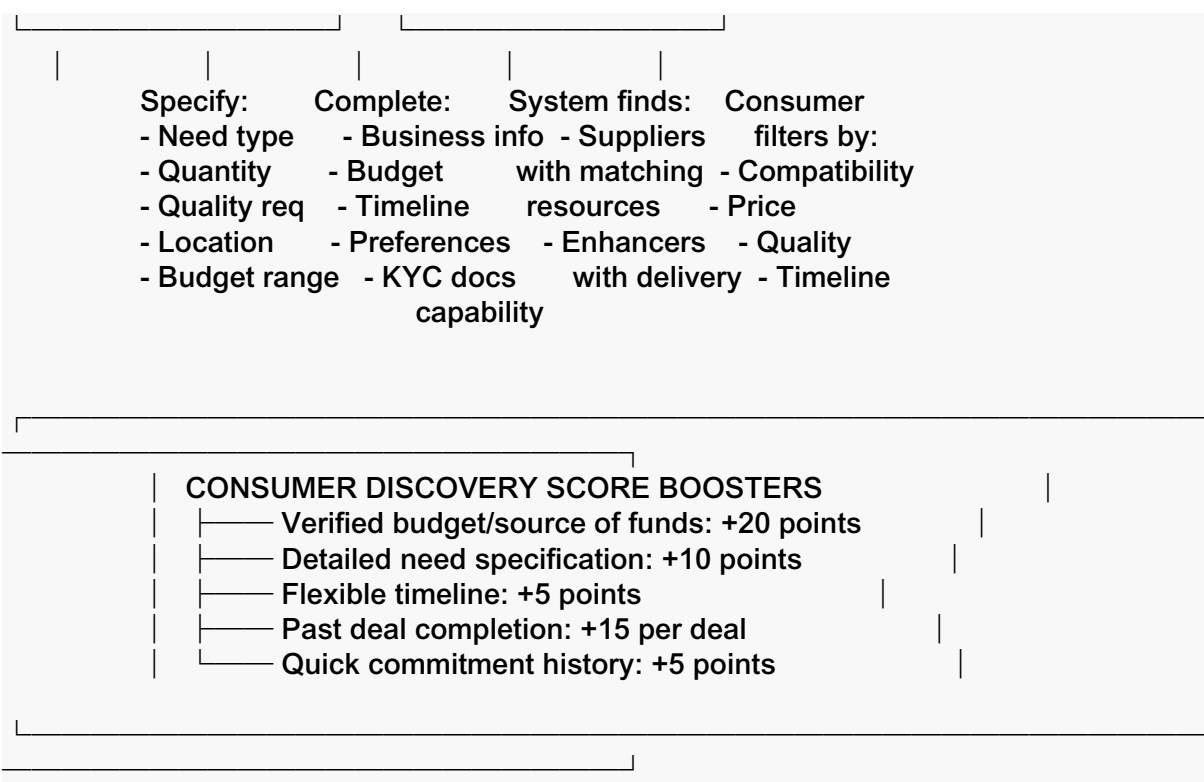
Specify: Complete: System finds: Supplier
 - Resource type - Capabilities - Consumers filters by:
 - Location - History with matching - Compatibility
 - Availability - Preferences needs - Distance
 - Output specs - Pricing hint - Enhancers - Value potential
 - KYC docs with relevant - Trust score
 capabilities

SUPPLIER DISCOVERY SCORE BOOSTERS

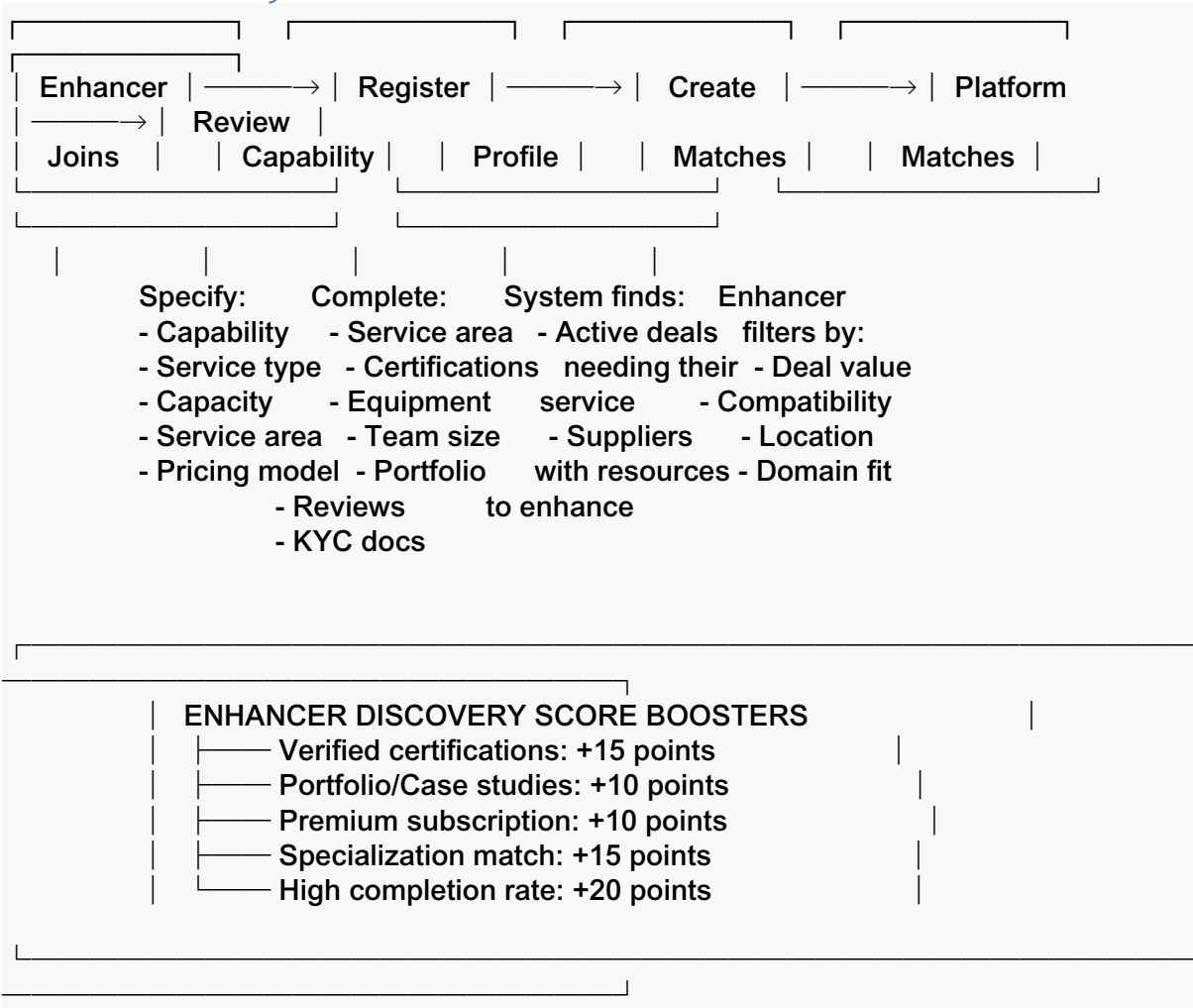
- Complete profile: +10 points
- Verified identity: +15 points
- Photos/documentation: +5 points
- Premium subscription: +10 points
- Quick response history: +5 points
- Successful past deals: +10 per deal

Consumer Discovery Flow

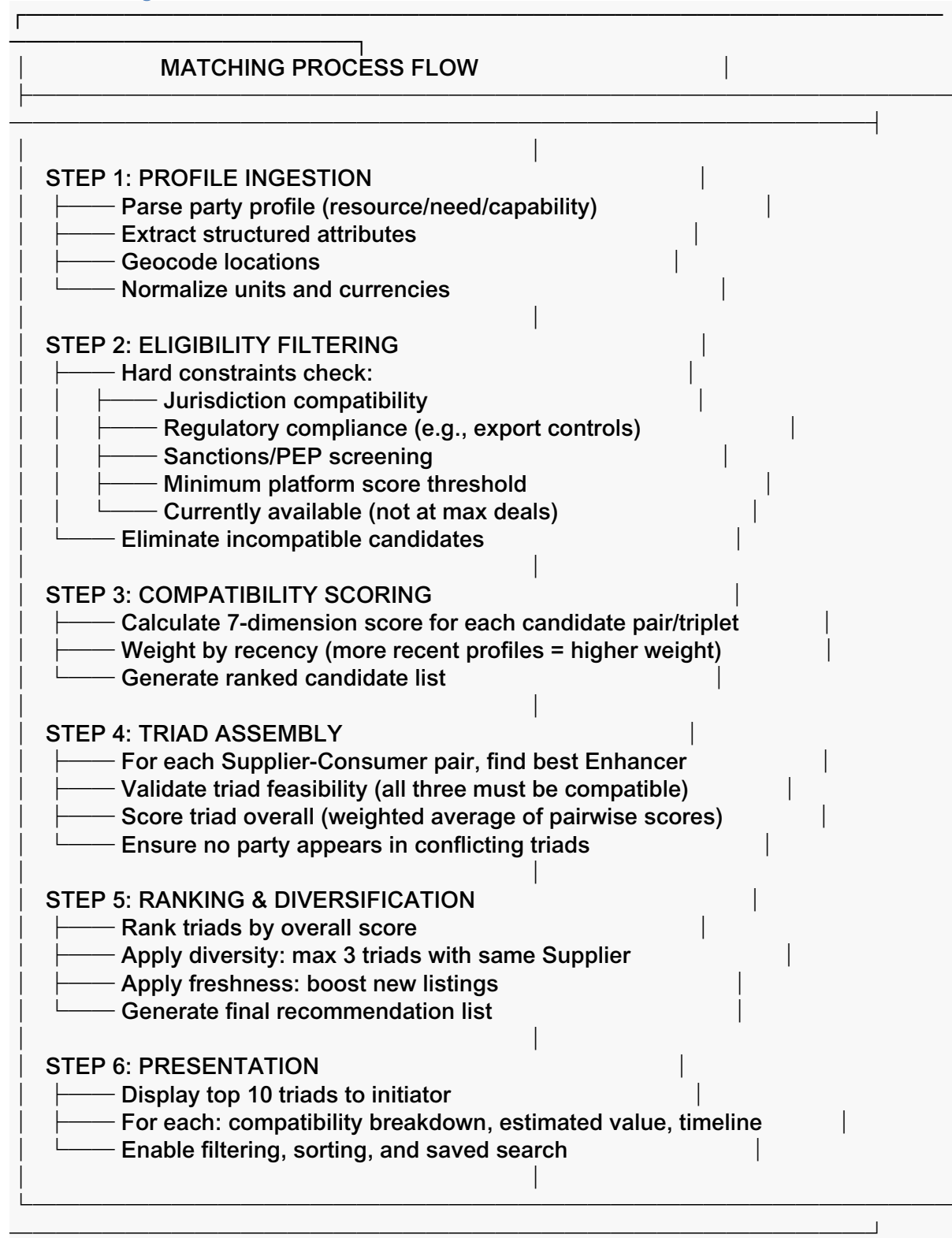




Enhancer Discovery Flow



4.4 Matching Process Flow



4.5 Deal Suggestion Engine

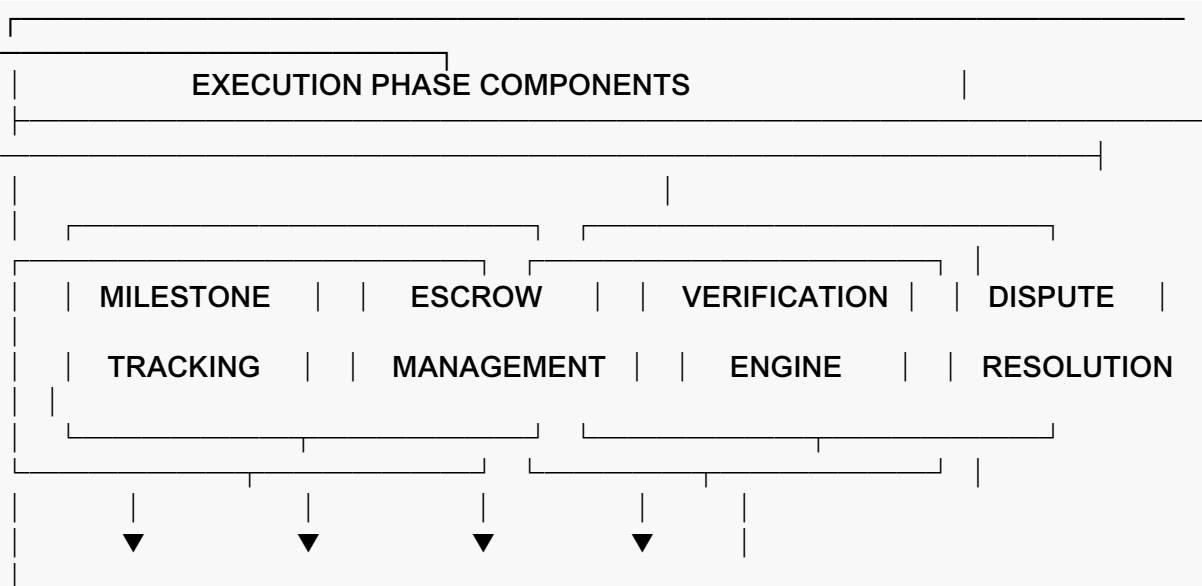
The platform proactively suggests deals based on:

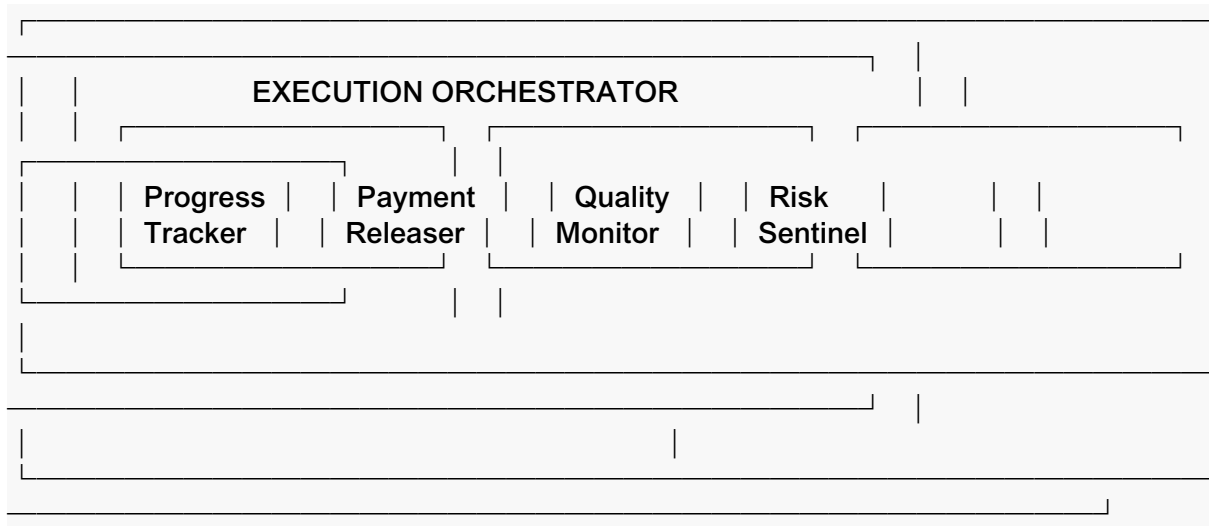
--

SUGGESTION TRIGGERS
TRIGGER 1: New resource/need posted - Immediate match against existing pool - Notify matched parties within 1 hour - Confidence threshold: 70+ for immediate, 50+ for digest TRIGGER 2: Market opportunity detected - Price/volume anomaly in market data - Match underutilized resources with unmet needs - Notify relevant parties in daily opportunity digest TRIGGER 3: Historical pattern match "Deals like this usually pair with..." - Seasonal patterns (e.g., harvest timing) - Proactive suggestion 30 days before optimal window TRIGGER 4: Network effect "Your partner X worked successfully with Y" - Transitive trust: $A \leftrightarrow B + B \leftrightarrow C \rightarrow \text{suggest } A \leftrightarrow C$ - Weighted by mutual connection strength TRIGGER 5: Enhancer alert - Enhancer signals availability - Platform finds best S-C pair for E - Triple-notification to all three parties

4. Execution & Settlement Workflow

4.1 Execution Architecture





4.2 Milestone Tracking System

Milestone Structure:

MILESTONE DEFINITION TEMPLATE	
Field	Description
milestone_id	Unique identifier (e.g., MS-001)
title	Short name (e.g., "Site Preparation")
description	Detailed deliverable description
owner	Party responsible for delivery
verifier	Party responsible for verification
due_date	Target completion date
predecessors	Milestones that must complete first
deliverables	Required evidence (photos, docs, signatures)
payment_trigger	Amount released on completion
completion_criteria	Measurable acceptance criteria
auto_verify	Can platform auto-verify? (Y/N)
dispute_window	Hours allowed for challenge post-submission

Milestone Status Flow:

stateDiagram-v2

[*] --> NOT_STARTED : Deal enters EXECUTING

NOT_STARTED --> IN_PROGRESS : Owner begins work

NOT_STARTED --> BLOCKED : Predecessor not complete

BLOCKED --> IN_PROGRESS : Predecessors complete

IN_PROGRESS --> SUBMITTED : Owner submits deliverables

IN_PROGRESS --> AT_RISK : < 24h to deadline, < 80% complete
IN_PROGRESS --> DELAYED : Past deadline

AT_RISK --> SUBMITTED : Owner submits
AT_RISK --> DELAYED : Deadline passes

DELAYED --> SUBMITTED : Owner submits late
DELAYED --> DISPUTED : Verifier challenges delay

SUBMITTED --> UNDER_REVIEW : Verifier begins review
SUBMITTED --> DISPUTED : Verifier challenges

UNDER_REVIEW --> ACCEPTED : Verifier approves (auto after 48h)
UNDER_REVIEW --> REJECTED : Verifier rejects with reason
UNDER_REVIEW --> DISPUTED : Verifier raises dispute

REJECTED --> IN_PROGRESS : Owner revises
REJECTED --> DISPUTED : Owner disputes rejection

ACCEPTED --> COMPLETED : Payment released
COMPLETED --> [*]

DISPUTED --> IN_PROGRESS : Resolved - continue
DISPUTED --> COMPLETED : Resolved - partial accept
DISPUTED --> CANCELLED : Resolved - terminate

Example Milestone Sequence (Farm-Dealer-Crop Deal):

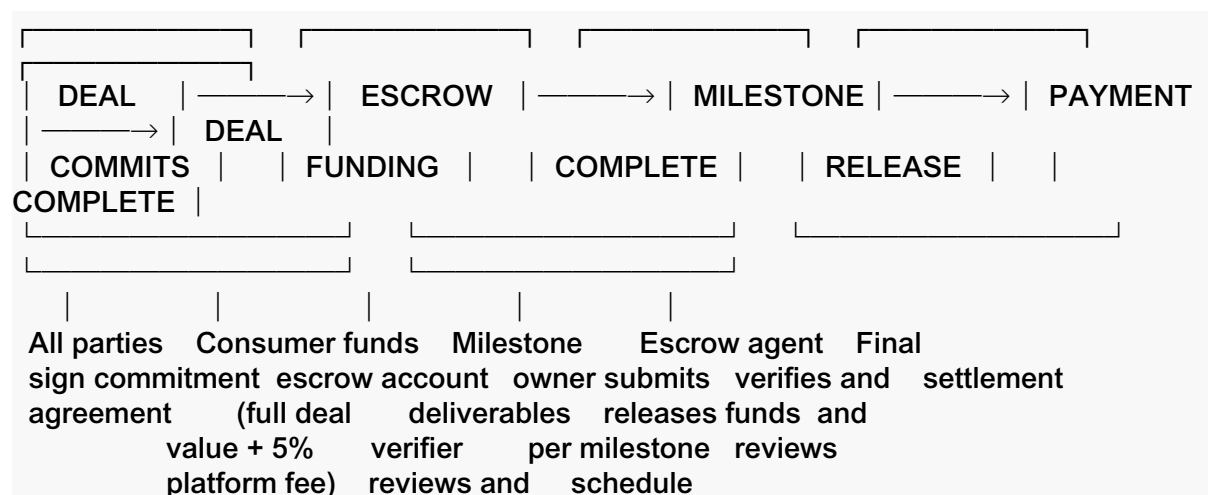
#	Milestone	Owner	Verifier	Payment	Duration
1	Land preparation	Supplier	Enhancer	10%	1 week
2	Seed/fertilizer delivery	Enhancer	Supplier	15%	3 days
3	Planting completed	Enhancer	Supplier	15%	1 week
4	Growth monitoring #1	Enhancer	Consumer	10%	4 weeks
5	Growth monitoring #2	Enhancer	Consumer	10%	4 weeks
6	Harvest completed	Supplier	Enhancer	20%	2 weeks
7	Quality inspection	Consumer	Platform	10%	3 days
8	Final delivery	Supplier	Consumer	10%	1 week

4.3 Payment / Value Transfer Flow

The platform supports multiple payment models:

PAYMENT MODEL OPTIONS	
Model	Mechanics
ESCROW (Default)	Platform holds funds; releases on milestone verification. 3-party approval for release. Best for: High-value, complex deals
STAGED_DIRECT	Consumer pays directly at milestones; platform tracks but doesn't hold funds. Auto-invoicing. Best for: Trusted repeat partners
PERFORMANCE_BOND	Enhancer posts bond; forfeits on non-delivery. Supplier and Consumer payments held in escrow. Best for: Enhancer-dependent deals
REVENUE_SHARE	Minimal upfront; automatic revenue splitting. Smart-contract style distribution. Best for: Joint ventures, ongoing operations
HYBRID	Combination: Escrow for key milestones + direct for minor milestones + revenue share for surplus. Best for: Large, multi-phase deals

Escrow Flow (Default Model):



accepts	
ESCROW RULES	
—	Funding must occur within 72h of COMMITTED state
—	Partial funding → AWAITING_PARTY until fully funded
—	Unfunded after 7 days → CANCELLED
—	Release requires: milestone ACCEPTED + 24h dispute window clear
—	Dispute freezes all releases until resolved
—	Platform fee (2-5%) deducted at each release
—	Refund on CANCELLED: pro-rata minus platform fees

4.4 Completion Criteria & Verification

COMPLETION VERIFICATION MATRIX	
Verification Method	Applicable Deal Types
PHOTO_EVIDENCE	Construction, agriculture, logistics GPS-tagged photos, before/after comparison
DOCUMENT_REVIEW	Professional services, IP deals, consulting Signed deliverables, reports, code review
THIRD_PARTY_INSPECTION	Manufacturing, quality-sensitive goods Certified inspector report
IoT_SENSOR_DATA	Equipment rental, logistics, smart buildings Automated readings, usage logs
COUNTER_PARTY_ACCEPT	All deal types (baseline) Verifier clicks "Accept" with optional comment
PLATFORM_AUTO_VERIFY	Low-risk, repetitive milestones All parties opted in, pattern matches history

Completion Workflow:

Step	Action	Actor	System	Notes
C1	Milestone owner completes work	Owner	Log completion timestamp	GPS/timestamp verified
C2	Owner uploads deliverables	Owner	Store evidence; Notify verifier	Required format check
C3	Verifier reviews evidence	Verifier	Display evidence; Show acceptance criteria	48h review window
C4	Verifier decides: accept / reject / dispute	Verifier	Route accordingly	Auto-accept after 48h
C5a	If accept → payment released	System	Trigger escrow release; Update milestone	Notification to all
C5b	If reject → owner revises	System	Return to IN_PROGRESS with feedback	Max 3 revision cycles
C5c	If dispute → DISPUTED state	System	Create dispute case; Assign mediator	See Section 5.5
C6	If all milestones COMPLETED	System	Trigger final settlement; Calculate bonuses	Generate completion report
C7	All parties rate each other	All	Collect ratings; Calculate reputation delta	Anonymous, mandatory
C8	Deal → COMPLETED	System	Archive deal; Update statistics; Release final payment	Terminal state

4.5 Dispute Handling Workflow

Dispute Classification:

DISPUTE TYPES & SEVERITY			
Type	Severity	Resolution Path	
DELIVERY_DELAY	Low-Medium	Mediation + penalty adjustment	
QUALITY_SHORTFALL	Medium	Inspection + rework/reduction	
PAYMENT_DISPUTE	Medium	Escrow review + mediation	
SCOPE_DISAGREEMENT	Medium	Terms review + renegotiation	
BREACH_OF_CONTRACT	High	Mediation + penalty + termination	
FORCE_MAJEURE	Variable	Clause review + mutual agreement	

FRAUD	Critical	Investigation + account action	
SEVERITY → RESPONSE TIME			
Low: 7 days			
Medium: 5 days			
High: 3 days			
Critical: 24 hours			

Dispute Resolution Flow:

Step	Action	Actor	System	Notes
D1	Party raises dispute with evidence	Disputant	Create dispute case; Assign ID; Freeze payments	2% good-faith deposit required
D2	Platform reviews dispute validity	Platform	Check: evidence provided, category valid, deposit paid	Invalid → reject, refund deposit
D3	Platform notifies all parties	System	Share dispute summary (not full evidence); Request response	48h response window
D4	Accused party submits defense	Respondent	Upload counter-evidence; Propose resolution	Optional
D5	Platform assigns mediator	System	Match to trained mediator by deal domain	Auto-escalation rules apply
D6	Mediator reviews all evidence	Mediator	Review case file; Schedule mediation session if needed	Within severity response time
D7	Mediation session (virtual)	All parties + Mediator	Facilitated discussion; Explore resolutions	Recorded, confidential
D8	Mediator proposes resolution	Mediator	Draft resolution: terms, penalties, next steps	Based on contract terms
D9	Parties vote: accept / reject	All parties	Collect votes; Check approval thresholds	Unanimous accept → resolved
D10a	If accepted → implement resolution	System	Execute: payment adjustment, milestone update,	Resume or terminate per resolution

Step	Action	Actor	System	Notes
D10b	If rejected → escalate	System	etc. Notify parties; Offer: re-mediation or arbitration	Arbitration requires mutual consent
D11	External arbitration (if agreed)	Arbitrator	Binding arbitration per platform terms	Result enforced by platform
D12	Resolution implemented	System	Close dispute case; Update deal state; Release/adjust escrow	Reputation impact applied

Dispute Outcome Types:

DISPUTE OUTCOMES	
Outcome	Effect
FAVOR_DISPUTANT	Disputant receives compensation/reduction Respondent reputation penalty Deal continues with adjusted terms
FAVOR_RESPONDENT	Disputant deposit forfeited to respondent Disputant reputation penalty Deal continues as originally specified
SPLIT_DECISION	Shared responsibility; partial adjustment Both parties minor reputation impact Deal continues with mutual compromise
MUTUAL_TERMINATION	Deal ends by agreement Pro-rata settlement from escrow No reputation penalty (if mutually agreed)
FORCED_TERMINATION	Deal ends by platform decision Escrow distributed per mediator ruling Breaching party reputation penalty
REVISED_TERMS	New terms agreed during mediation

	Deal continues; terms updated	
	Minimal reputation impact	

4. Swimlane Diagrams

4.1 Overall Deal Lifecycle Swimlane

```
%%{init: {'theme': 'base', 'themeVariables': { 'primaryColor': '#e1f5fe', 'primaryTextColor': '#01579b', 'primaryBorderColor': '#0288d1', 'lineColor': '#0288d1', 'secondaryColor': '#fff3e0', 'tertiaryColor': '#e8f5e9'}}}%%
```

```
sequenceDiagram
```

```
actor Initiator as Initiator
actor Counter1 as Counter-Party 1
actor Counter2 as Counter-Party 2
participant Platform as Platform Engine
participant Escrow as Escrow Service
```

Note over Initiator, Escrow: PHASE 1: INITIATION

```
Initiator->>Platform: Create Deal Draft
```

```
Platform-->>Initiator: Confirm DRAFT state
```

```
Initiator->>Platform: Submit Proposal
```

```
Platform->>Counter1: Send Invitation
```

```
Platform->>Counter2: Send Invitation
```

```
Platform-->>Initiator: SUGGESTED state
```

Note over Initiator, Escrow: PHASE 2: REVIEW & ACK

```
Counter1->>Platform: Acknowledge Proposal
```

```
Counter2->>Platform: Acknowledge Proposal
```

```
Platform-->>Initiator,Counter1,Counter2: NEGOTIATING state
```

Note over Initiator, Escrow: PHASE 3: NEGOTIATION

```
par Term Thread 1: Value Distribution
```

```
Initiator->>Platform: Propose 40/35/25 split
```

```
Platform->>Counter1: Notify proposal
```

```
Platform->>Counter2: Notify proposal
```

```
Counter1->>Platform: Counter: 42/33/25
```

```
Counter2->>Platform: Accept
```

```
Initiator->>Platform: Accept counter
```

```
Platform-->>Initiator,Counter1,Counter2: Value dim LOCKED
```

```
and Term Thread 2: Timeline
```

```
Counter1->>Platform: Propose 90-day timeline
```

```
Initiator->>Platform: Counter: 60 days
```

```
Counter2->>Platform: Accept 60 days
```

```
Counter1->>Platform: Accept 60 days
```

```
Platform-->>Initiator,Counter1,Counter2: Timeline dim LOCKED
```

```
end
```

Note over Initiator, Escrow: PHASE 4: COMMITMENT

```
Platform->>Initiator,Counter1,Counter2: All terms LOCKED
```

```
Initiator->>Platform: Sign Commitment
```

```
Counter1->>Platform: Sign Commitment
```


Counter2->>Platform: Sign Commitment
Platform->>Escrow: Initialize Escrow Account
Escrow-->>Platform: Account Ready
Platform-->>Initiator,Counter1,Counter2: COMMITTED state

Note over Initiator, Escrow: PHASE 5: EXECUTION
Counter1->>Escrow: Fund Escrow (100%)
Escrow-->>Platform: Confirm Funding
Platform-->>Initiator,Counter1,Counter2: EXECUTING state

loop Per Milestone
Initiator->>Platform: Submit Milestone Evidence
Platform->>Counter1: Request Verification
Counter1->>Platform: Verify & Accept
Platform->>Escrow: Release Milestone Payment
Escrow-->>Counter2: Transfer Payment
Platform-->>Initiator,Counter1,Counter2: Milestone Complete
end

Note over Initiator, Escrow: PHASE 6: COMPLETION
Platform->>Initiator,Counter1,Counter2: All Milestones Complete
Platform->>Escrow: Release Final Payment
Escrow-->>Initiator: Final Transfer
Platform->>Initiator,Counter1,Counter2: Request Ratings
Initiator->>Platform: Submit Ratings
Counter1->>Platform: Submit Ratings
Counter2->>Platform: Submit Ratings
Platform-->>Initiator,Counter1,Counter2: COMPLETED state

4.2 Supplier-Initiated Deal Flow Swimlane

sequenceDiagram

actor S as Supplier
actor C as Consumer
actor E as Enhancer
participant Platform as Platform
participant Matcher as Matching Engine
participant Escrow as Escrow Service

Note over S,Escrow: SUPPLIER INITIATION FLOW

S->>Platform: Login → "Create Deal"
Platform-->>S: Display Resource Template
S->>Platform: Describe Resource (idle farm, 50 acres)
S->>Platform: Set Availability & Output Specs
S->>Platform: Set Value Share Expectation (40%)

S->>Platform: "Find Matches"
Platform->>Matcher: Query: Suppliers with farm → Consumers needing crops
Matcher->>Platform: Return ranked Consumer list
Platform-->>S: Display Consumer matches with scores
S->>Platform: Select Consumer C (score: 92)

Platform->>Matcher: Query: Enhancers for farm+crop deals
Matcher->>Platform: Return ranked Enhancer list
Platform-->>S: Display Enhancer matches
S->>Platform: Select Enhancer E (score: 88)

S->>Platform: Propose Value Split: S=40%, C=35%, E=25%
S->>Platform: Propose Milestones (8 steps)
S->>Platform: Submit Deal Proposal
Platform->>Platform: Run Win-Win-Win Validation
Platform-->>S: Validation Passed → SUGGESTED

Platform->>C: Send Deal Invitation (email + in-app)
Platform->>E: Send Deal Invitation (email + in-app)

alt Consumer Accepts
 C->>Platform: Acknowledge & Review
 Platform-->>C: Display Full Proposal
 C->>Platform: Accept Negotiation
else Consumer Declines
 C->>Platform: Decline with Reason
 Platform-->>S: Notification: C declined, suggest alternatives
 S->>Platform: Select Alternative Consumer
end

alt Enhancer Accepts
 E->>Platform: Acknowledge & Review
 Platform-->>E: Display Full Proposal
 E->>Platform: Accept Negotiation
else Enhancer Declines
 E->>Platform: Decline with Reason
 Platform-->>S: Notification: E declined, suggest alternatives
 S->>Platform: Select Alternative Enhancer
end

Platform-->>S,C,E: All parties accepted → NEGOTIATING

Note over S,Escrow: NEGOTIATION
C->>Platform: Counter: Value Split S=38%, C=37%, E=25%
E->>Platform: Accept C's counter
S->>Platform: Accept C's counter
Platform-->>S,C,E: TERMS_LOCKED

S->>Platform: Sign Commitment
C->>Platform: Sign Commitment
E->>Platform: Sign Commitment
Platform->>Escrow: Initialize Escrow
Platform-->>S,C,E: COMMITTED → EXECUTING in 3 days

Note over S,Escrow: EXECUTION
C->>Escrow: Fund Escrow (\$100K)
Platform-->>S,C,E: EXECUTING

S->>Platform: Submit Milestone 1: Land Prep (photos)
 E->>Platform: Verify: Accept
 Platform->>Escrow: Release \$10K → S

E->>Platform: Submit Milestone 2: Seeds Delivered (invoice)
 S->>Platform: Verify: Accept
 Platform->>Escrow: Release \$15K → E

Note over S,Escrow: ... (milestones 3-8) ...

S->>Platform: Submit Milestone 8: Final Delivery
 E->>Platform: Verify: Accept
 C->>Platform: Verify Quality: Accept
 Platform->>Escrow: Release Final \$10K → S

Platform-->>S,C,E: COMPLETED
 S->>Platform: Rate C (5★) and E (5★)
 C->>Platform: Rate S (5★) and E (4★)
 E->>Platform: Rate S (5★) and C (5★)
 Platform-->>S,C,E: Deal Closed, Reputation Updated

4.3 Negotiation Flow Swimlane

sequenceDiagram

actor P1 as Party A
 actor P2 as Party B
 actor P3 as Party C
 participant Platform as Platform
 participant Mediator as Deal Coordinator

Note over P1,Mediator: NEGOTIATION PHASE ENTRY

Platform->>P1,P2,P3: All parties in NEGOTIATING state
 Platform->>P1,P2,P3: Display Negotiation Dashboard

Note over P1,Mediator: PARALLEL THREAD 1: VALUE DISTRIBUTION

P1->>Platform: Propose: P1=45%, P2=35%, P3=20%
 Platform->>P2: Notify: P1 proposes value split
 Platform->>P3: Notify: P1 proposes value split
 Platform-->>P1,P2,P3: Show: Gini=0.47, E is below market

P2->>Platform: Counter: P1=42%, P2=35%, P3=23%
 P3->>Platform: Request Info: "Can P1 reduce to 40%?"
 Platform-->>P1: P3 requests info on your position
 P1->>Platform: Respond: "42% is my floor"
 Platform-->>P3: P1 responds: 42% is floor

P3->>Platform: Accept P2's counter (42/35/23)
 P1->>Platform: Accept P2's counter
 Platform-->>P1,P2,P3: VALUE DISTRIBUTION LOCKED (42/35/23)

Note over P1,Mediator: PARALLEL THREAD 2: TIMELINE

P3->>Platform: Propose: 120-day timeline

P1->>Platform: Counter: 90-day timeline
 P2->>Platform: Counter: 100-day timeline
 P1->>Platform: Accept 100 days
 P3->>Platform: Accept 100 days
 Platform-->>P1,P2,P3: TIMELINE LOCKED (100 days)

Note over P1,Mediator: MERGE & LOCK
 Platform->>Platform: Check: All dimensions LOCKED?
 Platform->>Platform: Run Win-Win-Win Validation
 Platform-->>P1,P2,P3: VALIDATION PASSED

Note over P1,Mediator: TERMS LOCKED → COMMITMENT
 Platform->>P1,P2,P3: Present Final Term Sheet
 P1->>Platform: Review & Approve
 P2->>Platform: Review & Approve
 P3->>Platform: Review & Approve

alt Internal Approval Needed (P2 has 3 members)
 P2->>Platform: Initiate Internal Vote
 Platform-->>P2: Distribute to P2-M1, P2-M2, P2-M3
 P2-M1->>Platform: Approve
 P2-M2->>Platform: Approve
 P2-M3->>Platform: Reject (wants 38% for P2)
 Platform-->>P2: 2/3 majority → Approved (MAJORITY rule)

end

P1->>Platform: Digital Signature
 P2->>Platform: Digital Signature
 P3->>Platform: Digital Signature
 Platform-->>P1,P2,P3: TERMS_LOCKED → COMMITTED

4.4 Dispute Resolution Flow Swimlane

sequenceDiagram

actor P1 as Disputing Party
 actor P2 as Accused Party
 actor P3 as Third Party
 participant Platform as Platform
 participant Mediator as Mediator
 participant Arb as Arbitration (optional)

Note over P1,Arb: DISPUTE RAISED

P1->>Platform: Raise Dispute (Category: QUALITY_SHORTFALL)
 P1->>Platform: Upload Evidence (photos, docs)
 P1->>Platform: Pay Dispute Deposit (2%)
 Platform->>Platform: Validate Dispute
 Platform-->>P1: Dispute Valid → Case #D-1042 Created

Platform->>P2: Notify: Dispute Raised by P1
 Platform->>P3: Notify: Dispute Raised (informational)
 Platform->>Escrow: FREEZE all pending payments

P2->>Platform: Submit Defense + Counter-Evidence
Platform-->>P1: P2 has submitted defense (evidence summary)

Note over P1,Arb: MEDIATION
Platform->>Mediator: Assign Case #D-1042
Mediator->>Platform: Review Case File
Mediator->>P1: Request Clarification
P1->>Mediator: Respond
Mediator->>P2: Request Clarification
P2->>Mediator: Respond

Mediator->>P1,P2,P3: Schedule Mediation Session
P1->>Mediator: Join Session
P2->>Mediator: Join Session
P3->>Mediator: Join Session (optional)

Note over P1,Arb: within session
Mediator->>P1,P2: Facilitate Discussion
Mediator->>P1,P2: Propose Resolution: Partial refund + revised milestone

alt Resolution Accepted
P1->>Platform: Accept Resolution
P2->>Platform: Accept Resolution
P3->>Platform: Accept Resolution
Platform->>Platform: Implement: Adjust escrow, update milestone
Platform-->>P1: Refund dispute deposit
Platform-->>P1,P2,P3: DISPUTE RESOLVED → Resume EXECUTING
else Resolution Rejected
P1->>Platform: Reject Resolution
Platform-->>P1,P2,P3: Offer: Re-mediation or Arbitration

alt Choose Arbitration
P1->>Platform: Request Arbitration
P2->>Platform: Agree to Arbitration
Platform->>Arb: Submit Case
Arb->>Arb: Review Evidence
Arb->>P1,P2: Binding Ruling
Platform->>Platform: Implement Ruling
Platform-->>P1,P2,P3: DISPUTE RESOLVED
else Choose Re-mediation
P1->>Platform: Request Re-mediation
Note over P1,Arb: Loop to mediation with new mediator
end
end

Note over P1,Arb: POST-RESOLUTION
P1->>Platform: Rate Resolution Process
P2->>Platform: Rate Resolution Process
Platform->>Platform: Update reputation scores
Platform->>Platform: Close Case #D-1042
Platform-->>P1,P2,P3: Final Resolution Summary

4. Appendices

Appendix A: State Machine Formal Definition

```
STATES = { DRAFT, SUGGESTED, PENDING_REVIEW, NEGOTIATING,  
          AWAITING_PARTY,  
          TERMS_LOCKED, COMMITTED, EXECUTING, ON_HOLD, COMPLETED,  
          DISPUTED, CANCELLED, EXPIRED }
```

```
INITIAL_STATE = DRAFT
```

```
TERMINAL_STATES = { COMPLETED, CANCELLED, EXPIRED }
```

```
TRANSITIONS = {  
  (DRAFT, submit, SUGGESTED)      : [Initiator],  
  (DRAFT, abandon, CANCELLED)     : [Initiator],  
  (DRAFT, timeout, EXPIRED)       : [System],  
  
  (SUGGESTED, all_invited, PENDING_REVIEW) : [System],  
  (SUGGESTED, reject, CANCELLED)   : [CounterParty],  
  (SUGGESTED, timeout, EXPIRED)    : [System],  
  
  (PENDING_REVIEW, acknowledge, NEGOTIATING) : [AllParties],  
  (PENDING_REVIEW, withdraw, CANCELLED)      : [Initiator],  
  (PENDING_REVIEW, timeout, EXPIRED)         : [System],  
  
  (NEGOTIATING, lock_terms, TERMS_LOCKED)    : [AllParties],  
  (NEGOTIATING, pause, AWAITING_PARTY)       : [AnyParty, System],  
  (NEGOTIATING, hold, ON_HOLD)                : [AnyParty, System],  
  (NEGOTIATING, withdraw, CANCELLED)          : [AnyParty],  
  (NEGOTIATING, timeout, ON_HOLD)             : [System],  
  
  (AWAITING_PARTY, respond, NEGOTIATING)     : [TargetParty],  
  (AWAITING_PARTY, hold, ON_HOLD)             : [System],  
  (AWAITING_PARTY, withdraw, CANCELLED)       : [AnyParty],  
  (AWAITING_PARTY, timeout, ON_HOLD)          : [System],  
  
  (TERMS_LOCKED, commit, COMMITTED)          : [AllParties],  
  (TERMS_LOCKED, renegotiate, NEGOTIATING)    : [AnyParty],  
  (TERMS_LOCKED, withdraw, CANCELLED)        : [AnyParty],  
  (TERMS_LOCKED, timeout, CANCELLED)         : [System],  
  
  (COMMITTED, begin, EXECUTING)              : [System],  
  (COMMITTED, withdraw, CANCELLED)           : [Unanimous],  
  
  (EXECUTING, complete, COMPLETED)          : [AllParties, System],  
  (EXECUTING, dispute, DISPUTED)             : [AnyParty],  
  (EXECUTING, pause, AWAITING_PARTY)        : [AnyParty],  
  (EXECUTING, hold, ON_HOLD)                 : [AnyParty, System],  
  (EXECUTING, terminate, CANCELLED)          : [Mutual, System],  
  
  (ON_HOLD, resume_negotiating, NEGOTIATING) : [AnyParty],  
  (ON_HOLD, resume_executing, EXECUTING)     : [AnyParty],
```

```

(ON_HOLD, cancel, CANCELLED)      : [System, AnyParty],

(DISPUTED, resume, EXECUTING)      : [AllParties, Mediator],
(DISPUTED, partial_complete, COMPLETED) : [AllParties, Mediator],
(DISPUTED, terminate, CANCELLED)   : [AllParties, Mediator],
(DISPUTED, external_hold, ON_HOLD) : [Mediator]
}

```

Appendix B: Role Permissions Matrix

Action	Initiator	Supplier	Consumer	Enhancer	Platform
Create Draft	✓	✓	✓	✓	
Submit Proposal	✓				
Invite Parties	✓				
Acknowledge Proposal		✓	✓	✓	
Decline Proposal		✓	✓	✓	
Propose Terms	✓	✓	✓	✓	
Counter Terms	✓	✓	✓	✓	
Accept Terms	✓	✓	✓	✓	
Request Hold	✓	✓	✓	✓	✓
Sign Commitment	✓	✓	✓	✓	
Submit Milestone		✓	✓	✓	
Verify Milestone		✓	✓	✓	✓
Raise Dispute	✓	✓	✓	✓	
Mediate Dispute				✓	
Force Cancel (Breach)				✓	
Escrow Release				✓	
Rate Counter-Party		✓	✓	✓	

Appendix C: Notification Matrix

Event	Notifications
Deal submitted	Initiator: confirmation; Counter-parties: invite
Party acknowledges	All parties: "X joined negotiation"
Party declines	Initiator: "X declined, reason: Y"
Proposal submitted	Counter-parties: "New proposal from X"
Terms locked	All: "Terms locked - sign by [date]"
Deal committed	All: "Deal starts [date]"; Escrow: fund request
Milestone submitted	Verifier: "Please review milestone X"
Milestone verified	All: "Milestone X complete"; Owner: payment sent
Milestone delayed	All: "Milestone X delayed"; Coordinator: alert
Dispute raised	All: "Dispute case D-XXX opened"
Dispute resolved	All: "Case D-XXX resolved: [outcome]"

Deal completed	All: "Deal complete - please rate"	
Deal cancelled	All: "Deal cancelled, reason: [reason]"	
Timeout warning	Relevant party: "Action required within X days"	

Appendix D: Glossary

Term	Definition
3-Party Deal	A transaction involving Supplier, Consumer, and Enhancer roles
Commitment	Binding agreement signed by all parties, triggering execution
Escrow	Third-party holding of funds until conditions are met
Milestone	A defined checkpoint in execution with associated deliverables and payments
Win-Win-Win	The principle that all three parties must benefit from the deal
ZOPA	Zone of Possible Agreement - the overlap between party acceptable ranges
Gini Coefficient	Measure of inequality in value distribution (0 = equal, 1 = unequal)
Compatibility Score	Algorithmic measure of fit between parties (0-100)
Good-Faith Deposit	Payment required to raise a dispute, discouraging frivolous claims
Enhancer	Party that adds value to enable the Supplier-Consumer transaction

Document Version: 1.0 / Status: Draft / For Review

Version: 1.0

Status: Production-Ready Architecture Blueprint

Classification: Internal - Confidential

5. Executive Summary

This document presents the comprehensive system architecture for a **3-Party Deal Making Platform** - a domain-agnostic, scalable marketplace that connects Suppliers, Consumers, and Enhancers to create mutually beneficial deals. The architecture follows cloud-native principles, microservices patterns, and event-driven design to support complex multi-party negotiations across any industry domain.

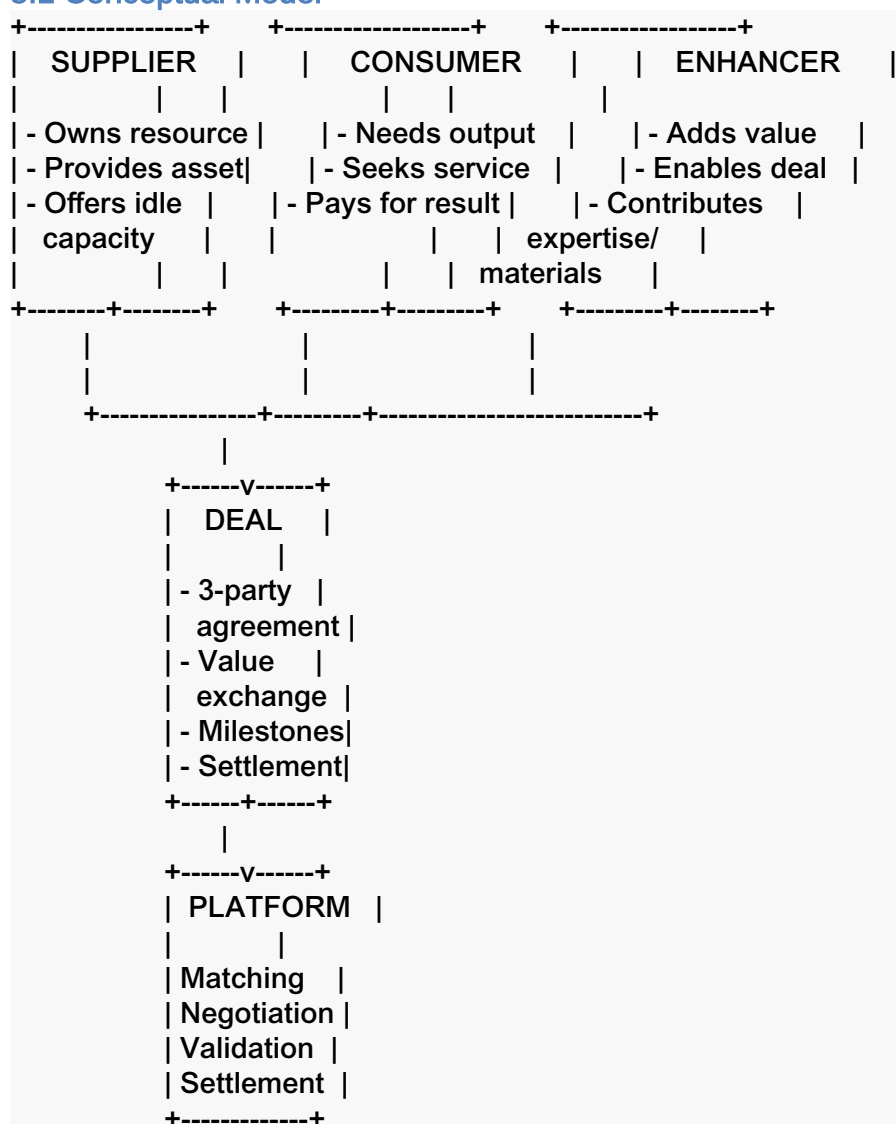
Key Architectural Principles

Principle	Description
Domain-Agnostic Design	Core services are generic; domain-specific logic is pluggable via configuration and extensions
Event-Driven Core	All state changes flow through events, enabling loose coupling and extensibility

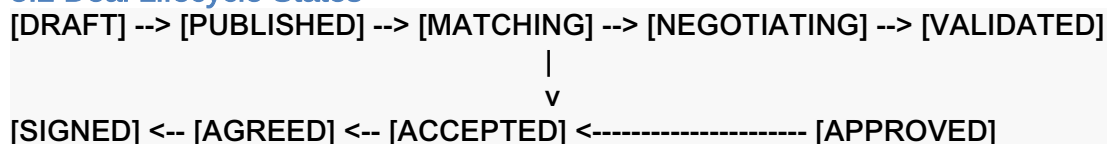
Principle	Description
Data Sovereignty	Each service owns its data; no direct database access between services
** eventual Consistency**	Accepted for cross-service operations; strong consistency within service boundaries
Auditability by Design	Complete event sourcing for deal lifecycle ensures full audit trail
API-First	All services expose versioned APIs; consumer-driven contract testing

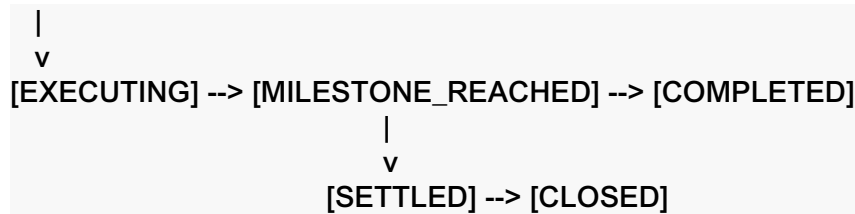
5. System Overview

5.1 Conceptual Model



5.2 Deal Lifecycle States





Alternative paths:

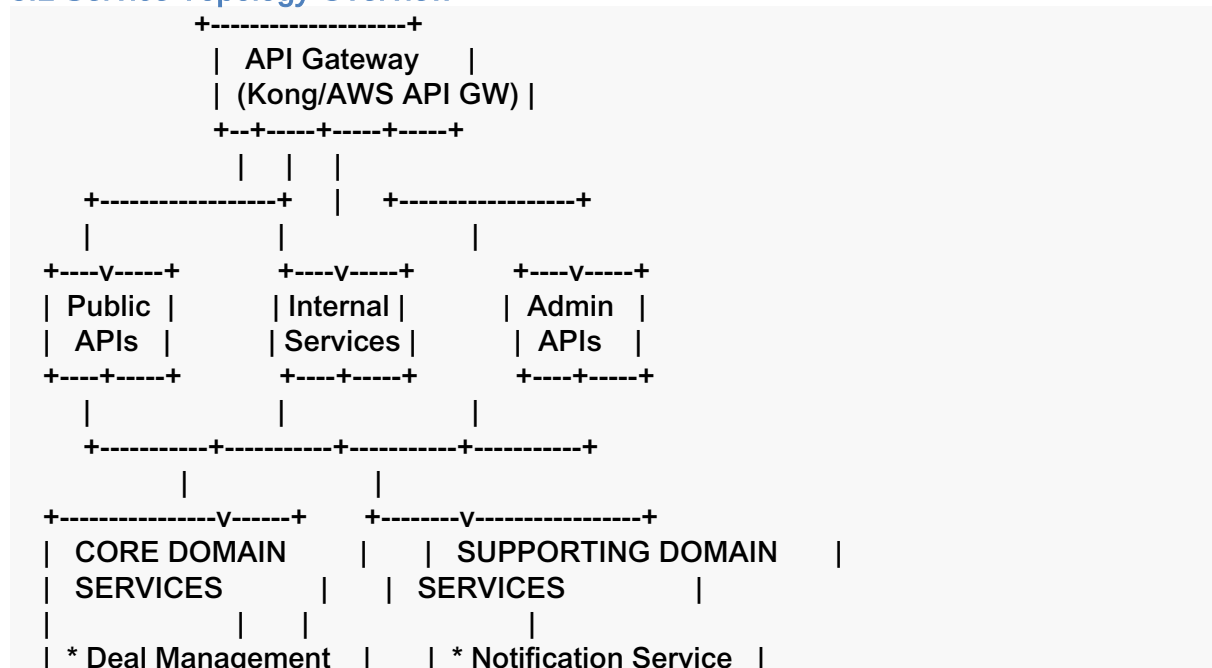
- Any state --> [CANCELLED] (by initiator)
- NEGOTIATING --> [REJECTED] --> [RE-NEGOTIATING] or [ARCHIVED]
- VALIDATED --> [INVALID] --> [REVISION_REQUIRED]
- EXECUTING --> [DISPUTED] --> [ARBITRATION] --> [RESOLVED] or [TERMINATED]

5.3 Platform Capabilities

Capability	Description
Multi-Party Matching	Algorithmic matching of compatible Suppliers, Consumers, and Enhancers
Win-Win Validation	Mathematical/model-based validation that all parties benefit
Flexible Negotiation	Multi-round, asynchronous negotiation with proposal versioning
Milestone Tracking	Progress tracking with conditional payment releases
Trust & Reputation	Rating, review, and reputation scoring across all party types
Domain Adaptability	Configurable deal templates, resource types, and value models

5. Service Decomposition (Microservices)

5.1 Service Topology Overview



* Party & Profile	* Trust & Reputation
* Matching & Disc.	* Analytics & Reporting
* Negotiation	* Media/Document
* Resource Catalog	* Admin & Moderation
* Value Distribution	
* Agreement/Contract	INFRASTRUCTURE
* Execution & Milest.	SERVICES
* Payment/Settlement	* User & Identity
* Search Service	* API Gateway
	* Event Bus
+-----+	+-----+

5.2 Service Catalog

5.2.1 User & Identity Service

Responsibility: Authentication, authorization, identity verification (KYC), session management, MFA

Aspect	Details
Service ID	user-identity-svc
Team Ownership	Platform Security
Deployment	3+ instances, critical path
Data Ownership	Users, credentials, KYC data, sessions, MFA tokens

Data Model:

```
User {
  user_id: UUID (PK)
  email: String (unique, encrypted)
  phone: String (encrypted)
  password_hash: String (bcrypt)
  status: Enum [PENDING_VERIFICATION, ACTIVE, SUSPENDED, DEACTIVATED]
  email_verified: Boolean
  phone_verified: Boolean
  kyc_status: Enum [NOT_REQUIRED, PENDING, VERIFIED, FAILED, EXPIRED]
  mfa_enabled: Boolean
  created_at: Timestamp
  updated_at: Timestamp
}
```

```
UserProfile {
  profile_id: UUID (PK)
  user_id: UUID (FK)
  display_name: String
  avatar_url: String
  locale: String
  timezone: String
  notification_preferences: JSON
}
```

```
KYCRecord {
  kyc_id: UUID (PK)
  user_id: UUID (FK)
  provider: String (e.g., "onfido", "jumio")
  provider_reference: String
  status: Enum
  document_type: String
  verification_result: JSON
  completed_at: Timestamp
}
```

APIs (REST + gRPC):

REST APIs:

```
POST /api/v1/auth/register
POST /api/v1/auth/login
POST /api/v1/auth/logout
POST /api/v1/auth/refresh
POST /api/v1/auth/mfa/enable
POST /api/v1/auth/mfa/verify
GET /api/v1/auth/me
PUT /api/v1/users/{id}/profile
POST /api/v1/users/{id}/kyc/initiate
GET /api/v1/users/{id}/kyc/status
```

gRPC Services:

```
service IdentityService {
  rpc Authenticate(TokenRequest) returns (UserIdentity);
  rpc Authorize(PermissionRequest) returns (AuthorizationResult);
  rpc ValidateSession(SessionRequest) returns (SessionValidity);
  rpc GetUserBulk(UserIdList) returns (UserList);
}
```

Events Published:

- user.registered
- user.activated
- user.suspended
- user.deactivated
- user.kyc.completed
- user.kyc.failed
- user.login.succeeded
- user.login.failed
- user.mfa.enabled

Technology Stack: - Runtime: Node.js / Go - Database: PostgreSQL (users), Redis (sessions/cache) - Auth: OAuth2/OIDC, JWT (RS256), bcrypt - KYC: Onfido/Jumio integration

5.2.2 Party & Profile Service

Responsibility: Party CRUD, party composition, role management, party relationships, verification status

Aspect	Details
Service ID	party-profile-svc
Team Ownership	Core Platform
Deployment	2+ instances
Data Ownership	Parties, party_members, party_roles, party_relationships, verifications

Key Concepts: - A **Party** is an entity (individual or organization) that participates in deals - A Party has a **Type**: **SUPPLIER**, **CONSUMER**, **ENHANCER**, or **MULTI** (can play multiple roles) - A Party consists of one or more **Members** (users or sub-entities) - Parties can have **Relationships** (parent-child, partnership, etc.)

Data Model:

```
Party {
  party_id: UUID (PK)
  party_type: Enum [SUPPLIER, CONSUMER, ENHANCER, MULTI]
  entity_type: Enum [INDIVIDUAL, ORGANIZATION, GROUP]
  display_name: String
  description: Text
  status: Enum [DRAFT, ACTIVE, SUSPENDED, VERIFIED, INACTIVE]
  verification_level: Enum [NONE, BASIC, VERIFIED, PREMIUM]
  metadata: JSON (domain-specific attributes)
  created_by: UUID (User FK)
  created_at: Timestamp
  updated_at: Timestamp
}
```

```
PartyMember {
  membership_id: UUID (PK)
  party_id: UUID (FK)
  user_id: UUID (FK to Identity Service)
  role_within_party: Enum [OWNER, ADMIN, MEMBER, VIEWER]
  permissions: JSON
  joined_at: Timestamp
}
```

```
PartyRoleCapability {
  capability_id: UUID (PK)
  party_id: UUID (FK)
  role_type: Enum [SUPPLIER, CONSUMER, ENHANCER]
  domain_tags: String[] (e.g., ["agriculture", "real-estate"])
  capabilities: JSON
  is_active: Boolean
}
```

APIs:

```
REST:
POST /api/v1/parties
GET /api/v1/parties/{partyId}
PUT /api/v1/parties/{partyId}
```

```
DELETE /api/v1/parties/{partyId}
POST  /api/v1/parties/{partyId}/members
GET   /api/v1/parties/{partyId}/members
DELETE /api/v1/parties/{partyId}/members/{userId}
POST  /api/v1/parties/{partyId}/roles
GET   /api/v1/parties/{partyId}/roles
GET   /api/v1/parties/search
GET   /api/v1/parties/{partyId}/deals
```

gRPC:

```
service PartyService {
  rpc GetParty(PartyRequest) returns (Party);
  rpc ValidatePartyExists(PartyIdRequest) returns (ValidationResult);
  rpc GetPartyCapabilities(PartyRequest) returns (CapabilityList);
  rpc CheckPartyRole(RoleCheckRequest) returns (RoleCheckResult);
  rpc GetPartiesByUser(UserIdRequest) returns (PartyList);
}
```

Events Published:

- party.created
- party.updated
- party.activated
- party.suspended
- party.verified
- party.member.added
- party.member.removed
- party.role.assigned
- party.role.revoked

5.2.3 Deal Management Service

Responsibility: Deal lifecycle orchestration, state machine management, deal metadata, deal-party associations

Aspect	Details
Service ID	deal-mgmt-svc
Team Ownership	Core Platform
Deployment	3+ instances, critical path
Data Ownership	Deals, deal_parties, deal_state_history, deal_templates

Data Model:

```
Deal {
  deal_id: UUID (PK)
  deal_reference: String (human-readable, e.g., "DL-2024-A7B3C2")
  title: String
  description: Text
  domain: String (e.g., "agriculture", "real-estate", "transport")
  template_id: UUID (FK, nullable)
  status: Enum [DRAFT, PUBLISHED, MATCHING, NEGOTIATING, VALIDATED,
```

```

        APPROVED, ACCEPTED, AGREED, SIGNED, EXECUTING,
        COMPLETED, SETTLED, CLOSED, CANCELLED, DISPUTED,
        ARCHIVED]
    initiated_by: UUID (Party FK)
    win_win_validated: Boolean
    value_distribution_id: UUID (FK)
    agreement_id: UUID (FK, nullable)
    execution_plan_id: UUID (FK, nullable)
    metadata: JSON (domain-specific)
    tags: String[]
    visibility: Enum [PUBLIC, PRIVATE, INVITATION_ONLY]
    expiry_date: Timestamp (nullable)
    created_at: Timestamp
    updated_at: Timestamp
    completed_at: Timestamp (nullable)
}

DealParty {
    deal_party_id: UUID (PK)
    deal_id: UUID (FK)
    party_id: UUID (FK to Party Service)
    role_in_deal: Enum [SUPPLIER, CONSUMER, ENHANCER]
    status: Enum [INVITED, PENDING, ACCEPTED, REJECTED, WITHDRAWN]
    joined_at: Timestamp (nullable)
    withdrew_at: Timestamp (nullable)
}

DealStateTransition {
    transition_id: UUID (PK)
    deal_id: UUID (FK)
    from_state: String
    to_state: String
    triggered_by: UUID (User FK)
    trigger_event: String
    reason: Text (nullable)
    metadata: JSON
    occurred_at: Timestamp
}

DealTemplate {
    template_id: UUID (PK)
    name: String
    domain: String
    description: Text
    default_structure: JSON
    required_party_roles: String[]
    default_milestones: JSON
    is_active: Boolean
    created_by: UUID
}

```

State Machine Definition:

states:

- DRAFT
- PUBLISHED
- MATCHING
- NEGOTIATING
- VALIDATED
- APPROVED
- ACCEPTED
- AGREED
- SIGNED
- EXECUTING
- MILESTONE_REACHED
- COMPLETED
- SETTLED
- CLOSED
- CANCELLED
- REJECTED
- DISPUTED
- ARCHIVED

transitions:

- { from: DRAFT, to: PUBLISHED, event: deal.published, auto: false }
- { from: DRAFT, to: ARCHIVED, event: deal.discarded, auto: false }
- { from: PUBLISHED, to: MATCHING, event: deal.matching.started, auto: true }
- }
- { from: MATCHING, to: NEGOTIATING, event: deal.parties.matched, auto: true }
- }
- { from: NEGOTIATING, to: VALIDATED, event: deal.terms.validated, auto: false }
- }
- { from: NEGOTIATING, to: REJECTED, event: deal.rejected, auto: false }
- { from: REJECTED, to: NEGOTIATING, event: deal.renegotiated, auto: false }
- { from: REJECTED, to: ARCHIVED, event: deal.archived, auto: false }
- { from: VALIDATED, to: APPROVED, event: deal.approved, auto: false }
- { from: VALIDATED, to: NEGOTIATING, event: deal.revision.required, auto: false }
- }
- { from: APPROVED, to: ACCEPTED, event: deal.accepted, auto: false }
- { from: ACCEPTED, to: AGREED, event: deal.agreed, auto: false }
- { from: AGREED, to: SIGNED, event: deal.signed, auto: false }
- { from: SIGNED, to: EXECUTING, event: deal.execution.started, auto: true }
- { from: EXECUTING, to: MILESTONE_REACHED, event: milestone.completed, auto: true }
- { from: MILESTONE_REACHED, to: COMPLETED, event: deal.all_milestones.done, auto: true }
- { from: COMPLETED, to: SETTLED, event: deal.settled, auto: true }
- { from: SETTLED, to: CLOSED, event: deal.closed, auto: true }
- { from: EXECUTING, to: DISPUTED, event: deal.disputed, auto: false }
- { from: DISPUTED, to: EXECUTING, event: dispute.resolved, auto: false }
- { from: DISPUTED, to: CANCELLED, event: deal.terminated, auto: false }
- { from: "*", to: CANCELLED, event: deal.cancelled, auto: false }

APIs:

REST:

```
POST /api/v1/deals                # Create deal
GET  /api/v1/deals/{dealId}       # Get deal
PUT  /api/v1/deals/{dealId}       # Update deal
POST /api/v1/deals/{dealId}/publish # Publish deal
POST /api/v1/deals/{dealId}/cancel # Cancel deal
POST /api/v1/deals/{dealId}/parties # Add party to deal
PUT  /api/v1/deals/{dealId}/parties/{partyId}/accept # Accept invitation
PUT  /api/v1/deals/{dealId}/parties/{partyId}/reject # Reject invitation
PUT  /api/v1/deals/{dealId}/parties/{partyId}/withdraw # Withdraw from deal
GET  /api/v1/deals/{dealId}/history # State history
GET  /api/v1/deals/{dealId}/timeline # Full timeline
GET  /api/v1/deals                # List/search deals
POST /api/v1/deals/{dealId}/clone  # Clone from template
```

gRPC:

```
service DealManagementService {
  rpc CreateDeal(DealRequest) returns (Deal);
  rpc GetDeal(DealIdRequest) returns (Deal);
  rpc TransitionState(StateTransitionRequest) returns (StateTransitionResult);
  rpc GetDealParties(DealIdRequest) returns (DealPartyList);
  rpc ValidateDealExists(DealIdRequest) returns (ValidationResult);
  rpc GetDealsByParty(PartyIdRequest) returns (DealList);
  rpc GetDealStateHistory(DealIdRequest) returns (StateHistory);
}
```

Events Published:

- deal.created
 - deal.updated
 - deal.published
 - deal.matching.started
 - deal.parties.matched
 - deal.negotiation.started
 - deal.terms.validated
 - deal.validation.failed
 - deal.approved
 - deal.accepted
 - deal.agreed
 - deal.signed
 - deal.execution.started
 - deal.milestone.reached
 - deal.completed
 - deal.settled
 - deal.closed
 - deal.cancelled
 - deal.rejected
 - deal.disputed
 - deal.archived
 - deal.party.joined
 - deal.party.left
-

5.2.4 Matching & Discovery Service

Responsibility: Algorithmic matching of compatible parties, deal recommendations, compatibility scoring

Aspect	Details
Service ID	matching-svc
Team Ownership	Data Science / Core Platform
Deployment	2+ instances, background workers
Data Ownership	Match_results, compatibility_scores, matching_rules, recommendation_cache

Core Matching Algorithm:

For a given deal requiring parties [S, C, E]:

1. Filter candidates by: domain, location, availability, verification level
2. Score each candidate by: profile completeness, reputation, past performance
3. Compute pairwise compatibility: $S \leftrightarrow C$, $S \leftrightarrow E$, $C \leftrightarrow E$
4. Compute triple compatibility: win-win-win score
5. Rank candidate combinations by aggregate score
6. Apply diversity and fairness constraints
7. Return top-N recommendations with explanation

Data Model:

```
MatchResult {
  match_id: UUID (PK)
  deal_id: UUID (FK)
  supplier_party_id: UUID (FK)
  consumer_party_id: UUID (FK)
  enhancer_party_id: UUID (FK)
  overall_score: Float (0.0 - 1.0)
  pairwise_scores: JSON {s_c: Float, s_e: Float, c_e: Float}
  win_win_win_score: Float
  match_reasons: JSON[]
  status: Enum [PENDING_REVIEW, ACCEPTED, REJECTED, EXPIRED]
  generated_at: Timestamp
  reviewed_at: Timestamp (nullable)
}
```

```
CompatibilityScore {
  score_id: UUID (PK)
  party_a_id: UUID (FK)
  party_b_id: UUID (FK)
  score: Float
  factors: JSON
  computed_at: Timestamp
}
```

```
MatchingRule {
  rule_id: UUID (PK)
  domain: String
}
```

```

rule_type: Enum [FILTER, SCORE, EXCLUDE, REQUIRE]
condition: JSON
weight: Float
priority: Int
is_active: Boolean
}

```

APIs:

REST:

```

POST /api/v1/matching/find          # Find matches for a deal
GET  /api/v1/matching/results/{matchId} # Get match details
POST /api/v1/matching/results/{matchId}/accept # Accept match
POST /api/v1/matching/results/{matchId}/reject # Reject match
GET  /api/v1/matching/recommendations # Get personalized recommendations
POST /api/v1/matching/refresh/{dealId} # Refresh matches for deal
GET  /api/v1/matching/compatibility/{partyA}/{partyB} # Check compatibility

```

gRPC:

```

service MatchingService {
  rpc FindMatches(MatchRequest) returns (MatchResultList);
  rpc GetCompatibilityScore(CompatibilityRequest) returns (ScoreResult);
  rpc ComputeWinWinWinScore(WinWinWinRequest) returns (ScoreResult);
  rpc GetRecommendations(RecommendationRequest) returns (RecommendationList);
}

```

Events Published:

- matching.completed
- matching.match.accepted
- matching.match.rejected
- matching.recommendation.generated

Events Consumed:

- deal.published (triggers matching)
- deal.negotiation.started
- party.created (updates candidate pool)
- party.updated (updates candidate pool)

5.2.5 Negotiation Service

Responsibility: Multi-round proposal exchange, proposal versioning, counter-proposals, negotiation timeline management

Aspect	Details
Service ID	negotiation-svc
Team Ownership	Core Platform
Deployment	2+ instances
Data Ownership	Proposals, proposal_versions, counter_proposals, negotiation_sessions, comments

Key Concepts: - **Negotiation Session:** Container for all proposals and counter-proposals for a deal - **Proposal:** A set of terms offered by one party, versioned - **Counter-Proposal:** Response to a proposal with modifications - **Proposal Thread:** Chain of proposals/counter-proposals on a specific topic

Data Model:

```
NegotiationSession {
  session_id: UUID (PK)
  deal_id: UUID (FK)
  status: Enum [ACTIVE, PAUSED, RESOLVED, EXPIRED, DEADLOCKED]
  started_at: Timestamp
  deadline: Timestamp (nullable)
  max_rounds: Int (default: 10)
  current_round: Int
  resolved_at: Timestamp (nullable)
}

Proposal {
  proposal_id: UUID (PK)
  session_id: UUID (FK)
  deal_id: UUID (FK)
  proposed_by: UUID (Party FK)
  parent_proposal_id: UUID (FK, nullable - for counter-proposals)
  version: Int
  proposal_type: Enum [INITIAL, COUNTER, FINAL, WITHDRAWAL]
  status: Enum [DRAFT, SUBMITTED, UNDER_REVIEW, ACCEPTED, REJECTED,
SUPERSEDED, WITHDRAWN]
  terms: JSON {
    value_distribution: {...},
    milestones: [...],
    deliverables: [...],
    timelines: {...},
    conditions: [...]
  }
  proposed_at: Timestamp
  response_deadline: Timestamp (nullable)
  responded_at: Timestamp (nullable)
}

ProposalComment {
  comment_id: UUID (PK)
  proposal_id: UUID (FK)
  author_party_id: UUID (FK)
  content: Text
  is_private: Boolean (visible to all or only own party)
  created_at: Timestamp
}

NegotiationVote {
  vote_id: UUID (PK)
  proposal_id: UUID (FK)
  party_id: UUID (FK)
```

```

vote: Enum [ACCEPT, REJECT, ABSTAIN]
conditions: JSON (nullable)
voted_at: Timestamp
}

```

APIs:

REST:

```

POST /api/v1/deals/{dealId}/negotiations          # Start negotiation
GET  /api/v1/deals/{dealId}/negotiations/{sessionId} # Get session
POST /api/v1/deals/{dealId}/negotiations/{sessionId}/proposals # Submit proposal
GET  /api/v1/deals/{dealId}/negotiations/{sessionId}/proposals # List proposals
GET  /api/v1/deals/{dealId}/negotiations/{sessionId}/proposals/{proposalId}
POST /api/v1/deals/{dealId}/negotiations/{sessionId}/proposals/{proposalId}/counter
POST /api/v1/deals/{dealId}/negotiations/{sessionId}/proposals/{proposalId}/accept
POST /api/v1/deals/{dealId}/negotiations/{sessionId}/proposals/{proposalId}/reject
POST /api/v1/deals/{dealId}/negotiations/{sessionId}/proposals/{proposalId}/comments
POST /api/v1/deals/{dealId}/negotiations/{sessionId}/vote
POST /api/v1/deals/{dealId}/negotiations/{sessionId}/pause
POST /api/v1/deals/{dealId}/negotiations/{sessionId}/resume
GET  /api/v1/deals/{dealId}/negotiations/{sessionId}/timeline

```

gRPC:

```

service NegotiationService {
  rpc StartNegotiation(StartRequest) returns (Session);
  rpc SubmitProposal(ProposalRequest) returns (Proposal);
  rpc AcceptProposal(AcceptRequest) returns (Result);
  rpc RejectProposal(RejectRequest) returns (Result);
  rpc GetNegotiationState(SessionRequest) returns (SessionState);
  rpc GetProposalHistory(ProposalRequest) returns (ProposalChain);
}

```

Events Published:

- negotiation.started
- negotiation.proposal.submitted
- negotiation.proposal.countered
- negotiation.proposal.accepted
- negotiation.proposal.rejected
- negotiation.proposal.withdrawn
- negotiation.proposal.superseded
- negotiation.paused
- negotiation.resumed
- negotiation.resolved
- negotiation.deadlocked
- negotiation.expired
- negotiation.comment.added

5.2.6 Resource/Asset Catalog Service

Responsibility: Resource taxonomy, asset registration, attribute schemas, resource availability, resource search

Aspect	Details
Service ID	resource-catalog-svc
Team Ownership	Core Platform
Deployment	2+ instances
Data Ownership	Resources, resource_types, attributes, categories, availability windows

Data Model:

```
Resource {
  resource_id: UUID (PK)
  owner_party_id: UUID (FK to Party Service)
  resource_type_id: UUID (FK)
  title: String
  description: Text
  status: Enum [DRAFT, AVAILABLE, RESERVED, IN_USE, UNAVAILABLE, RETIRED]
  location: JSON {type: "Point", coordinates: [lng, lat]}
  location_description: String
  availability_schedule: JSON (recurring + exceptions)
  attributes: JSON (type-specific, validated against schema)
  media_urls: String[]
  tags: String[]
  metadata: JSON
  created_at: Timestamp
  updated_at: Timestamp
}
```

```
ResourceType {
  type_id: UUID (PK)
  name: String
  category: String
  domain: String (e.g., "agriculture", "transport", "real-estate")
  description: Text
  attribute_schema: JSON (JSON Schema for validation)
  parent_type_id: UUID (FK, nullable - for inheritance)
  is_active: Boolean
}
```

```
ResourceAttributeDefinition {
  attr_id: UUID (PK)
  resource_type_id: UUID (FK)
  attribute_name: String
  display_name: String
  data_type: Enum [STRING, NUMBER, BOOLEAN, DATE, ENUM, JSON, GEO]
  is_required: Boolean
  is_filterable: Boolean
  is_searchable: Boolean
  validation_rules: JSON
  options: JSON (for ENUM type)
}
```

```

AvailabilityWindow {
  window_id: UUID (PK)
  resource_id: UUID (FK)
  start_time: Timestamp
  end_time: Timestamp
  recurrence: String (RRULE format, nullable)
  status: Enum [AVAILABLE, BLOCKED, RESERVED]
  reason: String (nullable)
}

```

APIs:

REST:

```

POST /api/v1/resources
GET /api/v1/resources/{resourceId}
PUT /api/v1/resources/{resourceId}
DELETE /api/v1/resources/{resourceId}
GET /api/v1/resources
POST /api/v1/resources/{resourceId}/availability
GET /api/v1/resources/{resourceId}/availability
GET /api/v1/resource-types
POST /api/v1/resource-types
GET /api/v1/resource-types/{typeId}
GET /api/v1/resource-types/{typeId}/attributes

```

gRPC:

```

service ResourceCatalogService {
  rpc RegisterResource(ResourceRequest) returns (Resource);
  rpc GetResource(ResourceIdRequest) returns (Resource);
  rpc ValidateAttributes(ValidationRequest) returns (ValidationResult);
  rpc CheckAvailability(AvailabilityRequest) returns (AvailabilityResult);
  rpc SearchResources(SearchRequest) returns (ResourceList);
  rpc GetResourcesByParty(PartyIdRequest) returns (ResourceList);
}

```

Events Published:

- resource.registered
- resource.updated
- resource.availability.changed
- resource.retired
- resource.type.created

5.2.7 Value Distribution & Pricing Service

Responsibility: Value modeling, pricing algorithms, win-win validation, cost/benefit analysis, dynamic pricing

Aspect	Details
Service ID	value-distribution-svc
Team Ownership	Data Science / Core Platform
Deployment	2+ instances

Aspect	Details
Data Ownership	Value_models, pricing_rules, win_win_analyses, market_benchmarks

Win-Win Validation Engine:

Inputs:

- Supplier's cost structure and minimum acceptable return
- Consumer's maximum willingness to pay
- Enhancer's cost and desired margin
- External market benchmarks

Validation Checks:

1. Supplier surplus > 0 (receives more than reservation price)
2. Consumer surplus > 0 (pays less than maximum willingness)
3. Enhancer surplus > 0 (profitable participation)
4. Total surplus > 0 (deal creates value)
5. No party is worse off than outside option
6. Incentive compatibility (truthful reporting is dominant strategy)

Output:

- Win-win-win score (0.0 - 1.0)
- Individual surplus estimates
- Risk-adjusted value distribution
- Confidence intervals

Data Model:

```

ValueDistribution {
  distribution_id: UUID (PK)
  deal_id: UUID (FK)
  status: Enum [DRAFT, VALIDATED, REJECTED, REVISED]
  model_type: Enum [FIXED, REVENUE_SHARE, COST_PLUS, HYBRID, CUSTOM]
  allocations: JSON [
    {party_id: UUID, role: String, value_type: String, amount: Decimal, currency: String,
conditions: JSON}
  ]
  win_win_analysis: JSON {
    supplier_surplus: Decimal,
    consumer_surplus: Decimal,
    enhancer_surplus: Decimal,
    total_surplus: Decimal,
    confidence_score: Float,
    risks: [...]
  }
  market_benchmarks_used: UUID[]
  created_at: Timestamp
  validated_at: Timestamp (nullable)
}

```

```

PricingRule {
  rule_id: UUID (PK)
  domain: String
}

```



```

    rule_type: Enum [BASE_PRICE, PREMIUM, DISCOUNT, MINIMUM, MAXIMUM,
DYNAMIC]
    condition: JSON
    formula: String
    priority: Int
    is_active: Boolean
}

```

```

MarketBenchmark {
    benchmark_id: UUID (PK)
    domain: String
    resource_type: String
    metric: String
    value: Decimal
    unit: String
    source: String
    region: String
    recorded_at: Timestamp
}

```

APIs:

REST:

```

POST /api/v1/deals/{dealId}/value-distribution # Create distribution
GET  /api/v1/deals/{dealId}/value-distribution # Get distribution
PUT  /api/v1/deals/{dealId}/value-distribution # Update distribution
POST /api/v1/deals/{dealId}/value-distribution/validate
GET  /api/v1/deals/{dealId}/value-distribution/surplus-analysis
GET  /api/v1/pricing/benchmarks # Get market benchmarks
POST /api/v1/pricing/calculate # Calculate price for resource

```

gRPC:

```

service ValueDistributionService {
    rpc CreateDistribution(DistributionRequest) returns (ValueDistribution);
    rpc ValidateWinWin(ValidationRequest) returns (ValidationResult);
    rpc CalculateSurplus(CalculationRequest) returns (SurplusResult);
    rpc GetMarketBenchmarks(BenchmarkRequest) returns (BenchmarkList);
    rpc SuggestValueModel(SuggestionRequest) returns (ValueModelSuggestion);
}

```

Events Published:

- value_distribution.created
- value_distribution.validated
- value_distribution.rejected
- value_distribution.revised
- pricing.benchmark.updated

5.2.8 Agreement/Contract Service

Responsibility: Contract template management, contract generation, e-signature orchestration, contract versioning

Aspect	Details
Service ID	agreement-svc
Team Ownership	Core Platform / Legal
Deployment	2+ instances
Data Ownership	Agreements, contract_templates, signatures, amendments

Data Model:

```

Agreement {
  agreement_id: UUID (PK)
  deal_id: UUID (FK)
  template_id: UUID (FK, nullable)
  status: Enum [DRAFT, PENDING_SIGNATURES, PARTIALLY_SIGNED,
FULLY_SIGNED, ACTIVE, AMENDED, EXPIRED, TERMINATED]
  document_url: String
  document_hash: String (SHA-256 for integrity)
  generated_content: Text (rendered contract text)
  effective_date: Timestamp (nullable)
  expiry_date: Timestamp (nullable)
  signature_deadline: Timestamp
  terms_snapshot: JSON (snapshot of agreed terms)
  created_at: Timestamp
  updated_at: Timestamp
}

Signature {
  signature_id: UUID (PK)
  agreement_id: UUID (FK)
  party_id: UUID (FK)
  signatory_user_id: UUID (FK to Identity)
  status: Enum [PENDING, SIGNED, DECLINED, EXPIRED]
  signature_method: Enum [E_SIGNATURE, DIGITAL_SIGNATURE, MANUAL]
  provider: String (e.g., "docusign", "hellosign")
  provider_signature_id: String
  signed_at: Timestamp (nullable)
  ip_address: String (nullable)
  geo_location: JSON (nullable)
}

ContractTemplate {
  template_id: UUID (PK)
  name: String
  domain: String
  jurisdiction: String
  language: String
  template_content: Text (with variable placeholders)
  variables: JSON[]
  required_clauses: JSON[]
  is_active: Boolean

```

```
version: Int
}
```

APIs:

REST:

```
POST /api/v1/deals/{dealId}/agreements      # Generate agreement
GET  /api/v1/agreements/{agreementId}
POST /api/v1/agreements/{agreementId}/send  # Send for signature
POST /api/v1/agreements/{agreementId}/sign  # Sign agreement
GET  /api/v1/agreements/{agreementId}/signatures
GET  /api/v1/agreements/{agreementId}/download
POST /api/v1/agreements/{agreementId}/amend # Create amendment
GET  /api/v1/contract-templates
POST /api/v1/contract-templates
```

gRPC:

```
service AgreementService {
  rpc GenerateAgreement(GenerateRequest) returns (Agreement);
  rpc SendForSignature(SendRequest) returns (Result);
  rpc RecordSignature(SignatureRequest) returns (Signature);
  rpc GetSignatureStatus(AgreementIdRequest) returns (SignatureStatus);
}
```

Events Published:

- agreement.created
- agreement.sent_for_signature
- agreement.signed
- agreement.fully_signed
- agreement.declined
- agreement.amended
- agreement.terminated

5.2.9 Execution & Milestone Service

Responsibility: Milestone definition, progress tracking, deliverable verification, execution timeline management

Aspect	Details
Service ID	execution-svc
Team Ownership	Core Platform
Deployment	2+ instances, background workers
Data Ownership	Milestones, deliverables, progress_updates, verification_records

Data Model:

```
Milestone {
  milestone_id: UUID (PK)
  deal_id: UUID (FK)
  title: String
}
```

```

description: Text
sequence_order: Int
responsible_party_id: UUID (FK, nullable)
status: Enum [PENDING, IN_PROGRESS, COMPLETED, OVERDUE, BLOCKED,
SKIPPED]
due_date: Timestamp (nullable)
completed_at: Timestamp (nullable)
completion_criteria: JSON
verification_method: Enum [MANUAL, AUTOMATIC, THIRD_PARTY, UPLOAD]
release_trigger: JSON (payment/release conditions)
dependencies: UUID[] (other milestone IDs)
metadata: JSON
}

Deliverable {
deliverable_id: UUID (PK)
milestone_id: UUID (FK)
title: String
description: Text
deliverable_type: Enum [DOCUMENT, MEDIA, PHYSICAL, SERVICE, CONFIRMATION]
status: Enum [PENDING, SUBMITTED, UNDER_REVIEW, APPROVED, REJECTED]
submitted_by: UUID (Party FK)
submitted_at: Timestamp (nullable)
content_url: String (nullable)
content_hash: String (nullable)
reviewed_by: UUID (User FK, nullable)
reviewed_at: Timestamp (nullable)
review_notes: Text (nullable)
}

ProgressUpdate {
update_id: UUID (PK)
milestone_id: UUID (FK)
reported_by: UUID (User FK)
progress_percentage: Int (0-100)
notes: Text
evidence_urls: String[]
reported_at: Timestamp
}

```

APIs:

REST:

```

POST /api/v1/deals/{dealId}/milestones
GET /api/v1/deals/{dealId}/milestones
GET /api/v1/milestones/{milestoneId}
PUT /api/v1/milestones/{milestoneId}
POST /api/v1/milestones/{milestoneId}/start
POST /api/v1/milestones/{milestoneId}/complete
POST /api/v1/milestones/{milestoneId}/deliverables
GET /api/v1/milestones/{milestoneId}/deliverables
POST /api/v1/milestones/{milestoneId}/deliverables/{deliverableId}/submit
POST /api/v1/milestones/{milestoneId}/deliverables/{deliverableId}/approve

```

```
POST /api/v1/milestones/{milestoneId}/deliverables/{deliverableId}/reject
POST /api/v1/milestones/{milestoneId}/progress
GET /api/v1/deals/{dealId}/progress
```

gRPC:

```
service ExecutionService {
  rpc CreateMilestone(MilestoneRequest) returns (Milestone);
  rpc CompleteMilestone(CompleteRequest) returns (Milestone);
  rpc SubmitDeliverable(DeliverableRequest) returns (Deliverable);
  rpc ApproveDeliverable(ApprovalRequest) returns (Deliverable);
  rpc GetDealProgress(DealIdRequest) returns (ProgressReport);
}
```

Events Published:

- milestone.created
- milestone.started
- milestone.completed
- milestone.overdue
- milestone.blocked
- deliverable.submitted
- deliverable.approved
- deliverable.rejected
- progress.updated

5.2.10 Payment/Settlement Service

Responsibility: Payment orchestration, escrow management, milestone-based releases, settlement, refund handling

Aspect	Details
Service ID	payment-svc
Team Ownership	Payments / Core Platform
Deployment	3+ instances, critical path
Data Ownership	Payment_orders, escrow_accounts, transactions, settlements, refunds, payment_methods

Payment Flow:

1. Deal agreed -> Payment plan created
2. Consumer funds escrow -> Funds held in escrow
3. Milestone completed -> Release condition triggered
4. Value Distribution Service calculates release amount
5. Escrow release executed -> Funds distributed to Supplier & Enhancer
6. Final milestone -> Remaining funds released
7. Settlement record created -> Deal settled

Data Model:

```
PaymentPlan {
  plan_id: UUID (PK)
```

```

    deal_id: UUID (FK)
    status: Enum [DRAFT, PENDING_FUNDING, FUNDED, EXECUTING, COMPLETED,
CANCELLED]
    total_amount: Decimal
    currency: String (ISO 4217)
    escrow_account_id: String
    consumer_party_id: UUID (FK)
    installments: JSON [
    {
        milestone_id: UUID,
        amount: Decimal,
        recipient_party_id: UUID,
        release_condition: String,
        status: Enum [PENDING, RELEASED, FAILED]
    }
    ]
    created_at: Timestamp
    updated_at: Timestamp
}

```

```

Transaction {
    transaction_id: UUID (PK)
    plan_id: UUID (FK)
    transaction_type: Enum [DEPOSIT, ESCROW_HOLD, RELEASE, REFUND, FEE,
ADJUSTMENT]
    amount: Decimal
    currency: String
    from_party_id: UUID (FK, nullable)
    to_party_id: UUID (FK, nullable)
    status: Enum [PENDING, PROCESSING, COMPLETED, FAILED, REVERSED]
    provider: String (e.g., "stripe", "flutterwave")
    provider_transaction_id: String
    metadata: JSON
    created_at: Timestamp
    completed_at: Timestamp (nullable)
}

```

```

Settlement {
    settlement_id: UUID (PK)
    deal_id: UUID (FK)
    status: Enum [PENDING, PROCESSING, SETTLED, FAILED]
    total_settled: Decimal
    breakdown: JSON [
        {party_id: UUID, role: String, amount: Decimal, method: String}
    ]
    settled_at: Timestamp (nullable)
}

```

```

PaymentMethod {
    method_id: UUID (PK)
    party_id: UUID (FK)
    method_type: Enum [BANK_TRANSFER, MOBILE_MONEY, CARD, CRYPTO, WALLET]
}

```

```

provider: String
account_details: JSON (encrypted)
is_default: Boolean
is_verified: Boolean
created_at: Timestamp
}

```

APIs:

REST:

```

POST /api/v1/deals/{dealId}/payment-plans      # Create payment plan
GET  /api/v1/deals/{dealId}/payment-plans      # Get payment plan
POST /api/v1/deals/{dealId}/fund               # Fund escrow
POST /api/v1/deals/{dealId}/release            # Manual release trigger
GET  /api/v1/transactions                       # List transactions
GET  /api/v1/transactions/{transactionId}
POST /api/v1/refunds                            # Request refund
GET  /api/v1/settlements/{settlementId}
POST /api/v1/parties/{partyId}/payment-methods
GET  /api/v1/parties/{partyId}/payment-methods

```

gRPC:

```

service PaymentService {
  rpc CreatePaymentPlan(PlanRequest) returns (PaymentPlan);
  rpc FundEscrow(FundRequest) returns (Transaction);
  rpc ReleaseEscrow(ReleaseRequest) returns (Transaction);
  rpc ProcessRefund(RefundRequest) returns (Transaction);
  rpc GetTransactionStatus(TransactionIdRequest) returns (TransactionStatus);
}

```

Events Published:

- payment.plan.created
- payment.escrow.funded
- payment.escrow.released
- payment.transaction.completed
- payment.transaction.failed
- payment.refund.processed
- payment.settlement.completed

5.2.11 Notification Service

Responsibility: Multi-channel notifications, preference management, templating, delivery tracking, rate limiting

Aspect	Details
Service ID	notification-svc
Team Ownership	Platform
Deployment	2+ instances, background workers
Data Ownership	Notifications, templates, delivery_records, preferences

Data Model:

```
Notification {
  notification_id: UUID (PK)
  recipient_user_id: UUID (FK)
  notification_type: Enum [DEAL_UPDATE, NEGOTIATION, MILESTONE, PAYMENT,
SYSTEM, MARKETING]
  channels: Enum[] [IN_APP, EMAIL, SMS, PUSH, WEBHOOK]
  subject: String
  content: Text
  template_id: UUID (FK, nullable)
  template_data: JSON
  priority: Enum [LOW, NORMAL, HIGH, URGENT]
  status: Enum [PENDING, SENT, DELIVERED, READ, FAILED]
  sent_at: Timestamp (nullable)
  read_at: Timestamp (nullable)
  related_entity_type: String (nullable)
  related_entity_id: UUID (nullable)
  created_at: Timestamp
}
```

```
NotificationTemplate {
  template_id: UUID (PK)
  name: String
  notification_type: String
  channels: JSON {email: {...}, sms: {...}, push: {...}, in_app: {...}}
  subject_template: String
  content_template: String (with variable placeholders)
  variables_schema: JSON
  locale: String
  is_active: Boolean
}
```

```
NotificationPreference {
  preference_id: UUID (PK)
  user_id: UUID (FK)
  notification_type: String
  channel_preferences: JSON {email: Boolean, sms: Boolean, push: Boolean, in_app:
Boolean}
  quiet_hours: JSON {enabled: Boolean, start: "22:00", end: "08:00", timezone: String}
  is_muted: Boolean
}
```

APIs:

REST:

```
GET    /api/v1/notifications          # List notifications
GET    /api/v1/notifications/{notificationId}
POST   /api/v1/notifications/{notificationId}/read # Mark as read
POST   /api/v1/notifications/read-all          # Mark all as read
GET    /api/v1/notification-preferences
PUT    /api/v1/notification-preferences
POST   /api/v1/admin/notifications/send        # Admin: send notification
```



```
GET /api/v1/admin/notification-templates
POST /api/v1/admin/notification-templates
```

gRPC:

```
service NotificationService {
  rpc SendNotification(SendRequest) returns (Result);
  rpc SendBulkNotification(BulkRequest) returns (Result);
  rpc GetUnreadCount(UserIdRequest) returns (CountResult);
  rpc UpdatePreferences(PreferenceRequest) returns (Result);
}
```

Events Consumed:

- deal.* (all deal events)
- negotiation.* (all negotiation events)
- milestone.* (all milestone events)
- payment.* (all payment events)
- user.* (user lifecycle events)
- trust.review.received

5.2.12 Trust & Reputation Service

Responsibility: Rating calculation, review management, reputation scoring, trust badges, fraud detection signals

Aspect	Details
Service ID	trust-reputation-svc
Team Ownership	Platform / Trust & Safety
Deployment	2+ instances
Data Ownership	Reviews, ratings, reputation_scores, trust_badges, fraud_signals

Reputation Score Calculation:

```
Composite Score = weighted_sum(
  - Deal completion rate: 25%
  - Average rating: 20%
  - Review sentiment: 15%
  - Response timeliness: 10%
  - Verification level: 10%
  - Platform tenure: 5%
  - Dispute history: 5%
  - Community contributions: 5%
  - Recency weighting: (reviews in last 90 days get 2x weight)
)
```

Score ranges:

- 0.0 - 0.3: New / Low reputation
- 0.3 - 0.6: Building reputation
- 0.6 - 0.8: Good reputation
- 0.8 - 0.95: Excellent reputation
- 0.95 - 1.0: Top-tier reputation

Data Model:

```
Review {
  review_id: UUID (PK)
  deal_id: UUID (FK)
  reviewer_party_id: UUID (FK)
  reviewed_party_id: UUID (FK)
  role_context: Enum [AS_SUPPLIER, AS_CONSUMER, AS_ENHANCER]
  overall_rating: Int (1-5)
  category_ratings: JSON {
    communication: Int,
    reliability: Int,
    quality: Int,
    fairness: Int,
    timeliness: Int
  }
  review_text: Text (nullable)
  is_anonymous: Boolean
  is_verified: Boolean (verified by platform)
  helpful_count: Int
  status: Enum [PUBLISHED, HIDDEN, UNDER_REVIEW, REMOVED]
  created_at: Timestamp
  updated_at: Timestamp
}
```

```
ReputationScore {
  score_id: UUID (PK)
  party_id: UUID (FK)
  overall_score: Float (0.0 - 1.0)
  deal_completion_rate: Float
  average_rating: Float
  total_reviews: Int
  positive_reviews: Int
  neutral_reviews: Int
  negative_reviews: Int
  detailed_scores: JSON
  score_history: JSON[]
  trust_badges: String[]
  computed_at: Timestamp
}
```

```
TrustBadge {
  badge_id: UUID (PK)
  name: String
  description: String
  icon_url: String
  criteria: JSON
  is_active: Boolean
}
```

APIs:

REST:

```
POST /api/v1/deals/{dealId}/reviews      # Submit review
GET  /api/v1/reviews/{reviewId}
GET  /api/v1/parties/{partyId}/reviews   # Get party reviews
GET  /api/v1/parties/{partyId}/reputation # Get reputation score
GET  /api/v1/parties/{partyId}/trust-badges # Get trust badges
POST /api/v1/reviews/{reviewId}/helpful  # Mark as helpful
GET  /api/v1/reviews                     # List reviews (admin)
```

gRPC:

```
service TrustReputationService {
  rpc SubmitReview(ReviewRequest) returns (Review);
  rpc GetReputationScore(PartyIdRequest) returns (Reputation);
  rpc GetTrustIndicators(PartyIdRequest) returns (TrustIndicators);
  rpc CalculateScore(PartyIdRequest) returns (ScoreResult);
}
```

Events Published:

- trust.review.submitted
- trust.review.published
- trust.score.updated
- trust.badge.awarded
- trust.badge.revoked

5.2.13 Analytics & Reporting Service

Responsibility: Metrics collection, dashboards, reports, data pipelines, business intelligence, predictive analytics

Aspect	Details
Service ID	analytics-svc
Team Ownership	Data / Platform
Deployment	2+ instances, background workers
Data Ownership	Metrics, reports, dashboards, data_snapshots, ml_features

Data Model:

```
MetricDefinition {
  metric_id: UUID (PK)
  name: String
  description: Text
  category: Enum [DEAL, USER, PLATFORM, FINANCIAL, OPERATIONAL]
  calculation_method: Enum [COUNT, SUM, AVERAGE, RATIO, CUSTOM]
  query_template: String
  dimensions: String[]
  granularity: Enum [REALTIME, HOURLY, DAILY, WEEKLY, MONTHLY]
}
```

```
Report {
  report_id: UUID (PK)
```

```

name: String
report_type: Enum [DEAL_SUMMARY, FINANCIAL, USER_ACTIVITY,
PLATFORM_HEALTH, CUSTOM]
parameters: JSON
scheduled: Boolean
schedule_cron: String (nullable)
last_generated_at: Timestamp (nullable)
created_by: UUID
}

DataSnapshot {
snapshot_id: UUID (PK)
snapshot_type: String
data: JSON
period_start: Timestamp
period_end: Timestamp
created_at: Timestamp
}

```

APIs:

REST:

```

GET    /api/v1/analytics/metrics          # List available metrics
GET    /api/v1/analytics/metrics/{metricId}  # Get metric data
POST   /api/v1/analytics/reports          # Create report
GET    /api/v1/analytics/reports/{reportId}  # Get report
GET    /api/v1/analytics/reports/{reportId}/download
GET    /api/v1/analytics/dashboard          # Get dashboard data
GET    /api/v1/analytics/platform-health    # Platform health metrics
GET    /api/v1/analytics/deals/summary      # Deal analytics

```

gRPC:

```

service AnalyticsService {
  rpc GetMetricData(MetricRequest) returns (MetricData);
  rpc GenerateReport(ReportRequest) returns (Report);
  rpc GetDashboardData(DashboardRequest) returns (DashboardData);
  rpc TrackEvent(TrackingEvent) returns (Result);
}

```

Events Consumed:

- All domain events (for metrics and analytics)

5.2.14 Search Service

Responsibility: Full-text search, faceted search, geo-search, search ranking, search analytics

Aspect	Details
Service ID	search-svc
Team Ownership	Platform
Deployment	2+ instances
Data Ownership	Search indices, query logs, search_analytics

Aspect	Details
	(derived from read models)

Search Domains: - **Deals:** Search public deals by keywords, domain, location, value range - **Resources:** Search available resources by type, location, attributes - **Parties:** Search parties by capabilities, domain, reputation - **Templates:** Search deal templates

APIs:

REST:

```
GET /api/v1/search/deals?q={query}&domain={d}&location={loc}&value_min={min}
&value_max={max}
GET /api/v1/search/resources?q={query}&type={type}&location={loc}
&available_from={date}
GET /api/v1/search/parties?q={query}&role={role}&domain={d}
&min_reputation={score}
GET /api/v1/search/templates?q={query}&domain={d}
GET /api/v1/search/suggestions?q={partial} # Autocomplete
GET /api/v1/search/filters/{domain} # Available filters
GET /api/v1/search/popular # Trending searches
```

gRPC:

```
service SearchService {
  rpc SearchDeals(SearchRequest) returns (SearchResult);
  rpc SearchResources(SearchRequest) returns (SearchResult);
  rpc SearchParties(SearchRequest) returns (SearchResult);
  rpc GetSuggestions(SuggestionRequest) returns (SuggestionList);
}
```

Events Consumed:

- deal.created (index deal)
- deal.updated (update index)
- deal.published (make searchable)
- deal.closed (remove from search)
- resource.registered (index resource)
- party.created (index party)
- party.verified (boost ranking)
- trust.score.updated (update ranking)

5.2.15 Media/Document Service

Responsibility: File storage, document management, image processing, content delivery, virus scanning

Aspect	Details
Service ID	media-svc
Team Ownership	Platform
Deployment	2+ instances
Data Ownership	Media metadata, document metadata, access_logs (files stored in object storage)

Data Model:

```
MediaItem {
  media_id: UUID (PK)
  owner_party_id: UUID (FK)
  uploaded_by: UUID (User FK)
  media_type: Enum [IMAGE, VIDEO, DOCUMENT, AUDIO, ARCHIVE]
  mime_type: String
  original_filename: String
  storage_path: String
  storage_provider: String (e.g., "s3", "gcs")
  size_bytes: Long
  checksum: String (SHA-256)
  status: Enum [UPLOADING, PROCESSING, READY, FAILED, QUARANTINED]
  metadata: JSON {width, height, duration, pages, etc.}
  access_control: Enum [PUBLIC, RESTRICTED, PRIVATE]
  allowed_parties: UUID[] (for RESTRICTED)
  created_at: Timestamp
}

Document {
  document_id: UUID (PK)
  media_id: UUID (FK)
  document_type: Enum [CONTRACT, INVOICE, RECEIPT, REPORT, KYC, OTHER]
  related_entity_type: String (e.g., "deal", "party")
  related_entity_id: UUID
  extracted_text: Text (OCR result, nullable)
  signatures: JSON[] (nullable)
  version: Int
}
```

APIs:

```
REST:
POST /api/v1/media/upload          # Upload file (multipart)
GET  /api/v1/media/{mediaId}      # Get media metadata
GET  /api/v1/media/{mediaId}/download # Download file
GET  /api/v1/media/{mediaId}/preview # Get preview/thumbnail
DELETE /api/v1/media/{mediaId}
POST /api/v1/media/{mediaId}/share # Share with parties
GET  /api/v1/documents/{documentId}
POST /api/v1/documents/{documentId}/ocr # Trigger OCR
GET  /api/v1/documents/{documentId}/text # Get extracted text
```

5.2.16 Admin & Moderation Service

Responsibility: Admin operations, content moderation, dispute resolution, user management, platform configuration

Aspect	Details
Service ID	admin-svc
Team Ownership	Platform / Operations

Aspect	Details
Deployment	2+ instances
Data Ownership	Admin_actions, moderation_decisions, disputes, platform_config, audit_logs

Data Model:

```
AdminAction {
  action_id: UUID (PK)
  admin_user_id: UUID (FK)
  action_type: Enum [USER_SUSPEND, USER_REINSTATE, DEAL_INTERVENE,
CONTENT_REMOVE, CONFIG_CHANGE, MANUAL_REFUND]
  target_entity_type: String
  target_entity_id: UUID
  action_reason: Text
  action_metadata: JSON
  created_at: Timestamp
}
```

```
Dispute {
  dispute_id: UUID (PK)
  deal_id: UUID (FK)
  raised_by: UUID (Party FK)
  against_party: UUID (Party FK)
  dispute_type: Enum [NON_PAYMENT, NON_DELIVERY, QUALITY,
BREACH_OF_TERMS, OTHER]
  description: Text
  evidence_media_ids: UUID[]
  status: Enum [OPEN, UNDER_REVIEW, MEDIATION, RESOLVED, CLOSED,
ESCALATED]
  resolution: JSON (nullable)
  resolved_by: UUID (Admin FK, nullable)
  created_at: Timestamp
  resolved_at: Timestamp (nullable)
}
```

```
PlatformConfig {
  config_key: String (PK)
  config_value: JSON
  config_category: String
  modified_by: UUID
  modified_at: Timestamp
}
```

APIs:

REST:

```
# User Management
GET  /api/v1/admin/users
POST /api/v1/admin/users/{userId}/suspend
POST /api/v1/admin/users/{userId}/reinstate
```

```

# Deal Management
GET  /api/v1/admin/deals
POST /api/v1/admin/deals/{dealId}/intervene
POST /api/v1/admin/deals/{dealId}/cancel

# Moderation
GET  /api/v1/admin/moderation/queue          # Content moderation queue
POST /api/v1/admin/moderation/{itemId}/approve
POST /api/v1/admin/moderation/{itemId}/reject

# Disputes
GET  /api/v1/admin/disputes
GET  /api/v1/admin/disputes/{disputeId}
POST /api/v1/admin/disputes/{disputeId}/mediate
POST /api/v1/admin/disputes/{disputeId}/resolve

# Platform Config
GET  /api/v1/admin/config
PUT  /api/v1/admin/config/{key}

# Audit
GET  /api/v1/admin/audit-log
GET  /api/v1/admin/actions
GET  /api/v1/admin/platform-health

```

5.3 Service Interaction Matrix

	U&I	P&P	DM	Mat	Neg	RC	VD	Agr	Exe	Pay	Not	Trst	Anl	Srch	Med	Adm
User & Identity	-	S	S	-	-	S	-	S	-	S	S	S	-	-	S	S
Party & Profile	C	-	S	S	S	S	S	S	S	S	S	S	-	S	S	S
Deal Management	-	C	-	C	C	C	C	C	C	C	P	C	P	P	-	-
Matching & Discovery	-	C	C	-	-	C	C	-	-	-	-	C	-	-	-	-
Negotiation	-	C	C	-	-	-	C	C	-	-	P	-	-	-	-	C
Resource Catalog	-	C	C	C	-	-	-	-	-	-	P	-	-	P	S	C
Value Distribution	-	C	C	-	C	-	-	C	-	C	P	-	-	-	-	-
Agreement/Contract	C	C	C	-	C	-	C	-	-	-	P	-	-	-	-	S
Execution & Milestone	-	C	C	-	-	-	-	-	-	C	P	-	-	-	-	S
Payment/Settlement	-	C	C	-	-	-	C	-	C	-	P	-	-	-	-	C
Notification	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
Trust & Reputation	-	C	C	C	-	-	-	-	-	-	P	-	-	-	-	C
Analytics	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
Search	-	C	C	-	-	C	-	-	-	-	C	-	-	-	-	-
Media/Document	-	-	-	-	-	-	-	S	S	-	-	-	-	-	-	S
Admin & Moderation	C	C	C	-	C	C	-	C	C	C	P	C	-	-	-	C

Legend: S = Synchronous call, C = Consumes events from, P = Publishes events to

5. Communication Patterns

5.1 Communication Strategy Overview

Pattern	Use Case	Implementation
Sync REST	External client-facing APIs, CRUD operations	HTTP/1.1 + HTTP/2, JSON
Sync gRPC	Inter-service communication, high-throughput internal APIs	HTTP/2, Protocol Buffers
Async Events	State changes, cross-service notifications, decoupled operations	Apache Kafka
Async CQRS	Read model updates, search indexing, analytics	Kafka + Materialized Views
Async Saga	Distributed transactions (deal creation, settlement)	Saga Orchestrator + Kafka

5.2 Synchronous Communication

REST APIs (External-Facing)

Standards:

- **OpenAPI 3.1** specification for all APIs
- **JSON:API** or standard REST conventions
- **HTTP status codes:** 200, 201, 204, 400, 401, 403, 404, 409, 422, 429, 500
- **Pagination:** cursor-based for large collections
- **Versioning:** URL path (/api/v1/, /api/v2/)
- **Rate limiting:** 100 req/min anonymous, 1000 req/min authenticated
- **Request ID:** X-Request-ID header for tracing
- **Content negotiation:** Accept/Content-Type headers
- **CORS:** Configured per environment
- **HATEOAS:** Optional, for discoverability

gRPC APIs (Internal Services)

// Internal service communication schema

```
syntax = "proto3";
```

```
package dealplatform.common;
```

// Shared types

```
message UUID {  
    string value = 1;  
}
```

```
message Timestamp {  
    int64 seconds = 1;  
    int32 nanos = 2;  
}
```

```

message Money {
  string currency_code = 1;
  int64 units = 2;
  int32 nanos = 3;
}

message PaginationRequest {
  int32 page_size = 1;
  string page_token = 2;
}

message PaginationResponse {
  string next_page_token = 1;
  int32 total_count = 2;
}

// Error handling
message ServiceError {
  string code = 1;
  string message = 2;
  map<string, string> details = 3;
}

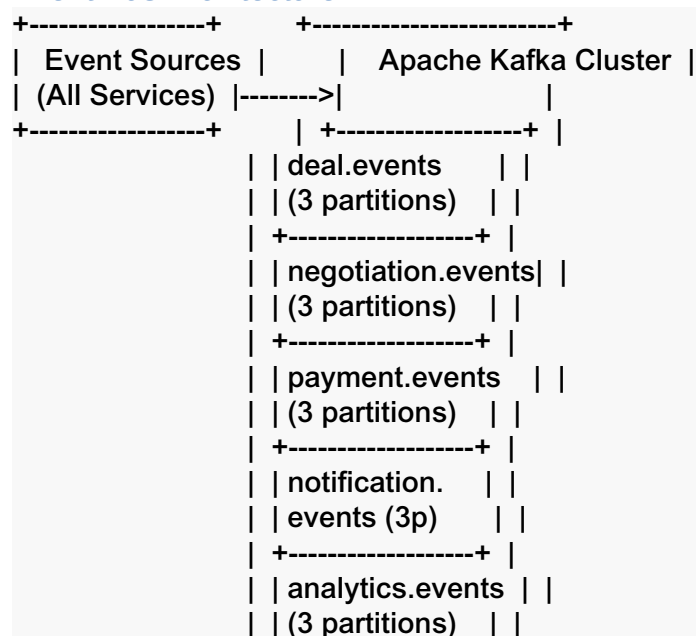
```

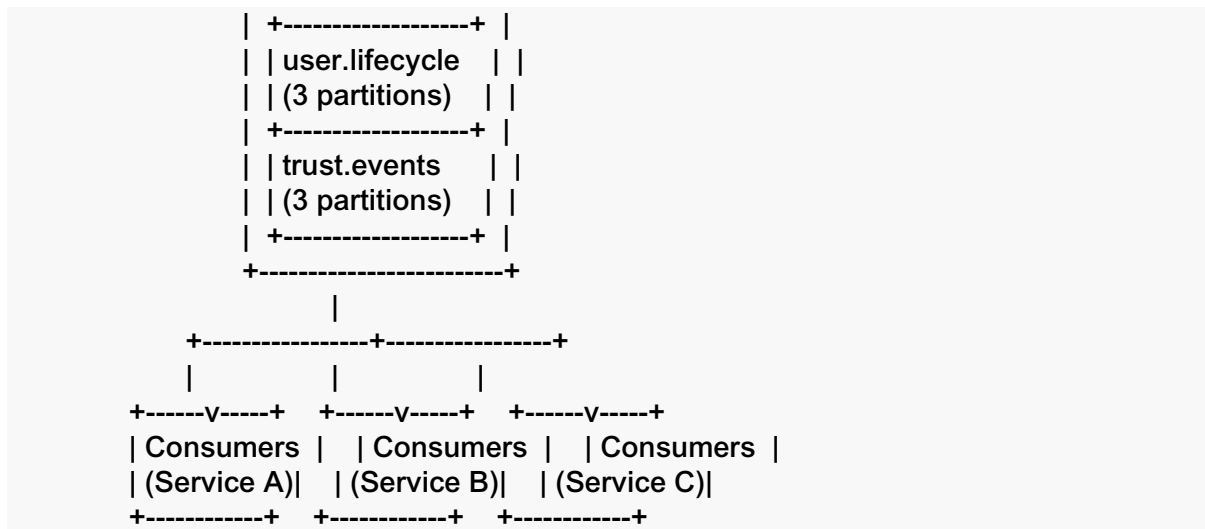
gRPC Service Definitions per domain: - dealplatform.identity.v1 - Identity operations - dealplatform.party.v1 - Party management - dealplatform.deal.v1 - Deal lifecycle - dealplatform.payment.v1 - Payment operations - dealplatform.matching.v1 - Matching & recommendations

Service Mesh Integration: - All gRPC traffic routed through service mesh (Istio/Linkerd) - Automatic mTLS between services - Circuit breaking at the mesh level - Distributed tracing headers propagated

5.3 Asynchronous Communication

Event Bus Architecture





Topic Design

Topic	Partitions	Retention	Description
deal.lifecycle.events	12 (by deal_id)	30 days	All deal state changes
negotiation.events	6 (by session_id)	30 days	Proposals, counter-proposals
payment.transaction.events	6 (by plan_id)	90 days	Payment lifecycle
user.lifecycle.events	3	Forever	Registration, KYC, status
notification.triggers.events	6	7 days	Notification triggers
trust.reputation.events	3	Forever	Reviews, scores
analytics.raw.events	6	90 days	All events for analytics
resource.catalog.events	3	30 days	Resource CRUD operations
milestone.execution.events	6 (by deal_id)	90 days	Milestone progress
matching.results.events	3	7 days	Match recommendations
platform.audit.events	6	Forever	Audit trail
dead-letter.events	1	30 days	Failed message processing

Event Schema Standards

```

{
  "$schema": "https://schemas.dealplatform.io/events/v1/event",

```

```

"event_id": "evt_a1b2c3d4e5f6",
"event_type": "deal.state.changed",
"event_version": "1.0",
"timestamp": "2024-01-15T10:30:00.000Z",
"source": {
  "service": "deal-mgmt-svc",
  "instance": "deal-mgmt-7b8c9d5e-f4g5h",
  "version": "2.3.1"
},
"correlation_id": "corr_x1y2z3a4b5c6",
"causation_id": "evt_prev_event_id",
"payload": {
  "deal_id": "dl_abc123",
  "from_state": "NEGOTIATING",
  "to_state": "VALIDATED",
  "triggered_by": "usr_xyz789",
  "reason": "All parties agreed on terms",
  "metadata": {}
},
"metadata": {
  "trace_id": "trace_123abc",
  "span_id": "span_456def",
  "region": "us-east-1",
  "environment": "production"
}
}

```

Event Catalog

Deal Lifecycle Events: | Event Name | Producer | Consumers | Description | |-----|

deal.created	Deal Mgmt	Search, Analytics, Notification	New deal drafted
deal.published	Deal Mgmt	Matching, Search, Notification	Deal made public
deal.state.changed	Deal Mgmt	All interested services	Generic state transition
deal.parties.matched	Matching	Deal Mgmt, Notification	Compatible parties found
deal.negotiation.started	Deal Mgmt	Negotiation, Notification	Negotiation begins
deal.terms.validated	Negotiation	Deal Mgmt, Value Dist	Terms validated
deal.agreed	Negotiation	Deal Mgmt, Agreement, Payment	All parties agreed
deal.signed	Agreement	Deal Mgmt, Execution, Payment	Contract signed
deal.execution.started	Deal Mgmt	Execution, Payment, Notification	Execution begins
deal.milestone.reached	Execution	Deal Mgmt, Payment, Notification	Milestone completed
deal.completed	Execution	Deal Mgmt, Payment, Notification	All milestones done
deal.settled	Payment	Deal Mgmt, Trust, Notification	Financial settlement
deal.closed	Deal Mgmt	Analytics, Search, Trust	Deal fully closed
deal.cancelled	Deal Mgmt	All services	Deal cancelled
deal.disputed	Execution	Admin, Trust, Notification	Dispute raised

Negotiation Events: | Event Name | Producer | Consumers | Description | |-----|

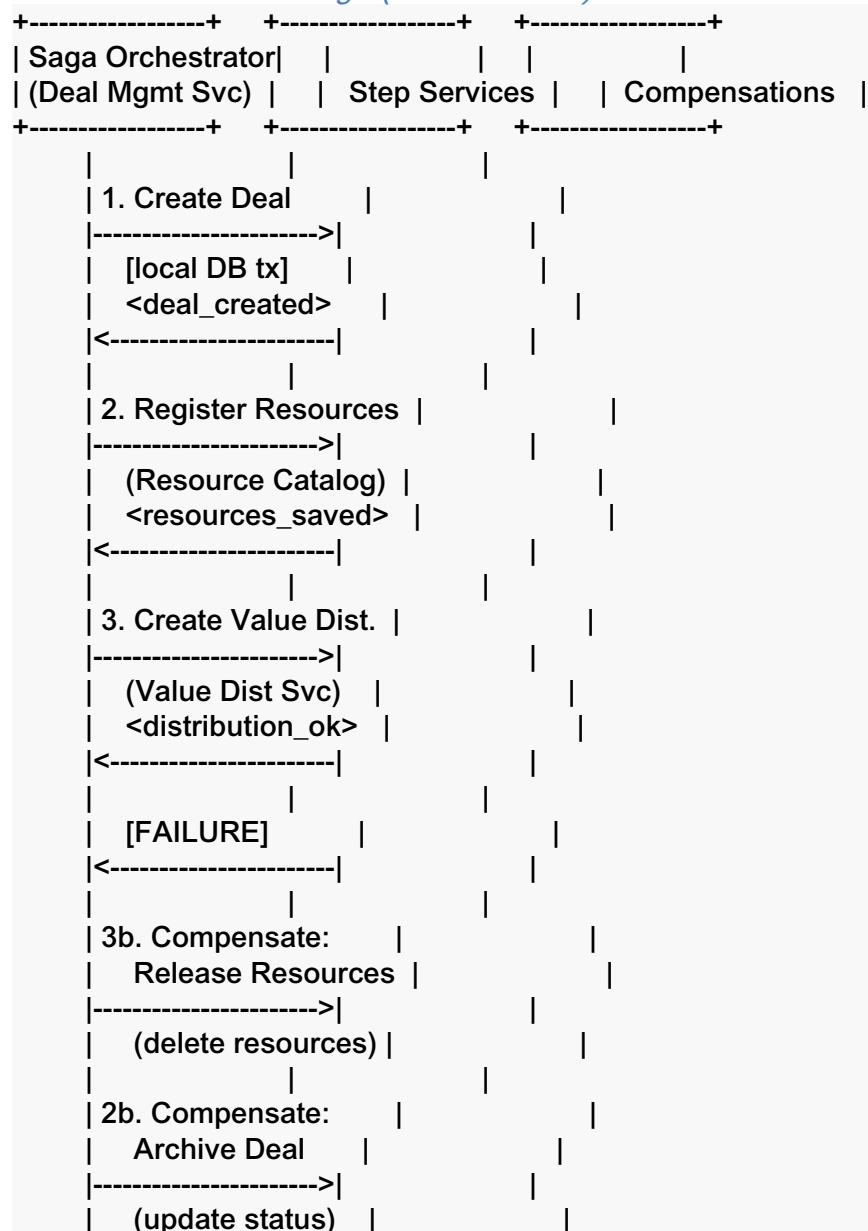
negotiation.started	Negotiation	Deal Mgmt, Notification	Session opened
negotiation.proposal.submitted	Negotiation	Deal Mgmt, Notification	New proposal
negotiation.proposal.accepted	Negotiation	Deal Mgmt, Value Dist	Proposal accepted
negotiation.resolved	Negotiation	Deal Mgmt	All terms agreed

Payment Events:	Event Name	Producer	Consumers	Description
	payment.plan.created	Payment	Deal Mgmt	Payment plan ready
	payment.escrow.funded	Payment	Deal Mgmt, Execution	Consumer funded escrow
	payment.escrow.released	Payment	Deal Mgmt, Trust	Funds released
	payment.transaction.completed	Payment	Deal Mgmt, Analytics	Transaction done
	payment.transaction.failed	Payment	Deal Mgmt, Notification	Transaction failed
	payment.settlement.completed	Payment	Deal Mgmt, Trust	Final settlement

Trust & Reputation Events:	Event Name	Producer	Consumers	Description
	trust.review.submitted	Trust	Analytics, Search, Notification	New review
	trust.score.updated	Trust	Search, Matching, Notification	Score changed

5.4 Saga Patterns

5.4.1 Deal Creation Saga (Orchestration)



Saga Definition (YAML):

```
saga:
  name: deal_creation
  version: "1.0"

  steps:
    - name: create_deal
      service: deal-mgmt-svc
      action: CreateDeal
      compensation: ArchiveDeal

    - name: register_resources
      service: resource-catalog-svc
      action: RegisterResources
      compensation: DeregisterResources
      depends_on: [create_deal]

    - name: create_value_distribution
      service: value-distribution-svc
      action: CreateDistribution
      compensation: DeleteDistribution
      depends_on: [create_deal]

    - name: validate_win_win
      service: value-distribution-svc
      action: ValidateWinWin
      compensation: null # no-op, validation is read-only
      depends_on: [create_value_distribution]

    - name: initialize_negotiation
      service: negotiation-svc
      action: StartNegotiation
      compensation: EndNegotiation
      depends_on: [create_value_distribution]

  completion:
    publish_event: deal.creation.completed

  failure_policy:
    max_retries: 3
    retry_delay: "5s"
    compensation_order: REVERSE
    timeout: "30s"
```

5.4.2 Deal Settlement Saga (Orchestration)

```
saga:
  name: deal_settlement
  version: "1.0"

  steps:
    - name: finalize_milestones
      service: execution-svc
      action: FinalizeAllMilestones
      compensation: null
```

- name: **calculate_final_settlement**
 service: **value-distribution-svc**
 action: **CalculateSettlement**
 compensation: **null**
 depends_on: **[finalize_milestones]**

- name: **release_escrow**
 service: **payment-svc**
 action: **ReleaseEscrow**
 compensation: **HoldEscrow** *# Reverse the release*
 depends_on: **[calculate_final_settlement]**

- name: **process_settlement**
 service: **payment-svc**
 action: **ProcessSettlement**
 compensation: **ReverseSettlement**
 depends_on: **[release_escrow]**

- name: **update_reputation**
 service: **trust-reputation-svc**
 action: **RecordDealCompletion**
 compensation: **null**
 depends_on: **[process_settlement]**

- name: **close_deal**
 service: **deal-mgmt-svc**
 action: **CloseDeal**
 compensation: **ReopenDeal**
 depends_on: **[update_reputation]**

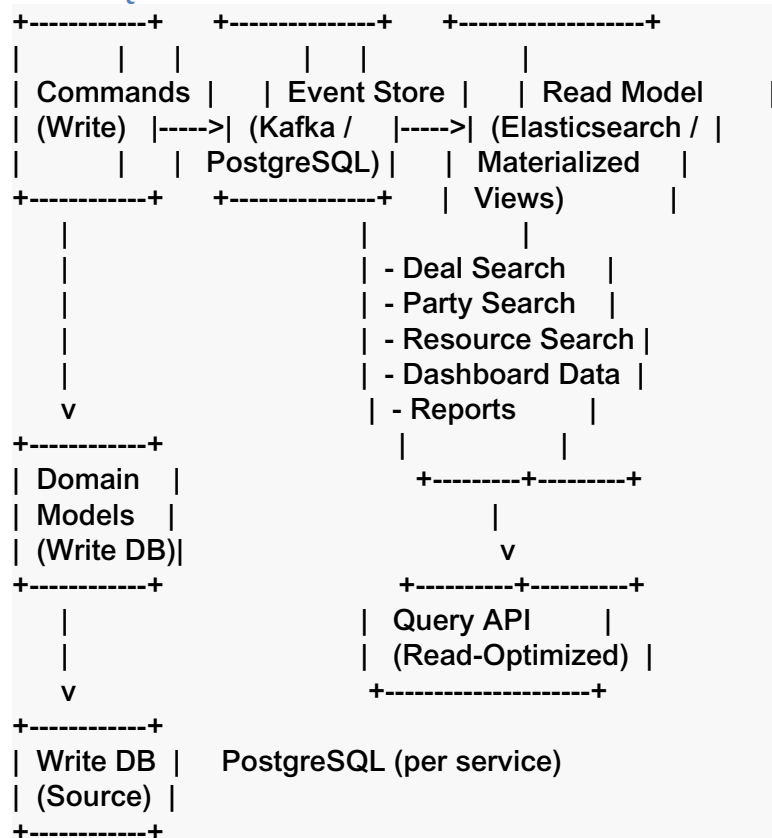
- name: **send_completion_notifications**
 service: **notification-svc**
 action: **SendSettlementNotifications**
 compensation: **null**
 depends_on: **[close_deal]**

5.4.3 Saga Implementation Details

Aspect	Implementation
Orchestrator	Saga state machine within Deal Management Service
State Store	Dedicated saga_instances table in Deal Mgmt DB
Event Coordination	Kafka topics for saga commands and responses
Compensation	Reverse transactions via compensating service calls
Idempotency	All saga steps are idempotent (saga_instance_id + step_name)
Timeouts	Per-step timeouts with configurable values
Monitoring	Saga status exposed via admin API and dashboards
DLQ	Failed saga steps go to dead-letter queue for manual intervention

5.5 CQRS Pattern

5.5.1 CQRS Architecture



5.5.2 Read Model Projections

Read Model	Source Events	Target Store	Refresh Strategy
Deal Search Index	deal., negotiation., trust.*	Elasticsearch	Event-driven, near-realtime
Party Search Index	party., trust.	Elasticsearch	Event-driven, near-realtime
Resource Search Index	resource.*	Elasticsearch	Event-driven, near-realtime
Deal Dashboard	deal., payment., execution.*	PostgreSQL MV + Cache	Event-driven + cached 5min
Platform Analytics	All events	ClickHouse/BigQuery	Batch + streaming
User Activity Feed	deal., negotiation., notification.*	Redis (time-series)	Event-driven

5.5.3 Projection Workers

```

class DealSearchProjection(Projection):
    source_topics = ["deal.lifecycle.events"]
    target_index = "deals"

    async def handle(self, event: DomainEvent):
        if event.type == "deal.created":

```



```

await self.index.create(event.payload.deal_id, {
    "title": event.payload.title,
    "domain": event.payload.domain,
    "status": event.payload.status,
    "created_at": event.timestamp,
    ...
})
elif event.type == "deal.state.changed":
    await self.index.update(event.payload.deal_id, {
        "status": event.payload.to_state,
        "updated_at": event.timestamp
    })
elif event.type == "deal.closed":
    await self.index.delete(event.payload.deal_id)

```

5. Data Architecture

5.1 Database Per Service

Service	Primary DB	Purpose	Backup Strategy
User & Identity	PostgreSQL	User accounts, KYC	Daily full + continuous WAL
Party & Profile	PostgreSQL	Party data, relationships	Daily full + continuous WAL
Deal Management	PostgreSQL + Event Store	Deal data, state history	Daily full + continuous WAL
Matching & Discovery	PostgreSQL + Redis	Match results, scores, cache	Daily full
Negotiation	PostgreSQL	Proposals, sessions	Daily full + continuous WAL
Resource Catalog	PostgreSQL + PostGIS	Resources, types, geo data	Daily full + continuous WAL
Value Distribution	PostgreSQL	Value models, benchmarks	Daily full
Agreement/Contract	PostgreSQL	Agreements, signatures	Daily full + continuous WAL
Execution & Milestone	PostgreSQL	Milestones, deliverables	Daily full + continuous WAL
Payment/Settlement	PostgreSQL	Transactions, settlements	Daily full + continuous WAL (critical)
Notification	PostgreSQL	Notification records	Daily full
Trust & Reputation	PostgreSQL	Reviews, scores	Daily full
Analytics	ClickHouse	Time-series metrics, events	Daily full
Search	Elasticsearch	Search indices	Snapshot every 6 hours

Service	Primary DB	Purpose	Backup Strategy
Media/ Document	PostgreSQL (metadata) + S3	File metadata + object storage	Cross-region replication
Admin & Moderation	PostgreSQL	Admin actions, disputes	Daily full + continuous WAL

5.2 Event Sourcing for Deal Lifecycle

5.2.1 Why Event Sourcing

Requirement	How Event Sourcing Helps
Complete audit trail	Every state change is an immutable event
State reconstruction	Rebuild any deal state at any point in time
Regulatory compliance	Immutable record of all decisions and changes
Dispute resolution	Full history for arbitration
Analytics & ML	Rich event data for pattern analysis
Debugging	Replay events to reproduce issues

5.2.2 Event Store Schema

-- Event store table (append-only)

```
CREATE TABLE deal_events (
  event_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_type        VARCHAR(100) NOT NULL,
  event_version     INT NOT NULL DEFAULT 1,
  aggregate_type    VARCHAR(50) NOT NULL, -- 'deal', 'negotiation', 'milestone'
  aggregate_id      UUID NOT NULL,       -- deal_id
  sequence_number   BIGINT NOT NULL,     -- ordering within aggregate

  -- Event payload
  payload           JSONB NOT NULL,

  -- Metadata
  occurred_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  recorded_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  correlation_id    UUID,
  causation_id      UUID,

  -- Source tracking
  source_service    VARCHAR(100),
  source_version    VARCHAR(20),

  -- Actor
  triggered_by      UUID,                -- user_id

  -- Integrity
  checksum          VARCHAR(64) NOT NULL, -- SHA-256 of serialized event

  UNIQUE (aggregate_id, sequence_number)
);
```

-- Indexes

```

CREATE INDEX idx_deal_events_aggregate
  ON deal_events(aggregate_type, aggregate_id, sequence_number);
CREATE INDEX idx_deal_events_type
  ON deal_events(event_type, occurred_at);
CREATE INDEX idx_deal_events_correlation
  ON deal_events(correlation_id);
CREATE INDEX idx_deal_events_occurred
  ON deal_events(occurred_at);

-- Partition by month for performance
CREATE TABLE deal_events_2024_01 PARTITION OF deal_events
  FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

-- Snapshot table for fast reconstruction
CREATE TABLE deal_snapshots (
  snapshot_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  aggregate_id     UUID NOT NULL,
  aggregate_type   VARCHAR(50) NOT NULL,
  sequence_number  BIGINT NOT NULL,
  snapshot_data    JSONB NOT NULL,
  created_at       TIMESTAMPTZ NOT NULL DEFAULT now(),

  UNIQUE (aggregate_id, sequence_number)
);

```

5.2.3 Deal Aggregate Reconstruction

// Event-sourced aggregate reconstruction

```

class DealAggregate {
  private events: DomainEvent[] = [];
  private snapshot: DealSnapshot | null = null;
  private state: DealState;

  async reconstruct(dealId: UUID): Promise<DealState> {
    // 1. Get latest snapshot
    this.snapshot = await snapshotStore.getLatest(dealId);
    const fromSequence = this.snapshot ? this.snapshot.sequenceNumber : 0;

    // 2. Get events after snapshot
    this.events = await eventStore.getEvents(dealId, fromSequence);

    // 3. Apply snapshot state if exists
    if (this.snapshot) {
      this.state = DealState.fromSnapshot(this.snapshot.data);
    } else {
      this.state = new DealState();
    }

    // 4. Apply events sequentially
    for (const event of this.events) {
      this.state.apply(event);
    }
  }
}

```

```

    return this.state;
}

async appendEvent(event: DomainEvent): Promise<void> {
    // Validate event against current state
    const validation = this.state.validateTransition(event);
    if (!validation.valid) {
        throw new InvalidTransitionError(validation.reason);
    }

    // Append to event store
    await eventStore.append(event);

    // Apply to state
    this.state.apply(event);

    // Check if snapshot needed (every 100 events)
    if (this.events.length % 100 === 0) {
        await snapshotStore.save(this.aggregateId, this.state.toSnapshot());
    }
}
}

```

5.3 Shared Read Models

5.3.1 Search Read Model (Elasticsearch)

```

{
  "deals_index": {
    "mappings": {
      "properties": {
        "deal_id": { "type": "keyword" },
        "deal_reference": { "type": "keyword" },
        "title": { "type": "text", "analyzer": "standard" },
        "description": { "type": "text" },
        "domain": { "type": "keyword" },
        "status": { "type": "keyword" },
        "visibility": { "type": "keyword" },
        "initiated_by": { "type": "keyword" },
        "parties": {
          "type": "nested",
          "properties": {
            "party_id": { "type": "keyword" },
            "role": { "type": "keyword" },
            "party_name": { "type": "text" }
          }
        },
      },
    },
    "value_range": {
      "properties": {
        "min": { "type": "scaled_float", "scaling_factor": 100 },
        "max": { "type": "scaled_float", "scaling_factor": 100 },
        "currency": { "type": "keyword" }
      }
    },
  },
}

```

```

    "location": { "type": "geo_point" },
    "tags": { "type": "keyword" },
    "created_at": { "type": "date" },
    "updated_at": { "type": "date" },
    "win_win_validated": { "type": "boolean" },
    "reputation_boost": { "type": "float" }
  }
}
}
}

```

5.3.2 Analytics Read Model (ClickHouse)

-- Deal events table for analytics

```

CREATE TABLE deal_analytics_events (
  event_id UUID,
  event_type String,
  deal_id UUID,
  deal_domain String,
  deal_status String,
  party_ids Array(UUID),
  event_timestamp DateTime64(3),
  event_date Date DEFAULT toDate(event_timestamp),

```

-- Value metrics

```

  deal_value Decimal(18, 2),
  currency String,

```

-- Performance metrics

```

  processing_time_ms UInt32,

```

-- Dimensions

```

  region String,
  platform_version String

```

```

) ENGINE = MergeTree()

```

```

ORDER BY (event_date, deal_domain, event_type)

```

```

PARTITION BY toYYYYMM(event_date);

```

-- Aggregated metrics materialized view

```

CREATE MATERIALIZED VIEW deal_metrics_mv

```

```

ENGINE = SummingMergeTree()

```

```

ORDER BY (event_date, deal_domain, deal_status)

```

```

AS SELECT

```

```

  event_date,
  deal_domain,
  deal_status,
  count() as deal_count,
  sum(deal_value) as total_value,
  avg(processing_time_ms) as avg_processing_time

```

```

FROM deal_analytics_events

```

```

GROUP BY event_date, deal_domain, deal_status;

```

5.4 Data Consistency Model

Data Type	Consistency Model	Implementation
User credentials	Strong	ACID transactions in User & Identity DB
Deal state transitions	Strong	Event-sourced with optimistic locking
Party profiles	Strong	ACID within Party & Profile DB
Payment transactions	Strong	Two-phase commit with external payment providers
Escrow balances	Strong	ACID with double-entry bookkeeping
Search indices	Eventual	Async projection from event stream (max 2s lag)
Analytics data	Eventual	Async batch + streaming (max 5min lag)
Notification delivery	Eventual	Async with at-least-once delivery
Reputation scores	Eventual	Async recomputation (max 1h lag)
Match recommendations	Eventual	Async computation (near-realtime)

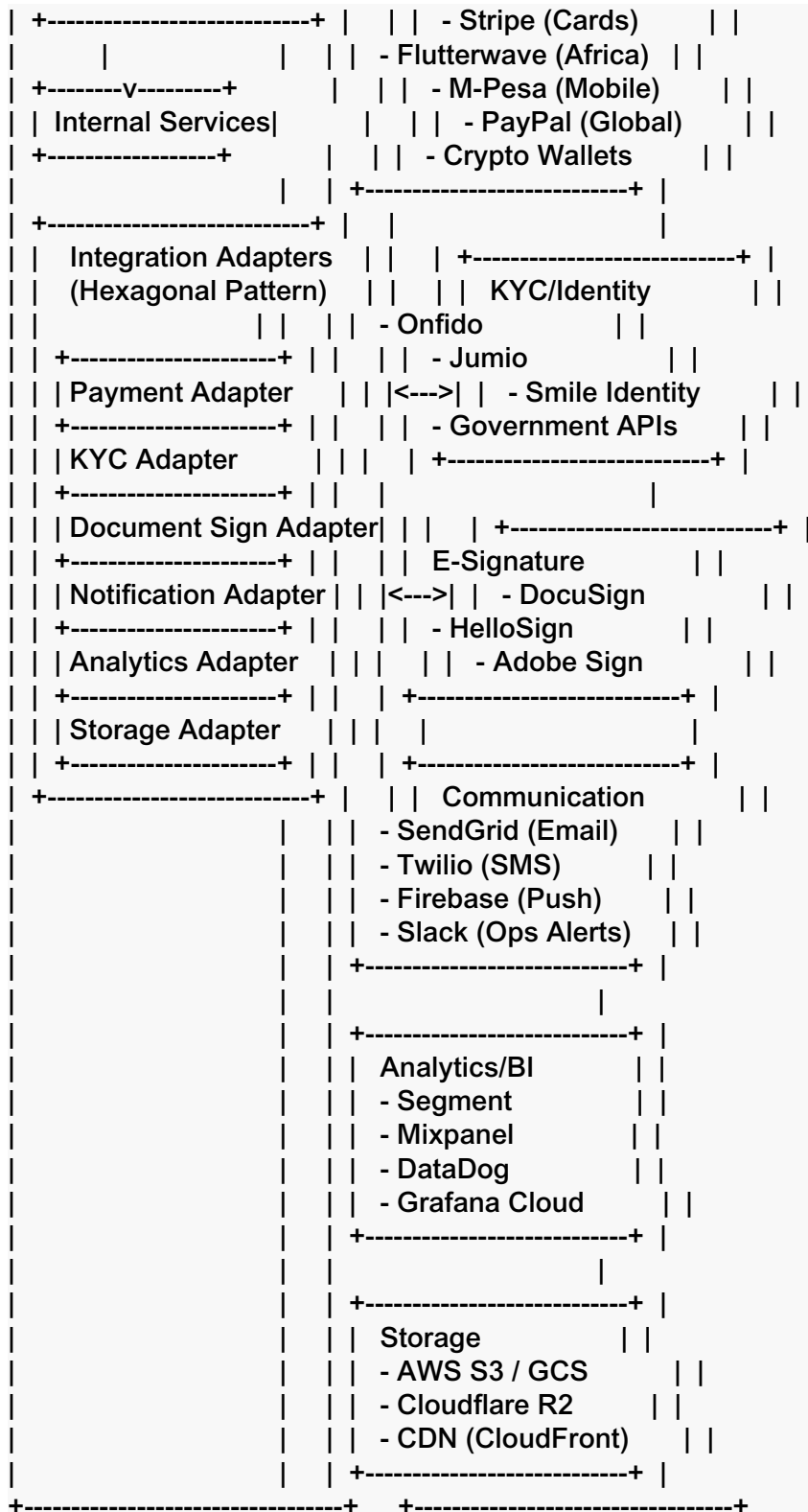
5.5 Data Retention & Archival

Data Category	Hot Storage	Warm Storage	Cold Storage	Retention
Active deals	PostgreSQL	-	-	2 years active
Closed deals	PostgreSQL	S3 (parquet)	Glacier	7 years
Events	PostgreSQL	S3 (parquet)	Glacier	10 years
Payment records	PostgreSQL	S3 (encrypted)	Glacier	10 years
Media files	S3 Standard	S3 IA	Glacier	Per owner config
Audit logs	ClickHouse	S3 (parquet)	Glacier	7 years
Analytics raw	ClickHouse	S3 (parquet)	-	2 years
Notifications	PostgreSQL	-	-	90 days
Search queries	Elasticsearch	-	-	30 days

5. Integration Architecture

5.1 Integration Topology





5.2 External Payment Providers

Payment Provider Adapter

// Abstract payment provider interface

```

interface PaymentProvider {
    name: string;
    supportedMethods: PaymentMethod[];
}
  
```

```

supportedCurrencies: string[];

// Core operations
initializePayment(request: PaymentRequest): Promise<PaymentInitialization>;
processPayment(paymentId: string): Promise<PaymentResult>;
verifyPayment(paymentId: string): Promise<PaymentVerification>;
refund(request: RefundRequest): Promise<RefundResult>;

// Escrow operations
createEscrow(dealId: string, amount: Money): Promise<EscrowAccount>;
releaseEscrow(escrowId: string, distribution: Distribution[]): Promise<ReleaseResult>;

// Webhook handling
handleWebhook(payload: unknown, signature: string): Promise<WebhookEvent>;
}

// Provider implementations
class StripeProvider implements PaymentProvider { ... }
class FlutterwaveProvider implements PaymentProvider { ... }
class MPesaProvider implements PaymentProvider { ... }
class CryptoWalletProvider implements PaymentProvider { ... }

// Provider factory
class PaymentProviderFactory {
  getProvider(method: PaymentMethod, currency: string): PaymentProvider {
    // Strategy: select best provider based on method + currency + region
  }
}

```

Payment Integration Architecture:

Layer	Component	Responsibility
API Layer	PaymentController	REST/gRPC endpoints, request validation
Service Layer	PaymentOrchestrator	Business logic, saga coordination
Adapter Layer	PaymentProviderAdapter	Abstract external provider interactions
Provider Layer	StripeAdapter, FlutterwaveAdapter	Provider-specific implementations
Event Layer	PaymentEventPublisher	Publish domain events to Kafka

Idempotency Pattern:

```

// All payment operations are idempotent
async processPayment(request: PaymentRequest): Promise<PaymentResult> {
  const idempotencyKey = request.idempotencyKey ||
    generateKey(request.dealId, request.amount, request.currency);

  // Check if already processed
  const existing = await this.transactionStore.getByIdempotencyKey(idempotencyKey);
  if (existing) {
    return { status: 'already_processed', transactionId: existing.id };
  }
}

```



```

}

// Acquire distributed lock
const lock = await this.lockManager.acquire(`payment:${idempotencyKey}`, 30);
try {
  // Process with provider
  const result = await this.provider.process(request);

  // Record transaction
  await this.transactionStore.save({
    idempotencyKey,
    ...result
  });

  return result;
} finally {
  await lock.release();
}
}

```

5.3 KYC/Identity Verification

KYC Integration Flow:

1. User initiates KYC in app
2. Platform calls KYC Adapter with user info
3. Adapter selects provider (Onfido/Jumio) based on region
4. Provider generates SDK token for client-side capture
5. User captures ID document + selfie (client-side)
6. Provider processes verification (async)
7. Provider sends webhook with result
8. KYC Adapter processes webhook
9. User & Identity Service updates KYC status
10. Event published: user.kyc.completed
11. Party & Profile Service may upgrade verification level
12. Search Service updates index

KYC Provider Adapter:

```

interface KYCProvider {
  createCheck(userId: UUID, documents: DocumentInfo[]): Promise<KYCCheck>;
  getCheckStatus(checkId: string): Promise<KYCStatus>;
  handleWebhook(payload: unknown): Promise<KYCResult>;
  getSupportedDocuments(country: string): Promise<DocumentType[]>;
}

```

5.4 Document Signing (E-Signatures)

E-Signature Integration Flow:

1. Deal reaches AGREED state
2. Agreement Service generates contract from template
3. Agreement Service calls E-Signature Adapter
4. Adapter creates envelope with DocuSign/HelloSign

5. Signing URLs generated for each party
6. Parties redirected to signing ceremony
7. Signatures captured by provider
8. Webhook received on completion
9. Agreement Service updates signature records
10. When all signed -> event: deal.signed
11. Deal Management advances to SIGNED state

5.5 Notification Channels

Notification Channel Strategy:

Channel	Provider	Use Cases	Priority
Email	SendGrid / AWS SES	Transactional, digest, marketing	All priorities
SMS	Twilio / Africa's Talking	Urgent alerts, OTP, reminders	High, Urgent
Push	Firebase Cloud Messaging	Real-time updates	Normal, High
In-App	WebSocket / SSE	Real-time UI updates	All priorities
Slack	Slack Webhook	Ops alerts, admin notifications	System only

Notification Routing:

```

interface NotificationRouter {
  route(notification: Notification): Promise<Channel[]> {
    const channels: Channel[] = [];

    // 1. Check user preferences
    const prefs = await this.preferenceService.get(notification.userId);

    // 2. Apply priority-based routing
    switch (notification.priority) {
      case 'URGENT':
        channels.push('in_app', 'push', 'sms', 'email');
        break;
      case 'HIGH':
        channels.push('in_app', 'push', 'email');
        if (prefs.smsForHighPriority) channels.push('sms');
        break;
      case 'NORMAL':
        channels.push('in_app');
        if (prefs.emailEnabled) channels.push('email');
        break;
      case 'LOW':
        if (prefs.emailEnabled) channels.push('email');
        break;
    }

    // 3. Apply quiet hours
    if (prefs.quietHours.enabled) {

```

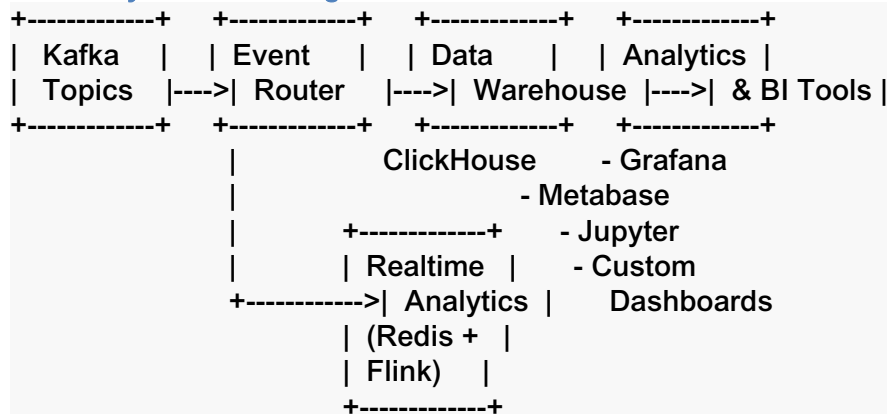
```

        channels = this.filterQuietHours(channels, prefs.quietHours);
    }

    // 4. Apply channel availability
    return channels.filter(c => prefs.channelEnabled(c));
}
}

```

5.6 Analytics & BI Integration



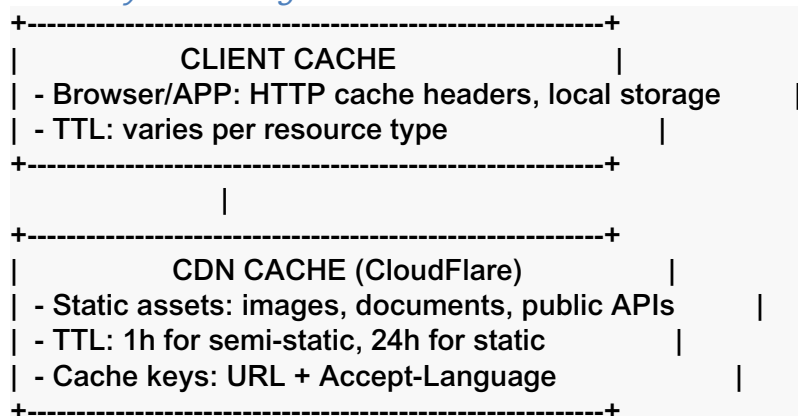
5. Scalability & Resilience

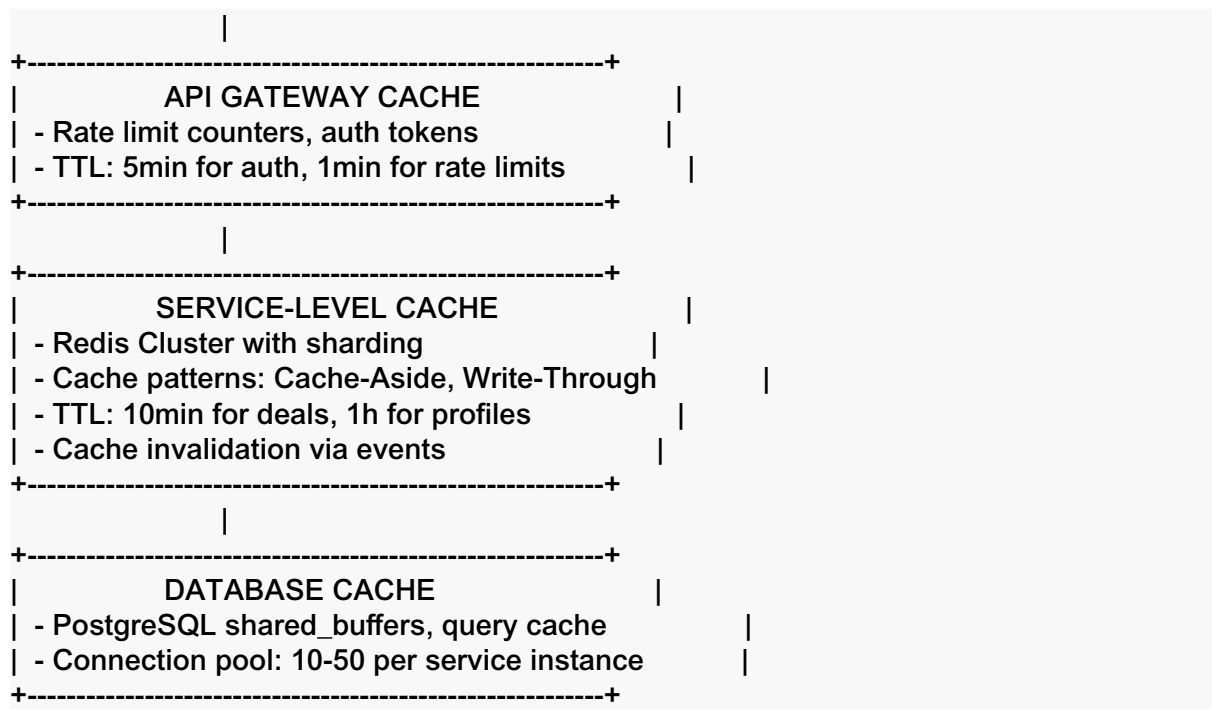
5.1 Load Balancing

Layer	Strategy	Implementation
Edge	Geo-DNS + Anycast	Cloudflare / AWS Route 53
API Gateway	L7 round-robin	Kong / AWS ALB
Service Mesh	Weighted routing + canary	Istio virtual services
Database	Read replicas + connection pooling	PgBouncer, RDS Proxy
Cache	Consistent hashing	Redis Cluster
Message Queue	Partition-aware routing	Kafka partition strategy

5.2 Caching Strategy

Multi-Layer Caching





Cache Key Strategy

Pattern: {entity_type}:{entity_id}:{aspect}:{version}

Examples:

- deal:dl_abc123:summary:v1
- party:pt_xyz789:profile:v1
- party:pt_xyz789:reputation:v2
- resource:rs_def456:availability:2024-01-15
- search:results:deals:agriculture:kenya:{hash_of_params}

Cache Invalidation

Invalidation Strategy	Use Case	Implementation
TTL-based	Non-critical, eventually consistent data	Redis EXPIRE
Event-based	Critical data that must be fresh	Listen to Kafka events
Write-through	Data that changes infrequently	Update cache on write
Explicit purge	Admin operations	Direct Redis DEL

// Event-based cache invalidation

```

class CacheInvalidationListener {
    @OnEvent('deal.updated')
    async handleDealUpdated(event: DealUpdatedEvent) {
        const keys = [
            `deal:${event.dealId}:summary:*`,
            `deal:${event.dealId}:details:*`,
            `search:results:deals:*`, // Invalidate search caches
        ]
    }
}
  
```

```

];

for (const pattern of keys) {
  await this.redis.delPattern(pattern);
}
}
}

```

5.3 Rate Limiting

Rate Limiting Tiers

Tier	Requests/Min	Scope	Applied To
Anonymous	30	IP address	Unauthenticated endpoints
Basic Auth	300	User ID	Authenticated users
Verified	1,000	User ID	KYC-verified users
Premium	5,000	User ID	Premium tier users
Internal	50,000	Service ID	Inter-service calls
Admin	10,000	Admin ID	Admin operations

Implementation:

plugins:

```

- name: rate-limiting
  config:
    minute: 300
    policy: redis
    redis_host: redis-cluster
    fault_tolerant: true
    hide_client_headers: false

- name: rate-limiting-advanced # For different tiers
  config:
    limit:
      - {tier: anonymous, minute: 30}
      - {tier: basic, minute: 300}
      - {tier: verified, minute: 1000}
      - {tier: premium, minute: 5000}
    identifier: jwt_claim_tier

```

5.4 Circuit Breaker Pattern

// Circuit breaker configuration per dependency

```

interface CircuitBreakerConfig {
  failureThreshold: number; // Open after N failures
  successThreshold: number; // Close after N successes
  timeoutDuration: number; // Request timeout (ms)
  resetTimeout: number; // Time before half-open (ms)
  halfOpenMaxCalls: number; // Max calls in half-open state
}

```

```

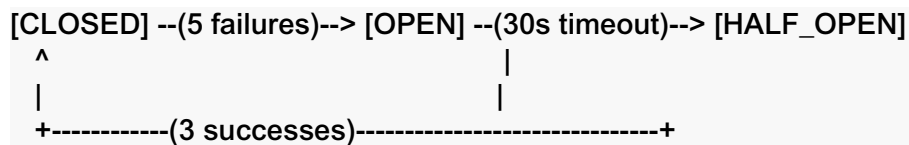
}

const defaultCBConfig: CircuitBreakerConfig = {
  failureThreshold: 5,
  successThreshold: 3,
  timeoutDuration: 10000,
  resetTimeout: 30000,
  halfOpenMaxCalls: 3,
};

// Service-specific configurations
const serviceCBConfigs: Record<string, CircuitBreakerConfig> = {
  'payment-svc': { ...defaultCBConfig, timeoutDuration: 15000, resetTimeout: 60000 },
  'kyc-provider': { ...defaultCBConfig, failureThreshold: 3, resetTimeout: 120000 },
  'email-provider': { ...defaultCBConfig, failureThreshold: 10 },
};

```

Circuit Breaker States:



CLOSED: Normal operation, requests pass through

OPEN: Fast-fail, return cached response or error

HALF_OPEN: Limited requests allowed to test recovery

5.5 Bulkhead Pattern

bulkheads:

deal-mgmt-svc:

max_concurrent_calls: 100

max_wait_duration: 5s

payment-svc:

max_concurrent_calls: 50 *# Lower for critical financial operations*

max_wait_duration: 10s

matching-svc:

max_concurrent_calls: 200 *# Higher for read-heavy operations*

max_wait_duration: 2s

notification-svc:

max_concurrent_calls: 300

max_wait_duration: 1s

5.6 Auto-Scaling

Horizontal Pod Autoscaler (HPA) Configuration

apiVersion: autoscaling/v2

kind: HorizontalPodAutoscaler

metadata:

name: deal-mgmt-svc-hpa

```

spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: deal-mgmt-svc
  minReplicas: 3
  maxReplicas: 30
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80
    - type: Pods
      pods:
        metric:
          name: http_requests_per_second
        target:
          type: AverageValue
          averageValue: "100"
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 60
      policies:
        - type: Percent
          value: 100
          periodSeconds: 60
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
        - type: Percent
          value: 10
          periodSeconds: 60

```

Scaling Triggers by Service

Service	Scale Trigger	Min	Max	Notes
API Gateway	CPU > 70%	2	10	Network-intensive
Deal Management	CPU > 70%, Latency > 200ms	3	30	Core business logic
Payment	CPU > 60%, Queue depth > 100	3	20	Critical path
Matching	CPU > 75%	2	15	Computationally intensive

Service	Scale Trigger	Min	Max	Notes
Search	Query latency > 100ms	2	10	Read-heavy
Notification	Queue depth > 500	2	20	Event-driven

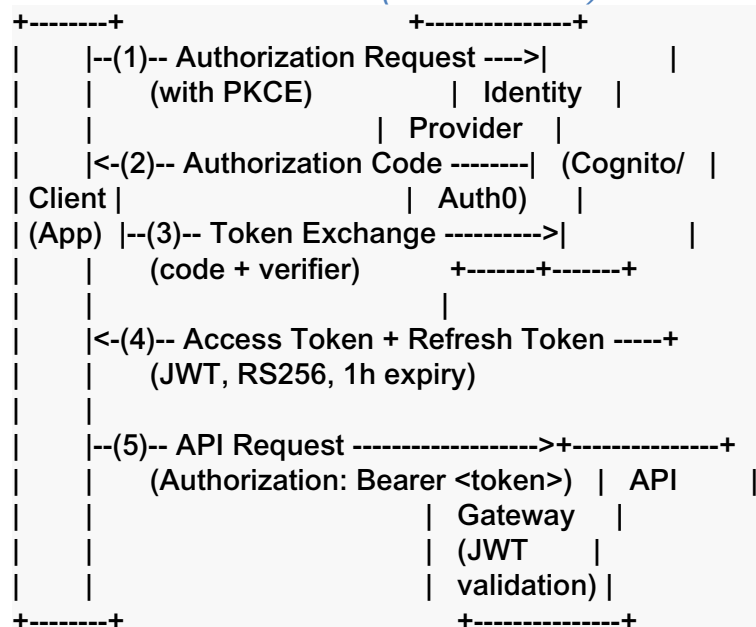
5.7 Chaos Engineering

Experiment	Scope	Frequency	Expected Behavior
Pod failure	Random deal-mgmt pod	Weekly	Traffic rerouted, no data loss
Network latency	payment-svc	Bi-weekly	Circuit breaker opens gracefully
Database failover	PostgreSQL primary	Monthly	Automatic replica promotion
Kafka partition loss	Random partition	Monthly	Consumer group rebalances
Full AZ failure	All services in one AZ	Quarterly	Multi-AZ failover within 60s

5. Security Architecture

5.1 Authentication & Authorization

5.1.1 Authentication Flow (OAuth2/OIDC)



5.1.2 Token Structure

```
{
  "header": {
    "alg": "RS256",
    "typ": "JWT",
    "kid": "key-2024-01"
  },

```



```

"payload": {
  "sub": "usr_a1b2c3d4",
  "iss": "https://auth.dealplatform.io",
  "aud": "dealplatform-api",
  "iat": 1705312800,
  "exp": 1705316400,
  "scope": "deals:read deals:write parties:read profile:write",
  "party_ids": ["pty_e5f6g7h8", "pty_i9j0k1l2"],
  "tier": "verified",
  "mfa_verified": true,
  "jti": "tok_unique_token_id"
}
}

```

5.1.3 Authorization Model (RBAC + ABAC)

Role-Based Access Control (RBAC):

Role	Permissions
guest	Read public deals, search
user	Create deals, manage own profile, negotiate
verified_user	All user + publish deals, accept matches
premium_user	All verified + advanced analytics, priority matching
party_admin	Manage party members, resources, deals for party
party_owner	Full party control, billing, deletion
moderator	Review content, resolve disputes, suspend users
admin	Full platform access, configuration, audit
system	Internal service-to-service communication

Attribute-Based Access Control (ABAC):

policies:

- name: **deal_access**
description: **Users can only access deals they participate in**
resource: **deal**
action: **[read, update]**
condition: |
subject.party_ids contains resource.participant_party_ids
OR subject.roles contains 'admin'
OR subject.roles contains 'moderator'
- name: **negotiation_write**
description: **Only deal participants can submit proposals**
resource: **negotiation_session**
action: **[submit_proposal, accept, reject]**
condition: |
subject.party_ids contains resource.participating_party_ids
- name: **payment_access**
description: **Users can only view payments for their deals**

```

resource: payment_plan
action: [read]
condition: |
  subject.party_ids contains resource.payer_party_id
  OR subject.party_ids contains resource.payee_party_ids

- name: admin_override
  description: Admins can access everything
  resource: '*'
  action: '*'
  condition: |
    subject.roles contains 'admin'

```

Policy Engine (OPA - Open Policy Agent):

```

package dealplatform.authz

import future.keywords.if
import future.keywords.in

default allow := false

allow if {
  input.user.roles[_] == "admin"
}

allow if {
  input.action == "read"
  input.resource.type == "deal"
  input.user.party_ids[_] == input.resource.party_ids[_]
}

allow if {
  input.action in ["update", "delete"]
  input.resource.type == "party"
  input.user.user_id == input.resource.owner_id
}

```

5.2 Data Encryption

Layer	Method	Implementation
In transit	TLS 1.3	All external and internal traffic
At rest (DB)	AES-256	Cloud provider managed encryption
At rest (files)	AES-256	S3 server-side encryption with CMK
Application-level	Field-level encryption	Sensitive PII (email, phone, KYC data)
Secrets	Vault	HashiCorp Vault for all secrets
Keys	HSM / KMS	AWS KMS / GCP Cloud KMS for key management

Field-Level Encryption:

```
// PII fields encrypted at application level
@Entity()
class User {
  @PrimaryColumn('uuid')
  userId: string;

  @Column({
    transformer: new FieldEncryptionTransformer('email')
  })
  email: string; // Encrypted in DB

  @Column({
    transformer: new FieldEncryptionTransformer('phone')
  })
  phone: string; // Encrypted in DB

  @Column()
  @Index()
  emailHash: string; // HMAC for lookup without decryption

  @Column()
  @Index()
  phoneHash: string; // HMAC for lookup
}
```

5.3 API Security

Control	Implementation
Input validation	JSON Schema validation at API Gateway + service level
SQL injection	Parameterized queries, ORM (TypeORM/Prisma)
XSS prevention	Output encoding, Content-Security-Policy headers
CSRF protection	SameSite cookies, CSRF tokens for state-changing ops
Content validation	File type validation, size limits, virus scanning
Rate limiting	Per-user, per-IP, per-endpoint (see 7.3)
CORS	Whitelist-based, strict origin validation
Security headers	HSTS, X-Frame-Options, X-Content-Type-Options
API versioning	URL-based versioning with deprecation notices

5.4 Audit Logging

Audit Event Schema:

```
{
  "audit_id": "aud_abc123",
  "timestamp": "2024-01-15T10:30:00.000Z",
  "event_type": "deal.state.changed",
  "severity": "info",
  "actor": {
    "type": "user",
    "user_id": "usr_xyz789",
  }
}
```

```

    "party_id": "pty_abc123",
    "ip_address": "192.168.1.100",
    "user_agent": "Mozilla/5.0...",
    "session_id": "sess_def456"
  },
  "resource": {
    "type": "deal",
    "id": "dl_abc123",
    "attributes": {
      "from_state": "NEGOTIATING",
      "to_state": "VALIDATED"
    }
  },
  "action": {
    "name": "transition_state",
    "status": "success",
    "result": "VALIDATED"
  },
  "context": {
    "request_id": "req_ghi789",
    "trace_id": "trace_jkl012",
    "service": "deal-mgmt-svc",
    "region": "us-east-1"
  },
  "compliance": {
    "data_classification": "business",
    "retention_period_days": 2555,
    "gdpr_relevant": false
  }
}

```

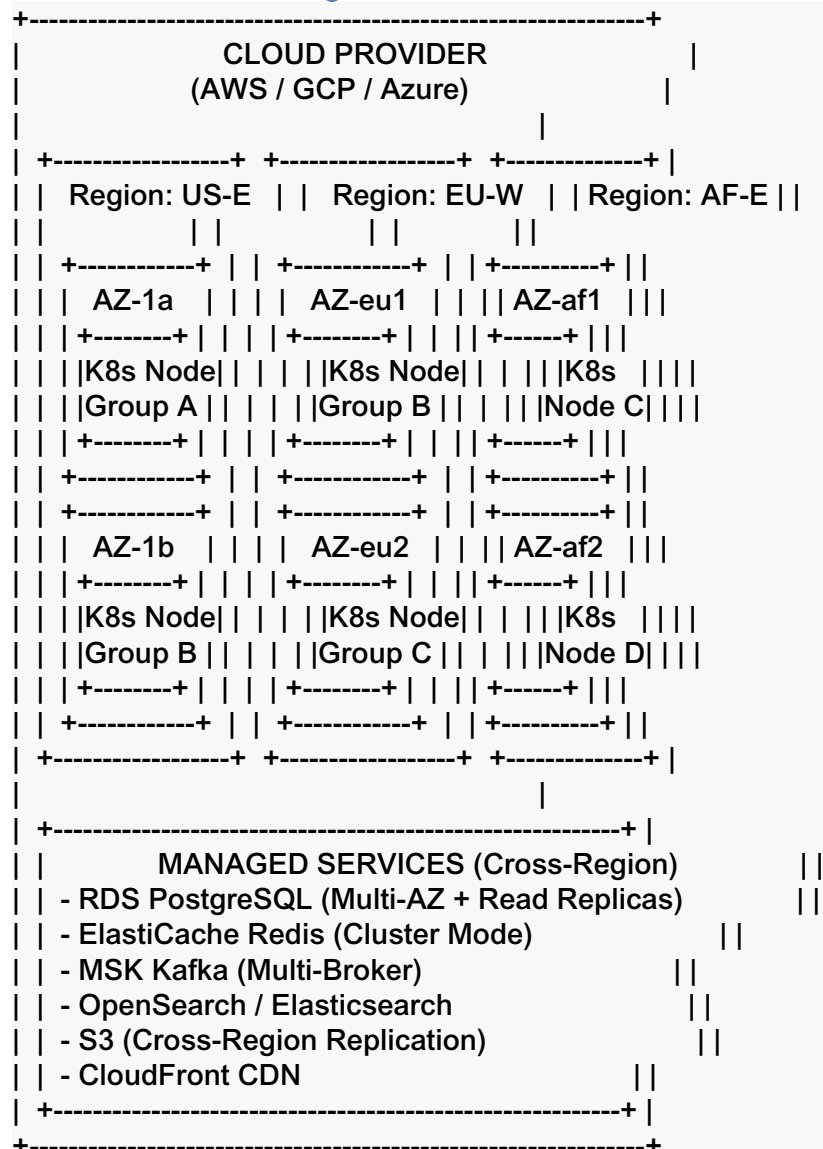
Audit Log Storage: - Hot: ClickHouse (90 days, queryable) - Warm: S3 Parquet (2 years) - Cold: Glacier (7 years, compliance)

5.5 Security Monitoring

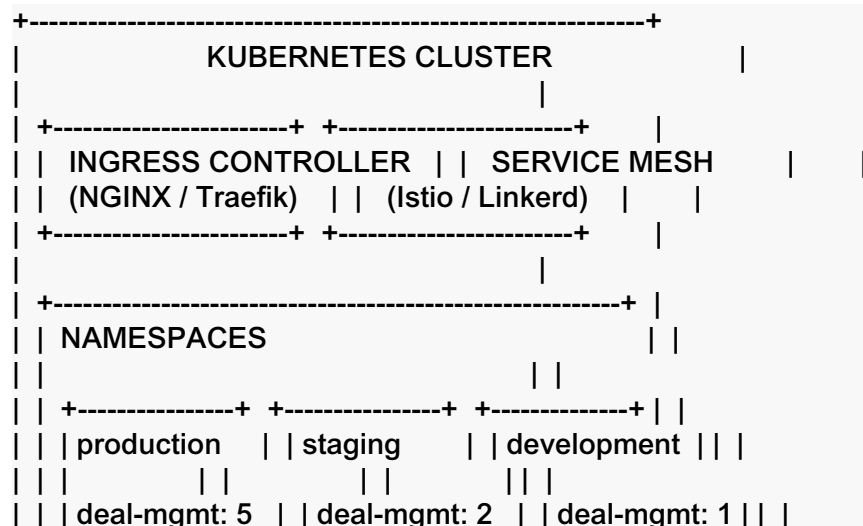
Control	Tool	Purpose
WAF	AWS WAF / Cloudflare	DDoS protection, rule-based filtering
Intrusion Detection	Falco (K8s)	Runtime threat detection
Vulnerability Scanning	Trivy, Snyk	Container and dependency scanning
Secret Scanning	GitLeaks, TruffleHog	Prevent secret leakage
SIEM	Datadog Security	Centralized security monitoring
Penetration Testing	External vendor	Quarterly security assessments

5. Deployment Architecture

5.1 Cloud-Native Design



5.2 Kubernetes Architecture



```

| | payment: 3 | | payment: 1 | | payment: 1 | | |
| | matching: 3 | | matching: 1 | | matching: 1 | | |
| | ... | | ... | | ... | | |
| | +-----+ +-----+ +-----+ | |
| | | | | | | |
| | +-----+ +-----+ | |
| | | istio-system | | monitoring | | |
| | | - Ingress GW | | - Prometheus | | |
| | | - Istiod | | - Grafana | | |
| | | - Sidecars | | - Jaeger | | |
| | +-----+ | - Alertmanager | | |
| | | | +-----+ | | |
| | +-----+ | |
| | | | |
| | +-----+ +-----+ | |
| | | INFRASTRUCTURE | | DATA STORES | | |
| | | - cert-manager | | - PostgreSQL (RDS) | | |
| | | - external-dns | | - Redis (Elasticache) | | |
| | | - sealed-secrets | | - Kafka (MSK) | | |
| | | - velero (backup) | | - S3 | | |
| | +-----+ +-----+ | |
| | +-----+ |

```

5.3 Service Deployment Configuration

apiVersion: apps/v1

kind: Deployment

metadata:

name: deal-mgmt-svc

namespace: production

labels:

app: deal-mgmt-svc

version: v2.3.1

tier: core

spec:

replicas: 5

strategy:

type: RollingUpdate

rollingUpdate:

maxSurge: 2

maxUnavailable: 1

selector:

matchLabels:

app: deal-mgmt-svc

template:

metadata:

labels:

app: deal-mgmt-svc

version: v2.3.1

annotations:

prometheus.io/scrape: "true"

prometheus.io/port: "9090"

spec:

serviceAccountName: deal-mgmt-svc

```
securityContext:
  runAsNonRoot: true
  runAsUser: 1000
  fsGroup: 1000
containers:
- name: deal-mgmt-svc
  image: registry.dealplatform.io/deal-mgmt-svc:v2.3.1
  imagePullPolicy: IfNotPresent
  ports:
    - containerPort: 8080
      name: http
    - containerPort: 9090
      name: metrics
    - containerPort: 50051
      name: grpc
  resources:
    requests:
      memory: "512Mi"
      cpu: "500m"
    limits:
      memory: "2Gi"
      cpu: "2000m"
  env:
    - name: NODE_ENV
      value: "production"
    - name: DB_HOST
      valueFrom:
        secretKeyRef:
          name: deal-mgmt-db-credentials
          key: host
    - name: KAFKA_BROKERS
      valueFrom:
        configMapKeyRef:
          name: platform-config
          key: kafka.brokers
    - name: LOG_LEVEL
      value: "info"
  livenessProbe:
    httpGet:
      path: /health/live
      port: 8080
    initialDelaySeconds: 30
    periodSeconds: 10
    timeoutSeconds: 5
    failureThreshold: 3
  readinessProbe:
    httpGet:
      path: /health/ready
      port: 8080
    initialDelaySeconds: 10
    periodSeconds: 5
    timeoutSeconds: 3
```

```

    failureThreshold: 3
  startupProbe:
    httpGet:
      path: /health/startup
      port: 8080
    initialDelaySeconds: 10
    periodSeconds: 5
    failureThreshold: 30
  volumeMounts:
    - name: tmp
      mountPath: /tmp
  volumes:
    - name: tmp
      emptyDir: {}
  affinity:
    podAntiAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            labelSelector:
              matchExpressions:
                - key: app
                  operator: In
                  values:
                    - deal-mgmt-svc
            topologyKey: kubernetes.io/hostname
---
apiVersion: v1
kind: Service
metadata:
  name: deal-mgmt-svc
  namespace: production
  labels:
    app: deal-mgmt-svc
spec:
  selector:
    app: deal-mgmt-svc
  ports:
    - port: 8080
      targetPort: 8080
      name: http
    - port: 9090
      targetPort: 9090
      name: metrics
    - port: 50051
      targetPort: 50051
      name: grpc
  type: ClusterIP

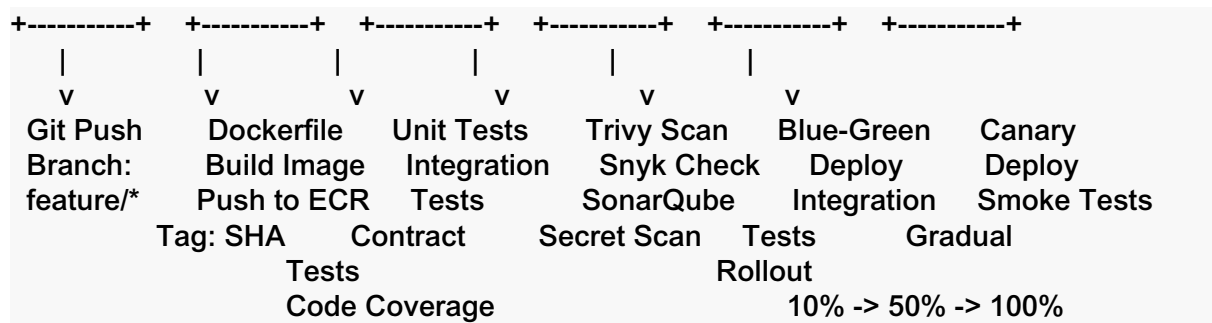
```

5.4 CI/CD Pipeline

```

+-----+ +-----+ +-----+ +-----+ +-----+ +-----+
| Develop |->| Build |->| Test  |->| Security |->| Stage  |->| Produce |
| Commit  | | & Lint | |      | | Scan   | | Deploy | | Deploy |

```

Pipeline Configuration (GitHub Actions):

name: **CI/CD Pipeline**

on:

push:

branches: [main, release/*]

pull_request:

branches: [main]

env:

REGISTRY: \${ secrets.ECR_REGISTRY }

IMAGE_NAME: deal-mgmt-svc

jobs:

build:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4
- name: Setup Node.js
 - uses: actions/setup-node@v4
 - with:
 - node-version: '20'
 - cache: 'npm'
- name: Install dependencies
 - run: npm ci
- name: Lint
 - run: npm run lint
- name: Build
 - run: npm run build
- name: Unit Tests
 - run: npm run test:unit -- --coverage
- name: Upload coverage
 - uses: codecov/codecov-action@v3

integration-tests:

needs: build

runs-on: ubuntu-latest

services:

postgres:

image: postgres:16

env:

POSTGRES_PASSWORD: test

```

redis:
  image: redis:7-alpine
kafka:
  image: confluentinc/cp-kafka:7.5
steps:
  - uses: actions/checkout@v4
  - name: Integration Tests
    run: npm run test:integration
  - name: Contract Tests
    run: npm run test:contract

security-scan:
  needs: build
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Trivy vulnerability scan
      uses: aquasecurity/trivy-action@master
      with:
        scan-type: 'fs'
        format: 'sarif'
        output: 'trivy-results.sarif'
    - name: Secret detection
      uses: trufflesecurity/trufflehog@main
      with:
        path: ./
        base: main

build-image:
  needs: [integration-tests, security-scan]
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Build and push
      uses: docker/build-push-action@v5
      with:
        push: true
        tags: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:${{ github.sha }}
        cache-from: type=gha
        cache-to: type=gha,mode=max

deploy-staging:
  needs: build-image
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Deploy to staging
      run: |
        kubectl set image deployment/deal-mgmt-svc \
          deal-mgmt-svc=${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:${{ github.sha }} \
          -n staging

```

```
kubectl rollout status deployment/deal-mgmt-svc -n staging --timeout=300s
```

deploy-production:

needs: **deploy-staging**

if: **github.ref == 'refs/heads/main'**

runs-on: **ubuntu-latest**

environment: **production**

steps:

- uses: **actions/checkout@v4**

- name: **Canary deploy**

run: |

./scripts/canary-deploy.sh \

--service deal-mgmt-svc \

--image "\${{ env.REGISTRY }}/\${{ env.IMAGE_NAME }}:\${{ github.sha }}" \

--namespace production \

--steps "10,25,50,100" \

--step-duration 300

5.5 Environment Strategy

Environment	Purpose	Data	Scaling	Access
Local	Developer workstations	Seeded test data	Docker Compose	Developer only
Development	Integration testing	Synthetic data	1 replica each	Team
Staging	Pre-production validation	Anonymized prod snapshot	Production-like	Team + QA
Production	Live traffic	Real user data	Auto-scaled	Ops only
DR	Disaster recovery	Async replication	Warm standby	Failover only

5. Architecture Diagrams

5.1 High-Level System Architecture

graph TD

subgraph "Client Layer"

WEB["Web App (React/Next.js)"]

MOBILE["Mobile App (React Native)"]

API_CLIENT["API Clients (Partners)"]

end

subgraph "Edge Layer"

CDN["CDN (CloudFront)"]

WAF["WAF (Cloudflare/AWS WAF)"]

DNS["DNS (Route 53)"]

end

subgraph "API & Gateway Layer"

```
    APIGW["API Gateway (Kong)"]
    AUTHZ["Authorization (OPA)"]
    RL["Rate Limiter"]
end

subgraph "Core Domain Services"
    DEAL["Deal Management Service"]
    PARTY["Party & Profile Service"]
    MATCH["Matching & Discovery Service"]
    NEG["Negotiation Service"]
    RES["Resource/Asset Catalog Service"]
    VAL["Value Distribution Service"]
    AGR["Agreement/Contract Service"]
    EXE["Execution & Milestone Service"]
    PAY["Payment/Settlement Service"]
end

subgraph "Supporting Services"
    SEARCH["Search Service"]
    NOTIF["Notification Service"]
    TRUST["Trust & Reputation Service"]
    ANAL["Analytics & Reporting Service"]
    MEDIA["Media/Document Service"]
    ADMIN["Admin & Moderation Service"]
end

subgraph "Infrastructure Services"
    IDENTITY["User & Identity Service"]
    CONFIG["Config Service"]
    DISCOV["Service Discovery"]
end

subgraph "Message Infrastructure"
    KAFKA["Apache Kafka"]
    SCHEMA["Schema Registry"]
    KAFKA_CONNECT["Kafka Connect"]
end

subgraph "Data Layer"
    PG[(PostgreSQL Cluster)]
    REDIS[(Redis Cluster)]
    ES[(Elasticsearch)]
    CH[(ClickHouse)]
    S3[(Object Storage)]
    EVT_STORE[(Event Store)]
end

subgraph "External Integrations"
    STRIPE["Stripe"]
    FLUTTER["Flutterwave"]
    MPESA["M-Pesa"]
    DOCUSIGN["DocuSign"]
end
```

```
ONFIDO["Onfido"]
SENDGRID["SendGrid"]
TWILIO["Twilio"]
FIREBASE["Firebase"]
VAULT["HashiCorp Vault"]
end
```

```
WEB --> CDN --> WAF --> APIGW
MOBILE --> WAF --> APIGW
API_CLIENT --> APIGW
```

```
APIGW --> AUTHZ
APIGW --> RL
APIGW --> DEAL
APIGW --> PARTY
APIGW --> MATCH
APIGW --> NEG
APIGW --> RES
APIGW --> VAL
APIGW --> AGR
APIGW --> EXE
APIGW --> PAY
APIGW --> SEARCH
APIGW --> MEDIA
APIGW --> ADMIN
APIGW --> IDENTITY
```

```
DEAL --> KAFKA
PARTY --> KAFKA
MATCH --> KAFKA
NEG --> KAFKA
RES --> KAFKA
VAL --> KAFKA
AGR --> KAFKA
EXE --> KAFKA
PAY --> KAFKA
IDENTITY --> KAFKA
ADMIN --> KAFKA
```

```
KAFKA --> NOTIF
KAFKA --> TRUST
KAFKA --> ANAL
KAFKA --> SEARCH
```

```
DEAL <--> PG
PARTY <--> PG
MATCH <--> PG
NEG <--> PG
RES <--> PG
VAL <--> PG
AGR <--> PG
EXE <--> PG
```

PAY <--> PG
IDENTITY <--> PG

DEAL <--> REDIS
MATCH <--> REDIS
SEARCH <--> ES
ANAL <--> CH
MEDIA <--> S3
DEAL <--> EVT_STORE

PAY --> STRIPE
PAY --> FLUTTER
PAY --> MPESA
AGR --> DOCUSIGN
IDENTITY --> ONFIDO
NOTIF --> SENDGRID
NOTIF --> TWILIO
NOTIF --> FIREBASE

style APIGW fill:#4a90e2,stroke:#333,color:#fff
style KAFKA fill:#e67e22,stroke:#333,color:#fff
style DEAL fill:#2ecc71,stroke:#333,color:#000
style PAY fill:#e74c3c,stroke:#333,color:#fff
style PG fill:#3498db,stroke:#333,color:#fff

5.2 Data Flow for Deal Creation

sequenceDiagram

autonumber
actor U as User (Party Owner)
participant API as API Gateway
participant DS as Deal Mgmt Service
participant RS as Resource Catalog Service
participant VS as Value Distribution Service
participant NS as Negotiation Service
participant KF as Kafka Event Bus
participant PS as Party & Profile Service
participant SS as Search Service
participant NT as Notification Service

U->>API: POST /api/v1/deals (deal data, resource IDs)
API->>API: Validate request, JWT auth
API->>DS: Forward request

DS->>PS: gRPC: ValidatePartyExists(initiator_party_id)
PS-->>DS: Party validated

DS->>DS: Create deal (local transaction)
DS->>DS: Record deal.created event
DS-->>API: Deal created (201)
API-->>U: Deal created with ID

par Async Event Processing

```

    DS->>KF: Publish: deal.created
    DS->>KF: Publish: deal.published (if auto-publish)
and Resource Registration
    DS->>RS: gRPC: RegisterResources(deal_id, resources)
    RS-->>DS: Resources registered
    RS->>KF: Publish: resource.registered
end

par Value Distribution
    DS->>VS: gRPC: CreateDistribution(deal_id, terms)
    VS-->>DS: Distribution created
    VS->>KF: Publish: value_distribution.created
and Negotiation Setup
    DS->>NS: gRPC: StartNegotiation(deal_id, parties)
    NS-->>DS: Negotiation session started
    NS->>KF: Publish: negotiation.started
end

DS->>DS: Update deal status: NEGOTIATING
DS->>KF: Publish: deal.negotiation.started

par Read Model Updates
    KF->>SS: Consume: deal.created
    SS->>SS: Index deal in Elasticsearch
and Notification
    KF->>NT: Consume: deal.created
    NT->>NT: Queue welcome notification
and Analytics
    KF->>SS: Consume: all events (for audit trail)
end

NT-->>U: Push: "Deal created, negotiation started"

```

5.3 Event Flow for Deal State Changes

```

graph LR
    subgraph "State Change Sources"
        USER["User Action"]
        SYS["System Trigger"]
        TIME["Time-Based"]
        WEBHOOK["External Webhook"]
    end

    subgraph "Deal State Machine"
        DRAFT["DRAFT"]
        PUBLISHED["PUBLISHED"]
        MATCHING["MATCHING"]
        NEGOTIATING["NEGOTIATING"]
        VALIDATED["VALIDATED"]
        AGREED["AGREED"]
        SIGNED["SIGNED"]
        EXECUTING["EXECUTING"]
        COMPLETED["COMPLETED"]
    end

```

```
SETTLED["SETTLED"]
CLOSED["CLOSED"]
CANCELLED["CANCELLED"]
DISPUTED["DISPUTED"]
end
```

```
subgraph "Event Consumers"
  NOTIFY["Notification Service"]
  PAYMENT["Payment Service"]
  TRUST["Trust & Reputation"]
  ANALYTICS["Analytics Service"]
  SEARCH["Search Service"]
  ADMIN["Admin Service"]
end
```

```
USER -->|publish| DRAFT
USER -->|submit_proposal| NEGOTIATING
USER -->|sign_contract| SIGNED
USER -->|cancel| CANCELLED
USER -->|raise_dispute| DISPUTED
```

```
SYS -->|auto-publish| PUBLISHED
SYS -->|matching_complete| MATCHING
SYS -->|validation_pass| VALIDATED
SYS -->|all_milestones| COMPLETED
```

```
TIME -->|expiry| CANCELLED
TIME -->|milestone_due| EXECUTING
```

```
WEBHOOK -->|payment_confirmed| SETTLED
WEBHOOK -->|signature_complete| SIGNED
```

```
DRAFT -->|deal.published| PUBLISHED
PUBLISHED -->|deal.matching.started| MATCHING
MATCHING -->|deal.parties.matched| NEGOTIATING
NEGOTIATING -->|deal.terms.validated| VALIDATED
NEGOTIATING -->|deal.rejected| CANCELLED
VALIDATED -->|deal.agreed| AGREED
AGREED -->|deal.signed| SIGNED
SIGNED -->|deal.execution.started| EXECUTING
EXECUTING -->|milestone.completed| EXECUTING
EXECUTING -->|deal.all_milestones.done| COMPLETED
EXECUTING -->|deal.disputed| DISPUTED
DISPUTED -->|dispute.resolved| EXECUTING
DISPUTED -->|deal.terminated| CANCELLED
COMPLETED -->|deal.settled| SETTLED
SETTLED -->|deal.closed| CLOSED
```

```
PUBLISHED -->|deal.created event| NOTIFY
PUBLISHED -->|deal.created event| SEARCH
NEGOTIATING -->|negotiation.events| NOTIFY
SIGNED -->|deal.signed event| PAYMENT
```


COMPLETED -->|deal.completed event| PAYMENT
COMPLETED -->|deal.completed event| TRUST
SETTLED -->|payment.settlement.completed| TRUST
SETTLED -->|payment.settlement.completed| ANALYTICS
CLOSED -->|deal.closed event| SEARCH
CANCELLED -->|deal.cancelled event| ADMIN
CANCELLED -->|deal.cancelled event| SEARCH

style DRAFT fill:#95a5a6,stroke:#333,color:#000
style NEGOTIATING fill:#f39c12,stroke:#333,color:#000
style EXECUTING fill:#3498db,stroke:#333,color:#fff
style COMPLETED fill:#2ecc71,stroke:#333,color:#000
style CLOSED fill:#27ae60,stroke:#333,color:#fff
style CANCELLED fill:#e74c3c,stroke:#333,color:#fff
style DISPUTED fill:#e67e22,stroke:#333,color:#000

5.4 Integration Architecture

graph TB

```
subgraph "Platform Core"
    direction TB
    SVC["Internal Services"]
    ADAPTER["Integration Adapters"]
    WEBHOOK["Webhook Receiver"]
    QUEUE["Outgoing Queue"]
end
```

```
subgraph "Payment Providers"
    direction LR
    STRIPE["Stripe"]
    FLUTTER["Flutterwave"]
    MPESA["M-Pesa"]
    PAYPAL["PayPal"]
end
```

```
subgraph "Identity Verification"
    direction LR
    ONFIDO["Onfido"]
    JUMIO["Jumio"]
    SMILE["Smile Identity"]
end
```

```
subgraph "E-Signature"
    direction LR
    DOCUSIGN["DocuSign"]
    HELLOSIGN["HelloSign"]
    ADOBESIGN["Adobe Sign"]
end
```

```
subgraph "Communications"
    direction LR
    SENDGRID["SendGrid Email"]
    TWILIO["Twilio SMS"]
end
```

```
    FCM["Firebase Push"]
    SLACK["Slack Alerts"]
end

subgraph "Storage & CDN"
    direction LR
    S3["S3 / GCS"]
    CDN["CloudFront CDN"]
    CLOUDFLARE["Cloudflare"]
end

subgraph "Observability"
    direction LR
    DATADOG["Datadog APM"]
    GRAFANA["Grafana"]
    JAEGER["Jaeger Tracing"]
end

subgraph "Security"
    direction LR
    VAULT["HashiCorp Vault"]
    KMSC["AWS KMS"]
    WAF2["WAF"]
end

SVC --> ADAPTER
ADAPTER --> STRIPE
ADAPTER --> FLUTTER
ADAPTER --> MPESA
ADAPTER --> PAYPAL

SVC --> ADAPTER
ADAPTER --> ONFIDO
ADAPTER --> JUMIO
ADAPTER --> SMILE

SVC --> ADAPTER
ADAPTER --> DOCUSIGN
ADAPTER --> HELLOSIGN
ADAPTER --> ADOBESIGN

SVC --> QUEUE
QUEUE --> SENDGRID
QUEUE --> TWILIO
QUEUE --> FCM
QUEUE --> SLACK

SVC --> S3
S3 --> CDN
CDN --> CLOUDFLARE
WEB_CLIENT["Web Client"] --> CLOUDFLARE
```

```

SVC --> DATADOG
SVC --> JAEGER
GRAFANA --> DATADOG

SVC --> VAULT
VAULT --> KMSC
EXTERNAL["External Traffic"] --> WAF2 --> SVC

STRIPE -->|webhook| WEBHOOK
FLUTTER -->|webhook| WEBHOOK
MPESA -->|webhook| WEBHOOK
ONFIDO -->|webhook| WEBHOOK
DOCUSIGN -->|webhook| WEBHOOK
SENDGRID -->|webhook| WEBHOOK

style SVC fill:#4a90e2,stroke:#333,color:#fff
style ADAPTER fill:#e67e22,stroke:#333,color:#fff
style QUEUE fill:#9b59b6,stroke:#333,color:#fff
style WEBHOOK fill:#e74c3c,stroke:#333,color:#fff

```

5. Cross-Cutting Concerns

5.1 Observability

Concern	Tool	Implementation
Metrics	Prometheus + Grafana	Service-level, business, infrastructure metrics
Logging	Fluentd + Elasticsearch	Structured JSON logging, centralized aggregation
Tracing	OpenTelemetry + Jaeger	Distributed tracing across all services
Alerting	Prometheus Alertmanager + PagerDuty	Multi-channel alerting with escalation
Error Tracking	Sentry	Real-time error tracking and aggregation
Uptime Monitoring	Datadog Synthetics	Endpoint monitoring from multiple locations

Key Metrics:

infrastructure:

- **cpu_usage_percent**
- **memory_usage_percent**
- **disk_usage_percent**
- **network_io_bytes**
- **pod_restart_count**

application:

- **http_request_duration_seconds** (histogram)
- **http_request_total** (counter, by status_code, method, path)
- **active_connections** (gauge)

- database_query_duration_seconds
- cache_hit_ratio
- message_queue_lag_seconds
- circuit_breaker_state

business:

- deals_created_total (counter, by domain, country)
- deal_lifecycle_duration_hours (histogram, by domain, status)
- negotiation_rounds_average (gauge)
- win_win_validation_success_rate
- payment_success_rate
- settlement_value_total (counter, by currency)
- user_active_daily (gauge)
- matching_accuracy_score

5.2 Configuration Management

config_sources:

- name: application.yaml
description: Default configuration, versioned in Git
- name: environment variables
description: Container-injected, for secrets and environment-specific values
- name: Consul / AWS Parameter Store
description: Dynamic configuration, runtime updatable
- name: Feature flags (LaunchDarkly)
description: Gradual rollouts, A/B testing

5.3 Feature Flags

Feature Flag	Default	Description
matching.v2_algorithm	false	Enable new matching algorithm
negotiation.ai_suggestions	false	AI-powered negotiation suggestions
payment.crypto_enabled	false	Enable cryptocurrency payments
value_distribution.ai_pricing	false	AI-driven dynamic pricing
trust.reputation.v2	false	New reputation scoring model
analytics.realtime_dashboard	true	Real-time analytics dashboard

5.4 Disaster Recovery

Scenario	RTO	RPO	Strategy
Single pod failure	< 30s	0	Kubernetes auto-restart
Single AZ failure	< 5 min	< 1 min	Multi-AZ deployment
Full region failure	< 1 hour	< 5 min	Cross-region DR with async replication
Database corruption	< 2 hours	< 5 min	Point-in-time recovery from WAL
Kafka partition loss	< 5 min	0	Replication factor 3
Complete data center loss	< 4 hours	< 5 min	Cross-region failover

5.5 Compliance Considerations

Regulation	Requirement	Implementation
GDPR	Data portability, right to erasure	Export APIs, anonymization jobs, data retention policies
PCI DSS	Payment card security	Tokenization, isolated payment service, regular audits
SOC 2	Security controls	Access controls, audit logging, change management
KYC/AML	Identity verification	Partner integrations, document verification, watchlist screening
eIDAS	Electronic signatures	Qualified e-signature provider integration
Local data residency	Data stays in country	Region-specific deployments, data classification

5. Appendices

Appendix A: Technology Stack Summary

Layer	Technology	Alternative
Runtime	Node.js 20 / Go 1.21	Python, Java
Framework	NestJS (Node) / Gin (Go)	Express, Fastify
API	REST + gRPC	GraphQL (future)
Database	PostgreSQL 16	CockroachDB (future geo-distribution)
Cache	Redis 7 Cluster	KeyDB
Search	Elasticsearch 8	OpenSearch
Analytics	ClickHouse	BigQuery
Events	Apache Kafka 3.6	NATS, Pulsar
Message Format	JSON / Protobuf	Avro (with Schema Registry)
Container	Docker	Podman
Orchestration	Kubernetes 1.28	AWS EKS / GKE
Service Mesh	Istio	Linkerd
API Gateway	Kong	AWS API Gateway
Auth	OAuth2/OIDC + OPA	Keycloak, Auth0
Secrets	HashiCorp Vault	AWS Secrets Manager
CI/CD	GitHub Actions + ArgoCD	GitLab CI, Jenkins
Monitoring	Prometheus + Grafana	Datadog
Tracing	OpenTelemetry + Jaeger	Zipkin
Logs	Fluentd + Elasticsearch	Loki

Appendix B: Glossary

Term	Definition
3-Party Deal	A transaction involving three distinct roles: Supplier, Consumer, and Enhancer
Party	An entity (individual or organization) that participates in a deal
Deal	A structured agreement between three parties facilitated by the platform
Win-Win-Win	A validated state where all three parties derive positive value from the deal
Escrow	A financial arrangement where funds are held by a third party until deal conditions are met
Saga	A sequence of local transactions that together achieve a global business transaction
Event Sourcing	A pattern where state changes are captured as a sequence of events
CQRS	Command Query Responsibility Segregation - separating read and write models
ABAC	Attribute-Based Access Control - authorization based on attributes of subjects, resources, and environment
Bulkhead	A pattern that isolates failures to prevent cascading failures

Appendix C: Capacity Estimates

Metric	Year 1	Year 2	Year 3
Registered Users	50,000	200,000	500,000
Active Deals (monthly)	5,000	25,000	80,000
Completed Deals (monthly)	2,000	12,000	40,000
API Requests (daily)	2M	10M	40M
Events (daily)	5M	25M	100M
Peak RPS	500	2,500	8,000
Storage (deals + events)	500 GB	2 TB	8 TB
Media Storage	2 TB	10 TB	50 TB

Appendix D: Risk Assessment

Risk	Probability	Impact	Mitigation
Payment processing failures	Medium	High	Multiple providers, automatic failover
Data breach	Low	Critical	Encryption, access

Risk	Probability	Impact	Mitigation
Service cascading failure	Medium	High	controls, regular audits Circuit breakers, bulkheads, rate limiting
Regulatory changes	Medium	Medium	Flexible compliance framework, legal review
Third-party provider outage	Medium	Medium	Multi-provider strategy, graceful degradation
Data loss	Low	Critical	Multi-region replication, point-in-time recovery
Performance degradation under load	Medium	Medium	Auto-scaling, caching, performance testing
Fraudulent deals/parties	Medium	High	KYC, reputation system, fraud detection

Document Control

Version	Date	Author	Changes
1.0	2024-01-15	Platform Architecture Team	Initial comprehensive architecture design

This document represents the authoritative architecture blueprint for the 3-Party Deal Making Platform. All implementation decisions should align with the patterns and principles defined herein. Changes require architecture review board approval.

Document Information

Property	Value
Version	1.0.0
Status	Draft
Date	2024
Classification	Internal

6. API Standards & Conventions

6.1 Base URL Structure

Production: <https://api.dealplatform.com/{version}>
 Staging: <https://api-staging.dealplatform.com/{version}>
 Sandbox: <https://api-sandbox.dealplatform.com/{version}>

URL Pattern:

`/api/{version}/{service-resource}/{resource-id}/{sub-resource}/{sub-resource-id}/{action}`

Examples:

```
/api/v1/deals/550e8400-e29b-41d4-a716-446655440000
/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/terms
/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/transitions/lock-terms
/api/v1/parties/7c9e6679-7425-40de-944b-e07fc1f90ae7/roles
```

6.2 Versioning Strategy

URL Path Versioning (primary): The API version is included in the URL path. - Current stable: **v1** - Beta features: **v1-beta** (may change without notice) - Deprecation: 6-month notice via **Sunset** header

Version Lifecycle: | Phase | Duration | Support Level | |———|———|———| | Preview | 1-3 months | No SLA, breaking changes possible | | Stable | 12+ months | Full SLA, no breaking changes | | Deprecated | 6 months | Security fixes only | | Retired | — | 410 Gone response |

Deprecation Headers:

```
Deprecation: @1735689600 ; Expires: Thu, 01 Jan 2026 00:00:00 GMT
Sunset: Sat, 01 Jul 2026 00:00:00 GMT
Link: </docs/api/v2/migration-guide>; rel="successor-version"
```

6.3 Authentication

Bearer Token (JWT):

Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...

Token contains claims:

```
{
  "sub": "user-uuid",
  "party_ids": ["party-uuid-1", "party-uuid-2"],
  "roles": ["SUPPLIER", "CONSUMER"],
  "scope": "deals:read deals:write payments:write",
  "ver": 1,
  "iat": 1704067200,
  "exp": 1704153600,
  "jti": "unique-token-id"
}
```

Multi-Party Authorization: When a user belongs to multiple parties, include the acting party in the X-Party-ID header:

```
Authorization: Bearer <token>
X-Party-ID: 550e8400-e29b-41d4-a716-446655440000
```

Idempotency (for mutation endpoints):

Idempotency-Key: f47ac10b-58cc-4372-a567-0e02b2c3d479

6.4 Authorization Model

RBAC + ABAC: Role-based access control with attribute-based extensions.

Permission Scope	Description	Roles
parties:read	Read party data	All authenticated
parties:write	Create/update party	Account owner
deals:read	Read deal data	Deal participants
deals:write	Create/update deal	Initiator, participants
deals:transition	Execute state transitions	Per deal role
terms:negotiate	Propose/counter terms	Active deal participants
payments:read	View payment data	Payer/payee
payments:write	Initiate payments	Authorized party
admin:*	Administrative functions	Platform admins

6.5 Pagination

Cursor-based pagination for all list endpoints with >50 potential results.

Request Parameters: | Parameter | Type | Default | Max | Description | |—————|————|————|
 —|—————-| | **limit** | Integer | 20 | 100 | Items per page | | **cursor** | String | — | — | Opaque
 cursor from previous response | | **sort** | String | **createdAt:desc** | — | Sort field and direction |

Response Format:

```
{
  "data": [...],
  "pagination": {
    "limit": 20,
    "nextCursor": "eyJpZCI6IjEyMyIsInNjb3JlIjo5OC43fQ==",
    "prevCursor": null,
    "total": null,
    "hasMore": true
  }
}
```

Offset pagination (available for analytics/reporting only):

```
{
  "data": [...],
  "pagination": {
    "limit": 20,
    "offset": 0,
    "total": 1542,
    "hasMore": true
  }
}
```

6.6 Rate Limiting

Rate Limit Headers (per client ID):

```
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 998
X-RateLimit-Reset: 1704153600
X-RateLimit-Window: 3600
```

Tiered Limits:	Tier	Requests/Min	Requests/Hour	Burst
Free	60	1,000	10	Pro
Pro	300	10,000	50	Enterprise
Enterprise	1,000	50,000	200	Internal
Internal	5,000	250,000	500	

429 Response:

```
{
  "type": "https://api.dealplatform.com/errors/rate-limit-exceeded",
  "title": "Rate Limit Exceeded",
  "status": 429,
  "detail": "Request rate exceeds tier limit. Retry after 45 seconds.",
  "retryAfter": 45
}
```

6.7 Error Response Format (RFC 7807 Problem Details)

All error responses use the `application/problem+json` content type.

Base Error Structure:

```
{
  "type": "https://api.dealplatform.com/errors/{error-code}",
  "title": "Human-readable title",
  "status": 400,
  "detail": "Detailed explanation of the error",
  "instance": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000",
  "errorCode": "DEAL_INVALID_TRANSITION",
  "timestamp": "2024-01-01T12:00:00Z",
  "requestId": "req-uuid-for-tracing",
  "errors": [
    {
      "field": "valueDistribution.supplierSharePercent",
      "message": "Must be between 0 and 100",
      "code": "FIELD_RANGE_INVALID"
    }
  ]
}
```

Common Error Codes:	Code	Status	Description	
UNAUTHORIZED	401	Authentication required or invalid	FORBIDDEN	
Insufficient permissions	NOT_FOUND	404	Resource not found	CONFLICT
Resource state conflict	VALIDATION_ERROR	422	Request validation failed	INVALID_TRANSITION
INVALID_TRANSITION	422	State transition not allowed	RATE_LIMITED	
RATE_LIMITED	429	Too many requests	INTERNAL_ERROR	
INTERNAL_ERROR	500	Internal server error	SERVICE_UNAVAILABLE	
SERVICE_UNAVAILABLE	503	Service temporarily unavailable		

6.8 Content Negotiation

Request:

```
Accept: application/json
Accept: application/problem+json ; for errors
Accept: text/event-stream ; for streaming endpoints
```

Request Body Content-Type:

Content-Type: application/json

Response Content-Type:

Content-Type: application/json; charset=utf-8

6.9 Common Request/Response Patterns

Creation Pattern (201):

// Request: POST /api/v1/deals

```
{
  "dealTitle": "Farm Share Crop Agreement",
  "dealDescription": "Seasonal crop sharing on 10-acre farmland",
  "domainCategoryId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "expectedStartDate": "2024-03-01",
  "expectedEndDate": "2024-08-31",
  "currency": "USD",
  "isPublic": true
}
```

// Response: 201 Created

```
{
  "dealId": "550e8400-e29b-41d4-a716-446655440000",
  "dealReference": "DL-2024-0001",
  "dealTitle": "Farm Share Crop Agreement",
  "dealStatus": "DRAFT",
  "initiatorPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "initiatorRole": "SUPPLIER",
  "createdAt": "2024-01-15T10:30:00Z",
  "updatedAt": "2024-01-15T10:30:00Z",
  "links": {
    "self": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000",
    "terms": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/terms",
    "participations":
"/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/participations",
    "transitions": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/transitions"
  }
}
```

List Pattern (200):

// Response: GET /api/v1/deals?limit=2

```
{
  "data": [
    { /* deal 1 */ },
    { /* deal 2 */ }
  ],
  "pagination": {
    "limit": 2,
    "nextCursor": "eyJpZCZlIjEYMyIsInNjb3JlIjo5OC43fQ==",
    "prevCursor": null,
    "hasMore": true
  }
}
```

```
}  
}
```

6.10 HTTP Method Usage

Method	Usage	Idempotent
GET	Retrieve resource(s)	Yes
POST	Create resource, execute action	No (use Idempotency-Key)
PUT	Full resource replacement	Yes
PATCH	Partial resource update	Yes
DELETE	Remove resource	Yes
POST /.../actions/{action}	Execute state transition/action	No (use Idempotency-Key)

6.11 Idempotency for Mutation Endpoints

All POST, PUT, PATCH, DELETE operations support idempotency via the **Idempotency-Key** header.

- Key: UUID v4, generated by the client
- Retention: 24 hours on the server
- Behavior: Duplicate requests with same key return the original response (including status code)
- Scope: Per-client (based on **sub** claim in JWT)

6.12 Request/Response Tracing

Correlation Headers:

X-Request-ID: req-uuid	; Client-generated (optional, will be generated if absent)
X-Correlation-ID: corr-uuid	; Cross-service correlation
X-B3-TraceId: trace-id	; Distributed tracing (Zipkin)
X-B3-SpanId: span-id	; Current span

Response includes:

X-Request-ID: req-uuid	; Echoed or generated
X-Response-Time: 145	; Milliseconds

6.13 Common Filtering & Query Parameters

Parameter	Type	Description	Example
status	String[]	Filter by status(es)	status=DRAFT&status=SUGGESTED
partyId	UUID	Filter by party involvement	partyId=abc-123
role	Enum	Filter by role	role=SUPPLIER
domainCategoryId	UUID	Filter by domain	domainCategoryId=xyz-789

Parameter	Type	Description	Example
fromDate	ISO 8601	Start date range	fromDate=2024-01-01
toDate	ISO 8601	End date range	toDate=2024-12-31
q	String	Full-text search query	q=farm+share
include	String[]	Eager load relations	include=parties,terms,milestones
fields	String[]	Sparse fieldset	fields=dealId,dealTitle,dealStatus

6.14 WebSocket Real-time Endpoints

For real-time updates, the platform provides WebSocket connections:

`wss://realtime.dealplatform.com/v1/connect?token=<jwt>`

Subscribe to channels:

```
{
  "action": "subscribe",
  "channels": [
    "deal:550e8400-e29b-41d4-a716-446655440000",
    "party:7c9e6679-7425-40de-944b-e07fc1f90ae7:notifications",
    "negotiation:550e8400-e29b-41d4-a716-446655440000:terms"
  ]
}
```

6. Party & Profile APIs

Service: party-profile-svc Base Path: /api/v1/parties

6.1 Party CRUD

Create Party

POST /api/v1/parties

Description: Register a new party (individual, organization, or group) on the platform. Associates with the authenticated user's identity.

Authorization: Authenticated user.

Request Body:

```
{
  "partyType": "INDIVIDUAL",
  "displayName": "Green Acres Farm Ltd",
  "email": "contact@greenacres.example.com",
  "phone": "+1-555-0123",
  "taxId": "12-3456789",
  "primaryDomain": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "location": {
```

```

"latitude": 37.7749,
"longitude": -122.4194,
"addressLine1": "123 Farm Road",
"city": "San Francisco",
"region": "CA",
"country": "US",
"postalCode": "94102"
},
"serviceRadiusKm": 100,
"roles": ["SUPPLIER", "CONSUMER"],
"profile": {
  "description": "Organic farm specializing in seasonal produce",
  "website": "https://greenacres.example.com",
  "logoUrl": "https://cdn.dealplatform.com/logos/greenacres.png",
  "languages": ["en", "es"],
  "timezone": "America/Los_Angeles"
}
}

```

Validation Rules: - **partyType:** Required. One of **INDIVIDUAL**, **ORGANIZATION**, **PARTY_GROUP** - **displayName:** 3-120 characters, required - **email:** Valid email format, unique across platform - **roles:** At least one role required (**SUPPLIER**, **CONSUMER**, **ENHANCER**) - **location.latitude/longitude:** Valid coordinate range

Success Response (201):

```

{
  "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "partyType": "INDIVIDUAL",
  "displayName": "Green Acres Farm Ltd",
  "email": "contact@greenacres.example.com",
  "phone": "+1-555-0123",
  "verificationStatus": "UNVERIFIED",
  "primaryDomain": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "location": {
    "latitude": 37.7749,
    "longitude": -122.4194,
    "addressLine1": "123 Farm Road",
    "city": "San Francisco",
    "region": "CA",
    "country": "US",
    "postalCode": "94102"
  },
  "serviceRadiusKm": 100,
  "trustScore": 0.0,
  "totalDealsCompleted": 0,
  "totalDealsInitiated": 0,
  "isActive": true,
  "roles": [
    {
      "roleType": "SUPPLIER",
      "isActive": true,
      "assignedAt": "2024-01-15T10:30:00Z"
    }
  ]
}

```

```

    },
    {
      "roleType": "CONSUMER",
      "isActive": true,
      "assignedAt": "2024-01-15T10:30:00Z"
    }
  ],
  "createdAt": "2024-01-15T10:30:00Z",
  "updatedAt": "2024-01-15T10:30:00Z",
  "links": {
    "self": "/api/v1/parties/7c9e6679-7425-40de-944b-e07fc1f90ae7",
    "profile": "/api/v1/parties/7c9e6679-7425-40de-944b-e07fc1f90ae7/profile",
    "roles": "/api/v1/parties/7c9e6679-7425-40de-944b-e07fc1f90ae7/roles",
    "deals": "/api/v1/deals?partyId=7c9e6679-7425-40de-944b-e07fc1f90ae7",
    "wallet": "/api/v1/payments/wallets/7c9e6679-7425-40de-944b-e07fc1f90ae7"
  }
}

```

Error Responses:

// 400 Bad Request

```

{
  "type": "https://api.dealplatform.com/errors/validation-error",
  "title": "Validation Error",
  "status": 400,
  "detail": "One or more fields failed validation",
  "errorCode": "VALIDATION_ERROR",
  "errors": [
    {
      "field": "displayName",
      "message": "Must be between 3 and 120 characters",
      "code": "FIELD_LENGTH_INVALID"
    },
    {
      "field": "email",
      "message": "Email address already registered",
      "code": "FIELD_DUPLICATE_VALUE"
    }
  ]
}

```

Get Party

GET /api/v1/parties/{partyId}

Description: Retrieve a party's details. Returns different field sets based on caller's relationship to the party (self, deal partner, or public).

Authorization: Authenticated. Public fields visible to all; private fields to party members.

Query Parameters: | Parameter | Type | Description | | include | String[] | Include related resources: **profile, roles, groups, trustScore, verification** |

Success Response (200):

```

{
  "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "partyType": "INDIVIDUAL",
  "displayName": "Green Acres Farm Ltd",
  "email": "contact@greenacres.example.com",
  "verificationStatus": "VERIFIED",
  "primaryDomain": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "location": {
    "latitude": 37.7749,
    "longitude": -122.4194,
    "addressLine1": "123 Farm Road",
    "city": "San Francisco",
    "region": "CA",
    "country": "US",
    "postalCode": "94102"
  },
  "serviceRadiusKm": 100,
  "trustScore": 4.52,
  "totalDealsCompleted": 12,
  "totalDealsInitiated": 5,
  "isActive": true,
  "roles": [
    {
      "roleType": "SUPPLIER",
      "isActive": true,
      "resourceTypes": ["farmland", "greenhouse"],
      "typicalCapacity": "10-50 acres",
      "preferredCompensation": ["REVENUE_SHARE", "HYBRID"],
      "insuranceVerified": true,
      "assignedAt": "2024-01-15T10:30:00Z"
    }
  ],
  "profile": {
    "description": "Organic farm specializing in seasonal produce",
    "website": "https://greenacres.example.com",
    "logoUrl": "https://cdn.dealplatform.com/logos/greenacres.png",
    "languages": ["en", "es"],
    "timezone": "America/Los_Angeles",
    "bannerImageUrl": "https://cdn.dealplatform.com/banners/greenacres.jpg",
    "socialLinks": {
      "linkedin": "https://linkedin.com/company/greenacres"
    },
    "foundedYear": 2015,
    "teamSize": "5-20"
  },
  "verification": {
    "status": "VERIFIED",
    "level": "PREMIUM",
    "verifiedAt": "2024-02-01T14:22:00Z",
    "provider": "onfido",
    "documents": ["identity", "address", "business_registration"]
  },
}

```



```
"createdAt": "2024-01-15T10:30:00Z",
"updatedAt": "2024-06-01T09:15:00Z",
"links": {
  "self": "/api/v1/parties/7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "deals": "/api/v1/deals?partyId=7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "resources": "/api/v1/resources?partyId=7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "trustScore": "/api/v1/trust/party/7c9e6679-7425-40de-944b-e07fc1f90ae7"
}
}
```

Update Party

PUT /api/v1/parties/{partyId}

Description: Full replacement of party data.

Authorization: Party owner or admin.

Request Body: (Full party object, same as Create Party but without auto-generated fields)

Success Response (200):

```
{
  "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "displayName": "Green Acres Organic Farm LLC",
  "email": "contact@greenacres.example.com",
  "phone": "+1-555-0456",
  "updatedAt": "2024-06-10T14:30:00Z",
  "links": {
    "self": "/api/v1/parties/7c9e6679-7425-40de-944b-e07fc1f90ae7"
  }
}
```

Partial Update Party

PATCH /api/v1/parties/{partyId}

Description: Partial update of party data using JSON Patch or merge patch semantics.

Authorization: Party owner or admin.

Request Body (JSON Merge Patch):

```
{
  "displayName": "Green Acres Organic Farm LLC",
  "phone": "+1-555-0456",
  "location": {
    "city": "Oakland"
  }
}
```

Success Response (200): Updated party object.

Delete Party (Soft Delete)

DELETE /api/v1/parties/{partyId}

Description: Soft-delete a party. Fails if party has active deals.

Authorization: Party owner or admin.

Query Parameters: | Parameter | Type | Description | | | | | **force** |
Boolean | Force delete (admin only, requires **admin:parties** scope) |

Success Response (204): No body.

Error Response (409):

```
{
  "type": "https://api.dealplatform.com/errors/party-has-active-deals",
  "title": "Party Has Active Deals",
  "status": 409,
  "detail": "Cannot deactivate party with 3 active deals. Complete or transfer deals first.",
  "errorCode": "PARTY_ACTIVE_DEALS_EXIST",
  "activeDealCount": 3,
  "activeDealIds": ["deal-1", "deal-2", "deal-3"]
}
```

6.2 Party Profile APIs

Get Profile

GET /api/v1/parties/{partyId}/profile

Authorization: Authenticated. Public profile visible to all.

Success Response (200):

```
{
  "profileId": "prof-uuid-1",
  "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "description": "Organic farm specializing in seasonal produce",
  "website": "https://greenacres.example.com",
  "logoUrl": "https://cdn.dealplatform.com/logos/greenacres.png",
  "bannerImageUrl": "https://cdn.dealplatform.com/banners/greenacres.jpg",
  "languages": ["en", "es"],
  "timezone": "America/Los_Angeles",
  "socialLinks": {
    "linkedin": "https://linkedin.com/company/greenacres",
    "twitter": "https://twitter.com/greenacres"
  },
  "foundedYear": 2015,
  "teamSize": "5-20",
  "annualRevenue": "1M-5M",
  "certifications": [
    {
      "name": "USDA Organic",
      "issuer": "USDA",
      "issuedDate": "2020-03-15",
      "expiryDate": "2025-03-15",
      "documentUrl": "https://cdn.../cert.pdf"
    }
  ]
}
```

```

],
"portfolio": [
  {
    "title": "Summer 2023 Harvest",
    "description": "Successful 3-month crop sharing deal",
    "dealId": "previous-deal-uuid",
    "imageUrls": ["https://cdn.../img1.jpg"]
  }
],
"operatingHours": {
  "monday": { "open": "06:00", "close": "18:00" },
  "tuesday": { "open": "06:00", "close": "18:00" }
},
"createdAt": "2024-01-15T10:30:00Z",
"updatedAt": "2024-06-01T09:15:00Z"
}

```

Update Profile

PUT /api/v1/parties/{partyId}/profile

Authorization: Party owner, admin, or member with **profile:write** permission.

Request Body: Profile object (as above, without **profileId**, **partyId**, **createdAt**).

Success Response (200): Updated profile object.

6.3 Role Management

List Roles

GET /api/v1/parties/{partyId}/roles

Authorization: Authenticated.

Success Response (200):

```

{
  "data": [
    {
      "roleType": "SUPPLIER",
      "isActive": true,
      "assignedAt": "2024-01-15T10:30:00Z",
      "resourceTypes": ["farmland", "greenhouse"],
      "typicalCapacity": "10-50 acres",
      "preferredCompensation": ["REVENUE_SHARE", "HYBRID"],
      "insuranceVerified": true,
      "supplierProfile": {
        "resourceTypes": ["farmland", "greenhouse"],
        "typicalCapacity": "10-50 acres",
        "availabilitySchedule": { /* ... */ },
        "preferredCompensation": ["REVENUE_SHARE", "HYBRID"],
        "insuranceVerified": true
      }
    }
  ],
}

```

```
{
  "roleType": "CONSUMER",
  "isActive": true,
  "assignedAt": "2024-01-15T10:30:00Z",
  "consumerProfile": {
    "needCategories": ["organic-produce", "fresh-vegetables"],
    "typicalVolume": "500-2000 lbs/month",
    "preferredQualityStandard": "USDA Organic",
    "budgetRangeMin": 5000.00,
    "budgetRangeMax": 25000.00,
    "preferredPaymentTerms": ["MILESTONE", "ON_DELIVERY"]
  }
}
```

Add Role

POST /api/v1/parties/{partyId}/roles

Description: Add a new role (SUPPLIER, CONSUMER, or ENHANCER) to an existing party.

Authorization: Party owner or admin.

Request Body:

```
{
  "roleType": "ENHANCER",
  "enhancerProfile": {
    "enhancementTypes": ["agricultural-consulting", "soil-analysis"],
    "skills": ["Precision Agriculture", "Organic Certification"],
    "certifications": [
      {
        "name": "Certified Crop Adviser",
        "issuer": "ASA",
        "issuedDate": "2022-01-01"
      }
    ],
    "hourlyRate": 150.00,
    "equipmentOwned": ["Soil Testing Kit", "Drone Survey Equipment"],
    "typicalEngagementDuration": "2-6 weeks"
  }
}
```

Success Response (201):

```
{
  "roleType": "ENHANCER",
  "isActive": true,
  "assignedAt": "2024-06-10T14:30:00Z",
  "enhancerProfile": { /* ... */ },
  "message": "Role ENHANCER added to party. You can now participate in deals as an
```

```
Enhancer."  
}
```

Error Response (409):

```
{  
  "type": "https://api.dealplatform.com/errors/role-already-assigned",  
  "title": "Role Already Assigned",  
  "status": 409,  
  "detail": "Party already has ENHANCER role assigned since 2024-01-15T10:30:00Z",  
  "errorCode": "ROLE_ALREADY_ASSIGNED",  
  "roleType": "ENHANCER",  
  "assignedAt": "2024-01-15T10:30:00Z"  
}
```

Remove Role

DELETE /api/v1/parties/{partyId}/roles/{roleType}

Description: Remove a role from a party. Fails if party has active deals in that role.

Authorization: Party owner or admin.

Query Parameters:

Parameter	Type	Description	force
Boolean		Force removal with active deals (admin only)	

Error Response (409):

```
{  
  "type": "https://api.dealplatform.com/errors/role-has-active-deals",  
  "title": "Role Has Active Deals",  
  "status": 409,  
  "detail": "Cannot remove SUPPLIER role: party has 2 active deals as supplier.",  
  "errorCode": "ROLE_ACTIVE_DEALS_EXIST",  
  "activeDealCount": 2,  
  "roleType": "SUPPLIER"  
}
```

Update Role Profile

PUT /api/v1/parties/{partyId}/roles/{roleType}

Authorization: Party owner, admin, or member with role-specific write permission.

Request Body: Role-specific profile (e.g., **SupplierProfile**, **ConsumerProfile**, **EnhancerProfile**).

Success Response (200): Updated role with profile.

6.4 Party Group Management

Create Party Group

POST /api/v1/parties/{partyId}/group

Description: Convert an existing party into a PartyGroup with governance rules.

Authorization: Party owner.

Request Body:

```
{
  "groupName": "Valley Farmers Cooperative",
  "groupType": "COOPERATIVE",
  "governanceModel": "Each member has one vote. Decisions require majority approval (51%+1).",
  "isOpenToNewMembers": true,
  "minimumMemberApproval": 3,
  "sharedBankAccount": true,
  "members": [
    {
      "memberPartyId": "member-uuid-1",
      "roleInGroup": "LEADER",
      "contributionPercentage": 35.0
    },
    {
      "memberPartyId": "member-uuid-2",
      "roleInGroup": "MEMBER",
      "contributionPercentage": 25.0
    },
    {
      "memberPartyId": "member-uuid-3",
      "roleInGroup": "MEMBER",
      "contributionPercentage": 40.0
    }
  ]
}
```

Success Response (201):

```
{
  "groupId": "group-uuid-1",
  "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "groupName": "Valley Farmers Cooperative",
  "groupType": "COOPERATIVE",
  "governanceModel": "Each member has one vote...",
  "memberCount": 3,
  "isOpenToNewMembers": true,
  "minimumMemberApproval": 3,
  "sharedBankAccount": true,
  "members": [
    {
      "membershipId": "mem-uuid-1",
      "memberPartyId": "member-uuid-1",
      "displayName": "John Smith",
      "roleInGroup": "LEADER",
      "joinedAt": "2024-01-15T10:30:00Z",
      "contributionPercentage": 35.0,
      "isActive": true
    }
  ]
}
```

```
// ...  
],  
"createdAt": "2024-01-15T10:30:00Z"  
}
```

List Group Members

GET /api/v1/parties/{partyId}/group/members

Authorization: Authenticated (public for group info; private details for members only).

Query Parameters: | Parameter | Type | Description | |—————|—————|—————| | **status** | String | Filter by **active**, **pending**, **inactive** | | **role** | String | Filter by group role |

Success Response (200):

```
{  
  "data": [  
    {  
      "membershipId": "mem-uuid-1",  
      "memberPartyId": "member-uuid-1",  
      "displayName": "John Smith",  
      "roleInGroup": "LEADER",  
      "joinedAt": "2024-01-15T10:30:00Z",  
      "contributionPercentage": 35.0,  
      "isActive": true,  
      "dealsParticipated": 8  
    }  
  ],  
  "pagination": {  
    "limit": 20,  
    "nextCursor": null,  
    "hasMore": false  
  }  
}
```

Add Group Member

POST /api/v1/parties/{partyId}/group/members

Description: Invite/add a member to the group.

Authorization: Group leader or admin. May require member approval if `minimumMemberApproval > 1`.

Request Body:

```
{  
  "memberPartyId": "new-member-uuid",  
  "roleInGroup": "MEMBER",  
  "contributionPercentage": 15.0  
}
```

Success Response (201):

```
{
  "membershipId": "mem-uuid-new",
  "memberPartyId": "new-member-uuid",
  "roleInGroup": "MEMBER",
  "joinedAt": "2024-06-10T14:30:00Z",
  "contributionPercentage": 15.0,
  "isActive": true,
  "status": "PENDING_APPROVAL",
  "approvalVotesRequired": 2,
  "approvalVotesReceived": 0
}
```

Remove Group Member

DELETE /api/v1/parties/{partyId}/group/members/{memberPartyId}

Authorization: Group leader or admin.

Query Parameters:

Parameter	Type	Description	reason
String	Reason for removal		

Success Response (204): No body.

Update Member Role

PATCH /api/v1/parties/{partyId}/group/members/{memberPartyId}

Authorization: Group leader.

Request Body:

```
{
  "roleInGroup": "ADMIN",
  "contributionPercentage": 30.0
}
```

Success Response (200): Updated membership record.

Vote on Member Action

POST /api/v1/parties/{partyId}/group/members/{actionId}/vote

Description: Cast a vote on a pending group action (add member, approve deal, etc.).

Authorization: Active group member.

Request Body:

```
{
  "vote": "APPROVE",
  "comment": "New member brings valuable expertise"
}
```

6.5 KYC & Verification APIs

Initiate Verification

POST /api/v1/parties/{partyId}/verification/initiate

Authorization: Party owner or admin.

Request Body:

```
{
  "verificationLevel": "PREMIUM",
  "provider": "onfido",
  "documents": [
    {
      "type": "IDENTITY",
      "documentUrl": "https://cdn.../passport.pdf"
    },
    {
      "type": "ADDRESS_PROOF",
      "documentUrl": "https://cdn.../utility_bill.pdf"
    },
    {
      "type": "BUSINESS_REGISTRATION",
      "documentUrl": "https://cdn.../registration.pdf"
    }
  ]
}
```

Success Response (202):

```
{
  "verificationId": "verif-uuid-1",
  "status": "PENDING",
  "provider": "onfido",
  "providerReference": "check-uuid-from-provider",
  "submittedAt": "2024-01-20T10:00:00Z",
  "estimatedCompletion": "2024-01-22T10:00:00Z",
  "message": "Verification initiated. You will be notified when complete.",
  "webhookEndpoint": "https://api.dealplatform.com/webhooks/kyc/onfido"
}
```

Get Verification Status

GET /api/v1/parties/{partyId}/verification

Authorization: Party owner/admin (full details); authenticated (public status).

Success Response (200):

```
{
  "verificationId": "verif-uuid-1",
  "status": "VERIFIED",
  "level": "PREMIUM",
  "provider": "onfido",
  "providerReference": "check-uuid-from-provider",
}
```

```
"documents": [  
  {  
    "type": "IDENTITY",  
    "status": "VERIFIED",  
    "verifiedAt": "2024-01-21T08:30:00Z"  
  },  
  {  
    "type": "ADDRESS_PROOF",  
    "status": "VERIFIED",  
    "verifiedAt": "2024-01-21T09:15:00Z"  
  },  
  {  
    "type": "BUSINESS_REGISTRATION",  
    "status": "VERIFIED",  
    "verifiedAt": "2024-01-21T14:00:00Z"  
  }  
],  
"verifiedAt": "2024-01-21T14:00:00Z",  
"expiresAt": "2026-01-21T14:00:00Z"  
}
```

List My Parties

GET /api/v1/parties/me

Description: Get all parties associated with the authenticated user.

Authorization: Authenticated.

Success Response (200):

```
{  
  "data": [  
    {  
      "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",  
      "displayName": "Green Acres Farm Ltd",  
      "partyType": "INDIVIDUAL",  
      "roles": ["SUPPLIER", "CONSUMER"],  
      "verificationStatus": "VERIFIED",  
      "trustScore": 4.52,  
      "memberRole": "OWNER",  
      "isActive": true  
    }  
  ]  
}
```

Search Parties

GET /api/v1/parties/search

Description: Search for parties by name, role, domain, location, or trust score.

Authorization: Authenticated.

Query Parameters: | Parameter | Type | Description | | q | String | Free-text search query | | role | Enum[] | Filter by role: **SUPPLIER**, **CONSUMER**, **ENHANCER** | | domainCategoryId | UUID | Filter by domain category | | verificationStatus | String | **VERIFIED**, **UNVERIFIED**, etc. | | minTrustScore | Decimal | Minimum trust score (0-5) | | location | String | Geo search: lat,long,radiusKm | | available | Boolean | Only parties available for new deals |

Success Response (200):

```
{
  "data": [
    {
      "partyId": "search-result-uuid-1",
      "displayName": "Sunset Valley Farm",
      "partyType": "INDIVIDUAL",
      "roles": ["SUPPLIER"],
      "trustScore": 4.80,
      "verificationStatus": "VERIFIED",
      "location": { "city": "San Jose", "region": "CA" },
      "primaryDomainName": "Agriculture",
      "matchScore": 0.92,
      "resourceSummary": "25 acres farmland, organic certified"
    }
  ],
  "pagination": { /* ... */ }
}
```

6. Deal Management APIs

Service: deal-mgmt-svc Base Path: /api/v1/deals

6.1 Create Deal

Create Deal (Generic Entry Point)

POST /api/v1/deals

Description: Create a new deal. The initiator party and role are determined from the **X-Party-ID** header and request body. Any party type (Supplier, Consumer, or Enhancer) can initiate.

Authorization: Authenticated with **deals:write** scope. **X-Party-ID** header required.

Idempotency: Supported via **Idempotency-Key** header.

Request Body:

```
{
  "dealTitle": "Farm Share Crop Agreement - Summer 2024",
  "dealDescription": "Collaborative farming agreement for organic vegetable production on 10-acre irrigated farmland. Expected yield: 15,000 lbs over 6-month season.",
  "domainCategoryId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "expectedStartDate": "2024-03-01",
  "expectedEndDate": "2024-08-31",
  "currency": "USD",
  "isPublic": true,
}
```

```
"location": {
  "latitude": 37.4419,
  "longitude": -122.1430,
  "addressLine1": "500 Farm Lane",
  "city": "Palo Alto",
  "region": "CA",
  "country": "US",
  "postalCode": "94301"
},
"participations": [
  {
    "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
    "role": "SUPPLIER",
    "isInitiator": true
  },
  {
    "partyId": "consumer-party-uuid-1",
    "role": "CONSUMER",
    "isInitiator": false
  },
  {
    "partyId": "enhancer-party-uuid-1",
    "role": "ENHANCER",
    "isInitiator": false
  }
],
"resource": {
  "resourceName": "Irrigated Farmland - 10 Acres",
  "resourceType": "resource-type-farmland-uuid",
  "description": "Flat, irrigated farmland with drip irrigation system installed. Soil tested and certified organic.",
  "quantity": 10,
  "quantityUnit": "acres",
  "condition": "GOOD",
  "availabilityStart": "2024-03-01",
  "availabilityEnd": "2024-08-31",
  "location": {
    "latitude": 37.4419,
    "longitude": -122.1430,
    "addressLine1": "500 Farm Lane",
    "city": "Palo Alto",
    "region": "CA",
    "country": "US"
  }
},
"need": {
  "needDescription": "Organic mixed vegetables: tomatoes, lettuce, peppers, cucumbers. Total: 15,000 lbs over season.",
  "needCategoryId": "need-category-produce-uuid",
  "requiredQuantity": 15000,
  "quantityUnit": "lbs",
  "qualityRequirements": "USDA Organic certified. Grade A. No pesticides. Traceable
```

```

origin.",
  "requiredByDate": "2024-08-31",
  "maxBudget": 50000.00,
  "budgetCurrency": "USD",
  "priority": "HIGH"
},
"enhancement": {
  "enhancementName": "Full Season Agricultural Support Package",
  "enhancementTypeId": "enh-type-agri-support-uuid",
  "description": "Seed supply, organic fertilizer, pest management, weekly crop
monitoring, harvest coordination.",
  "inputQuantity": 1,
  "quantityUnit": "package",
  "serviceDurationHours": 320,
  "estimatedCompletionDays": 180,
  "deliverables": "Organic seeds, fertilizers, 24 weekly visits, harvest labor, yield report",
  "prerequisites": "Access to water supply, storage facility for inputs"
},
"valueDistribution": {
  "totalValue": 45000.00,
  "currency": "USD",
  "distributionModel": "REVENUE_SHARE",
  "supplierSharePercent": 40.0,
  "consumerCostPercent": 100.0,
  "enhancerSharePercent": 35.0,
  "platformFeePercent": 5.0,
  "paymentSchedule": [
    {
      "milestoneName": "Planting Complete",
      "triggerPercentage": 20.0,
      "dueDate": "2024-03-15"
    },
    {
      "milestoneName": "Mid-Season Harvest",
      "triggerPercentage": 40.0,
      "dueDate": "2024-06-01"
    },
    {
      "milestoneName": "Final Harvest & Delivery",
      "triggerPercentage": 40.0,
      "dueDate": "2024-08-31"
    }
  ]
}
}

```

Validation Rules: - Exactly 3 participations required (one per role: SUPPLIER, CONSUMER, ENHANCER) - **participations** must include the initiator party with **isInitiator: true** - At least one of resource/need/enhancement must be provided (based on initiator role) - **valueDistribution** must pass Win-Win-Win validation (each party $\geq 5\%$, Gini ≤ 0.5) - **expectedEndDate** $\geq$ **expectedStartDate**

Success Response (201):

```
{
  "dealId": "550e8400-e29b-41d4-a716-446655440000",
  "dealReference": "DL-2024-0001",
  "dealTitle": "Farm Share Crop Agreement - Summer 2024",
  "dealDescription": "Collaborative farming agreement for organic vegetable
production...",
  "dealStatus": "DRAFT",
  "domainCategoryId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "domainCategoryName": "Agriculture",
  "initiatorPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "initiatorRole": "SUPPLIER",
  "expectedStartDate": "2024-03-01",
  "expectedEndDate": "2024-08-31",
  "currency": "USD",
  "isPublic": true,
  "location": { /* ... */ },
  "winWinWinValidated": false,
  "participations": [
    {
      "participationId": "part-uuid-1",
      "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
      "partyDisplayName": "Green Acres Farm Ltd",
      "role": "SUPPLIER",
      "participationStatus": "ACCEPTED",
      "isInitiator": true,
      "valueSharePercentage": 40.0,
      "valueShareAmount": 18000.00,
      "invitedAt": null,
      "respondedAt": "2024-01-15T10:30:00Z"
    },
    {
      "participationId": "part-uuid-2",
      "partyId": "consumer-party-uuid-1",
      "partyDisplayName": "FreshMart Grocery Chain",
      "role": "CONSUMER",
      "participationStatus": "INVITED",
      "isInitiator": false,
      "valueSharePercentage": 0.0,
      "valueShareAmount": 0.00,
      "invitedAt": "2024-01-15T10:30:00Z",
      "respondedAt": null
    },
    {
      "participationId": "part-uuid-3",
      "partyId": "enhancer-party-uuid-1",
      "partyDisplayName": "AgriTech Solutions",
      "role": "ENHANCER",
      "participationStatus": "INVITED",
      "isInitiator": false,
      "valueSharePercentage": 35.0,
      "valueShareAmount": 15750.00,
      "invitedAt": "2024-01-15T10:30:00Z",

```

```

    "respondedAt": null
  },
  ],
  "totalDealValue": 45000.00,
  "platformFeePercentage": 5.0,
  "platformFeeAmount": 2250.00,
  "createdAt": "2024-01-15T10:30:00Z",
  "updatedAt": "2024-01-15T10:30:00Z",
  "links": {
    "self": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000",
    "terms": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/terms",
    "negotiation": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/negotiation",
    "valueDistribution": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/value-distribution",
    "milestones": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/milestones",
    "agreement": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/agreement",
    "transitions": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/transitions",
    "submit": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/actions/submit"
  }
}

```

Error Response (422):

```

{
  "type": "https://api.dealplatform.com/errors/win-win-validation-failed",
  "title": "Win-Win-Win Validation Failed",
  "status": 422,
  "detail": "The proposed value distribution does not satisfy the win-win-win principle. Each party must receive at least 5% of total value.",
  "errorCode": "WIN_WIN_VALIDATION_FAILED",
  "validationDetails": {
    "supplierSharePercent": 40.0,
    "consumerCostPercent": 100.0,
    "enhancerSharePercent": 35.0,
    "platformFeePercent": 5.0,
    "issues": [
      {
        "party": "ENHANCER",
        "issue": "Share below minimum threshold",
        "minimum": 5.0,
        "actual": 35.0
      }
    ],
    "giniCoefficient": 0.15,
    "passed": false
  }
}

```

Create Deal - Supplier Initiated
 POST /api/v1/deals/initiate/supplier

Description: Convenience endpoint optimized for supplier-initiated deals. Pre-validates that the initiator has SUPPLIER role.

Authorization: Authenticated with **deals:write** and **X-Party-ID** of a party with SUPPLIER role.

Request Body: Same as generic create, but **resource** is required and the initiator's participation role must be SUPPLIER.

Success Response: Same as generic create.

Create Deal - Consumer Initiated

POST /api/v1/deals/initiate/consumer

Description: Convenience endpoint optimized for consumer-initiated deals. Pre-validates that the initiator has CONSUMER role.

Authorization: Authenticated with **deals:write** and **X-Party-ID** of a party with CONSUMER role.

Request Body: Same as generic create, but **need** is required and the initiator's participation role must be CONSUMER.

Create Deal - Enhancer Initiated

POST /api/v1/deals/initiate/enhancer

Description: Convenience endpoint optimized for enhancer-initiated deals. Pre-validates that the initiator has ENHANCER role.

Authorization: Authenticated with **deals:write** and **X-Party-ID** of a party with ENHANCER role.

Request Body: Same as generic create, but **enhancement** is required and the initiator's participation role must be ENHANCER.

6.2 Get Deal

GET /api/v1/deals/{dealId}

Description: Retrieve a deal with all related data. Field visibility depends on caller's role in the deal (participant, public viewer, admin).

Authorization: Authenticated. Deal participants see full details; public sees limited fields if **isPublic: true**; otherwise 403.

Query Parameters: | Parameter | Type | Description | |-----|-----|-----| | **include** | String[] | Eager load: **participations, terms, valueDistribution, milestones, agreement, resource, need, enhancement, messages, reviews** | | **fields** | String[] | Sparse fieldset for reduced payload |

Success Response (200):

```
{
  "dealId": "550e8400-e29b-41d4-a716-446655440000",
```



```
"dealReference": "DL-2024-0001",
"dealTitle": "Farm Share Crop Agreement - Summer 2024",
"dealDescription": "Collaborative farming agreement for organic vegetable
production...",
"dealStatus": "NEGOTIATING",
"dealStatusDisplay": "Negotiating",
"domainCategoryId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
"domainCategoryName": "Agriculture",
"domainCategoryPath": "Agriculture > Crop Production > Organic Vegetables",
"initiatorPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
"initiatorRole": "SUPPLIER",
"expectedStartDate": "2024-03-01",
"expectedEndDate": "2024-08-31",
"actualStartDate": null,
"actualEndDate": null,
"location": {
  "latitude": 37.4419,
  "longitude": -122.1430,
  "addressLine1": "500 Farm Lane",
  "city": "Palo Alto",
  "region": "CA",
  "country": "US",
  "postalCode": "94301"
},
"currency": "USD",
"totalDealValue": 45000.00,
"platformFeePercentage": 5.0,
"platformFeeAmount": 2250.00,
"winWinWinValidated": true,
"validationCheckedAt": "2024-01-16T08:45:00Z",
"isPublic": true,
"currentStateEnteredAt": "2024-01-18T14:20:00Z",
"availableActions": [
  {
    "action": "PROPOSE_TERM",
    "description": "Propose a new term or counter existing term",
    "allowed": true,
    "requires": ["terms:negotiate"]
  },
  {
    "action": "LOCK_TERMS",
    "description": "Lock all terms when all parties agree",
    "allowed": false,
    "reason": "Not all terms are accepted yet. 2 of 5 terms still under negotiation.",
    "requires": ["terms:negotiate"]
  },
  {
    "action": "WITHDRAW",
    "description": "Withdraw from this deal",
    "allowed": true,
    "requires": ["deals:transition"]
  }
]
```

```

],
"participations": [
  {
    "participationId": "part-uuid-1",
    "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
    "partyDisplayName": "Green Acres Farm Ltd",
    "partyTrustScore": 4.52,
    "role": "SUPPLIER",
    "participationStatus": "ACCEPTED",
    "isInitiator": true,
    "valueSharePercentage": 40.0,
    "valueShareAmount": 18000.00
  },
  {
    "participationId": "part-uuid-2",
    "partyId": "consumer-party-uuid-1",
    "partyDisplayName": "FreshMart Grocery Chain",
    "partyTrustScore": 4.75,
    "role": "CONSUMER",
    "participationStatus": "ACCEPTED",
    "isInitiator": false,
    "valueSharePercentage": 0.0,
    "valueShareAmount": 0.00
  },
  {
    "participationId": "part-uuid-3",
    "partyId": "enhancer-party-uuid-1",
    "partyDisplayName": "AgriTech Solutions",
    "partyTrustScore": 4.30,
    "role": "ENHANCER",
    "participationStatus": "ACCEPTED",
    "isInitiator": false,
    "valueSharePercentage": 35.0,
    "valueShareAmount": 15750.00
  }
],
"resource": {
  "resourceId": "res-uuid-1",
  "resourceName": "Irrigated Farmland - 10 Acres",
  "resourceTypeName": "Farmland",
  "quantity": 10,
  "quantityUnit": "acres",
  "condition": "GOOD",
  "verifiedByPlatform": true
},
"need": {
  "needId": "need-uuid-1",
  "needDescription": "Organic mixed vegetables: tomatoes, lettuce, peppers, cucumbers...",
  "needCategoryName": "Fresh Produce",
  "requiredQuantity": 15000,
  "quantityUnit": "lbs",

```

```
"priority": "HIGH",
},
"enhancement": {
  "enhancementId": "enh-uuid-1",
  "enhancementName": "Full Season Agricultural Support Package",
  "enhancementTypeName": "Agricultural Services",
  "serviceDurationHours": 320,
  "estimatedCompletionDays": 180
},
"valueDistribution": {
  "distributionId": "vd-uuid-1",
  "totalValue": 45000.00,
  "currency": "USD",
  "supplierSharePercent": 40.0,
  "supplierShareAmount": 18000.00,
  "consumerCostPercent": 100.0,
  "consumerCostAmount": 45000.00,
  "enhancerSharePercent": 35.0,
  "enhancerShareAmount": 15750.00,
  "platformFeePercent": 5.0,
  "platformFeeAmount": 2250.00,
  "distributionModel": "REVENUE_SHARE",
  "winWinWinScore": 87.5,
  "paymentSchedule": [ /* ... */ ]
},
"termsSummary": {
  "totalTerms": 5,
  "accepted": 3,
  "proposed": 0,
  "countered": 2,
  "rejected": 0,
  "locked": 0
},
"milestonesSummary": {
  "total": 4,
  "completed": 0,
  "inProgress": 0,
  "pending": 4
},
"createdAt": "2024-01-15T10:30:00Z",
"updatedAt": "2024-01-18T14:20:00Z",
"links": {
  "self": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000",
  "terms": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/terms",
  "negotiation": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/negotiation",
  "valueDistribution": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/value-
distribution",
  "milestones": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/milestones",
  "history": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/history",
  "messages": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/messages"
}
}
```

6.3 List Deals

GET /api/v1/deals

Description: List deals with advanced filtering. Returns deals visible to the caller (participant, public, or admin).

Authorization: Authenticated.

Query Parameters: | Parameter | Type | Description | |-----|-----|-----| | **status** | String[] | Filter by deal status(es) | | **partyId** | UUID | Filter by party involvement | | **myRole** | Enum | Filter by caller's role: **SUPPLIER**, **CONSUMER**, **ENHANCER** | | **domainCategoryId** | UUID | Filter by domain | | **initiatorRole** | Enum | Filter by initiator role | | **fromDate** | ISO 8601 | Deals created after | | **toDate** | ISO 8601 | Deals created before | | **minValue** | Decimal | Minimum deal value | | **maxValue** | Decimal | Maximum deal value | | **currency** | String | Currency filter | | **location** | String | Geo filter: **lat, long, radiusKm** | | **isPublic** | Boolean | Public vs private deals | | **q** | String | Full-text search | | **include** | String[] | Eager load relations | | **limit** | Integer | Page size (default: 20, max: 100) | | **cursor** | String | Pagination cursor | | **sort** | String | Sort field:direction |

Success Response (200):

```
{
  "data": [
    {
      "dealId": "550e8400-e29b-41d4-a716-446655440000",
      "dealReference": "DL-2024-0001",
      "dealTitle": "Farm Share Crop Agreement - Summer 2024",
      "dealStatus": "NEGOTIATING",
      "domainCategoryName": "Agriculture",
      "initiatorRole": "SUPPLIER",
      "participations": [
        {
          "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
          "partyDisplayName": "Green Acres Farm Ltd",
          "role": "SUPPLIER",
          "participationStatus": "ACCEPTED"
        },
        {
          "partyId": "consumer-party-uuid-1",
          "partyDisplayName": "FreshMart Grocery Chain",
          "role": "CONSUMER",
          "participationStatus": "ACCEPTED"
        },
        {
          "partyId": "enhancer-party-uuid-1",
          "partyDisplayName": "AgriTech Solutions",
          "role": "ENHANCER",
          "participationStatus": "ACCEPTED"
        }
      ]
    },
    {
      "totalDealValue": 45000.00,
      "currency": "USD",
    }
  ]
}
```

```
"expectedStartDate": "2024-03-01",  
"expectedEndDate": "2024-08-31",  
"location": { "city": "Palo Alto", "region": "CA" },  
"createdAt": "2024-01-15T10:30:00Z",  
"updatedAt": "2024-01-18T14:20:00Z"  
}  
],  
"pagination": {  
  "limit": 20,  
  "nextCursor":  
    "eyJpZCI6IjQ1MCI6InVwZGF0ZWRBdCI6IjIwMDEtMTUUMTMQ6MjA6MDBaIn0=",  
  "prevCursor": null,  
  "hasMore": true  
},  
"facets": {  
  "status": {  
    "DRAFT": 12,  
    "SUGGESTED": 8,  
    "NEGOTIATING": 25,  
    "TERMS_LOCKED": 5,  
    "EXECUTING": 18,  
    "COMPLETED": 142  
  },  
  "domain": {  
    "Agriculture": 45,  
    "Real Estate": 28,  
    "Transportation": 15  
  }  
}
```

6.4 Update Deal

Full Update

PUT /api/v1/deals/{dealId}

Description: Replace deal metadata. Only allowed in DRAFT and NEGOTIATING states. Cannot modify participations via this endpoint.

Authorization: Deal initiator or admin.

Request Body: Deal object without read-only fields.

Validation: Same as create deal, but participations cannot be modified.

Partial Update

PATCH /api/v1/deals/{dealId}

Description: Partial update of deal metadata. Only allowed in DRAFT and NEGOTIATING states.

Authorization: Deal initiator or admin.

Request Body (JSON Merge Patch):

```
{
  "dealTitle": "Updated Title",
  "expectedStartDate": "2024-03-15",
  "isPublic": false
}
```

6.5 Deal Lifecycle Transitions

Execute State Transition

POST /api/v1/deals/{dealId}/transitions

Description: Execute a state transition on a deal. Validates that the transition is valid from the current state and that the caller has permission.

Authorization: Deal participant with appropriate permission for the transition.

Request Body:

```
{
  "transition": "SUBMIT",
  "reason": "All parties have been identified and initial terms are ready for review",
  "metadata": {
    "submittedByUserId": "user-uuid-1",
    "clientVersion": "web-2.3.1"
  }
}
```

Available Transitions:

Transition	From State	To State	Required Permission	Who Can Trigger
SUBMIT	DRAFT	SUGGESTED	deals:transition	Initiator
ACKNOWLEDGE	SUGGESTED	PENDING_REVIEW	deals:transition	Any invited party
DECLINE_INVITATION	SUGGESTED	CANCELLED	deals:transition	Any invited party
ALL_ACKNOWLEDGED	PENDING_REVIEW	NEGOTIATING	system	System (auto when all ack)
WITHDRAW	*	CANCELLED	deals:transition	Any participant
REQUEST_PAUSE	NEGOTIATING	AWAITING_PARTY	deals:transition	Any participant
RESPOND	AWAITING_PARTY	NEGOTIATING	deals:transition	Requested party
REQUEST_HOLD	NEGOTIATING	ON_HOLD	deals:transition	Any participant
RESUME	ON_HOLD	NEGOTIATING	deals:transition	Party who requested hold
LOCK_TERMS	NEGOTIATING	TERMS_LOCKED	deals:transition	Any participant (all must accept)
REQUEST_RENEGOTIATE	TERMS_LOCKED	NEGOTIATING	deals:transition	Any participant
COMMIT	TERMS_LOCKED	COMMITTED	deals:transition	All parties (each signs)
BEGIN_EXECUTION	COMMITTED	EXECUTING	system	System (auto after 3 days)
BEGIN_EARLY	COMMITTED	EXECUTING	deals:transition	All parties (unanimous)
MILESTONE_COMPLETE	EXECUTING	EXECUTING	system	System (when milestone verified)
ALL_MILESTONES_DONE	EXECUTING	COMPLETED	system	System (auto when all done)
RAISE_DISPUTE	EXECUTING	DISPUTED	deals:transition	Any participant
RESOLVE_DISPUTE	DISPUTED	EXECUTING	system	System (after dispute resolution)
EXPIRE	SUGGESTED	EXPIRED	system	System (auto timeout)

Success Response (200):

```
{
  "dealId": "550e8400-e29b-41d4-a716-446655440000",
  "previousStatus": "DRAFT",
  "newStatus": "SUGGESTED",
  "transition": "SUBMIT",
  "triggeredByPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "reason": "All parties have been identified...",
  "occurredAt": "2024-01-15T14:00:00Z",
  "notificationsSent": [
    {
      "partyId": "consumer-party-uuid-1",
      "channel": "EMAIL",
      "type": "DEAL_INVITE"
    },
    {
      "partyId": "enhancer-party-uuid-1",
      "channel": "EMAIL",
      "type": "DEAL_INVITE"
    }
  ],
  "nextExpectedActions": [
    {
      "partyId": "consumer-party-uuid-1",
      "action": "ACKNOWLEDGE",
      "deadline": "2024-01-29T14:00:00Z"
    },
    {
      "partyId": "enhancer-party-uuid-1",
      "action": "ACKNOWLEDGE",
      "deadline": "2024-01-29T14:00:00Z"
    }
  ],
  "availableActions": [
    {
      "action": "WITHDRAW",
      "description": "Withdraw the deal proposal",
      "allowed": true
    }
  ],
  "links": {
    "self": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000",
    "history": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/history"
  }
}
```

Error Response (422):

```
{
  "type": "https://api.dealplatform.com/errors/invalid-state-transition",
  "title": "Invalid State Transition",
  "status": 422,
}
```

```
"detail": "Transition 'LOCK_TERMS' is not valid from current state 'DRAFT'. Current state must be NEGOTIATING.",
"errorCode": "INVALID_TRANSITION",
"currentStatus": "DRAFT",
"requestedTransition": "LOCK_TERMS",
"validTransitions": [
  {
    "transition": "SUBMIT",
    "toStatus": "SUGGESTED",
    "description": "Submit deal for party review"
  },
  {
    "transition": "ABANDON",
    "toStatus": "CANCELLED",
    "description": "Abandon draft deal"
  }
]
}
```

Get Available Transitions

GET /api/v1/deals/{dealId}/transitions

Description: Get all valid transitions from the current state for the calling party.

Authorization: Deal participant.

Success Response (200):

```
{
  "dealId": "550e8400-e29b-41d4-a716-446655440000",
  "currentStatus": "NEGOTIATING",
  "transitions": [
    {
      "transition": "LOCK_TERMS",
      "toStatus": "TERMS_LOCKED",
      "description": "Lock all terms - requires all parties to accept",
      "allowed": false,
      "reason": "2 of 5 terms still under negotiation",
      "requiredPermission": "deals:transition"
    },
    {
      "transition": "REQUEST_PAUSE",
      "toStatus": "AWAITING_PARTY",
      "description": "Pause negotiation awaiting clarification",
      "allowed": true,
      "requiredPermission": "deals:transition"
    },
    {
      "transition": "WITHDRAW",
      "toStatus": "CANCELLED",
      "description": "Withdraw from deal",
      "allowed": true,
      "requiredPermission": "deals:transition"
    }
  ]
}
```



```
}  
]  
}
```

6.6 Deal History

Get Deal History

GET /api/v1/deals/{dealId}/history

Description: Get full audit trail of all state transitions and significant events.

Authorization: Deal participants and admins.

Query Parameters: | Parameter | Type | Description | |-----|-----|-----| | **eventType** | String[] | Filter: **transition**, **term_change**, **milestone**, **payment**, **message** | | **fromDate** | ISO 8601 | Start date | | **toDate** | ISO 8601 | End date | | **limit** | Integer | Page size | | **cursor** | String | Pagination cursor |

Success Response (200):

```
{  
  "data": [  
    {  
      "eventId": "evt-uuid-1",  
      "eventType": "DEAL_CREATED",  
      "timestamp": "2024-01-15T10:30:00Z",  
      "actorPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",  
      "actorDisplayName": "Green Acres Farm Ltd",  
      "details": {  
        "dealTitle": "Farm Share Crop Agreement - Summer 2024",  
        "initialStatus": "DRAFT",  
        "initiatorRole": "SUPPLIER"  
      }  
    },  
    {  
      "eventId": "evt-uuid-2",  
      "eventType": "STATE_TRANSITION",  
      "timestamp": "2024-01-15T14:00:00Z",  
      "actorPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",  
      "actorDisplayName": "Green Acres Farm Ltd",  
      "details": {  
        "fromStatus": "DRAFT",  
        "toStatus": "SUGGESTED",  
        "transition": "SUBMIT",  
        "reason": "All parties have been identified and initial terms are ready"  
      }  
    },  
    {  
      "eventId": "evt-uuid-3",  
      "eventType": "PARTY_RESPONDED",  
      "timestamp": "2024-01-16T09:30:00Z",  
      "actorPartyId": "consumer-party-uuid-1",  
      "actorDisplayName": "FreshMart Grocery Chain",  
    }  
  ]  
}
```

```

"details": {
  "response": "ACKNOWLEDGED",
  "role": "CONSUMER",
  "notes": "Reviewed and acknowledged. Ready to negotiate terms."
},
{
  "eventId": "evt-uuid-4",
  "eventType": "STATE_TRANSITION",
  "timestamp": "2024-01-17T11:00:00Z",
  "actorPartyId": "system",
  "details": {
    "fromStatus": "SUGGESTED",
    "toStatus": "NEGOTIATING",
    "transition": "ALL_ACKNOWLEDGED",
    "reason": "All parties have acknowledged the deal proposal"
  }
},
{
  "eventId": "evt-uuid-5",
  "eventType": "TERM_PROPOSED",
  "timestamp": "2024-01-17T14:30:00Z",
  "actorPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "actorDisplayName": "Green Acres Farm Ltd",
  "details": {
    "termId": "term-uuid-1",
    "termType": "PAYMENT_SCHEDULE",
    "termName": "Payment Schedule - 3 Installments"
  }
}
],
"pagination": {
  "limit": 20,
  "nextCursor": null,
  "hasMore": false
}
}

```

6.7 Deal Search / Discovery

Search Deals

GET /api/v1/deals/search

Description: Full-text search across deals with relevance scoring.

Authorization: Authenticated. Public deals + deals where caller is a participant.

Query Parameters: | Parameter | Type | Description | |———|———|———| | **q** | String | Search query (searches title, description, domain, location) | | **status** | String[] | Filter by status | | **domainCategoryId** | UUID | Domain filter | | **location** | String | Geo search: lat,long,radiusKm | | **minValue** | Decimal | Minimum deal value | | **maxValue** | Decimal | Maximum deal value | | **roleNeeded** | Enum | Find deals needing this role | | **sort** | String |

relevance, value_desc, value_asc, newest, ending_soon | | limit | Integer | Page size | |
cursor | String | Pagination cursor |

Success Response (200):

```
{
  "data": [
    {
      "dealId": "deal-uuid-1",
      "dealReference": "DL-2024-0001",
      "dealTitle": "Farm Share Crop Agreement - Summer 2024",
      "dealStatus": "NEGOTIATING",
      "domainCategoryName": "Agriculture",
      "location": { "city": "Palo Alto", "region": "CA" },
      "totalDealValue": 45000.00,
      "currency": "USD",
      "relevanceScore": 0.95,
      "matchReasons": [
        "Domain match: Agriculture",
        "Location within 50km of your area",
        "Seeking supplier role that matches your profile"
      ],
      "participations": [
        { "role": "SUPPLIER", "partyDisplayName": "Green Acres Farm", "status": "ACCEPTED" },
        { "role": "CONSUMER", "partyDisplayName": "FreshMart", "status": "ACCEPTED" },
        { "role": "ENHANCER", "status": "INVITED" }
      ],
      "createdAt": "2024-01-15T10:30:00Z"
    }
  ],
  "pagination": { /* ... */ }
}
```

6.8 Deal Cloning

Clone Deal

POST /api/v1/deals/{dealId}/clone

Description: Create a new deal by cloning an existing deal as a template.

Authorization: Deal participant (of source deal).

Request Body:

```
{
  "dealTitle": "Farm Share Crop Agreement - Fall 2024",
  "expectedStartDate": "2024-09-01",
  "expectedEndDate": "2025-02-28",
  "participations": [
    {
      "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
      "role": "SUPPLIER",

```

```

    "isInitiator": true
  },
  ],
  "cloneOptions": {
    "copyTerms": true,
    "copyMilestones": true,
    "copyValueDistribution": true,
    "copyResource": true,
    "copyNeed": true,
    "copyEnhancement": true,
    "adjustDates": true,
    "dateOffsetDays": 180
  }
}

```

Success Response (201): New deal object in DRAFT state.

6.9 Deal Participation Management

Update Participation Response

PATCH /api/v1/deals/{dealId}/participations/{participationId}

Description: Accept, decline, or withdraw from a deal participation.

Authorization: The party whose participation is being updated.

Request Body:

```

{
  "participationStatus": "ACCEPTED",
  "responseNotes": "Excited to collaborate. Ready to negotiate terms."
}

```

Valid Status Values: - **ACCEPTED** - Accept the deal invitation - **DECLINED** - Decline the deal invitation - **WITHDRAWN** - Withdraw from an active deal

Success Response (200):

```

{
  "participationId": "part-uuid-2",
  "partyId": "consumer-party-uuid-1",
  "role": "CONSUMER",
  "participationStatus": "ACCEPTED",
  "respondedAt": "2024-01-16T09:30:00Z",
  "responseNotes": "Excited to collaborate. Ready to negotiate terms.",
  "dealStatus": "SUGGESTED",
  "allPartiesResponded": false,
  "pendingResponses": ["enhancer-party-uuid-1"]
}

```

6.10 Deal Summary / Dashboard

Get Deal Dashboard

GET /api/v1/deals/dashboard

Description: Get summary statistics for the authenticated party's deal activity.

Authorization: Authenticated. Uses **X-Party-ID** header.

Query Parameters: | Parameter | Type | Description | |-----|-----|-----| | **period** | String | 7d, 30d, 90d, 1y, all | | **partyId** | UUID | Override X-Party-ID |

Success Response (200):

```
{
  "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "period": "30d",
  "summary": {
    "totalDeals": 25,
    "byStatus": {
      "DRAFT": 3,
      "SUGGESTED": 2,
      "NEGOTIATING": 5,
      "TERMS_LOCKED": 1,
      "EXECUTING": 4,
      "COMPLETED": 8,
      "CANCELLED": 2
    },
    "byRole": {
      "SUPPLIER": 15,
      "CONSUMER": 8,
      "ENHANCER": 2
    },
    "totalValue": 1250000.00,
    "valueReceived": 480000.00,
    "valuePending": 320000.00,
    "dealsInitiated": 5,
    "dealsJoined": 20,
    "averageDealValue": 50000.00,
    "completionRate": 0.80,
    "averageNegotiationDays": 5.2
  },
  "upcomingMilestones": [
    {
      "milestoneId": "ms-uuid-1",
      "dealId": "deal-uuid-1",
      "dealTitle": "Farm Share Crop Agreement",
      "milestoneName": "Planting Complete",
      "dueDate": "2024-03-15",
      "daysRemaining": 5,
      "status": "IN_PROGRESS"
    }
  ],
  "pendingActions": [
```

```

{
  "dealId": "deal-uuid-2",
  "dealTitle": "Warehouse Renovation Project",
  "action": "REVIEW_TERMS",
  "description": "3 new terms proposed requiring your review",
  "deadline": "2024-01-25T10:00:00Z",
  "priority": "HIGH"
},
"recentActivity": [
  {
    "eventType": "MILESTONE_COMPLETED",
    "dealId": "deal-uuid-1",
    "dealTitle": "Farm Share Crop Agreement",
    "description": "Milestone 'Soil Preparation' completed",
    "timestamp": "2024-01-14T16:30:00Z"
  }
]
}

```

6. Resource, Need & Enhancement APIs

Service: resource-catalog-svc Base Path: /api/v1/resources, /api/v1/needs, /api/v1/enhancements

6.1 Resource APIs (Supplier Assets)

Create Resource

POST /api/v1/resources

Description: Register a resource that a supplier party can contribute to deals.

Authorization: Authenticated with X-Party-ID of a party with SUPPLIER role.

Request Body:

```

{
  "resourceName": "Irrigated Farmland - 10 Acres",
  "resourceType": "resource-type-farmland-uuid",
  "description": "Flat, irrigated farmland with drip irrigation system. Soil tested and certified organic since 2019. Full sun exposure, excellent drainage.",
  "quantity": 10,
  "quantityUnit": "acres",
  "condition": "GOOD",
  "location": {
    "latitude": 37.4419,
    "longitude": -122.1430,
    "addressLine1": "500 Farm Lane",
    "city": "Palo Alto",
    "region": "CA",
    "country": "US",
    "postalCode": "94301"
  }
},

```

```

"availabilityStart": "2024-03-01",
"availabilityEnd": "2024-08-31",
"documentUrls": [
  "https://cdn.dealplatform.com/docs/soil-test-2024.pdf",
  "https://cdn.dealplatform.com/docs/organic-cert.pdf"
],
"metadata": {
  "soilType": "Loam",
  "waterSource": "Municipal + Rainwater Collection",
  "previousCrops": ["Tomatoes", "Lettuce", "Peppers"],
  "irrigationType": "Drip",
  "fencing": true,
  "storageAvailable": "500 sq ft barn"
},
"isActive": true
}

```

Success Response (201):

```

{
  "resourceId": "res-uuid-1",
  "supplierPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "supplierDisplayName": "Green Acres Farm Ltd",
  "resourceName": "Irrigated Farmland - 10 Acres",
  "resourceTypeId": "resource-type-farmland-uuid",
  "resourceTypeName": "Farmland",
  "resourceCategoryPath": "Agriculture > Land > Farmland",
  "description": "Flat, irrigated farmland with drip irrigation system...",
  "quantity": 10,
  "quantityUnit": "acres",
  "condition": "GOOD",
  "location": { /* ... */ },
  "availabilityStart": "2024-03-01",
  "availabilityEnd": "2024-08-31",
  "documentUrls": [ /* ... */ ],
  "metadata": { /* ... */ },
  "verifiedByPlatform": false,
  "isActive": true,
  "dealCount": 0,
  "createdAt": "2024-01-15T10:30:00Z",
  "updatedAt": "2024-01-15T10:30:00Z",
  "links": {
    "self": "/api/v1/resources/res-uuid-1",
    "supplier": "/api/v1/parties/7c9e6679-7425-40de-944b-e07fc1f90ae7",
    "deals": "/api/v1/deals?resourceId=res-uuid-1"
  }
}

```

Get Resource

GET /api/v1/resources/{resourceId}

Authorization: Authenticated.

Query Parameters: | Parameter | Type | Description | | | | | include | String[] | deals, supplier, category |

List Resources

GET /api/v1/resources

Description: List resources with filtering. Public endpoint for discovery.

Query Parameters: | Parameter | Type | Description | | | | | partyId | UUID | Filter by supplier party | | resourceTypeId | UUID | Filter by type | | domainCategoryId | UUID | Filter by domain | | location | String | Geo search: lat,long,radiusKm | | condition | Enum | NEW, GOOD, FAIR, POOR, VARIABLE | | availableFrom | Date | Available from date | | availableTo | Date | Available to date | | minQuantity | Decimal | Minimum quantity | | verified | Boolean | Platform verified only | | q | String | Full-text search | | sort | String | newest, trustScore, nearest, value_desc |

Success Response (200): List of resources with pagination.

Update Resource

PUT /api/v1/resources/{resourceId}

Authorization: Supplier party owner/admin.

Partial Update Resource

PATCH /api/v1/resources/{resourceId}

Authorization: Supplier party owner/admin.

Delete Resource

DELETE /api/v1/resources/{resourceId}

Description: Delete a resource. Fails if resource is in use by an active deal.

Authorization: Supplier party owner/admin.

Get Resource Categories

GET /api/v1/resources/categories

Description: Browse the resource category hierarchy.

Query Parameters: | Parameter | Type | Description | | | | | parentId | UUID | Get children of this category | | depth | Integer | Recursion depth (default: 1) |

Success Response (200):

```
{
  "data": [
    {
      "categoryId": "cat-agriculture",
      "categoryName": "Agriculture",
      "categoryCode": "agriculture",
```



```

    "description": "Agricultural resources including land, crops, livestock",
    "categoryType": "DOMAIN",
    "iconUrl": "https://cdn.../icons/agriculture.svg",
    "childCount": 5,
    "resourceCount": 1280,
    "children": [
      {
        "categoryId": "cat-farmland",
        "categoryName": "Farmland",
        "categoryCode": "farmland",
        "categoryType": "RESOURCE_TYPE",
        "resourceCount": 450
      },
      {
        "categoryId": "cat-greenhouse",
        "categoryName": "Greenhouse Space",
        "categoryCode": "greenhouse",
        "categoryType": "RESOURCE_TYPE",
        "resourceCount": 180
      }
    ],
    {
      "categoryId": "cat-real-estate",
      "categoryName": "Real Estate",
      "categoryCode": "real-estate",
      "description": "Property and real estate assets",
      "categoryType": "DOMAIN",
      "iconUrl": "https://cdn.../icons/real-estate.svg",
      "childCount": 4,
      "resourceCount": 890
    }
  ]
}

```

Search Resources

GET /api/v1/resources/search

Description: Full-text search across resources with relevance scoring.

Query Parameters: | Parameter | Type | Description | |———|———|———| | **q** | String | Search query | | **resourceTypeId** | UUID | Filter by type | | **location** | String | Geo search | | **minQuantity** | Decimal | Minimum quantity | | **sort** | String | **relevance, nearest, newest** |

6.2 Need APIs (Consumer Requirements)

Create Need

POST /api/v1/needs

Description: Register a need that a consumer party wants fulfilled.

Authorization: Authenticated with **X-Party-ID** of a party with CONSUMER role.

Request Body:

```
{
  "needDescription": "Organic mixed vegetables: tomatoes, lettuce, peppers, cucumbers. Total 15,000 lbs over 6-month season.",
  "needCategoryId": "need-category-produce-uuid",
  "requiredQuantity": 15000,
  "quantityUnit": "lbs",
  "qualityRequirements": "USDA Organic certified. Grade A. No pesticides. Traceable origin. Weekly deliveries preferred.",
  "requiredByDate": "2024-08-31",
  "maxBudget": 50000.00,
  "budgetCurrency": "USD",
  "acceptableVariants": "Can substitute with seasonal varieties. Open to mixed root vegetables if yield is comparable.",
  "priority": "HIGH",
  "location": {
    "latitude": 37.4419,
    "longitude": -122.1430,
    "city": "Palo Alto",
    "region": "CA",
    "country": "US"
  },
  "deliveryPreferences": {
    "schedule": "WEEKLY",
    "preferredDay": "TUESDAY",
    "deliveryWindow": "08:00-12:00",
    "deliveryMethod": "SUPPLIER_DELIVERY"
  },
  "isActive": true
}
```

Success Response (201):

```
{
  "needId": "need-uuid-1",
  "consumerPartyId": "consumer-party-uuid-1",
  "consumerDisplayName": "FreshMart Grocery Chain",
  "needDescription": "Organic mixed vegetables: tomatoes, lettuce...",
  "needCategoryId": "need-category-produce-uuid",
  "needCategoryName": "Fresh Produce",
  "requiredQuantity": 15000,
  "quantityUnit": "lbs",
  "qualityRequirements": "USDA Organic certified...",
  "requiredByDate": "2024-08-31",
  "maxBudget": 50000.00,
  "budgetCurrency": "USD",
  "acceptableVariants": "Can substitute with seasonal varieties...",
  "priority": "HIGH",
  "location": { /* ... */ },
  "deliveryPreferences": { /* ... */ },
  "isActive": true,
  "dealCount": 0,
}
```

```
"createdAt": "2024-01-15T10:30:00Z",  
"updatedAt": "2024-01-15T10:30:00Z"  
}
```

Get Need

GET /api/v1/needs/{needId}

Authorization: Authenticated.

List Needs

GET /api/v1/needs

Description: List needs with filtering. Public endpoint for discovery.

Query Parameters: | Parameter | Type | Description | |-----|-----|-----| | **partyId** | UUID | Filter by consumer party | | **needCategoryId** | UUID | Filter by category | | **domainCategoryId** | UUID | Filter by domain | | **location** | String | Geo search | | **minBudget** | Decimal | Minimum budget | | **maxBudget** | Decimal | Maximum budget | | **requiredByDate** | Date | Required by date | | **priority** | Enum | LOW, MEDIUM, HIGH, URGENT | | **q** | String | Full-text search |

Update Need

PUT /api/v1/needs/{needId}

PATCH /api/v1/needs/{needId}

DELETE /api/v1/needs/{needId}

Same patterns as Resource APIs. Authorization: Consumer party owner/admin.

6.3 Enhancement APIs (Enhancer Services)

Create Enhancement

POST /api/v1/enhancements

Description: Register an enhancement/service that an enhancer party can provide.

Authorization: Authenticated with **X-Party-ID** of a party with ENHANCER role.

Request Body:

```
{  
  "enhancementName": "Full Season Agricultural Support Package",  
  "enhancementType": "enh-type-agri-support-uuid",  
  "description": "Complete agricultural support from planting through harvest. Includes  
organic seeds, fertilizer, pest management, weekly monitoring, and harvest labor.",  
  "inputQuantity": 1,  
  "quantityUnit": "package",  
  "serviceDurationHours": 320,  
  "estimatedCompletionDays": 180,  
  "deliverables": "Organic seeds, organic fertilizers, integrated pest management (24  
weekly visits), harvest labor crew (4 people), yield analysis report",  
  "prerequisites": "Access to water supply, storage facility for inputs, clear property  
boundaries",  
}
```

```

"skills": ["Precision Agriculture", "Organic Farming", "IPM", "Soil Science"],
"certifications": [
  {
    "name": "Certified Crop Adviser",
    "issuer": "American Society of Agronomy",
    "issuedDate": "2022-01-01",
    "expiryDate": "2026-01-01"
  }
],
"equipment": ["Soil Testing Kit", "Drone Survey Equipment", "Irrigation Monitoring System"],
"pricing": {
  "model": "PROJECT_BASED",
  "fixedRate": 15750.00,
  "currency": "USD",
  "estimateValidDays": 30
},
"availability": {
  "availableFrom": "2024-03-01",
  "availableTo": "2024-12-31",
  "leadTimeDays": 14,
  "maxConcurrentDeals": 3
},
"serviceArea": {
  "latitude": 37.4419,
  "longitude": -122.1430,
  "radiusKm": 75
},
"isActive": true
}

```

Success Response (201):

```

{
  "enhancementId": "enh-uuid-1",
  "enhancerPartyId": "enhancer-party-uuid-1",
  "enhancerDisplayName": "AgriTech Solutions",
  "enhancementName": "Full Season Agricultural Support Package",
  "enhancementTypeId": "enh-type-agri-support-uuid",
  "enhancementTypeName": "Agricultural Services",
  "description": "Complete agricultural support from planting through harvest...",
  "inputQuantity": 1,
  "quantityUnit": "package",
  "serviceDurationHours": 320,
  "estimatedCompletionDays": 180,
  "deliverables": "Organic seeds, organic fertilizers...",
  "prerequisites": "Access to water supply...",
  "skills": ["Precision Agriculture", "Organic Farming"],
  "certifications": [ /* ... */ ],
  "equipment": ["Soil Testing Kit", "Drone Survey Equipment"],
  "pricing": {
    "model": "PROJECT_BASED",
    "fixedRate": 15750.00,

```

```
"currency": "USD",
},
"availability": { /* ... */ },
"serviceArea": { /* ... */ },
"isActive": true,
"dealCount": 0,
"createdAt": "2024-01-15T10:30:00Z",
"updatedAt": "2024-01-15T10:30:00Z"
}
```

Get Enhancement

GET /api/v1/enhancements/{enhancementId}

Authorization: Authenticated.

List Enhancements

GET /api/v1/enhancements

Query Parameters: | Parameter | Type | Description | |-----|-----|-----| | **partyId** | UUID | Filter by enhancer party | | **enhancementTypeId** | UUID | Filter by type | | **domainCategoryId** | UUID | Filter by domain | | **location** | String | Geo search | | **skills** | String[] | Filter by skills | | **minPrice** | Decimal | Minimum price | | **maxPrice** | Decimal | Maximum price | | **availableFrom** | Date | Available from | | **q** | String | Full-text search |

Update Enhancement

PUT /api/v1/enhancements/{enhancementId}

PATCH /api/v1/enhancements/{enhancementId}

DELETE /api/v1/enhancements/{enhancementId}

Same patterns as Resource APIs. **Authorization:** Enhancer party owner/admin.

6.4 Deal-Bound Resource/Need/Enhancement APIs

Resources, needs, and enhancements can be created as standalone catalog entries OR directly bound to a deal. The following endpoints manage deal-bound instances.

Create Deal-Bound Resource

POST /api/v1/deals/{dealId}/resource

Description: Create or link a resource to a specific deal.

Authorization: Deal participant with SUPPLIER role.

Request Body:

```
{
  "catalogResourceId": "res-uuid-1",
  "overrides": {
    "quantity": 8,
    "availabilityStart": "2024-04-01",
    "notes": "Using only 8 of 10 acres this season"
  }
}
```

```
}  
}
```

OR create a new resource directly (without catalog reference):

```
{  
  "resourceName": "Greenhouse Space - 2000 sq ft",  
  "resourceTypeId": "resource-type-greenhouse-uuid",  
  "quantity": 2000,  
  "quantityUnit": "sqft",  
  "condition": "NEW",  
  "availabilityStart": "2024-03-01",  
  "availabilityEnd": "2024-09-30",  
  "description": "Climate-controlled greenhouse space"  
}
```

Get Deal Resource

GET /api/v1/deals/{dealId}/resource

Authorization: Deal participants.

Update Deal Resource

PATCH /api/v1/deals/{dealId}/resource

Authorization: Deal participant with SUPPLIER role.

Create Deal-Bound Need

POST /api/v1/deals/{dealId}/need

Authorization: Deal participant with CONSUMER role.

Get Deal Need

GET /api/v1/deals/{dealId}/need

Authorization: Deal participants.

Create Deal-Bound Enhancement

POST /api/v1/deals/{dealId}/enhancement

Authorization: Deal participant with ENHANCER role.

Get Deal Enhancement

GET /api/v1/deals/{dealId}/enhancement

Authorization: Deal participants.

6. Negotiation APIs

Service: negotiation-svc Base Path: /api/v1/deals/{dealId}

6.1 Negotiation Overview

Get Negotiation Thread

GET /api/v1/deals/{dealId}/negotiation

Description: Get the full negotiation context including all terms, messages, current status per dimension, and available actions for the calling party.

Authorization: Deal participants.

Query Parameters: | Parameter | Type | Description | |-----|-----|-----| | **include** | String[] | **terms, messages, valueDistribution, history** | | **termStatus** | String[] | Filter terms by status | | **termType** | String[] | Filter terms by type |

Success Response (200):

```
{
  "dealId": "550e8400-e29b-41d4-a716-446655440000",
  "dealStatus": "NEGOTIATING",
  "negotiationStatus": "ACTIVE",
  "negotiationStartedAt": "2024-01-17T11:00:00Z",
  "negotiationTimeoutAt": "2024-02-16T11:00:00Z",
  "daysRemaining": 25,
  "negotiationMode": "SEQUENTIAL",
  "dimensions": [
    {
      "dimension": "VALUE_DISTRIBUTION",
      "status": "LOCKED",
      "lockedAt": "2024-01-18T10:00:00Z",
      "lockedByAllAt": "2024-01-18T10:00:00Z",
      "roundCount": 2,
      "currentProposal": {
        "proposalId": "prop-vd-2",
        "proposedByPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
        "proposedAt": "2024-01-18T09:00:00Z",
        "acceptedBy": ["consumer-party-uuid-1", "enhancer-party-uuid-1"],
        "content": { /* value distribution */ }
      }
    },
    {
      "dimension": "DELIVERABLES",
      "status": "COUNTERED",
      "roundCount": 3,
      "currentProposal": {
        "proposalId": "prop-del-3",
        "proposedByPartyId": "enhancer-party-uuid-1",
        "proposedAt": "2024-01-19T14:30:00Z",
        "acceptedBy": [],
        "pendingReviewBy": ["consumer-party-uuid-1", "7c9e6679-7425-40de-944b-e07fc1f90ae7"]
      }
    },
    {
      "dimension": "TIMELINE",
```

```
"status": "PROPOSED",
"roundCount": 1,
"currentProposal": {
  "proposalId": "prop-tl-1",
  "proposedByPartyId": "consumer-party-uuid-1",
  "proposedAt": "2024-01-19T16:00:00Z",
  "acceptedBy": [],
  "pendingReviewBy": ["7c9e6679-7425-40de-944b-e07fc1f90ae7", "enhancer-party-
uuid-1"]
},
},
{
  "dimension": "RISK_ALLOCATION",
  "status": "OPEN",
  "roundCount": 0,
  "currentProposal": null
},
{
  "dimension": "TERMINATION",
  "status": "OPEN",
  "roundCount": 0,
  "currentProposal": null
}
],
"terms": [
  /* See term listing below */
],
"valueDistribution": {
  /* Current value distribution proposal */
},
"recentMessages": [
  {
    "messageId": "msg-uuid-1",
    "senderPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
    "senderDisplayName": "Green Acres Farm Ltd",
    "content": "I've updated the payment schedule. Please review and let me know if the
milestone dates work for everyone.",
    "messageType": "TEXT",
    "createdAt": "2024-01-19T10:00:00Z"
  }
],
"availableActions": [
  {
    "action": "PROPOSE_TERM",
    "allowed": true,
    "description": "Propose a new term or counter existing term"
  },
  {
    "action": "ACCEPT_TERM",
    "allowed": true,
    "targetTermIds": ["term-vd-2"],
    "description": "Accept pending terms"
  }
]
```



```

    },
    {
      "action": "COUNTER_TERM",
      "allowed": true,
      "targetTermIds": ["term-del-3", "term-tl-1"],
      "description": "Counter pending terms"
    },
    {
      "action": "REQUEST_INFO",
      "allowed": true,
      "description": "Request clarification from another party"
    },
    {
      "action": "LOCK_TERMS",
      "allowed": false,
      "reason": "Only 1 of 5 dimensions locked. 4 dimensions still open or under negotiation.",
      "requiredDimensionsLocked": 5,
      "currentDimensionsLocked": 1
    }
  ],
  "winWinWinValidation": {
    "lastChecked": "2024-01-19T17:00:00Z",
    "status": "PENDING",
    "score": null,
    "message": "Will be validated when all terms are locked"
  },
  "statistics": {
    "totalTerms": 5,
    "totalProposals": 6,
    "totalMessages": 23,
    "averageResponseTimeHours": 8.5,
    "mostActiveParty": "7c9e6679-7425-40de-944b-e07fc1f90ae7"
  }
}

```

6.2 Term Management

List Terms

GET /api/v1/deals/{dealId}/terms

Description: Get all terms for a deal, with their negotiation status.

Authorization: Deal participants.

Query Parameters:

Parameter	Type	Description
termType	String[]	Filter by term type
status	String[]	Filter by negotiation status
proposedBy	UUID	Filter by proposing party
sort	String	newest, oldest, status

Success Response (200):

```

{
  "data": [

```

```
{
  "termId": "term-vd-1",
  "termType": "VALUE_DISTRIBUTION",
  "termName": "Revenue Share Split",
  "description": "Revenue distribution: 40% Supplier, 35% Enhancer, 25% surplus pool",
  "proposedByPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "proposedByDisplayName": "Green Acres Farm Ltd",
  "negotiationStatus": "ACCEPTED",
  "version": 1,
  "parentTermId": null,
  "isMandatory": false,
  "proposedAt": "2024-01-17T14:30:00Z",
  "resolvedAt": "2024-01-18T10:00:00Z",
  "resolution": "Accepted by all parties. Consumer approved cost structure.",
  "responses": [
    {
      "partyId": "consumer-party-uuid-1",
      "displayName": "FreshMart",
      "response": "ACCEPTED",
      "respondedAt": "2024-01-18T08:00:00Z",
      "comment": "Budget works for us."
    },
    {
      "partyId": "enhancer-party-uuid-1",
      "displayName": "AgriTech",
      "response": "ACCEPTED",
      "respondedAt": "2024-01-18T10:00:00Z",
      "comment": "Fair split. Ready to proceed."
    }
  ],
},
{
  "termId": "term-del-3",
  "termType": "QUALITY",
  "termName": "Quality Standards and Inspection",
  "description": "Weekly quality inspections by certified inspector. USDA Organic certification maintained throughout. Grade A output required (minimum 85% Grade A).",
  "proposedByPartyId": "enhancer-party-uuid-1",
  "proposedByDisplayName": "AgriTech Solutions",
  "negotiationStatus": "COUNTERED",
  "version": 3,
  "parentTermId": "term-del-2",
  "isMandatory": false,
  "proposedAt": "2024-01-19T14:30:00Z",
  "resolvedAt": null,
  "responses": [
    {
      "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
      "displayName": "Green Acres Farm",
      "response": "PENDING",
      "respondedAt": null,
    }
  ]
}
```

```

        "comment": null
      },
      {
        "partyId": "consumer-party-uuid-1",
        "displayName": "FreshMart",
        "response": "PENDING",
        "respondedAt": null,
        "comment": null
      }
    ]
  },
  "pagination": { /* ... */ },
  "summary": {
    "total": 5,
    "byStatus": {
      "PROPOSED": 1,
      "UNDER_REVIEW": 0,
      "ACCEPTED": 2,
      "REJECTED": 0,
      "COUNTERED": 2
    }
  }
}

```

Propose Term

POST /api/v1/deals/{dealId}/terms

Description: Propose a new term or counter-proposal. If **parentTermId** is provided, this is a counter-proposal.

Authorization: Deal participants with **terms:negotiate** permission.

Request Body:

```

{
  "termType": "PAYMENT_SCHEDULE",
  "termName": "3-Stage Payment with Escrow",
  "description": "Payment released in 3 stages via platform escrow: (1) 20% on planting completion, (2) 40% on mid-season harvest, (3) 40% on final delivery. Each release requires all-party confirmation.",
  "parentTermId": null,
  "isMandatory": false,
  "metadata": {
    "stages": [
      {
        "stage": 1,
        "percentage": 20,
        "trigger": "Planting complete - all seeds in ground",
        "estimatedDate": "2024-03-15"
      },
      {
        "stage": 2,

```

```

    "percentage": 40,
    "trigger": "Mid-season harvest - 50% of yield delivered",
    "estimatedDate": "2024-06-01"
  },
  {
    "stage": 3,
    "percentage": 40,
    "trigger": "Final harvest - 100% yield delivered and inspected",
    "estimatedDate": "2024-08-31"
  }
],
"escrowRequired": true,
"penaltyLateDelivery": "1% per week overdue",
"earlyCompletionBonus": "2% of stage value"
}
}

```

Success Response (201):

```

{
  "termId": "term-uuid-new",
  "termType": "PAYMENT_SCHEDULE",
  "termName": "3-Stage Payment with Escrow",
  "description": "Payment released in 3 stages via platform escrow...",
  "proposedByPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "proposedByDisplayName": "Green Acres Farm Ltd",
  "negotiationStatus": "PROPOSED",
  "version": 1,
  "parentTermId": null,
  "isMandatory": false,
  "proposedAt": "2024-01-20T09:00:00Z",
  "pendingReviewBy": ["consumer-party-uuid-1", "enhancer-party-uuid-1"],
  "responses": [
    {
      "partyId": "consumer-party-uuid-1",
      "response": "PENDING",
      "deadline": "2024-01-23T09:00:00Z"
    },
    {
      "partyId": "enhancer-party-uuid-1",
      "response": "PENDING",
      "deadline": "2024-01-23T09:00:00Z"
    }
  ],
  "links": {
    "self": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/terms/term-uuid-new",
    "accept": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/terms/term-uuid-new/actions/accept",
    "counter": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/terms/term-uuid-new/actions/counter",
    "reject": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/terms/term-uuid-new/actions/reject"
  }
}

```

```
}  
}
```

Get Term

GET /api/v1/deals/{dealId}/terms/{termId}

Authorization: Deal participants.

Success Response (200): Full term object with version history.

Accept Term

POST /api/v1/deals/{dealId}/terms/{termId}/actions/accept

Description: Accept a proposed term. When all parties accept, the term status changes to ACCEPTED.

Authorization: Deal participant with **terms:negotiate** permission.

Request Body:

```
{  
  "comment": "This payment schedule works well for our cash flow. Approved.",  
  "metadata": {  
    "reviewedByUserId": "user-uuid-1"  
  }  
}
```

Success Response (200):

```
{  
  "termId": "term-uuid-new",  
  "negotiationStatus": "ACCEPTED",  
  "yourResponse": "ACCEPTED",  
  "allPartiesAccepted": true,  
  "termLocked": true,  
  "responses": [  
    {  
      "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",  
      "response": "ACCEPTED",  
      "isProposer": true  
    },  
    {  
      "partyId": "consumer-party-uuid-1",  
      "response": "ACCEPTED",  
      "respondedAt": "2024-01-21T10:00:00Z",  
      "comment": "Works for us too."  
    },  
    {  
      "partyId": "enhancer-party-uuid-1",  
      "response": "ACCEPTED",  
      "respondedAt": "2024-01-21T14:30:00Z",  
      "comment": "Confirmed. Escrow process is clear."  
    }  
  ]  
}
```

```
],  
  "message": "All parties have accepted this term. Term is now locked."  
}
```

Counter Term

POST /api/v1/deals/{dealId}/terms/{termId}/actions/counter

Description: Counter a proposed term with a modified version. Creates a new term with `parentTermId` referencing the original.

Authorization: Deal participant with `terms:negotiate` permission.

Request Body:

```
{  
  "termType": "PAYMENT_SCHEDULE",  
  "termName": "3-Stage Payment with Escrow - Counter Proposal",  
  "description": "Counter-proposal: Payment released in 3 stages: (1) 15% on planting, (2) 35% on mid-season, (3) 50% on final delivery. Requesting lower upfront to align with our seasonal revenue pattern.",  
  "isMandatory": false,  
  "metadata": {  
    "stages": [  
      { "stage": 1, "percentage": 15, "trigger": "Planting complete", "estimatedDate": "2024-03-15" },  
      { "stage": 2, "percentage": 35, "trigger": "Mid-season harvest", "estimatedDate": "2024-06-01" },  
      { "stage": 3, "percentage": 50, "trigger": "Final delivery", "estimatedDate": "2024-08-31" }  
    ],  
    "escrowRequired": true,  
    "counterReason": "Seasonal revenue pattern requires lower upfront payment"  
  },  
  "comment": "We'd prefer a 15/35/50 split due to our seasonal revenue pattern. The total amount remains the same."  
}
```

Success Response (201):

```
{  
  "termId": "term-counter-uuid",  
  "parentTermId": "term-uuid-new",  
  "termType": "PAYMENT_SCHEDULE",  
  "termName": "3-Stage Payment with Escrow - Counter Proposal",  
  "negotiationStatus": "PROPOSED",  
  "version": 2,  
  "proposedAt": "2024-01-22T11:00:00Z",  
  "pendingReviewBy": ["7c9e6679-7425-40de-944b-e07fc1f90ae7", "consumer-party-uuid-1"],  
  "previousTermStatus": "COUNTERED",  
  "round": 2,  
  "maxRoundsWarning": "Round 2 of 5. Approaching negotiation limit."  
}
```

Reject Term

POST /api/v1/deals/{dealId}/terms/{termId}/actions/reject

Description: Reject a proposed term. A rejected term can still be countered or a new term proposed.

Authorization: Deal participant with **terms:negotiate** permission.

Request Body:

```
{
  "reason": "REJECTED",
  "comment": "The upfront payment is too high for our seasonal cash flow. We'd need max 15% upfront.",
  "suggestedAlternative": "Please consider a 15/35/50 split instead."
}
```

Success Response (200):

```
{
  "termId": "term-uuid-new",
  "negotiationStatus": "REJECTED",
  "yourResponse": "REJECTED",
  "allPartiesResponded": false,
  "responses": [
    {
      "partyId": "consumer-party-uuid-1",
      "response": "REJECTED",
      "comment": "Upfront too high. Need max 15% upfront."
    },
    {
      "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
      "response": "PENDING"
    },
    {
      "partyId": "enhancer-party-uuid-1",
      "response": "PENDING"
    }
  ],
  "termCanBeReproposed": true
}
```

Get Term Version History

GET /api/v1/deals/{dealId}/terms/{termId}/history

Authorization: Deal participants.

Success Response (200):

```
{
  "data": [
    {
      "version": 1,

```

```

    "termId": "term-vd-1",
    "termName": "Revenue Share Split v1",
    "description": "Original: 45% Supplier, 30% Enhancer, 25% surplus",
    "proposedBy": "Green Acres Farm",
    "proposedAt": "2024-01-17T14:30:00Z",
    "status": "COUNTERED",
    "counteredBy": "AgriTech Solutions",
    "counteredAt": "2024-01-17T16:00:00Z"
  },
  {
    "version": 2,
    "termId": "term-vd-2",
    "termName": "Revenue Share Split v2",
    "description": "Revised: 40% Supplier, 35% Enhancer, 25% surplus",
    "proposedBy": "Green Acres Farm",
    "proposedAt": "2024-01-18T09:00:00Z",
    "status": "ACCEPTED",
    "acceptedAt": "2024-01-18T10:00:00Z"
  }
]
}

```

6.3 Value Distribution APIs

Get Value Distribution

GET /api/v1/deals/{dealId}/value-distribution

Authorization: Deal participants.

Success Response (200):

```

{
  "distributionId": "vd-uuid-1",
  "dealId": "550e8400-e29b-41d4-a716-446655440000",
  "totalValue": 45000.00,
  "currency": "USD",
  "distributionModel": "REVENUE_SHARE",
  "parties": [
    {
      "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
      "role": "SUPPLIER",
      "displayName": "Green Acres Farm Ltd",
      "sharePercent": 40.0,
      "shareAmount": 18000.00,
      "adjustmentNotes": "Base revenue share for land and water resource provision"
    },
    {
      "partyId": "consumer-party-uuid-1",
      "role": "CONSUMER",
      "displayName": "FreshMart Grocery Chain",
      "costPercent": 100.0,
      "costAmount": 45000.00,
      "adjustmentNotes": "Total cost including all shares and platform fee"
    }
  ]
}

```



```
},
{
  "partyId": "enhancer-party-uuid-1",
  "role": "ENHANCER",
  "displayName": "AgriTech Solutions",
  "sharePercent": 35.0,
  "shareAmount": 15750.00,
  "adjustmentNotes": "Revenue share for full-season agricultural support services"
},
],
"platformFee": {
  "feePercent": 5.0,
  "feeAmount": 2250.00,
  "calculationMethod": "PERCENTAGE_OF_TOTAL"
},
"paymentSchedule": [
  {
    "milestoneName": "Planting Complete",
    "triggerPercentage": 20.0,
    "triggerAmount": 9000.00,
    "dueDate": "2024-03-15",
    "status": "PENDING"
  },
  {
    "milestoneName": "Mid-Season Harvest",
    "triggerPercentage": 40.0,
    "triggerAmount": 18000.00,
    "dueDate": "2024-06-01",
    "status": "PENDING"
  },
  {
    "milestoneName": "Final Harvest & Delivery",
    "triggerPercentage": 40.0,
    "triggerAmount": 18000.00,
    "dueDate": "2024-08-31",
    "status": "PENDING"
  }
],
"winWinWinScore": 87.5,
"winWinWinAnalysis": {
  "economicCheck": "PASSED",
  "balanceCheck": "PASSED",
  "feasibilityCheck": "PASSED",
  "riskCheck": "PASSED",
  "complianceCheck": "PASSED",
  "giniCoefficient": 0.15,
  "minPartySharePercent": 35.0,
  "maxPartySharePercent": 40.0
},
"isLocked": true,
"lockedAt": "2024-01-18T10:00:00Z",
"createdAt": "2024-01-15T10:30:00Z",
```

```
"updatedAt": "2024-01-18T10:00:00Z"
}
```

Propose Value Distribution

POST /api/v1/deals/{dealId}/value-distribution/proposals

Description: Propose or counter-propose a value distribution. Follows the same negotiation pattern as terms.

Authorization: Deal participants.

Request Body:

```
{
  "totalValue": 45000.00,
  "currency": "USD",
  "distributionModel": "REVENUE_SHARE",
  "parties": [
    {
      "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
      "role": "SUPPLIER",
      "sharePercent": 40.0
    },
    {
      "partyId": "consumer-party-uuid-1",
      "role": "CONSUMER",
      "costPercent": 100.0
    },
    {
      "partyId": "enhancer-party-uuid-1",
      "role": "ENHANCER",
      "sharePercent": 35.0
    }
  ],
  "platformFeePercent": 5.0,
  "paymentSchedule": [
    { "milestoneName": "Planting Complete", "triggerPercentage": 20.0, "dueDate": "2024-03-15" },
    { "milestoneName": "Mid-Season Harvest", "triggerPercentage": 40.0, "dueDate": "2024-06-01" },
    { "milestoneName": "Final Harvest & Delivery", "triggerPercentage": 40.0, "dueDate": "2024-08-31" }
  ],
  "comment": "Proposing 40/35 split between supplier and enhancer. This reflects the significant time investment from AgriTech and the prime farmland location from Green Acres."
}
```

Validation: Must pass Win-Win-Win validation (each party $\geq 5\%$, Gini ≤ 0.5).

Success Response (201): Proposal object with validation results.

Accept Value Distribution

POST /api/v1/deals/{dealId}/value-distribution/proposals/{proposalId}/accept

Authorization: Deal participant.

Lock Terms

POST /api/v1/deals/{dealId}/actions/lock-terms

Description: Lock all terms when all parties agree on all dimensions. Moves deal to TERMS_LOCKED state.

Authorization: All parties must have accepted all terms. Any party can trigger after unanimous acceptance.

Request Body:

```
{
  "confirmation": true,
  "comment": "All terms reviewed and accepted. Ready to proceed to commitment."
}
```

Success Response (200):

```
{
  "dealId": "550e8400-e29b-41d4-a716-446655440000",
  "previousStatus": "NEGOTIATING",
  "newStatus": "TERMS_LOCKED",
  "allTermsLocked": true,
  "termsLockedAt": "2024-01-20T16:00:00Z",
  "termsLockedBy": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "winWinWinValidation": {
    "status": "PASSED",
    "score": 87.5,
    "checkedAt": "2024-01-20T16:00:01Z"
  },
  "nextSteps": [
    {
      "step": "Generate commitment agreement from locked terms",
      "automatic": true,
      "estimatedTime": "2 minutes"
    },
    {
      "step": "All parties digitally sign commitment",
      "automatic": false,
      "deadline": "2024-02-03T16:00:00Z"
    },
    {
      "step": "Fund escrow accounts",
      "automatic": false,
      "deadline": "2024-02-03T16:00:00Z"
    }
  ],
  "agreementPreviewUrl":
}
```

```
"/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/agreement/preview",
  "links": {
    "agreement": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/agreement",
    "commit": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/actions/commit"
  }
}
```

Unlock Terms (Request Renegotiation)

POST /api/v1/deals/{dealId}/actions/unlock-terms

Description: Request to re-open terms for renegotiation. Only valid in TERMS_LOCKED state.

Authorization: Any participant can request. Must be approved by all other parties.

Request Body:

```
{
  "reason": "DISCOVERED_NEW_REQUIREMENT",
  "description": "Consumer has requested additional organic certification (Non-GMO Project) that was not originally scoped. This affects the enhancer's cost structure.",
  "termsToUnlock": ["QUALITY"],
  "impactEstimate": "Estimated cost increase of $2,000-$3,000 for additional certification"
}
```

Success Response (202):

```
{
  "requestId": "unlock-req-uuid",
  "status": "PENDING_APPROVAL",
  "requestedBy": "enhancer-party-uuid-1",
  "approvalsRequired": 2,
  "approvalsReceived": 0,
  "deadline": "2024-01-25T10:00:00Z",
  "message": "Unlock request sent to other parties for approval. All parties must approve to re-open terms."
}
```

6.4 Negotiation Messages

List Negotiation Messages

GET /api/v1/deals/{dealId}/negotiation/messages

Authorization: Deal participants.

Query Parameters:

Parameter	Type	Description
messageType	String[]	TEXT, FILE, SYSTEM, OFFER, COUNTER_OFFER
fromPartyId	UUID	Filter by sender
termId	UUID	Messages related to specific term
limit	Integer	Page size

Send Negotiation Message

POST /api/v1/deals/{dealId}/negotiation/messages

Authorization: Deal participants.

Request Body:

```
{
  "content": "I've reviewed the counter-proposal on payment terms. The 15% upfront works for us if we can agree on a 2-week grace period for the first milestone payment.",
  "messageType": "COUNTER_OFFER",
  "relatedTermId": "term-counter-uuid",
  "attachmentUrls": []
}
```

6.5 Request Information / Clarification

Request Information

POST /api/v1/deals/{dealId}/negotiation/info-requests

Description: Request clarification or additional information from another party.

Authorization: Deal participants.

Request Body:

```
{
  "targetPartyId": "enhancer-party-uuid-1",
  "subject": "Clarification on pest management scope",
  "question": "Does the pest management service include organic certification compliance documentation? We need this for our retail compliance.",
  "relatedTermId": "term-del-3",
  "responseDeadline": "2024-01-23T10:00:00Z"
}
```

Success Response (201):

```
{
  "requestId": "info-req-uuid",
  "targetPartyId": "enhancer-party-uuid-1",
  "status": "PENDING_RESPONSE",
  "subject": "Clarification on pest management scope",
  "responseDeadline": "2024-01-23T10:00:00Z",
  "createdAt": "2024-01-21T09:00:00Z"
}
```

Respond to Information Request

POST /api/v1/deals/{dealId}/negotiation/info-requests/{requestId}/respond

Authorization: Target party of the info request.

Request Body:

```
{
  "response": "Yes, our Integrated Pest Management (IPM) service includes full organic certification compliance documentation. We provide monthly compliance reports that satisfy USDA Organic and Non-GMO Project requirements.",
}
```

```
"attachments": ["https://cdn.../sample-compliance-report.pdf"]
}
```

6. Agreement APIs

Service: agreement-contract-svc Base Path: /api/v1/deals/{dealId}/agreement

6.1 Generate Agreement

Generate from Locked Terms

POST /api/v1/deals/{dealId}/agreement/generate

Description: Generate a formal agreement document from locked terms. Auto-triggered when terms are locked, but can also be manually initiated. Creates the agreement in **DRAFT** status.

Authorization: Deal participants.

Request Body:

```
{
  "templateId": "template-standard-agriculture-uuid",
  "governingLaw": "California, United States",
  "disputeResolution": "MEDIATION",
  "language": "en",
  "additionalClauses": [
    {
      "title": "Force Majeure - Drought Exception",
      "text": "In the event of severe drought conditions as classified by the USDA Drought Monitor at D3 (Extreme) or higher affecting the farm location, the supplier may request a 60-day extension on all milestone deadlines without penalty."
    }
  ],
  "customizations": {
    "insuranceRequirements": "General liability minimum $1M per occurrence",
    "confidentialityLevel": "STANDARD",
    "terminationNoticeDays": 30
  }
}
```

Success Response (201):

```
{
  "agreementId": "agr-uuid-1",
  "dealId": "550e8400-e29b-41d4-a716-446655440000",
  "agreementStatus": "DRAFT",
  "agreementReference": "AGR-2024-0001",
  "agreementText": "FULL AGREEMENT TEXT... [truncated for brevity] ...",
  "agreementTextUrl": "https://cdn.dealplatform.com/agreements/agr-uuid-1.pdf",
  "governingLaw": "California, United States",
  "disputeResolution": "MEDIATION",
  "effectiveDate": null,
  "terminationDate": null,
  "autoRenew": false,
  "version": 1,
}
```

```
"templateUsed": "Standard Agriculture Agreement",
"generatedFromTerms": true,
"parties": [
  {
    "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
    "displayName": "Green Acres Farm Ltd",
    "role": "SUPPLIER",
    "signatureStatus": "PENDING",
    "signatureRequired": true
  },
  {
    "partyId": "consumer-party-uuid-1",
    "displayName": "FreshMart Grocery Chain",
    "role": "CONSUMER",
    "signatureStatus": "PENDING",
    "signatureRequired": true
  },
  {
    "partyId": "enhancer-party-uuid-1",
    "displayName": "AgriTech Solutions",
    "role": "ENHANCER",
    "signatureStatus": "PENDING",
    "signatureRequired": true
  }
],
"includedTerms": [
  {
    "termId": "term-vd-2",
    "termType": "VALUE_DISTRIBUTION",
    "termName": "Revenue Share Split",
    "includedAt": "2024-01-20T16:02:00Z"
  },
  {
    "termId": "term-ps-3",
    "termType": "PAYMENT_SCHEDULE",
    "termName": "3-Stage Payment with Escrow",
    "includedAt": "2024-01-20T16:02:00Z"
  }
],
"milestones": [
  {
    "milestoneName": "Planting Complete",
    "dueDate": "2024-03-15",
    "paymentTriggerAmount": 9000.00
  },
  {
    "milestoneName": "Mid-Season Harvest",
    "dueDate": "2024-06-01",
    "paymentTriggerAmount": 18000.00
  },
  {
    "milestoneName": "Final Harvest & Delivery",
```

```

    "dueDate": "2024-08-31",
    "paymentTriggerAmount": 18000.00
  },
  ],
  "generatedAt": "2024-01-20T16:02:00Z",
  "createdAt": "2024-01-20T16:02:00Z",
  "links": {
    "self": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/agreement",
    "preview":
"/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/agreement/preview",
    "download":
"/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/agreement/download",
    "sign": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/agreement/sign"
  }
}

```

6.2 Agreement Preview

Preview Agreement

GET /api/v1/deals/{dealId}/agreement/preview

Description: Get a rendered preview of the agreement (HTML or PDF) before signing.

Authorization: Deal participants.

Query Parameters: | Parameter | Type | Description | |———|———|———| | **format** |
String | **html** (default) or **pdf** |

Success Response (200): - **format=html:** Returns **text/html** with rendered agreement -
format=pdf: Returns **application/pdf** with binary PDF content

6.3 Sign Agreement

Sign Agreement

POST /api/v1/deals/{dealId}/agreement/sign

Description: Sign the agreement on behalf of the acting party (from **X-Party-ID** header).
Supports multiple signature types.

Authorization: Deal participant. Must be an authorized representative of the party
(owner/admin/member with signing permission).

Request Body:

```

{
  "signatureType": "E_SIGN",
  "confirmation": true,
  "legalAcknowledgment": "I have read and agree to all terms of this agreement. I am
authorized to sign on behalf of my party.",
  "metadata": {
    "ipAddress": "203.0.113.45",
    "deviceInfo": "Chrome 120, macOS",
    "timestampUtc": "2024-01-21T09:30:00Z"
  }
}

```



```
}  
}
```

Signature Types: | Type | Description | Security Level | |-----|-----|-----| |
E_SIGN | Electronic signature via platform | Standard | | **DIGITAL** | PKI-based digital
signature | High | | **BIOMETRIC** | Biometric-verified signature | Very High | |
MANUAL_UPLOAD | Uploaded scanned signed document | Low (requires verification) |

Success Response (200):

```
{  
  "signatureId": "sig-uuid-1",  
  "agreementId": "agr-uuid-1",  
  "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",  
  "partyDisplayName": "Green Acres Farm Ltd",  
  "signedByUserId": "user-uuid-1",  
  "signatureType": "E_SIGN",  
  "signatureData": "eyJhbGciOiJSUzI1NiIs...base64-encoded-signature...",  
  "ipAddress": "203.0.113.45",  
  "signedAt": "2024-01-21T09:30:00Z",  
  "allPartiesSigned": false,  
  "signaturesReceived": 1,  
  "signaturesRequired": 3,  
  "pendingSignatures": [  
    {  
      "partyId": "consumer-party-uuid-1",  
      "displayName": "FreshMart Grocery Chain",  
      "status": "PENDING",  
      "notifiedAt": "2024-01-21T09:30:05Z"  
    },  
    {  
      "partyId": "enhancer-party-uuid-1",  
      "displayName": "AgriTech Solutions",  
      "status": "PENDING",  
      "notifiedAt": "2024-01-21T09:30:05Z"  
    }  
  ],  
  "nextDeadline": "2024-02-03T16:00:00Z",  
  "message": "Signature recorded. Awaiting 2 more signatures."  
}
```

Success Response (200) — When Last Party Signs (All Signed):

```
{  
  "signatureId": "sig-uuid-3",  
  "agreementId": "agr-uuid-1",  
  "partyId": "enhancer-party-uuid-1",  
  "signedByUserId": "user-uuid-3",  
  "signatureType": "E_SIGN",  
  "signedAt": "2024-01-22T14:00:00Z",  
  "allPartiesSigned": true,  
  "signaturesReceived": 3,  
  "signaturesRequired": 3,  
}
```

```
"agreementStatus": "FULLY_SIGNED",
"dealStatus": "COMMITTED",
"dealTransition": {
  "fromStatus": "TERMS_LOCKED",
  "toStatus": "COMMITTED",
  "transition": "COMMIT",
  "occurredAt": "2024-01-22T14:00:01Z"
},
"effectiveDate": "2024-01-25T00:00:00Z",
"executionStartsAt": "2024-01-25T00:00:00Z",
"message": "All parties have signed. Agreement is now binding. Execution phase begins in 3 days (2024-01-25). Escrow funding required before execution starts.",
"nextActions": [
  {
    "action": "FUND_ESCROW",
    "description": "Fund escrow account for milestone payments",
    "deadline": "2024-01-24T23:59:59Z",
    "amount": 45000.00,
    "currency": "USD"
  }
]
```

Get Signature Status

GET /api/v1/deals/{dealId}/agreement/signatures

Authorization: Deal participants.

Success Response (200):

```
{
  "agreementId": "agr-uuid-1",
  "status": "FULLY_SIGNED",
  "signatures": [
    {
      "signatureId": "sig-uuid-1",
      "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
      "partyDisplayName": "Green Acres Farm Ltd",
      "signedByUserId": "user-uuid-1",
      "signedByUserName": "Jane Smith",
      "signatureType": "E_SIGN",
      "signedAt": "2024-01-21T09:30:00Z",
      "status": "VALID"
    },
    {
      "signatureId": "sig-uuid-2",
      "partyId": "consumer-party-uuid-1",
      "partyDisplayName": "FreshMart Grocery Chain",
      "signedByUserId": "user-uuid-2",
      "signedByUserName": "Robert Johnson",
      "signatureType": "E_SIGN",
      "signedAt": "2024-01-22T10:15:00Z",

```

```
{
  "status": "VALID",
},
{
  "signatureId": "sig-uuid-3",
  "partyId": "enhancer-party-uuid-1",
  "partyDisplayName": "AgriTech Solutions",
  "signedByUserId": "user-uuid-3",
  "signedByUserName": "Maria Garcia",
  "signatureType": "E_SIGN",
  "signedAt": "2024-01-22T14:00:00Z",
  "status": "VALID"
}
],
"fullySignedAt": "2024-01-22T14:00:00Z",
"effectiveDate": "2024-01-25T00:00:00Z"
}
```

6.4 Agreement Management

Get Agreement

GET /api/v1/deals/{dealId}/agreement

Authorization: Deal participants and admins.

Success Response (200): Full agreement object.

Download Agreement PDF

GET /api/v1/deals/{dealId}/agreement/download

Authorization: Deal participants.

Success Response (200): Returns application/pdf binary.

Void Agreement

POST /api/v1/deals/{dealId}/agreement/actions/void

Description: Void a signed agreement. Only possible in COMMITTED state (before execution begins). Requires all parties to consent or a dispute resolution outcome.

Authorization: Deal participants (unanimous consent required) or dispute resolution authority.

Request Body:

```
{
  "reason": "MUTUAL_CONSENT",
  "description": "All parties agree to void this agreement due to changed circumstances.
No disputes between parties.",
  "mutualConsent": true,
  "refundDistribution": {
    "consumerRefundPercent": 100.0,
    "platformFeeRefund": true
  }
}
```

```
}  
}
```

Success Response (200):

```
{  
  "agreementId": "agr-uuid-1",  
  "previousStatus": "FULLY_SIGNED",  
  "newStatus": "VOIDED",  
  "voidedAt": "2024-01-23T10:00:00Z",  
  "voidedBy": "mutual_consent",  
  "dealStatus": "CANCELLED",  
  "refundStatus": "INITIATED",  
  "refundAmount": 45000.00,  
  "refundCurrency": "USD",  
  "message": "Agreement voided by mutual consent. Full refund to consumer initiated.  
Deal cancelled."  
}
```

Expire Agreement

POST /api/v1/deals/{dealId}/agreement/actions/expire

Description: Mark an agreement as expired. System-triggered when **terminationDate** passes or timeout occurs.

Authorization: System (auto) or admin.

6.5 Agreement Templates

List Agreement Templates

GET /api/v1/agreement-templates

Authorization: Authenticated.

Query Parameters:

Parameter	Type	Description
domainCategoryId	UUID	Filter by domain
jurisdiction	String	Filter by governing law

Success Response (200):

```
{  
  "data": [  
    {  
      "templateId": "template-standard-agriculture-uuid",  
      "name": "Standard Agriculture Agreement",  
      "domain": "Agriculture",  
      "jurisdictions": ["US-CA", "US-NY", "US-TX"],  
      "description": "Standard agreement template for agricultural deals covering land use,  
crop sharing, and seasonal arrangements.",  
      "defaultMilestones": ["Planting", "Cultivation", "Harvest", "Delivery"],  
      "requiredPartyRoles": ["SUPPLIER", "CONSUMER", "ENHANCER"],  
      "isActive": true,  
      "usageCount": 342  
    }  
  ]  
}
```

```
}  
]  
}
```

Get Agreement Template

GET /api/v1/agreement-templates/{templateId}

Authorization: Authenticated.

6. Execution & Milestone APIs

Service: execution-milestone-svc Base Path: /api/v1/deals/{dealId}/milestones

6.1 Milestone Management

Create Milestone

POST /api/v1/deals/{dealId}/milestones

Description: Create a milestone for a deal. Typically auto-created from locked terms, but can also be added manually during NEGOTIATING or EXECUTING states.

Authorization: Deal participants.

Request Body:

```
{  
  "milestoneName": "Soil Preparation & Planting Complete",  
  "description": "All soil amendments applied, irrigation system tested and operational, all  
seeds planted per agreed schedule.",  
  "assignedToPartyId": "enhancer-party-uuid-1",  
  "dueDate": "2024-03-15",  
  "completionCriteria": "1. Soil pH tested and between 6.0-7.0. 2. Irrigation system  
operational with >95% coverage. 3. All seeds planted in designated rows. 4. Photos  
uploaded showing completed planting.",  
  "paymentTriggerAmount": 9000.00,  
  "displayOrder": 1,  
  "dependencies": [],  
  "deliverables": [  
    "Soil test report",  
    "Irrigation system test report",  
    "Planting completion photos (min 10)",  
    "Seed variety documentation"  
  ]  
}
```

Success Response (201):

```
{  
  "milestoneId": "ms-uuid-1",  
  "dealId": "550e8400-e29b-41d4-a716-446655440000",  
  "milestoneName": "Soil Preparation & Planting Complete",  
  "description": "All soil amendments applied, irrigation system tested..."
```

```
"assignedToPartyId": "enhancer-party-uuid-1",
"assignedToPartyName": "AgriTech Solutions",
"dueDate": "2024-03-15",
"completionCriteria": "1. Soil pH tested and between 6.0-7.0...",
"milestoneStatus": "PENDING",
"completionPercentage": 0,
"paymentTriggerAmount": 9000.00,
"displayOrder": 1,
"dependencies": [],
"deliverables": [ /* ... */ ],
"completedAt": null,
"verifiedByPartyId": null,
"createdAt": "2024-01-22T16:00:00Z",
"updatedAt": "2024-01-22T16:00:00Z",
"links": {
  "self": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/milestones/ms-uuid-1",
  "complete": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/milestones/ms-uuid-1/actions/complete",
  "verify": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/milestones/ms-uuid-1/actions/verify"
}
```

List Milestones

GET /api/v1/deals/{dealId}/milestones

Authorization: Deal participants.

Query Parameters: | Parameter | Type | Description | | status |
String[] | Filter by status | | assignedTo | UUID | Filter by assigned party | | overdue |
Boolean | Only overdue milestones |

Success Response (200):

```
{
  "data": [
    {
      "milestoneId": "ms-uuid-1",
      "milestoneName": "Soil Preparation & Planting Complete",
      "assignedToPartyId": "enhancer-party-uuid-1",
      "assignedToPartyName": "AgriTech Solutions",
      "dueDate": "2024-03-15",
      "milestoneStatus": "IN_PROGRESS",
      "completionPercentage": 65,
      "paymentTriggerAmount": 9000.00,
      "daysUntilDue": 15,
      "isOverdue": false
    },
    {
      "milestoneId": "ms-uuid-2",
      "milestoneName": "Mid-Season Harvest",
      "assignedToPartyId": "enhancer-party-uuid-1",
      "assignedToPartyName": "AgriTech Solutions",

```

```
{
  "dueDate": "2024-06-01",
  "milestoneStatus": "PENDING",
  "completionPercentage": 0,
  "paymentTriggerAmount": 18000.00,
  "daysUntilDue": 92,
  "isOverdue": false
},
{
  "milestoneId": "ms-uuid-3",
  "milestoneName": "Final Harvest & Delivery",
  "assignedToPartyId": "enhancer-party-uuid-1",
  "assignedToPartyName": "AgriTech Solutions",
  "dueDate": "2024-08-31",
  "milestoneStatus": "PENDING",
  "completionPercentage": 0,
  "paymentTriggerAmount": 18000.00,
  "daysUntilDue": 183,
  "isOverdue": false
}
],
"summary": {
  "total": 3,
  "completed": 0,
  "inProgress": 1,
  "pending": 2,
  "overdue": 0,
  "totalPaymentTriggers": 45000.00
}
}
```

Get Milestone

GET /api/v1/deals/{dealId}/milestones/{milestoneId}

Authorization: Deal participants.

Success Response (200): Full milestone with progress history.

Update Milestone

PUT /api/v1/deals/{dealId}/milestones/{milestoneId}

PATCH /api/v1/deals/{dealId}/milestones/{milestoneId}

Authorization: Milestone assignee or deal admin.

Delete Milestone

DELETE /api/v1/deals/{dealId}/milestones/{milestoneId}

Authorization: Deal initiator or admin. Only allowed for PENDING milestones.

6.2 Milestone Completion & Verification

Mark Milestone Complete

POST /api/v1/deals/{dealId}/milestones/{milestoneId}/actions/complete

Description: Mark a milestone as complete. Submits deliverables and evidence for review.

Authorization: Party assigned to the milestone.

Request Body:

```
{
  "completionPercentage": 100,
  "evidence": {
    "description": "All planting completed on schedule. Soil tests passed. Irrigation system fully operational.",
    "documentUrls": [
      "https://cdn.dealplatform.com/evidence/soil-test-2024-03-14.pdf",
      "https://cdn.dealplatform.com/evidence/irrigation-test-report.pdf",
      "https://cdn.dealplatform.com/evidence/planting-photos.zip"
    ],
    "photos": [
      "https://cdn.dealplatform.com/photos/planting-row1.jpg",
      "https://cdn.dealplatform.com/photos/planting-row2.jpg",
      "https://cdn.dealplatform.com/photos/irrigation-running.jpg"
    ],
    "notes": "Planted all 10 acres as agreed. Varieties: Roma tomatoes (3 acres), iceberg lettuce (2 acres), bell peppers (3 acres), cucumbers (2 acres)."
  },
  "metadata": {
    "completedByUserId": "user-uuid-3",
    "actualCompletionDate": "2024-03-14",
    "weatherConditions": "Favorable - light rain in morning, clear afternoon"
  }
}
```

Success Response (200):

```
{
  "milestoneId": "ms-uuid-1",
  "milestoneStatus": "COMPLETED",
  "completionPercentage": 100,
  "completedAt": "2024-03-14T18:00:00Z",
  "completedByPartyId": "enhancer-party-uuid-1",
  "verificationStatus": "PENDING_VERIFICATION",
  "evidenceSubmitted": {
    "description": "All planting completed on schedule...",
    "documentCount": 3,
    "photoCount": 10,
    "submittedAt": "2024-03-14T18:00:00Z"
  },
  "paymentTrigger": {
    "amount": 9000.00,
    "currency": "USD",
  }
}
```



```

    "status": "PENDING_VERIFICATION",
    "willReleaseOnVerification": true
  },
  "verificationRequiredBy": [
    {
      "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
      "displayName": "Green Acres Farm Ltd",
      "role": "SUPPLIER",
      "verified": false
    },
    {
      "partyId": "consumer-party-uuid-1",
      "displayName": "FreshMart Grocery Chain",
      "role": "CONSUMER",
      "verified": false
    }
  ],
  "verificationDeadline": "2024-03-17T18:00:00Z",
  "message": "Milestone marked complete. Verification required from at least one other party before payment is released."
}

```

Verify Milestone

POST /api/v1/deals/{dealId}/milestones/{milestoneId}/actions/verify

Description: Verify a completed milestone. After the required number of verifications, payment is released from escrow.

Authorization: Any party OTHER than the one who completed the milestone.

Request Body:

```

{
  "verification": "APPROVED",
  "comment": "Verified in person. Soil tests look good, irrigation is running well, all rows planted as specified. Photos match actual conditions.",
  "issueFlags": []
}

```

Verification Options: | Value | Description | |———|—————| | **APPROVED** | Milestone verified, no issues | | **APPROVED_WITH_NOTES** | Verified with minor notes (non-blocking) | | **REJECTED** | Milestone not completed to criteria |

Success Response (200):

```

{
  "milestoneId": "ms-uuid-1",
  "verificationId": "verif-uuid-1",
  "verifiedByPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "verification": "APPROVED",
  "verifiedAt": "2024-03-15T10:00:00Z",
  "sufficientVerifications": true,
  "requiredVerifications": 1,
}

```

```
"receivedVerifications": 1,
"paymentReleased": true,
"payment": {
  "transactionId": "txn-uuid-1",
  "amount": 9000.00,
  "currency": "USD",
  "fromEscrow": true,
  "toPartyId": "enhancer-party-uuid-1",
  "status": "COMPLETED",
  "releasedAt": "2024-03-15T10:00:05Z"
},
"message": "Milestone verified. Payment of $9,000.00 released from escrow to AgriTech Solutions.",
"nextMilestone": {
  "milestoneId": "ms-uuid-2",
  "milestoneName": "Mid-Season Harvest",
  "dueDate": "2024-06-01",
  "status": "PENDING"
}
}
```

Reject Milestone Verification

POST /api/v1/deals/{dealId}/milestones/{milestoneId}/actions/verify

Request Body (Rejection):

```
{
  "verification": "REJECTED",
  "comment": "Soil pH test shows 5.8 which is below the agreed 6.0 minimum. Also, 2 acres of cucumbers were not planted. Need these issues resolved before I can approve.",
  "issueFlags": [
    {
      "issue": "Soil pH below minimum",
      "severity": "BLOCKING",
      "criteriaReference": "1. Soil pH tested and between 6.0-7.0",
      "evidenceRequired": "Re-test showing pH >= 6.0"
    },
    {
      "issue": "Incomplete planting",
      "severity": "BLOCKING",
      "criteriaReference": "3. All seeds planted in designated rows",
      "evidenceRequired": "Photos showing remaining 2 acres planted"
    }
  ]
}
```

Success Response (200):

```
{
  "milestoneId": "ms-uuid-1",
  "verification": "REJECTED",
  "milestoneStatus": "IN_PROGRESS",
  "completionPercentage": 70,
}
```

```
"rejectionReason": "Soil pH test shows 5.8...",
"issueFlags": [ /* ... */ ],
"reworkRequired": true,
"reworkDeadline": "2024-03-20T18:00:00Z",
"message": "Milestone verification rejected. 2 blocking issues identified. Rework
required by 2024-03-20.",
"paymentTriggerStatus": "ON_HOLD"
}
```

Update Milestone Progress

POST /api/v1/deals/{dealId}/milestones/{milestoneId}/actions/progress

Description: Update milestone progress percentage without marking complete.

Authorization: Milestone assignee.

Request Body:

```
{
  "completionPercentage": 65,
  "progressNote": "Soil prep complete. Irrigation installed. 6.5 of 10 acres planted.
Remaining 3.5 acres scheduled for tomorrow.",
  "photos": ["https://cdn.../progress-march13.jpg"]
}
```

6.3 Execution Status

Get Execution Status

GET /api/v1/deals/{dealId}/execution

Authorization: Deal participants.

Success Response (200):

```
{
  "dealId": "550e8400-e29b-41d4-a716-446655440000",
  "dealStatus": "EXECUTING",
  "executionStartedAt": "2024-01-25T00:00:00Z",
  "expectedCompletionDate": "2024-08-31",
  "daysInExecution": 50,
  "daysRemaining": 132,
  "overallProgress": {
    "percentage": 22,
    "completedMilestones": 0,
    "totalMilestones": 3,
    "inProgressMilestones": 1
  },
  "milestones": [
    {
      "milestoneId": "ms-uuid-1",
      "milestoneName": "Soil Preparation & Planting Complete",
      "status": "IN_PROGRESS",
      "completionPercentage": 65,
    }
  ]
}
```

```

    "dueDate": "2024-03-15",
    "daysUntilDue": 15,
    "isOverdue": false,
    "paymentTriggerAmount": 9000.00,
    "paymentStatus": "NOT_TRIGGERED"
  },
  {
    "milestoneId": "ms-uuid-2",
    "milestoneName": "Mid-Season Harvest",
    "status": "PENDING",
    "completionPercentage": 0,
    "dueDate": "2024-06-01",
    "daysUntilDue": 92,
    "isOverdue": false,
    "paymentTriggerAmount": 18000.00,
    "paymentStatus": "NOT_TRIGGERED"
  },
  {
    "milestoneId": "ms-uuid-3",
    "milestoneName": "Final Harvest & Delivery",
    "status": "PENDING",
    "completionPercentage": 0,
    "dueDate": "2024-08-31",
    "daysUntilDue": 183,
    "isOverdue": false,
    "paymentTriggerAmount": 18000.00,
    "paymentStatus": "NOT_TRIGGERED"
  }
],
"payments": {
  "totalValue": 45000.00,
  "released": 0.00,
  "pending": 0.00,
  "inEscrow": 45000.00
},
"nextActions": [
  {
    "action": "UPDATE_PROGRESS",
    "milestoneId": "ms-uuid-1",
    "description": "Update planting progress",
    "deadline": "2024-03-15",
    "assignedTo": "enhancer-party-uuid-1"
  }
],
"alerts": []
}

```

6.4 Milestone Reassignment

Reassign Milestone

POST /api/v1/deals/{dealId}/milestones/{milestoneId}/actions/reassign

Description: Reassign a milestone to a different party. Requires agreement from both current and new assignee.

Authorization: Deal participants (unanimous or admin).

Request Body:

```
{
  "newAssignedPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "reason": "Supplier has more capacity for this task. Enhancer will focus on monitoring.",
  "mutualConsent": true
}
```

6. Payment & Settlement APIs

Service: payment-settlement-svc Base Path: /api/v1/payments

6.1 Wallet Management

Get Wallet

GET /api/v1/payments/wallets/{partyId}

Authorization: Wallet owner or admin.

Success Response (200):

```
{
  "walletId": "wallet-uuid-1",
  "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "partyDisplayName": "Green Acres Farm Ltd",
  "balance": 18500.00,
  "escrowBalance": 0.00,
  "pendingBalance": 2000.00,
  "totalDeposited": 75000.00,
  "totalWithdrawn": 56500.00,
  "currency": "USD",
  "isActive": true,
  "createdAt": "2024-01-15T10:30:00Z",
  "updatedAt": "2024-03-15T10:05:00Z"
}
```

List Transactions

GET /api/v1/payments/wallets/{partyId}/transactions

Authorization: Wallet owner or admin.

Query Parameters:

Parameter	Type	Description	type
String[]	Filter by transaction type	status	String[]
Filter by status	dealId	UUID	
Filter by deal	fromDate	ISO 8601	Start date
toDate	ISO 8601	End date	limit
Integer	Page size	cursor	String
			Pagination cursor

Success Response (200):

```

{
  "data": [
    {
      "transactionId": "txn-uuid-1",
      "dealId": "550e8400-e29b-41d4-a716-446655440000",
      "dealTitle": "Farm Share Crop Agreement",
      "transactionType": "ESCROW_RELEASE",
      "fromPartyId": null,
      "toPartyId": "enhancer-party-uuid-1",
      "toPartyName": "AgriTech Solutions",
      "amount": 9000.00,
      "currency": "USD",
      "description": "Milestone payment: Soil Preparation & Planting Complete",
      "status": "COMPLETED",
      "paymentMethod": "ESCROW",
      "externalReference": "escrow-release-uuid-1",
      "milestoneId": "ms-uuid-1",
      "milestoneName": "Soil Preparation & Planting Complete",
      "executedAt": "2024-03-15T10:00:05Z",
      "createdAt": "2024-03-15T10:00:00Z"
    },
    {
      "transactionId": "txn-uuid-2",
      "dealId": "550e8400-e29b-41d4-a716-446655440000",
      "transactionType": "PLATFORM_FEE",
      "fromPartyId": "consumer-party-uuid-1",
      "toPartyId": null,
      "amount": 2250.00,
      "currency": "USD",
      "description": "Platform fee (5% of deal value)",
      "status": "COMPLETED",
      "paymentMethod": "ESCROW",
      "executedAt": "2024-01-24T12:00:00Z"
    }
  ],
  "pagination": { /* ... */ },
  "summary": {
    "totalIn": 18500.00,
    "totalOut": 5000.00,
    "totalEscrowReceived": 18000.00,
    "totalFeesPaid": 225.00
  }
}

```

6.2 Escrow Management

Create Escrow

POST /api/v1/payments/escrow

Description: Create an escrow account for a deal. Auto-created when agreement is signed.

Authorization: Deal participants. Consumer party typically funds the escrow.

Request Body:

```
{
  "dealId": "550e8400-e29b-41d4-a716-446655440000",
  "totalAmount": 45000.00,
  "currency": "USD",
  "fundingPartyId": "consumer-party-uuid-1",
  "beneficiaryPartyIds": [
    {
      "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
      "scheduledAmount": 18000.00,
      "releaseTrigger": "MILESTONE_COMPLETION"
    },
    {
      "partyId": "enhancer-party-uuid-1",
      "scheduledAmount": 15750.00,
      "releaseTrigger": "MILESTONE_COMPLETION"
    }
  ],
  "platformFee": {
    "amount": 2250.00,
    "deductFrom": "FIRST_MILESTONE"
  },
  "paymentSchedule": [
    {
      "milestoneId": "ms-uuid-1",
      "releaseAmount": 9000.00,
      "conditions": ["MILESTONE_VERIFIED"]
    },
    {
      "milestoneId": "ms-uuid-2",
      "releaseAmount": 18000.00,
      "conditions": ["MILESTONE_VERIFIED"]
    },
    {
      "milestoneId": "ms-uuid-3",
      "releaseAmount": 18000.00,
      "conditions": ["MILESTONE_VERIFIED"]
    }
  ],
  "expiryDate": "2024-12-31T23:59:59Z"
}
```

Success Response (201):

```
{
  "escrowId": "escrow-uuid-1",
  "dealId": "550e8400-e29b-41d4-a716-446655440000",
  "status": "PENDING_FUNDING",
  "totalAmount": 45000.00,
  "fundedAmount": 0.00,
  "releasedAmount": 0.00,
  "currency": "USD",
}
```

```
"fundingPartyId": "consumer-party-uuid-1",
"createdAt": "2024-01-22T14:00:05Z",
"fundingDeadline": "2024-01-24T23:59:59Z",
"paymentUrl": "https://pay.dealplatform.com/escrow/fund/escrow-uuid-1",
"message": "Escrow created. Consumer must fund $45,000.00 by 2024-01-24 23:59:59 UTC."
}
```

Fund Escrow

POST /api/v1/payments/escrow/{escrowId}/fund

Description: Fund an escrow account. Supports multiple payment methods.

Authorization: Funding party.

Request Body:

```
{
  "amount": 45000.00,
  "currency": "USD",
  "paymentMethod": "BANK_TRANSFER",
  "sourceAccountId": "acct-uuid-1",
  "metadata": {
    "reference": "Farm Deal Escrow Funding",
    "initiatedByUserId": "user-uuid-2"
  }
}
```

Payment Methods: | Method | Code | Processing Time | | Bank Transfer | **BANK_TRANSFER** | 1-3 business days | | Credit/Debit Card | **CARD** | Instant | | Platform Wallet | **PLATFORM_WALLET** | Instant | | Cryptocurrency | **CRYPTO** | 10-60 minutes |

Success Response (202):

```
{
  "escrowId": "escrow-uuid-1",
  "status": "FUNDING_IN_PROGRESS",
  "fundedAmount": 0.00,
  "requestedAmount": 45000.00,
  "paymentMethod": "BANK_TRANSFER",
  "transactionId": "txn-fund-uuid-1",
  "estimatedCompletion": "2024-01-24T12:00:00Z",
  "message": "Bank transfer initiated. Funds will be available in escrow in 1-2 business days. You will be notified when funding is complete."
}
```

Get Escrow Status

GET /api/v1/payments/escrow/{escrowId}

Authorization: Deal participants.

Success Response (200):


```
{
  "escrowId": "escrow-uuid-1",
  "dealId": "550e8400-e29b-41d4-a716-446655440000",
  "status": "FULLY_FUNDED",
  "totalAmount": 45000.00,
  "fundedAmount": 45000.00,
  "releasedAmount": 9000.00,
  "heldAmount": 36000.00,
  "currency": "USD",
  "fundingPartyId": "consumer-party-uuid-1",
  "fundedAt": "2024-01-24T10:30:00Z",
  "releases": [
    {
      "releaseId": "rel-uuid-1",
      "milestoneId": "ms-uuid-1",
      "milestoneName": "Soil Preparation & Planting Complete",
      "amount": 9000.00,
      "toPartyId": "enhancer-party-uuid-1",
      "status": "COMPLETED",
      "releasedAt": "2024-03-15T10:00:05Z",
      "trigger": "MILESTONE_VERIFIED"
    },
    {
      "releaseId": "rel-uuid-2",
      "milestoneId": "ms-uuid-2",
      "milestoneName": "Mid-Season Harvest",
      "amount": 18000.00,
      "toPartyId": "enhancer-party-uuid-1",
      "status": "PENDING",
      "trigger": "MILESTONE_VERIFIED"
    }
  ],
  "createdAt": "2024-01-22T14:00:05Z",
  "expiresAt": "2024-12-31T23:59:59Z"
}
```

6.3 Payment Operations

Release Payment

POST /api/v1/payments/escrow/{escrowId}/releases

Description: Release a payment from escrow (typically auto-triggered by milestone verification). Can also be manually triggered by dispute resolution.

Authorization: System (auto) or dispute resolution authority.

Request Body:

```
{
  "milestoneId": "ms-uuid-1",
  "amount": 9000.00,
  "toPartyId": "enhancer-party-uuid-1",
  "reason": "MILESTONE_VERIFIED",
}
```

```
"verificationId": "verif-uuid-1"
}
```

Success Response (200):

```
{
  "releaseId": "rel-uuid-1",
  "escrowId": "escrow-uuid-1",
  "transactionId": "txn-uuid-1",
  "amount": 9000.00,
  "currency": "USD",
  "toPartyId": "enhancer-party-uuid-1",
  "toWalletId": "wallet-enhancer-1",
  "status": "COMPLETED",
  "releasedAt": "2024-03-15T10:00:05Z",
  "milestoneId": "ms-uuid-1",
  "remainingEscrow": 36000.00
}
```

Rollback Payment

POST /api/v1/payments/transactions/{transactionId}/rollback

Description: Rollback a payment (e.g., on dispute resolution). Creates a reversal transaction.

Authorization: Dispute resolution authority or admin.

Request Body:

```
{
  "reason": "DISPUTE_RESOLUTION",
  "disputeId": "dispute-uuid-1",
  "rollbackAmount": 4500.00,
  "redistribution": {
    "toPartyId": "consumer-party-uuid-1",
    "amount": 4500.00
  }
}
```

Success Response (200):

```
{
  "rollbackTransactionId": "txn-rollback-uuid-1",
  "originalTransactionId": "txn-uuid-1",
  "rollbackAmount": 4500.00,
  "status": "COMPLETED",
  "reason": "DISPUTE_RESOLUTION",
  "redistribution": {
    "toPartyId": "consumer-party-uuid-1",
    "amount": 4500.00,
    "transactionId": "txn-redist-uuid-1"
  }
}
```

Refund

POST /api/v1/payments/escrow/{escrowId}/refund

Description: Refund remaining escrow funds (e.g., on deal cancellation).

Authorization: Deal participants (unanimous) or dispute resolution authority.

Request Body:

```
{
  "reason": "DEAL_CANCELLED",
  "refundToPartyId": "consumer-party-uuid-1",
  "refundAmount": 36000.00,
  "deductions": [
    {
      "reason": "PLATFORM_FEE",
      "amount": 1125.00,
      "description": "Partial platform fee for completed work"
    }
  ]
}
```

6.4 Payment Webhooks (Inbound)

Payment Provider Webhook

POST /webhooks/payments/{provider}

Description: Receive payment status updates from external payment providers (Stripe, Adyen, etc.).

Authorization: HMAC-SHA256 signature verification per provider.

Request Body (Stripe example):

```
{
  "id": "evt_stripe_webhook_123",
  "object": "event",
  "type": "payment_intent.succeeded",
  "data": {
    "object": {
      "id": "pi_stripe_123",
      "amount": 4500000,
      "currency": "usd",
      "status": "succeeded",
      "metadata": {
        "escrowId": "escrow-uuid-1",
        "dealId": "550e8400-e29b-41d4-a716-446655440000"
      }
    }
  }
}
```

Success Response (200): Acknowledged.

6. Matching & Discovery APIs

Service: matching-discovery-svc Base Path: /api/v1/matches

6.1 Get Suggested Matches

GET /api/v1/matches

Description: Get match suggestions for the authenticated party. Returns triadic matches (Supplier-Consumer-Enhancer combinations).

Authorization: Authenticated. X-Party-ID header determines which party to match for.

Query Parameters: | Parameter | Type | Description | |-----|-----|-----| | **role** | Enum | Role to match for: **SUPPLIER**, **CONSUMER**, **ENHANCER** | | **domainCategoryId** | UUID | Filter by domain | | **location** | String | Geo search: **lat, long, radiusKm** | | **minScore** | Decimal | Minimum compatibility score (0-1) | | **maxResults** | Integer | Max results to return (default: 10, max: 50) | | **dealValueMin** | Decimal | Minimum deal value | | **dealValueMax** | Decimal | Maximum deal value | | **resourceTypeId** | UUID | Filter by resource type (when role=SUPPLIER) | | **needCategoryId** | UUID | Filter by need category (when role=CONSUMER) | | **enhancementTypeId** | UUID | Filter by enhancement type (when role=ENHANCER) |

Success Response (200):

```
{
  "data": [
    {
      "matchId": "match-uuid-1",
      "matchScore": 0.92,
      "matchScoreBreakdown": {
        "resourceNeedMatch": 0.95,
        "valueAlignment": 0.88,
        "trustScore": 0.90,
        "geographicFit": 0.85,
        "temporalAlignment": 0.92,
        "historicalSuccess": 0.85,
        "riskProfileMatch": 0.95
      },
      "matchReason": "High compatibility based on: organic farming expertise, geographic proximity (12km), complementary value expectations, strong trust scores (all > 4.2), and aligned seasonal timelines.",
      "suggestedDealValue": 42000.00,
      "suggestedValueCurrency": "USD",
      "parties": {
        "supplier": {
          "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
          "displayName": "Green Acres Farm Ltd",
          "trustScore": 4.52,
          "resource": {
            "resourceId": "res-uuid-1",
            "resourceName": "Irrigated Farmland - 10 Acres",
            "resourceTypeName": "Farmland",
```

```

    "location": { "city": "Palo Alto", "region": "CA" },
    "quantity": 10,
    "quantityUnit": "acres"
  },
  "consumer": {
    "partyId": "consumer-party-uuid-1",
    "displayName": "FreshMart Grocery Chain",
    "trustScore": 4.75,
    "need": {
      "needId": "need-uuid-1",
      "needDescription": "Organic mixed vegetables - 15,000 lbs",
      "needCategoryName": "Fresh Produce",
      "maxBudget": 50000.00,
      "budgetCurrency": "USD"
    }
  },
  "enhancer": {
    "partyId": "enhancer-party-uuid-1",
    "displayName": "AgriTech Solutions",
    "trustScore": 4.30,
    "enhancement": {
      "enhancementId": "enh-uuid-1",
      "enhancementName": "Full Season Agricultural Support Package",
      "enhancementTypeName": "Agricultural Services",
      "hourlyRate": 150.00,
      "skills": ["Precision Agriculture", "Organic Farming"]
    }
  },
  "generatedBy": "ALGORITHM",
  "expiresAt": "2024-02-15T10:00:00Z",
  "matchStatus": "PENDING",
  "createdAt": "2024-01-15T10:00:00Z",
  "actions": {
    "accept": "/api/v1/matches/match-uuid-1/actions/accept",
    "decline": "/api/v1/matches/match-uuid-1/actions/decline",
    "counter": "/api/v1/matches/match-uuid-1/actions/counter"
  },
  {
    "matchId": "match-uuid-2",
    "matchScore": 0.87,
    "matchReason": "Good compatibility with slightly different value distribution suggestion.",
    "suggestedDealValue": 38000.00,
    "parties": { /* ... */ }
  },
  ],
  "pagination": {
    "limit": 10,
    "hasMore": true,

```

```
"nextCursor": "eyJzY29yZSI6MC44NX0="
}
```

Request Match (Find Partners)

POST /api/v1/matches/request

Description: Request the platform to find matching partners. Party specifies what they're looking for.

Authorization: Authenticated. X-Party-ID header required.

Request Body:

```
{
  "myRole": "SUPPLIER",
  "resourceId": "res-uuid-1",
  "lookingFor": {
    "consumer": {
      "needCategoryIds": ["need-category-produce-uuid"],
      "minBudget": 30000.00,
      "locationRadiusKm": 100
    },
    "enhancer": {
      "enhancementTypeIds": ["enh-type-agri-support-uuid"],
      "requiredSkills": ["Organic Farming"],
      "locationRadiusKm": 75
    }
  },
  "preferences": {
    "minMatchScore": 0.75,
    "minTrustScore": 4.0,
    "preferredDealValue": 40000.00,
    "timeline": {
      "availableFrom": "2024-03-01",
      "availableTo": "2024-08-31"
    }
  },
  "autoCreateDeal": false,
  "notificationPreferences": {
    "email": true,
    "push": true
  }
}
```

Success Response (202):

```
{
  "matchRequestId": "match-req-uuid-1",
  "status": "PROCESSING",
  "estimatedCompletion": "2024-01-15T10:05:00Z",
  "message": "Match request received. Searching for compatible Consumer and Enhancer partners. You will be notified when matches are ready.",
}
```

```
"searchParameters": {  
  "consumerCandidates": 45,  
  "enhancerCandidates": 12,  
  "estimatedMatches": 5  
}  
}
```

Accept Match

POST /api/v1/matches/{matchId}/actions/accept

Description: Accept a match suggestion. When all parties accept, a deal can be created.

Authorization: Authenticated. Only the party being matched can accept.

Request Body:

```
{  
  "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",  
  "response": "ACCEPTED",  
  "notes": "Looks like a good match. Interested in proceeding.",  
  "preferredValueSharePercent": 42.0,  
  "suggestModifications": false  
}
```

Success Response (200):

```
{  
  "matchId": "match-uuid-1",  
  "responseId": "resp-uuid-1",  
  "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",  
  "response": "ACCEPTED",  
  "matchStatus": "PENDING",  
  "allPartiesResponded": false,  
  "responses": [  
    {  
      "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",  
      "role": "SUPPLIER",  
      "response": "ACCEPTED",  
      "respondedAt": "2024-01-15T14:00:00Z"  
    },  
    {  
      "partyId": "consumer-party-uuid-1",  
      "role": "CONSUMER",  
      "response": "PENDING"  
    },  
    {  
      "partyId": "enhancer-party-uuid-1",  
      "role": "ENHANCER",  
      "response": "PENDING"  
    }  
  ],  
  "message": "Your acceptance recorded. Awaiting responses from 2 other parties."  
}
```

Success Response (200) — When All Accept:

```
{
  "matchId": "match-uuid-1",
  "response": "ACCEPTED",
  "matchStatus": "ACCEPTED_BY_ALL",
  "allPartiesResponded": true,
  "convertedToDeal": true,
  "dealId": "new-deal-uuid-1",
  "dealReference": "DL-2024-0002",
  "dealStatus": "DRAFT",
  "message": "All parties accepted! A new deal (DL-2024-0002) has been created in DRAFT status. All parties have been notified."
}
```

Decline Match

POST /api/v1/matches/{matchId}/actions/decline

Authorization: Authenticated.

Request Body:

```
{
  "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "response": "DECLINED",
  "reason": "BUDGET_MISMATCH",
  "notes": "The suggested deal value is lower than our minimum threshold for this resource.",
  "suggestAlternativeValue": 50000.00
}
```

Decline Reasons: | Reason | Description | |———|—————| | **BUDGET_MISMATCH** | Deal value doesn't meet expectations | | **TIMELINE_MISMATCH** | Timeline doesn't work | | **TRUST_CONCERNS** | Concerns about counter-party trust score | | **CAPACITY** | At maximum deal capacity | | **LOCATION** | Location not suitable | | **OTHER** | Other reason |

Counter-Propose Match

POST /api/v1/matches/{matchId}/actions/counter

Authorization: Authenticated.

Request Body:

```
{
  "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "response": "COUNTER_PROPOSED",
  "counterProposal": {
    "suggestedDealValue": 52000.00,
    "valueSharePercent": 42.0,
    "timelineAdjustment": {
      "expectedStartDate": "2024-03-15",
      "expectedEndDate": "2024-09-15"
    }
  },
}
```



```
    "notes": "Willing to proceed at higher value with extended timeline for fall harvest."
  }
}
```

6.2 Compatibility Scoring

Get Compatibility Score

POST /api/v1/matches/compatibility-score

Description: Calculate a compatibility score between specific parties without creating a match.

Authorization: Authenticated.

Request Body:

```
{
  "supplierPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "consumerPartyId": "consumer-party-uuid-1",
  "enhancerPartyId": "enhancer-party-uuid-1",
  "resourceId": "res-uuid-1",
  "needId": "need-uuid-1",
  "enhancementId": "enh-uuid-1"
}
```

Success Response (200):

```
{
  "overallScore": 0.92,
  "scoreOutOf100": 92,
  "breakdown": {
    "resourceNeedMatch": {
      "score": 0.95,
      "weight": 0.25,
      "weightedScore": 0.2375,
      "details": {
        "typeCompatibility": 1.0,
        "quantityMatch": 0.95,
        "qualityAlignment": 0.90,
        "capacityUtilization": 0.95
      }
    },
    "valueAlignment": {
      "score": 0.88,
      "weight": 0.20,
      "weightedScore": 0.176,
      "details": {
        "priceOverlap": 0.90,
        "valueDistributionCompatibility": 0.85,
        "paymentTermsAlignment": 0.88
      }
    },
    "trustScore": {
```

```
"score": 0.90,
"weight": 0.15,
"weightedScore": 0.135,
"details": {
  "platformTenure": 0.80,
  "completionRate": 0.95,
  "ratingAverage": 0.92,
  "verificationLevel": 0.90,
  "disputeHistory": 0.95
}
},
"geographicFit": {
  "score": 0.85,
  "weight": 0.10,
  "weightedScore": 0.085,
  "details": {
    "distance": 0.90,
    "logisticsFeasibility": 0.80,
    "regulatoryCompatibility": 1.0
  }
},
"temporalAlignment": {
  "score": 0.92,
  "weight": 0.10,
  "weightedScore": 0.092,
  "details": {
    "availabilityOverlap": 0.95,
    "urgencyMatch": 0.85,
    "seasonalFit": 0.95
  }
},
"historicalSuccess": {
  "score": 0.85,
  "weight": 0.10,
  "weightedScore": 0.085,
  "details": {
    "priorDealsTogether": 0.80,
    "similarDealSuccess": 0.90,
    "mutualConnections": 0.85
  }
},
"riskProfileMatch": {
  "score": 0.95,
  "weight": 0.10,
  "weightedScore": 0.095,
  "details": {
    "dealSizeComfort": 0.95,
    "riskTolerance": 0.90,
    "financialHealth": 1.0
  }
}
},
```

```
"recommendation": "STRONG_MATCH",
"recommendationText": "This is a strong match with high compatibility across all
dimensions. Proceed with confidence.",
"suggestedImprovements": [
  "Consider extending the service radius slightly to improve geographic score",
  "Consumer could increase budget slightly for optimal value alignment"
]
}
```

6.3 Discovery Browsing

Browse by Domain

GET /api/v1/discovery/domains

Authorization: Authenticated.

Success Response (200):

```
{
  "data": [
    {
      "categoryId": "cat-agriculture",
      "categoryName": "Agriculture",
      "categoryCode": "agriculture",
      "description": "Farming, crop production, livestock, forestry",
      "resourceCount": 1280,
      "needCount": 890,
      "enhancementCount": 450,
      "activeDeals": 234,
      "iconUrl": "https://cdn.../icons/agriculture.svg"
    },
    {
      "categoryId": "cat-real-estate",
      "categoryName": "Real Estate",
      "categoryCode": "real-estate",
      "description": "Property, buildings, renovation, property management",
      "resourceCount": 890,
      "needCount": 1200,
      "enhancementCount": 670,
      "activeDeals": 189,
      "iconUrl": "https://cdn.../icons/real-estate.svg"
    },
    {
      "categoryId": "cat-transportation",
      "categoryName": "Transportation",
      "categoryCode": "transportation",
      "description": "Vehicles, logistics, fleet management, delivery",
      "resourceCount": 560,
      "needCount": 780,
      "enhancementCount": 340,
      "activeDeals": 156,
      "iconUrl": "https://cdn.../icons/transportation.svg"
    }
  ]
}
```

```
]
}
```

Browse Domain Detail

GET /api/v1/discovery/domains/{domainCategoryId}

Authorization: Authenticated.

Success Response (200):

```
{
  "categoryId": "cat-agriculture",
  "categoryName": "Agriculture",
  "statistics": {
    "totalResources": 1280,
    "totalNeeds": 890,
    "totalEnhancements": 450,
    "activeDeals": 234,
    "completedDeals": 1567,
    "averageDealValue": 38000.00
  },
  "trending": {
    "topResourceTypes": ["Farmland", "Greenhouse", "Irrigation Equipment"],
    "topNeedTypes": ["Organic Produce", "Grain", "Dairy"],
    "topEnhancementTypes": ["Agricultural Consulting", "Crop Monitoring", "Harvest Labor"]
  },
  "featuredDeals": [
    {
      "dealId": "featured-deal-1",
      "dealTitle": "Organic Dairy Partnership",
      "matchScore": 0.95,
      "dealValue": 75000.00
    }
  ]
}
```

6. Trust & Reputation APIs

Service: trust-reputation-svc **Base Path:** /api/v1/trust

6.1 Trust Score

Get Trust Score

GET /api/v1/trust/party/{partyId}

Description: Get detailed trust score with full breakdown.

Authorization: Authenticated. Public overview for all; full details for party members.

Success Response (200):

```
{
  "trustScoreId": "ts-uuid-1",
  "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "partyDisplayName": "Green Acres Farm Ltd",
  "overallScore": 4.52,
  "scoreOutOf5": 4.52,
  "roleScores": {
    "asSupplier": {
      "score": 4.65,
      "dealsCompleted": 12,
      "dealsCancelled": 1,
      "averageRating": 4.7,
      "topStrengths": ["Reliable delivery", "Quality resources", "Good communication"]
    },
    "asConsumer": {
      "score": 4.35,
      "dealsCompleted": 3,
      "dealsCancelled": 0,
      "averageRating": 4.3,
      "topStrengths": ["Timely payment", "Clear requirements"]
    },
    "asEnhancer": {
      "score": null,
      "dealsCompleted": 0,
      "dealsCancelled": 0,
      "averageRating": null,
      "topStrengths": []
    }
  },
  "detailedMetrics": {
    "dealsCompletedCount": 15,
    "dealsCancelledCount": 1,
    "dealsDisputedCount": 0,
    "completionRate": 0.94,
    "averageResponseHours": 4.5,
    "profileCompleteness": 95.0,
    "verificationLevel": 4,
    "longevityDays": 380,
    "totalReviews": 18,
    "averageRating": 4.5,
    "onTimeDeliveryRate": 0.92,
    "budgetAccuracy": 0.88
  },
  "scoreHistory": [
    { "date": "2024-01-01", "score": 4.45 },
    { "date": "2024-02-01", "score": 4.48 },
    { "date": "2024-03-01", "score": 4.52 }
  ],
  "lastCalculatedAt": "2024-03-01T00:00:00Z",
  "nextCalculationAt": "2024-04-01T00:00:00Z",
  "calculationFormula": {
    "weights": {
```

```

    "completionRate": 0.25,
    "averageRating": 0.25,
    "responseTime": 0.15,
    "profileCompleteness": 0.10,
    "verificationLevel": 0.10,
    "longevity": 0.05,
    "disputeHistory": 0.10
  },
  "trustLevel": "HIGH",
  "trustLevelDescription": "This party has a high trust score. They have consistently delivered on their commitments with minimal disputes.",
  "badges": [
    {
      "badgeId": "badge-verified",
      "name": "Verified Identity",
      "description": "Identity verified by platform",
      "earnedAt": "2024-02-01T14:22:00Z"
    },
    {
      "badgeId": "badge-top-rated",
      "name": "Top Rated Supplier",
      "description": "Rated 4.5+ as supplier for 5+ deals",
      "earnedAt": "2024-03-01T00:00:00Z"
    }
  ]
}

```

6.2 Reviews & Ratings

Submit Review

POST /api/v1/trust/reviews

Description: Submit a review for a party after a deal is completed. Reviews are verified to ensure the reviewer participated in the deal.

Authorization: Deal participant (only after deal is COMPLETED).

Request Body:

```

{
  "dealId": "550e8400-e29b-41d4-a716-446655440000",
  "reviewedPartyId": "enhancer-party-uuid-1",
  "reviewedRole": "ENHANCER",
  "overallRating": 5,
  "communicationRating": 5,
  "reliabilityRating": 5,
  "qualityRating": 4,
  "timelinessRating": 5,
  "reviewText": "AgriTech Solutions delivered exceptional service throughout the season. Their expertise in organic farming was evident from day one. The only minor issue was slightly lower yield than projected on peppers, but overall quality exceeded expectations. Would highly recommend for any agricultural enhancement needs."
}

```

```
"isPublic": true
}
```

Validation Rules: - Reviewer must have participated in the deal - Deal must be in COMPLETED status - One review per reviewer-reviewer pair per deal - Minimum 10 characters for review text (if provided) - Ratings 1-5 scale

Success Response (201):

```
{
  "reviewId": "rev-uuid-1",
  "dealId": "550e8400-e29b-41d4-a716-446655440000",
  "reviewerPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "reviewerDisplayName": "Green Acres Farm Ltd",
  "reviewedPartyId": "enhancer-party-uuid-1",
  "reviewedDisplayName": "AgriTech Solutions",
  "reviewedRole": "ENHANCER",
  "overallRating": 5,
  "communicationRating": 5,
  "reliabilityRating": 5,
  "qualityRating": 4,
  "timelinessRating": 5,
  "reviewText": "AgriTech Solutions delivered exceptional service...",
  "isVerified": true,
  "isPublic": true,
  "trustScoreImpact": {
    "previousScore": 4.30,
    "newScore": 4.35,
    "change": 0.05
  },
  "createdAt": "2024-09-05T10:00:00Z"
}
```

Get Reviews

GET /api/v1/trust/party/{partyId}/reviews

Authorization: Authenticated. Public reviews visible to all.

Query Parameters: | Parameter | Type | Description | |-----|-----|-----| | **role** | Enum | Filter by reviewed role | | **rating** | Integer[] | Filter by overall rating | | **verifiedOnly** | Boolean | Only verified reviews | | **sort** | String | **newest, highest, lowest** | | **limit** | Integer | Page size |

Success Response (200):

```
{
  "data": [
    {
      "reviewId": "rev-uuid-1",
      "dealId": "550e8400-e29b-41d4-a716-446655440000",
      "dealTitle": "Farm Share Crop Agreement - Summer 2024",
      "reviewerPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
      "reviewerDisplayName": "Green Acres Farm Ltd",
      "reviewerTrustScore": 4.52,

```

```

    "reviewedRole": "ENHANCER",
    "overallRating": 5,
    "communicationRating": 5,
    "reliabilityRating": 5,
    "qualityRating": 4,
    "timelinessRating": 5,
    "reviewText": "AgriTech Solutions delivered exceptional service...",
    "isVerified": true,
    "isPublic": true,
    "createdAt": "2024-09-05T10:00:00Z"
  }
],
"pagination": { /* ... */ },
"summary": {
  "totalReviews": 18,
  "averageRating": 4.5,
  "distribution": {
    "5": 12,
    "4": 4,
    "3": 2,
    "2": 0,
    "1": 0
  }
}
}
}

```

Get My Reviews

GET /api/v1/trust/reviews/me

Authorization: Authenticated. Returns reviews given and received by the caller's party.

Query Parameters: | Parameter | Type | Description | |———|———|———| | **direction** |
String | **given** or **received** | | **limit** | Integer | Page size |

6.3 Report Issue

Report Issue

POST /api/v1/trust/issues

Description: Report an issue with another party (non-dispute, for trust/reputation purposes).

Authorization: Authenticated.

Request Body:

```

{
  "reportedPartyId": "party-uuid-1",
  "dealId": "550e8400-e29b-41d4-a716-446655440000",
  "issueType": "LATE_RESPONSE",
  "description": "Party took 5 days to respond to milestone completion instead of the agreed 48 hours.",
  "severity": "LOW",

```



```
"evidenceUrls": []
}
```

Issue Types: | Type | Severity Range | Description | |——|——|——|——| |

LATE_RESPONSE | LOW-MEDIUM | Delayed communication | | **MISREPRESENTATION** | MEDIUM-HIGH | Inaccurate profile information | | **BREACH_OF_AGREEMENT** | HIGH | Minor breach not worth formal dispute | | **POOR_COMMUNICATION** | LOW | Unprofessional communication | | **QUALITY_ISSUE** | MEDIUM | Quality below expectations |

Success Response (201):

```
{
  "issueId": "issue-uuid-1",
  "status": "UNDER_REVIEW",
  "reportedAt": "2024-09-06T10:00:00Z",
  "estimatedResolution": "2024-09-10T10:00:00Z",
  "message": "Issue reported. Platform will review and may contact the reported party."
}
```

6. Notification APIs

Service: notification-svc **Base Path:** /api/v1/notifications

6.1 Notification Management

Get Notifications

GET /api/v1/notifications

Description: Get notifications for the authenticated user's party.

Authorization: Authenticated. Uses X-Party-ID header.

Query Parameters: | Parameter | Type | Description | |——|——|——|——| | **type** | String[] | Filter by notification type | | **isRead** | Boolean | Read/unread filter | | **isActioned** | Boolean | Actioned/unactioned filter | | **limit** | Integer | Page size (default: 20) | | **cursor** | String | Pagination cursor |

Success Response (200):

```
{
  "data": [
    {
      "notificationId": "notif-uuid-1",
      "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
      "notificationType": "DEAL_INVITE",
      "title": "New Deal Invitation",
      "body": "FreshMart Grocery Chain has invited you to participate in 'Farm Share Crop Agreement - Summer 2024' as Supplier.",
      "relatedEntityType": "DEAL",
      "relatedEntityId": "550e8400-e29b-41d4-a716-446655440000",
      "channel": ["IN_APP", "EMAIL"],
      "isRead": false,

```

```
"isActioned": false,
"actionUrl": "/deals/550e8400-e29b-41d4-a716-446655440000",
"actions": [
  {
    "label": "View Deal",
    "action": "NAVIGATE",
    "url": "/deals/550e8400-e29b-41d4-a716-446655440000"
  },
  {
    "label": "Accept",
    "action": "API_CALL",
    "method": "POST",
    "url": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/participations/
{partId}/accept"
  },
  {
    "label": "Decline",
    "action": "API_CALL",
    "method": "POST",
    "url": "/api/v1/deals/550e8400-e29b-41d4-a716-446655440000/participations/
{partId}/decline"
  }
],
"createdAt": "2024-01-15T10:30:00Z",
"expiresAt": "2024-01-29T10:30:00Z"
},
{
  "notificationId": "notif-uuid-2",
  "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "notificationType": "MILESTONE_DUE",
  "title": "Milestone Due Soon",
  "body": "'Soil Preparation & Planting Complete' is due in 5 days (March 15). Update
your progress to keep the deal on track.",
  "relatedEntityType": "MILESTONE",
  "relatedEntityId": "ms-uuid-1",
  "channel": ["IN_APP", "PUSH"],
  "isRead": true,
  "isActioned": false,
  "actionUrl": "/deals/550e8400-e29b-41d4-a716-446655440000/milestones/ms-uuid-1",
  "createdAt": "2024-03-10T09:00:00Z",
  "expiresAt": "2024-03-16T09:00:00Z"
},
{
  "notificationId": "notif-uuid-3",
  "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "notificationType": "PAYMENT_RECEIVED",
  "title": "Payment Received",
  "body": "$9,000.00 has been released to your wallet for milestone 'Soil Preparation &
Planting Complete'.",
  "relatedEntityType": "TRANSACTION",
  "relatedEntityId": "txn-uuid-1",
  "channel": ["IN_APP", "EMAIL"],
```

```
"isRead": false,
"isActioned": false,
"actionUrl": "/payments/transactions/txn-uuid-1",
"createdAt": "2024-03-15T10:00:05Z",
"expiresAt": null
},
],
"pagination": { /* ... */ },
"unreadCount": 2,
"totalCount": 47
}
```

Mark Notification Read

PATCH /api/v1/notifications/{notificationId}

Authorization: Notification recipient.

Request Body:

```
{
  "isRead": true,
  "readAt": "2024-03-10T10:00:00Z"
}
```

Mark All Notifications Read

POST /api/v1/notifications/actions/mark-all-read

Authorization: Authenticated.

Request Body:

```
{
  "type": "DEAL_INVITE",
  "beforeDate": "2024-03-10T00:00:00Z"
}
```

Success Response (200):

```
{
  "markedRead": 12,
  "typesAffected": ["DEAL_INVITE", "MILESTONE_DUE"]
}
```

Delete Notification

DELETE /api/v1/notifications/{notificationId}

Authorization: Notification recipient.

6.2 Subscription Management

Subscribe to Topic

POST /api/v1/notifications/subscriptions

Description: Subscribe to notification topics for real-time updates.

Authorization: Authenticated.

Request Body:

```
{
  "topics": [
    {
      "topic": "deal:550e8400-e29b-41d4-a716-446655440000",
      "channels": ["IN_APP", "PUSH", "EMAIL"]
    },
    {
      "topic": "party:7c9e6679-7425-40de-944b-e07fc1f90ae7:matches",
      "channels": ["EMAIL"]
    }
  ],
  "deliveryPreferences": {
    "emailFrequency": "IMMEDIATE",
    "quietHours": {
      "enabled": true,
      "start": "22:00",
      "end": "07:00",
      "timezone": "America/Los_Angeles"
    }
  }
}
```

Topics: | Topic Pattern | Description | |—————|—————| | **deal:{dealId}** | All events for a specific deal | | **party:{partyId}:deals** | Deal events for a party | | **party:{partyId}:matches** | Match suggestions for a party | | **party:{partyId}:payments** | Payment events for a party | | **party:{partyId}:trust** | Trust score changes | | **domain:{domainId}** | New deals/matches in domain |

Unsubscribe from Topic

DELETE /api/v1/notifications/subscriptions

Authorization: Authenticated.

Request Body:

```
{
  "topics": ["deal:550e8400-e29b-41d4-a716-446655440000"]
}
```

6.3 Notification Preferences

Get Preferences

GET /api/v1/notifications/preferences

Authorization: Authenticated.

Success Response (200):

```
{
  "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "channels": {
    "IN_APP": { "enabled": true },
    "EMAIL": {
      "enabled": true,
      "frequency": "IMMEDIATE",
      "digestDay": null
    },
    "SMS": {
      "enabled": true,
      "onlyUrgent": true
    },
    "PUSH": { "enabled": true }
  },
  "notificationTypes": {
    "MATCH_SUGGESTION": { "channels": ["EMAIL", "PUSH"], "enabled": true },
    "DEAL_INVITE": { "channels": ["EMAIL", "IN_APP", "PUSH"], "enabled": true },
    "TERM_UPDATE": { "channels": ["IN_APP", "EMAIL"], "enabled": true },
    "PAYMENT_DUE": { "channels": ["EMAIL", "SMS"], "enabled": true },
    "MILESTONE_DUE": { "channels": ["IN_APP", "PUSH"], "enabled": true },
    "REVIEW_REQUEST": { "channels": ["EMAIL"], "enabled": true },
    "DISPUTE_ALERT": { "channels": ["EMAIL", "SMS", "PUSH"], "enabled": true },
    "TRUST_UPDATE": { "channels": ["IN_APP"], "enabled": false },
    "SYSTEM": { "channels": ["EMAIL", "IN_APP"], "enabled": true }
  },
  "quietHours": {
    "enabled": true,
    "start": "22:00",
    "end": "07:00",
    "timezone": "America/Los_Angeles",
    "exceptUrgent": true
  }
}
```

Update Preferences

PUT /api/v1/notifications/preferences

Authorization: Authenticated.

Request Body: Full preferences object (same structure as GET response).

Success Response (200): Updated preferences.

6. Event Schema (Async APIs)

Events are published on a Kafka/RabbitMQ event bus using the CloudEvents 1.0 specification. All events follow the **application/cloudevents+json** format.

6.1 Event Envelope

```
{
  "specversion": "1.0",
  "type": "com.dealplatform.{service}.{eventName}",
```

```

"source": "{service-name}/{service-instance-id}",
"id": "{uuid-v4}",
"time": "2024-01-15T10:30:00Z",
"datacontenttype": "application/json",
"data": { /* event payload */ },
"traceparent": "00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01"
}

```

6.2 Event Topics

Topic Pattern	Service	Consumers
deals.events	deal-mgmt-svc	All services
negotiation.events	negotiation-svc	deal-mgmt-svc, notification-svc
agreement.events	agreement-contract-svc	deal-mgmt-svc, payment-settlement-svc, notification-svc
execution.events	execution-milestone-svc	deal-mgmt-svc, payment-settlement-svc, notification-svc
payments.events	payment-settlement-svc	deal-mgmt-svc, execution-milestone-svc, notification-svc
matching.events	matching-discovery-svc	deal-mgmt-svc, notification-svc
trust.events	trust-reputation-svc	matching-discovery-svc, party-profile-svc
party.events	party-profile-svc	matching-discovery-svc, trust-reputation-svc
notifications.events	notification-svc	External integrations

6.3 Deal Events

DealCreated

```

{
  "specversion": "1.0",
  "type": "com.dealplatform.deal.created",
  "source": "deal-mgmt-svc/instance-1",
  "id": "evt-uuid-1",
  "time": "2024-01-15T10:30:00Z",
  "datacontenttype": "application/json",
  "data": {
    "eventId": "evt-uuid-1",
    "eventType": "DEAL_CREATED",
    "dealId": "550e8400-e29b-41d4-a716-446655440000",
    "dealReference": "DL-2024-0001",
    "dealTitle": "Farm Share Crop Agreement - Summer 2024",
    "dealStatus": "DRAFT",
    "initiatorPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
    "initiatorRole": "SUPPLIER",

```

```
"domainCategoryId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
"totalDealValue": 45000.00,
"currency": "USD",
"participations": [
  {
    "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
    "role": "SUPPLIER",
    "isInitiator": true,
    "participationStatus": "ACCEPTED"
  },
  {
    "partyId": "consumer-party-uuid-1",
    "role": "CONSUMER",
    "isInitiator": false,
    "participationStatus": "INVITED"
  },
  {
    "partyId": "enhancer-party-uuid-1",
    "role": "ENHANCER",
    "isInitiator": false,
    "participationStatus": "INVITED"
  }
],
"createdAt": "2024-01-15T10:30:00Z",
"correlationId": "req-uuid-1"
}
```

DealStateChanged

```
{
  "specversion": "1.0",
  "type": "com.dealplatform.deal.state_changed",
  "source": "deal-mgmt-svc/instance-1",
  "id": "evt-uuid-2",
  "time": "2024-01-15T14:00:00Z",
  "datacontenttype": "application/json",
  "data": {
    "eventId": "evt-uuid-2",
    "eventType": "DEAL_STATE_CHANGED",
    "dealId": "550e8400-e29b-41d4-a716-446655440000",
    "dealReference": "DL-2024-0001",
    "previousStatus": "DRAFT",
    "newStatus": "SUGGESTED",
    "transition": "SUBMIT",
    "triggeredByPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
    "triggeredByUserId": "user-uuid-1",
    "reason": "All parties have been identified and initial terms are ready for review",
    "occurredAt": "2024-01-15T14:00:00Z",
    "nextExpectedActions": [
      {
        "partyId": "consumer-party-uuid-1",
        "action": "ACKNOWLEDGE",

```

```
    "deadline": "2024-01-29T14:00:00Z"
  },
  {
    "partyId": "enhancer-party-uuid-1",
    "action": "ACKNOWLEDGE",
    "deadline": "2024-01-29T14:00:00Z"
  }
],
"autoTransitions": [],
"correlationId": "req-uuid-2"
}
}
```

DealPartyResponded

```
{
  "specversion": "1.0",
  "type": "com.dealplatform.deal.party_responded",
  "source": "deal-mgmt-svc/instance-1",
  "id": "evt-uuid-3",
  "time": "2024-01-16T09:30:00Z",
  "datacontenttype": "application/json",
  "data": {
    "eventId": "evt-uuid-3",
    "eventType": "DEAL_PARTY_RESPONDED",
    "dealId": "550e8400-e29b-41d4-a716-446655440000",
    "dealReference": "DL-2024-0001",
    "participationId": "part-uuid-2",
    "partyId": "consumer-party-uuid-1",
    "partyDisplayName": "FreshMart Grocery Chain",
    "role": "CONSUMER",
    "response": "ACCEPTED",
    "previousStatus": "INVITED",
    "newStatus": "ACCEPTED",
    "responseNotes": "Excited to collaborate. Ready to negotiate terms.",
    "allPartiesResponded": false,
    "pendingResponses": ["enhancer-party-uuid-1"],
    "respondedAt": "2024-01-16T09:30:00Z"
  }
}
```

DealCancelled

```
{
  "specversion": "1.0",
  "type": "com.dealplatform.deal.cancelled",
  "source": "deal-mgmt-svc/instance-1",
  "id": "evt-uuid-cancel",
  "time": "2024-01-20T10:00:00Z",
  "datacontenttype": "application/json",
  "data": {
    "eventId": "evt-uuid-cancel",
    "eventType": "DEAL_CANCELLED",

```



```
"dealId": "550e8400-e29b-41d4-a716-446655440000",
"dealReference": "DL-2024-0001",
"previousStatus": "NEGOTIATING",
"cancelledByPartyId": "consumer-party-uuid-1",
"cancelledByPartyName": "FreshMart Grocery Chain",
"reason": "WITHDRAW",
"description": "Changed business priorities. No longer pursuing this arrangement.",
"cancellationImpact": {
  "escrowRefundRequired": true,
  "refundAmount": 45000.00,
  "platformFeeRefund": true
},
"cancelledAt": "2024-01-20T10:00:00Z"
}
}
```

6.4 Negotiation Events

TermProposed

```
{
  "specversion": "1.0",
  "type": "com.dealplatform.negotiation.term_proposed",
  "source": "negotiation-svc/instance-1",
  "id": "evt-uuid-4",
  "time": "2024-01-17T14:30:00Z",
  "datacontenttype": "application/json",
  "data": {
    "eventId": "evt-uuid-4",
    "eventType": "TERM_PROPOSED",
    "dealId": "550e8400-e29b-41d4-a716-446655440000",
    "dealReference": "DL-2024-0001",
    "termId": "term-uuid-1",
    "termType": "PAYMENT_SCHEDULE",
    "termName": "3-Stage Payment with Escrow",
    "proposedByPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
    "proposedByPartyName": "Green Acres Farm Ltd",
    "version": 1,
    "parentTermId": null,
    "isCounter": false,
    "negotiationDimension": "VALUE_DISTRIBUTION",
    "proposedAt": "2024-01-17T14:30:00Z",
    "responseDeadline": "2024-01-20T14:30:00Z",
    "partiesToRespond": ["consumer-party-uuid-1", "enhancer-party-uuid-1"]
  }
}
```

TermAccepted

```
{
  "specversion": "1.0",
  "type": "com.dealplatform.negotiation.term_accepted",
  "source": "negotiation-svc/instance-1",
```

```
"id": "evt-uuid-5",
"time": "2024-01-18T10:00:00Z",
"datacontenttype": "application/json",
"data": {
  "eventId": "evt-uuid-5",
  "eventType": "TERM_ACCEPTED",
  "dealId": "550e8400-e29b-41d4-a716-446655440000",
  "termId": "term-uuid-1",
  "termName": "3-Stage Payment with Escrow",
  "acceptedByPartyId": "enhancer-party-uuid-1",
  "acceptedByPartyName": "AgriTech Solutions",
  "allPartiesAccepted": true,
  "termLocked": true,
  "comment": "Fair split. Ready to proceed.",
  "dimensionStatus": {
    "dimension": "VALUE_DISTRIBUTION",
    "status": "LOCKED",
    "termsInDimension": 1,
    "allAccepted": true
  },
  "acceptedAt": "2024-01-18T10:00:00Z"
}
```

TermsLocked

```
{
  "specversion": "1.0",
  "type": "com.dealplatform.negotiation.terms_locked",
  "source": "negotiation-svc/instance-1",
  "id": "evt-uuid-6",
  "time": "2024-01-20T16:00:00Z",
  "datacontenttype": "application/json",
  "data": {
    "eventId": "evt-uuid-6",
    "eventType": "TERMS_LOCKED",
    "dealId": "550e8400-e29b-41d4-a716-446655440000",
    "dealReference": "DL-2024-0001",
    "previousStatus": "NEGOTIATING",
    "newStatus": "TERMS_LOCKED",
    "lockedByPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
    "lockedAt": "2024-01-20T16:00:00Z",
    "totalTerms": 5,
    "lockedTerms": 5,
    "winWinWinValidation": {
      "status": "PASSED",
      "score": 87.5,
      "giniCoefficient": 0.15,
      "checkedAt": "2024-01-20T16:00:01Z"
    },
    "agreementGeneration": {
      "triggered": true,
      "estimatedCompletion": "2024-01-20T16:02:00Z"
    }
  }
}
```

```
    },  
    "commitmentDeadline": "2024-02-03T16:00:00Z"  
  }  
}
```

6.5 Agreement Events

AgreementSigned

```
{  
  "specversion": "1.0",  
  "type": "com.dealplatform.agreement.signed",  
  "source": "agreement-contract-svc/instance-1",  
  "id": "evt-uuid-7",  
  "time": "2024-01-22T14:00:00Z",  
  "datacontenttype": "application/json",  
  "data": {  
    "eventId": "evt-uuid-7",  
    "eventType": "AGREEMENT_SIGNED",  
    "agreementId": "agr-uuid-1",  
    "dealId": "550e8400-e29b-41d4-a716-446655440000",  
    "dealReference": "DL-2024-0001",  
    "signedByPartyId": "enhancer-party-uuid-1",  
    "signedByPartyName": "AgriTech Solutions",  
    "signedByUserId": "user-uuid-3",  
    "signatureType": "E_SIGN",  
    "signedAt": "2024-01-22T14:00:00Z",  
    "allPartiesSigned": true,  
    "totalSignatures": 3,  
    "agreementStatus": "FULLY_SIGNED",  
    "dealTransition": {  
      "fromStatus": "TERMS_LOCKED",  
      "toStatus": "COMMITTED",  
      "occurredAt": "2024-01-22T14:00:01Z"  
    },  
    "effectiveDate": "2024-01-25T00:00:00Z",  
    "executionStartDate": "2024-01-25T00:00:00Z",  
    "escrowFundingRequired": true,  
    "escrowFundingDeadline": "2024-01-24T23:59:59Z",  
    "escrowAmount": 45000.00  
  }  
}
```

AgreementVoided

```
{  
  "specversion": "1.0",  
  "type": "com.dealplatform.agreement.voided",  
  "source": "agreement-contract-svc/instance-1",  
  "id": "evt-uuid-void",  
  "time": "2024-01-23T10:00:00Z",  
  "datacontenttype": "application/json",  
  "data": {
```

```
"eventId": "evt-uuid-void",
"eventType": "AGREEMENT_VOIDED",
"agreementId": "agr-uuid-1",
"dealId": "550e8400-e29b-41d4-a716-446655440000",
"voidedBy": "mutual_consent",
"voidedAt": "2024-01-23T10:00:00Z",
"reason": "MUTUAL_CONSENT",
"refundTriggered": true,
"refundAmount": 45000.00,
"dealStatus": "CANCELLED"
}
}
```

6.6 Execution Events

MilestoneCompleted

```
{
  "specversion": "1.0",
  "type": "com.dealplatform.execution.milestone_completed",
  "source": "execution-milestone-svc/instance-1",
  "id": "evt-uuid-8",
  "time": "2024-03-14T18:00:00Z",
  "datacontenttype": "application/json",
  "data": {
    "eventId": "evt-uuid-8",
    "eventType": "MILESTONE_COMPLETED",
    "milestoneId": "ms-uuid-1",
    "dealId": "550e8400-e29b-41d4-a716-446655440000",
    "dealReference": "DL-2024-0001",
    "milestoneName": "Soil Preparation & Planting Complete",
    "completedByPartyId": "enhancer-party-uuid-1",
    "completedByPartyName": "AgriTech Solutions",
    "completionPercentage": 100,
    "completedAt": "2024-03-14T18:00:00Z",
    "evidenceSubmitted": {
      "documentCount": 3,
      "photoCount": 10,
      "description": "All planting completed on schedule"
    },
    "verificationRequired": true,
    "verificationRequiredFrom": ["7c9e6679-7425-40de-944b-e07fc1f90ae7", "consumer-
party-uuid-1"],
    "verificationDeadline": "2024-03-17T18:00:00Z",
    "paymentTrigger": {
      "amount": 9000.00,
      "currency": "USD",
      "status": "PENDING_VERIFICATION",
      "willReleaseOnVerification": true
    }
  }
}
```

MilestoneVerified

```
{
  "specversion": "1.0",
  "type": "com.dealplatform.execution.milestone_verified",
  "source": "execution-milestone-svc/instance-1",
  "id": "evt-uuid-9",
  "time": "2024-03-15T10:00:00Z",
  "datacontenttype": "application/json",
  "data": {
    "eventId": "evt-uuid-9",
    "eventType": "MILESTONE_VERIFIED",
    "milestoneId": "ms-uuid-1",
    "dealId": "550e8400-e29b-41d4-a716-446655440000",
    "milestoneName": "Soil Preparation & Planting Complete",
    "verifiedByPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
    "verifiedByPartyName": "Green Acres Farm Ltd",
    "verification": "APPROVED",
    "sufficientVerifications": true,
    "paymentReleased": true,
    "payment": {
      "transactionId": "txn-uuid-1",
      "amount": 9000.00,
      "currency": "USD",
      "toPartyId": "enhancer-party-uuid-1",
      "status": "COMPLETED",
      "releasedAt": "2024-03-15T10:00:05Z"
    },
    "overallDealProgress": {
      "completedMilestones": 1,
      "totalMilestones": 3,
      "percentage": 33
    },
    "nextMilestone": {
      "milestoneId": "ms-uuid-2",
      "milestoneName": "Mid-Season Harvest",
      "dueDate": "2024-06-01"
    }
  }
}
```

DealExecutionStarted

```
{
  "specversion": "1.0",
  "type": "com.dealplatform.execution.started",
  "source": "execution-milestone-svc/instance-1",
  "id": "evt-uuid-exec",
  "time": "2024-01-25T00:00:00Z",
  "datacontenttype": "application/json",
  "data": {
    "eventId": "evt-uuid-exec",
    "eventType": "DEAL_EXECUTION_STARTED",
    "dealId": "550e8400-e29b-41d4-a716-446655440000",

```

```
"dealReference": "DL-2024-0001",
"previousStatus": "COMMITTED",
"newStatus": "EXECUTING",
"startedAt": "2024-01-25T00:00:00Z",
"expectedCompletionDate": "2024-08-31",
"milestones": [
  {
    "milestoneId": "ms-uuid-1",
    "milestoneName": "Soil Preparation & Planting Complete",
    "dueDate": "2024-03-15",
    "assignedTo": "enhancer-party-uuid-1"
  },
  {
    "milestoneId": "ms-uuid-2",
    "milestoneName": "Mid-Season Harvest",
    "dueDate": "2024-06-01",
    "assignedTo": "enhancer-party-uuid-1"
  },
  {
    "milestoneId": "ms-uuid-3",
    "milestoneName": "Final Harvest & Delivery",
    "dueDate": "2024-08-31",
    "assignedTo": "enhancer-party-uuid-1"
  }
],
"escrowStatus": "FULLY_FUNDED",
"totalEscrowAmount": 45000.00
}
```

DealCompleted

```
{
  "specversion": "1.0",
  "type": "com.dealplatform.execution.deal_completed",
  "source": "execution-milestone-svc/instance-1",
  "id": "evt-uuid-complete",
  "time": "2024-08-31T20:00:00Z",
  "datacontenttype": "application/json",
  "data": {
    "eventId": "evt-uuid-complete",
    "eventType": "DEAL_COMPLETED",
    "dealId": "550e8400-e29b-41d4-a716-446655440000",
    "dealReference": "DL-2024-0001",
    "dealTitle": "Farm Share Crop Agreement - Summer 2024",
    "previousStatus": "EXECUTING",
    "newStatus": "COMPLETED",
    "completedAt": "2024-08-31T20:00:00Z",
    "actualStartDate": "2024-01-25T00:00:00Z",
    "actualEndDate": "2024-08-31T20:00:00Z",
    "durationDays": 219,
    "totalDealValue": 45000.00,
    "totalReleased": 45000.00,
  }
}
```

```

"milestoneSummary": {
  "total": 3,
  "completedOnTime": 2,
  "completedLate": 1,
  "averageDelayDays": 3
},
"allMilestones": [
  { "milestoneId": "ms-uuid-1", "name": "Soil Preparation & Planting", "status":
"COMPLETED", "completedAt": "2024-03-14T18:00:00Z", "onTime": true },
  { "milestoneId": "ms-uuid-2", "name": "Mid-Season Harvest", "status":
"COMPLETED", "completedAt": "2024-06-03T10:00:00Z", "onTime": false, "delayDays": 2
},
  { "milestoneId": "ms-uuid-3", "name": "Final Harvest & Delivery", "status":
"COMPLETED", "completedAt": "2024-08-31T20:00:00Z", "onTime": true }
],
"reviewRequestsSent": true,
"nextActions": [
  {
    "action": "SUBMIT_REVIEW",
    "description": "Submit reviews for your deal partners",
    "deadline": "2024-09-14T20:00:00Z"
  }
]
}

```

6.7 Payment Events

PaymentReleased

```

{
  "specversion": "1.0",
  "type": "com.dealplatform.payment.released",
  "source": "payment-settlement-svc/instance-1",
  "id": "evt-uuid-10",
  "time": "2024-03-15T10:00:05Z",
  "datacontenttype": "application/json",
  "data": {
    "eventId": "evt-uuid-10",
    "eventType": "PAYMENT_RELEASED",
    "transactionId": "txn-uuid-1",
    "escrowId": "escrow-uuid-1",
    "dealId": "550e8400-e29b-41d4-a716-446655440000",
    "dealReference": "DL-2024-0001",
    "amount": 9000.00,
    "currency": "USD",
    "fromEscrow": true,
    "toPartyId": "enhancer-party-uuid-1",
    "toPartyName": "AgriTech Solutions",
    "toWalletId": "wallet-enhancer-1",
    "trigger": {
      "type": "MILESTONE_VERIFIED",
      "milestoneId": "ms-uuid-1",

```

```
"milestoneName": "Soil Preparation & Planting Complete"
},
"status": "COMPLETED",
"releasedAt": "2024-03-15T10:00:05Z",
"remainingEscrow": 36000.00
}
}
```

EscrowFunded

```
{
  "specversion": "1.0",
  "type": "com.dealplatform.payment.escrow_funded",
  "source": "payment-settlement-svc/instance-1",
  "id": "evt-uuid-escrow",
  "time": "2024-01-24T10:30:00Z",
  "datacontenttype": "application/json",
  "data": {
    "eventId": "evt-uuid-escrow",
    "eventType": "ESCROW_FUNDED",
    "escrowId": "escrow-uuid-1",
    "dealId": "550e8400-e29b-41d4-a716-446655440000",
    "fundedAmount": 45000.00,
    "totalRequired": 45000.00,
    "fundingPartyId": "consumer-party-uuid-1",
    "fundingPartyName": "FreshMart Grocery Chain",
    "paymentMethod": "BANK_TRANSFER",
    "status": "FULLY_FUNDED",
    "fundedAt": "2024-01-24T10:30:00Z",
    "dealTransition": {
      "toStatus": "EXECUTING",
      "trigger": "Begin execution on committed date"
    }
  }
}
```

6.8 Matching Events

MatchSuggested

```
{
  "specversion": "1.0",
  "type": "com.dealplatform.matching.suggested",
  "source": "matching-discovery-svc/instance-1",
  "id": "evt-uuid-11",
  "time": "2024-01-15T10:00:00Z",
  "datacontenttype": "application/json",
  "data": {
    "eventId": "evt-uuid-11",
    "eventType": "MATCH_SUGGESTED",
    "matchId": "match-uuid-1",
    "matchScore": 0.92,
    "supplierPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",

```



```
"consumerPartyId": "consumer-party-uuid-1",
"enhancerPartyId": "enhancer-party-uuid-1",
"suggestedDealValue": 42000.00,
"matchReason": "High compatibility based on: organic farming expertise, geographic
proximity (12km)...",
"generatedBy": "ALGORITHM",
"expiresAt": "2024-02-15T10:00:00Z",
"notifiedParties": ["7c9e6679-7425-40de-944b-e07fc1f90ae7"],
"createdAt": "2024-01-15T10:00:00Z"
}
}
```

MatchAccepted

```
{
  "specversion": "1.0",
  "type": "com.dealplatform.matching.accepted",
  "source": "matching-discovery-svc/instance-1",
  "id": "evt-uuid-12",
  "time": "2024-01-15T14:00:00Z",
  "datacontenttype": "application/json",
  "data": {
    "eventId": "evt-uuid-12",
    "eventType": "MATCH_ACCEPTED",
    "matchId": "match-uuid-1",
    "acceptedByPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
    "acceptedByPartyName": "Green Acres Farm Ltd",
    "allPartiesAccepted": false,
    "pendingResponses": ["consumer-party-uuid-1", "enhancer-party-uuid-1"],
    "matchStatus": "PENDING",
    "autoCreateDealOnFullAccept": true,
    "acceptedAt": "2024-01-15T14:00:00Z"
  }
}
```

MatchConvertedToDeal

```
{
  "specversion": "1.0",
  "type": "com.dealplatform.matching.converted_to_deal",
  "source": "matching-discovery-svc/instance-1",
  "id": "evt-uuid-match-convert",
  "time": "2024-01-15T18:00:00Z",
  "datacontenttype": "application/json",
  "data": {
    "eventId": "evt-uuid-match-convert",
    "eventType": "MATCH_CONVERTED_TO_DEAL",
    "matchId": "match-uuid-1",
    "dealId": "new-deal-uuid-1",
    "dealReference": "DL-2024-0002",
    "allPartiesAccepted": true,
    "convertedAt": "2024-01-15T18:00:00Z"
  }
}
```

```
}  
}
```

6.9 Trust & Review Events

ReviewSubmitted

```
{  
  "specversion": "1.0",  
  "type": "com.dealplatform.trust.review_submitted",  
  "source": "trust-reputation-svc/instance-1",  
  "id": "evt-uuid-13",  
  "time": "2024-09-05T10:00:00Z",  
  "datacontenttype": "application/json",  
  "data": {  
    "eventId": "evt-uuid-13",  
    "eventType": "REVIEW_SUBMITTED",  
    "reviewId": "rev-uuid-1",  
    "dealId": "550e8400-e29b-41d4-a716-446655440000",  
    "reviewerPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",  
    "reviewerPartyName": "Green Acres Farm Ltd",  
    "reviewedPartyId": "enhancer-party-uuid-1",  
    "reviewedPartyName": "AgriTech Solutions",  
    "reviewedRole": "ENHANCER",  
    "overallRating": 5,  
    "isVerified": true,  
    "trustScoreUpdated": true,  
    "previousTrustScore": 4.30,  
    "newTrustScore": 4.35,  
    "submittedAt": "2024-09-05T10:00:00Z"  
  }  
}
```

TrustScoreUpdated

```
{  
  "specversion": "1.0",  
  "type": "com.dealplatform.trust.score_updated",  
  "source": "trust-reputation-svc/instance-1",  
  "id": "evt-uuid-ts",  
  "time": "2024-09-05T10:05:00Z",  
  "datacontenttype": "application/json",  
  "data": {  
    "eventId": "evt-uuid-ts",  
    "eventType": "TRUST_SCORE_UPDATED",  
    "partyId": "enhancer-party-uuid-1",  
    "partyName": "AgriTech Solutions",  
    "previousOverallScore": 4.30,  
    "newOverallScore": 4.35,  
    "change": 0.05,  
    "previousRoleScores": {  
      "asSupplierScore": null,  
      "asConsumerScore": null,  
    }  
  }  
}
```

```

    "asEnhancerScore": 4.30
  },
  "newRoleScores": {
    "asSupplierScore": null,
    "asConsumerScore": null,
    "asEnhancerScore": 4.35
  },
  "trigger": {
    "type": "REVIEW_SUBMITTED",
    "reviewId": "rev-uuid-1",
    "rating": 5
  },
  "updatedAt": "2024-09-05T10:05:00Z",
  "nextScheduledUpdate": "2024-10-05T10:05:00Z"
}
}

```

6.10 Dispute Events

DisputeOpened

```

{
  "specversion": "1.0",
  "type": "com.dealplatform.dispute.opened",
  "source": "deal-mgmt-svc/instance-1",
  "id": "evt-uuid-dispute",
  "time": "2024-05-15T10:00:00Z",
  "datacontenttype": "application/json",
  "data": {
    "eventId": "evt-uuid-dispute",
    "eventType": "DISPUTE_OPENED",
    "disputeId": "dispute-uuid-1",
    "dealId": "550e8400-e29b-41d4-a716-446655440000",
    "dealReference": "DL-2024-0001",
    "raisedByPartyId": "consumer-party-uuid-1",
    "raisedByPartyName": "FreshMart Grocery Chain",
    "againstPartyId": "enhancer-party-uuid-1",
    "againstPartyName": "AgriTech Solutions",
    "disputeType": "QUALITY_ISSUE",
    "description": "Tomato yield is 20% below projected. Quality of lettuce is acceptable but pepper delivery is 3 weeks late.",
    "disputeStatus": "OPEN",
    "goodFaithDepositPaid": true,
    "depositAmount": 900.00,
    "dealStatusChanged": true,
    "previousDealStatus": "EXECUTING",
    "newDealStatus": "DISPUTED",
    "openedAt": "2024-05-15T10:00:00Z",
    "mediatorAssigned": false,
    "resolutionDeadline": "2024-05-29T10:00:00Z"
  }
}

```

DisputeResolved

```
{
  "specversion": "1.0",
  "type": "com.dealplatform.dispute.resolved",
  "source": "deal-mgmt-svc/instance-1",
  "id": "evt-uuid-resolved",
  "time": "2024-05-22T14:00:00Z",
  "datacontenttype": "application/json",
  "data": {
    "eventId": "evt-uuid-resolved",
    "eventType": "DISPUTE_RESOLVED",
    "disputeId": "dispute-uuid-1",
    "dealId": "550e8400-e29b-41d4-a716-446655440000",
    "disputeType": "QUALITY_ISSUE",
    "resolutionType": "PARTIAL_REFUND",
    "resolution": "Enhancer to refund $3,000 for yield shortfall. Remaining deal continues with adjusted milestones. Late pepper delivery accepted with 5% discount on final payment.",
    "resolvedBy": "mediator-uuid-1",
    "resolvedByName": "Platform Mediator",
    "compensationAmount": 3000.00,
    "compensatedToPartyId": "consumer-party-uuid-1",
    "goodFaithDepositRefund": true,
    "dealStatusChanged": true,
    "previousDealStatus": "DISPUTED",
    "newDealStatus": "EXECUTING",
    "resolvedAt": "2024-05-22T14:00:00Z",
    "daysToResolve": 7
  }
}
```

6.11 Party Events

PartyCreated

```
{
  "specversion": "1.0",
  "type": "com.dealplatform.party.created",
  "source": "party-profile-svc/instance-1",
  "id": "evt-uuid-party",
  "time": "2024-01-15T10:30:00Z",
  "datacontenttype": "application/json",
  "data": {
    "eventId": "evt-uuid-party",
    "eventType": "PARTY_CREATED",
    "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
    "partyType": "INDIVIDUAL",
    "displayName": "Green Acres Farm Ltd",
    "roles": ["SUPPLIER", "CONSUMER"],
    "createdByUserId": "user-uuid-1",
    "createdAt": "2024-01-15T10:30:00Z"
  }
}
```

PartyRoleAssigned

```
{
  "specversion": "1.0",
  "type": "com.dealplatform.party.role_assigned",
  "source": "party-profile-svc/instance-1",
  "id": "evt-uuid-role",
  "time": "2024-06-10T14:30:00Z",
  "datacontenttype": "application/json",
  "data": {
    "eventId": "evt-uuid-role",
    "eventType": "PARTY_ROLE_ASSIGNED",
    "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
    "roleType": "ENHANCER",
    "assignedAt": "2024-06-10T14:30:00Z",
    "totalActiveRoles": 3
  }
}
```

PartyVerified

```
{
  "specversion": "1.0",
  "type": "com.dealplatform.party.verified",
  "source": "party-profile-svc/instance-1",
  "id": "evt-uuid-verify",
  "time": "2024-02-01T14:22:00Z",
  "datacontenttype": "application/json",
  "data": {
    "eventId": "evt-uuid-verify",
    "eventType": "PARTY_VERIFIED",
    "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
    "verificationLevel": "PREMIUM",
    "provider": "onfido",
    "verifiedAt": "2024-02-01T14:22:00Z",
    "documents": ["identity", "address", "business_registration"],
    "trustScoreImpact": {
      "previousScore": 3.50,
      "newScore": 4.20,
      "verificationBonus": 0.70
    }
  }
}
```

6.12 Event Consumption Patterns

Consumer Service	Events Consumed	Action
notification-svc	All events	Generate and deliver notifications
search-svc	deal., party., matching.*	Update search indexes

Consumer Service	Events Consumed	Action
analytics-svc	All events	Compute analytics and reports
trust-reputation-svc	execution., <i>dispute.</i>	Recalculate trust scores
matching-discovery-svc	party., <i>trust.</i>	Update match candidate pools
payment-settlement-svc	agreement.signed, milestone.verified	Trigger payments
admin-svc	dispute., <i>trust.</i>	Admin dashboards and alerts

6. Webhook Contracts

Webhooks allow external systems to receive real-time notifications about platform events. All webhooks use HTTPS POST with JSON payloads and HMAC-SHA256 signature verification.

6.1 Webhook Configuration

Register Webhook

POST /api/v1/webhooks

Authorization: Authenticated with **webhooks:manage** scope.

Request Body:

```
{
  "url": "https://client-crm.example.com/webhooks/dealplatform",
  "events": [
    "deal.state_changed",
    "agreement.signed",
    "payment.released",
    "milestone.completed"
  ],
  "description": "CRM integration for deal tracking",
  "active": true,
  "secret": "whsec_your_webhook_secret_here",
  "metadata": {
    "integration": "salesforce",
    "environment": "production"
  }
}
```

Success Response (201):

```
{
  "webhookId": "wh-uuid-1",
  "url": "https://client-crm.example.com/webhooks/dealplatform",
  "events": ["deal.state_changed", "agreement.signed", "payment.released",
```

```
"milestone.completed"],
"description": "CRM integration for deal tracking",
"active": true,
"createdAt": "2024-01-15T10:30:00Z",
"secretHint": "whsec_****_here"
}
```

6.2 Webhook Delivery Format

All webhook deliveries follow this envelope format:

```
{
  "webhookId": "wh-uuid-1",
  "eventId": "evt-uuid-1",
  "eventType": "deal.state_changed",
  "timestamp": "2024-01-15T14:00:00Z",
  "deliveryAttempt": 1,
  "data": { /* event payload - see Event Schema section */ }
}
```

Headers:

```
Content-Type: application/json
X-Webhook-Id: wh-uuid-1
X-Event-Id: evt-uuid-1
X-Event-Type: deal.state_changed
X-Delivery-Attempt: 1
X-Webhook-Signature: v1=sha256=hex-digest
X-Webhook-Timestamp: 1705333200
```

Signature Verification:

```
import hmac
import hashlib
import base64

def verify_signature(payload_body, secret, signature_header, timestamp_header):
    # Expected signature format: "v1=sha256=hex-digest"
    timestamp = timestamp_header
    signed_payload = f"{timestamp}.{payload_body}"
    expected_signature = hmac.new(
        secret.encode('utf-8'),
        signed_payload.encode('utf-8'),
        hashlib.sha256
    ).hexdigest()

    # Compare using constant-time comparison
    return hmac.compare_digest(
        f"v1=sha256={expected_signature}",
        signature_header
    )
```

Retry Policy: | Attempt | Delay | Action | |———|———|———| | 1 | Immediate | First delivery
| 2 | 5 seconds | Retry on 5xx or timeout | | 3 | 30 seconds | Retry | | 4 | 2 minutes | Retry | | 5 |
10 minutes | Final retry | | >5 | — | Mark as failed, alert webhook owner |

6.3 Payment Provider Webhooks (Inbound)

The platform receives webhooks from external payment providers.

Stripe Webhook

POST /webhooks/payments/stripe

Authorization: Stripe signature verification (**Stripe-Signature** header).

Payload Examples:

Payment Intent Succeeded:

```
{
  "id": "evt_stripe_1",
  "object": "event",
  "type": "payment_intent.succeeded",
  "data": {
    "object": {
      "id": "pi_1234567890",
      "amount": 4500000,
      "amount_received": 4500000,
      "currency": "usd",
      "status": "succeeded",
      "charges": {
        "data": [{"id": "ch_1234567890", "receipt_url": "https://pay.stripe.com/receipts/..."}]
      },
      "metadata": {
        "escrowId": "escrow-uuid-1",
        "dealId": "550e8400-e29b-41d4-a716-446655440000",
        "partyId": "consumer-party-uuid-1"
      }
    }
  }
}
```

Payment Intent Failed:

```
{
  "id": "evt_stripe_2",
  "object": "event",
  "type": "payment_intent.payment_failed",
  "data": {
    "object": {
      "id": "pi_1234567890",
      "amount": 4500000,
      "currency": "usd",
      "status": "requires_payment_method",
      "last_payment_error": {
        "message": "Your card was declined."
      }
    }
  }
}
```



```
    "decline_code": "insufficient_funds"
  },
  "metadata": {
    "escrowId": "escrow-uuid-1",
    "dealId": "550e8400-e29b-41d4-a716-446655440000"
  }
}
```

Platform Response:

```
{
  "received": true,
  "processed": true,
  "escrowId": "escrow-uuid-1",
  "actionTaken": "MARKED_AS_FUNDED"
}
```

Adyen Webhook

POST /webhooks/payments/adyen

Authorization: Adyen HMAC signature verification.

Payload Example:

```
{
  "live": "false",
  "notificationItems": [
    {
      "NotificationRequestItem": {
        "amount": {
          "currency": "USD",
          "value": 4500000
        },
        "eventCode": "AUTHORISATION",
        "merchantAccount": "DealPlatformMerchant",
        "merchantReference": "escrow-uuid-1",
        "paymentMethod": "visa",
        "pspReference": "adyen-ref-123",
        "success": "true"
      }
    }
  ]
}
```

6.4 E-Signature Webhooks (Inbound)

DocuSign Webhook

POST /webhooks/signatures/docusign

Authorization: DocuSign HMAC verification.

Payload Examples:

Envelope Completed (All Signed):

```
{
  "event": "envelope-completed",
  "apiVersion": "v2.1",
  "uri": "/restapi/v2.1/accounts/12345/envelopes/env-uuid-1",
  "retryCount": 0,
  "configurationId": 12345,
  "generatedDateTime": "2024-01-22T14:00:00.0000000Z",
  "data": {
    "accountId": "12345",
    "userId": "user-uuid-1",
    "envelopeId": "env-uuid-1",
    "envelopeSummary": {
      "status": "completed",
      "documentsUri": "/envelopes/env-uuid-1/documents",
      "recipientsUri": "/envelopes/env-uuid-1/recipients",
      "envelopeUri": "/envelopes/env-uuid-1",
      "envelopeId": "env-uuid-1",
      "customFieldsUri": "/envelopes/env-uuid-1/custom_fields",
      "notificationUri": "/envelopes/env-uuid-1/notification",
      "statusChangedDateTime": "2024-01-22T14:00:00.0000000Z",
      "documentsCombinedUri": "/envelopes/env-uuid-1/documents/combined",
      "certificateUri": "/envelopes/env-uuid-1/documents/certificate",
      "templatesUri": "/envelopes/env-uuid-1/templates",
      "recipients": {
        "signers": [
          {
            "signatureInfo": {
              "signatureName": "Jane Smith",
              "signatureInitials": "JS",
              "fontStyle": "DancingScript_Medium"
            },
            "recipientId": "1",
            "email": "jane@greenacres.example.com",
            "userId": "docusign-user-1",
            "routingOrder": "1",
            "deliveredDateTime": "2024-01-21T09:30:00.0000000Z",
            "signedDateTime": "2024-01-21T09:30:00.0000000Z",
            "status": "completed",
            "completedCount": "1",
            "name": "Jane Smith"
          }
        ]
      }
    }
  }
}
```

Platform Response:

```
{
  "received": true,
  "processed": true,
  "agreementId": "agr-uuid-1",
  "signatureStatus": "PARTIALLY_SIGNED",
  "signaturesReceived": 1,
  "signaturesRequired": 3
}
```

6.5 KYC Verification Webhooks (Inbound)

Onfido Webhook

POST /webhooks/kyc/onfido

Authorization: Onfido signature token verification.

Payload Examples:

Check Completed:

```
{
  "payload": {
    "resource_type": "check",
    "action": "check.completed",
    "object": {
      "id": "check-uuid-1",
      "created_at": "2024-01-20T10:00:00Z",
      "href": "/v3/checks/check-uuid-1",
      "status": "complete",
      "result": "clear",
      "form_uri": null,
      "redirect_uri": null,
      "results_uri": "https://dashboard.onfido.com/checks/check-uuid-1",
      "applicant_id": "applicant-uuid-1",
      "tags": [],
      "applicant_provides_data": false,
      "reports": [
        {
          "id": "report-uuid-1",
          "name": "document",
          "created_at": "2024-01-20T10:00:00Z",
          "href": "/v3/reports/report-uuid-1",
          "status": "complete",
          "result": "clear",
          "sub_result": "clear",
          "breakdown": {},
          "documents": ["doc-uuid-1"],
          "check_id": "check-uuid-1"
        }
      ],
      "check_type": "standard"
    }
  }
}
```

```
}  
}
```

Platform Response:

```
{  
  "received": true,  
  "processed": true,  
  "verificationId": "verif-uuid-1",  
  "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",  
  "status": "VERIFIED",  
  "result": "clear"  
}
```

Jumio Webhook

POST /webhooks/kyc/jumio

Payload Example:

```
{  
  "transactionReference": "jumio-ref-123",  
  "customerInternalReference": "verif-uuid-1",  
  "merchantReportingCriteria": "party-verification",  
  "callbackType": "NETVERIFYID",  
  "verificationStatus": "APPROVED_VERIFIED",  
  "document": {  
    "status": "APPROVED_VERIFIED",  
    "type": "ID_CARD",  
    "firstName": "Jane",  
    "lastName": "Smith"  
  }  
}
```

6.6 Enterprise Client Webhooks (Outbound)

For enterprise clients, the platform can deliver structured webhooks to their systems.

Deal Lifecycle Webhook

```
{  
  "webhookId": "wh-uuid-1",  
  "eventId": "evt-uuid-2",  
  "eventType": "deal.state_changed",  
  "timestamp": "2024-01-15T14:00:00Z",  
  "deliveryAttempt": 1,  
  "data": {  
    "dealId": "550e8400-e29b-41d4-a716-446655440000",  
    "dealReference": "DL-2024-0001",  
    "dealTitle": "Farm Share Crop Agreement - Summer 2024",  
    "previousStatus": "DRAFT",  
    "newStatus": "SUGGESTED",  
    "transition": "SUBMIT",  
    "triggeredByPartyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",  
    "triggeredByPartyName": "Green Acres Farm Ltd",  
  }  
}
```

```

"parties": [
  {
    "partyId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
    "displayName": "Green Acres Farm Ltd",
    "role": "SUPPLIER",
    "externalRefId": "CRM-SUPPLIER-001"
  },
  {
    "partyId": "consumer-party-uuid-1",
    "displayName": "FreshMart Grocery Chain",
    "role": "CONSUMER",
    "externalRefId": "CRM-CONSUMER-001"
  },
  {
    "partyId": "enhancer-party-uuid-1",
    "displayName": "AgriTech Solutions",
    "role": "ENHANCER",
    "externalRefId": "CRM-ENHANCER-001"
  }
],
"totalDealValue": 45000.00,
"currency": "USD",
"occurredAt": "2024-01-15T14:00:00Z",
"nextExpectedActions": [
  {
    "partyId": "consumer-party-uuid-1",
    "action": "ACKNOWLEDGE",
    "deadline": "2024-01-29T14:00:00Z"
  }
]
}

```

Note: Enterprise webhooks can include **externalRefId** fields that map platform parties to the client's internal CRM/ERP system IDs.

Payment Webhook

```

{
  "webhookId": "wh-uuid-1",
  "eventId": "evt-uuid-10",
  "eventType": "payment.released",
  "timestamp": "2024-03-15T10:00:05Z",
  "deliveryAttempt": 1,
  "data": {
    "transactionId": "txn-uuid-1",
    "escrowId": "escrow-uuid-1",
    "dealId": "550e8400-e29b-41d4-a716-446655440000",
    "dealReference": "DL-2024-0001",
    "amount": 9000.00,
    "currency": "USD",
    "toPartyId": "enhancer-party-uuid-1",
    "toPartyName": "AgriTech Solutions",
  }
}

```

```
"toPartyExternalRefId": "CRM-ENHANCER-001",
"trigger": {
  "type": "MILESTONE_VERIFIED",
  "milestoneId": "ms-uuid-1",
  "milestoneName": "Soil Preparation & Planting Complete"
},
"invoiceReference": "INV-2024-0042",
"releasedAt": "2024-03-15T10:00:05Z",
"remainingEscrow": 36000.00
}
}
```

6.7 Webhook Delivery Log

Get Webhook Delivery History

GET /api/v1/webhooks/{webhookId}/deliveries

Authorization: Webhook owner.

Success Response (200):

```
{
  "data": [
    {
      "deliveryId": "del-uuid-1",
      "eventId": "evt-uuid-2",
      "eventType": "deal.state_changed",
      "status": "DELIVERED",
      "httpStatus": 200,
      "responseBody": "{\"received\":true}",
      "attemptNumber": 1,
      "sentAt": "2024-01-15T14:00:01Z",
      "deliveredAt": "2024-01-15T14:00:02Z",
      "latencyMs": 450
    },
    {
      "deliveryId": "del-uuid-2",
      "eventId": "evt-uuid-failed",
      "eventType": "payment.released",
      "status": "FAILED",
      "httpStatus": 500,
      "responseBody": "Internal Server Error",
      "attemptNumber": 5,
      "sentAt": "2024-03-15T10:15:00Z",
      "failedAt": "2024-03-15T10:15:05Z",
      "error": "Remote server returned 500"
    }
  ]
}
```

Redeliver Failed Webhook

POST /api/v1/webhooks/{webhookId}/deliveries/{deliveryId}/redeliver

Authorization: Webhook owner.

Success Response (202): Queued for redelivery.

6. GraphQL Schema

The platform provides a GraphQL API for complex read operations that require nested data fetching, flexible filtering, and aggregation queries.

Endpoint: POST /graphql/v1 **Authentication:** Same as REST API (Bearer token + X-Party-ID header) **Content-Type:** application/json **Rate Limiting:** Shared with REST API quota

6.1 Schema Overview

```
schema {  
  query: Query  
  mutation: Mutation  
  subscription: Subscription  
}
```

6.2 Query Types

```
type Query {  
  # Deal queries  
  deal(dealId: ID!): Deal  
  deals(  
    filter: DealFilterInput  
    sort: DealSortInput  
    pagination: PaginationInput  
  ): DealConnection!  
  
  # Party queries  
  party(partyId: ID!): Party  
  parties(  
    filter: PartyFilterInput  
    sort: PartySortInput  
    pagination: PaginationInput  
  ): PartyConnection!  
  
  # Search  
  search(  
    query: String  
    filter: SearchFilterInput  
    sort: SearchSortInput  
    pagination: PaginationInput  
  ): SearchResultConnection!  
  
  # Analytics  
  analytics(  
    type: AnalyticsType!  
    filter: AnalyticsFilterInput  
    granularity: Granularity  
  ): AnalyticsResult!
```

```

# Matching
suggestedMatches(
  partyId: ID!
  role: Role
  filter: MatchFilterInput
  pagination: PaginationInput
): MatchConnection!

compatibilityScore(
  supplierPartyId: ID!
  consumerPartyId: ID!
  enhancerPartyId: ID!
): CompatibilityScore!

# Trust & Reviews
trustScore(partyId: ID!): TrustScore
reviews(
  partyId: ID!
  filter: ReviewFilterInput
  pagination: PaginationInput
): ReviewConnection!

# Notifications
notifications(
  filter: NotificationFilterInput
  pagination: PaginationInput
): NotificationConnection!

# Dashboard
dashboard(
  partyId: ID!
  period: DashboardPeriod
): Dashboard!
}

```

6.3 Deal Types

```

type Deal {
  dealId: ID!
  dealReference: String!
  dealTitle: String!
  dealDescription: String
  dealStatus: DealStatus!
  domainCategory: Category
  initiatorParty: Party!
  initiatorRole: Role!

  # Dates
  expectedStartDate: Date
  expectedEndDate: Date
  actualStartDate: Date
  actualEndDate: Date
}

```


Financials

totalDealValue: Decimal
currency: String!
platformFeePercentage: Decimal
platformFeeAmount: Decimal

Validation

winWinWinValidated: Boolean!
validationCheckedAt: DateTime

Related entities (nested)

participations: [DealParticipation!]!
resource: Resource
need: Need
enhancement: Enhancement
terms(
 filter: TermFilterInput
 sort: TermSortInput
): [Term!]!
valueDistribution: ValueDistribution
milestones(
 filter: MilestoneFilterInput
 sort: MilestoneSortInput
): [Milestone!]!
agreement: Agreement
transactions: [Transaction!]!
reviews: [Review!]!
disputes: [Dispute!]!
messages: [Message!]!
history(
 filter: HistoryFilterInput
 pagination: PaginationInput
): [HistoryEvent!]!

Computed fields

availableActions: [AvailableAction!]!
progressPercentage: Decimal
daysInCurrentStatus: Int
isOverdue: Boolean

Metadata

isPublic: Boolean!
location: Location
createdAt: DateTime!
updatedAt: DateTime!

}

type DealParticipation {
 participationId: ID!
 party: Party!
 role: Role!

```
participationStatus: ParticipationStatus!  
isInitiator: Boolean!  
valueSharePercentage: Decimal  
valueShareAmount: Decimal  
invitedAt: DateTime  
respondedAt: DateTime  
}
```

```
type Term {  
  termId: ID!  
  termType: TermType!  
  termName: String!  
  description: String!  
  proposedByParty: Party!  
  negotiationStatus: NegotiationStatus!  
  version: Int!  
  parentTerm: Term  
  isMandatory: Boolean!  
  proposedAt: DateTime!  
  resolvedAt: DateTime  
  resolution: String  
  responses: [TermResponse!]!  
  versionHistory: [Term!]!  
}
```

```
type TermResponse {  
  party: Party!  
  response: ResponseType!  
  respondedAt: DateTime  
  comment: String  
}
```

```
type ValueDistribution {  
  distributionId: ID!  
  totalValue: Decimal!  
  currency: String!  
  distributionModel: DistributionModel!  
  parties: [ValueShare!]!  
  platformFee: PlatformFee!  
  paymentSchedule: [PaymentScheduleItem!]!  
  winWinWinScore: Decimal  
  isLocked: Boolean!  
  lockedAt: DateTime  
}
```

```
type ValueShare {  
  party: Party!  
  role: Role!  
  sharePercent: Decimal  
  shareAmount: Decimal  
  costPercent: Decimal  
  costAmount: Decimal
```

```

}

type Milestone {
  milestoneId: ID!
  milestoneName: String!
  description: String
  assignedToParty: Party!
  dueDate: Date
  completionCriteria: String!
  milestoneStatus: MilestoneStatus!
  completionPercentage: Decimal!
  paymentTriggerAmount: Decimal
  deliverables: [String!]!
  completedAt: DateTime
  verifiedByParty: Party
  verification: MilestoneVerification
  dependencies: [Milestone!]!
  displayOrder: Int!
}

type MilestoneVerification {
  verificationId: ID
  status: VerificationStatus!
  verifiedBy: Party
  verificationNotes: String
  issueFlags: [IssueFlag!]!
  verifiedAt: DateTime
}

type Agreement {
  agreementId: ID!
  agreementStatus: AgreementStatus!
  agreementReference: String!
  agreementTextUrl: String!
  governingLaw: String!
  disputeResolution: DisputeResolutionMethod!
  effectiveDate: Date
  terminationDate: Date
  autoRenew: Boolean!
  version: Int!
  parties: [AgreementParty!]!
  signatures: [Signature!]!
  includedTerms: [Term!]!
  milestones: [Milestone!]!
  generatedAt: DateTime!
}

type AgreementParty {
  party: Party!
  role: Role!
  signatureStatus: SignatureStatus!
  signatureRequired: Boolean!
}

```

```

}

type Signature {
  signatureId: ID!
  party: Party!
  signedByUser: User!
  signatureType: SignatureType!
  signedAt: DateTime!
  ipAddress: String!
}

type Transaction {
  transactionId: ID!
  deal: Deal
  transactionType: TransactionType!
  fromParty: Party
  toParty: Party
  amount: Decimal!
  currency: String!
  description: String!
  status: TransactionStatus!
  paymentMethod: PaymentMethod!
  milestone: Milestone
  executedAt: DateTime
  createdAt: DateTime!
}

```

6.4 Party Types

```

type Party {
  partyId: ID!
  partyType: PartyType!
  displayName: String!
  email: String!
  phone: String!
  verificationStatus: VerificationStatus!
  primaryDomain: Category
  location: Location
  serviceRadiusKm: Decimal
  trustScore: Decimal
  totalDealsCompleted: Int!
  totalDealsInitiated: Int!
  isActive: Boolean!

  # Nested
  roles: [PartyRole!]!
  profile: PartyProfile
  group: PartyGroup
  wallet: Wallet
  trustDetails: TrustScore
  reviewsReceived(
    filter: ReviewFilterInput
    pagination: PaginationInput
  )
}

```

```

): ReviewConnection!
reviewsGiven(
  filter: ReviewFilterInput
  pagination: PaginationInput
): ReviewConnection!
deals(
  filter: DealFilterInput
  pagination: PaginationInput
): DealConnection!
resources(
  filter: ResourceFilterInput
  pagination: PaginationInput
): ResourceConnection!
needs(
  filter: NeedFilterInput
  pagination: PaginationInput
): NeedConnection!
enhancements(
  filter: EnhancementFilterInput
  pagination: PaginationInput
): EnhancementConnection!

createdAt: DateTime!
updatedAt: DateTime!
}

```

```

type PartyRole {
  roleType: Role!
  isActive: Boolean!
  assignedAt: DateTime!
  profile: RoleProfile
}

```

```

type RoleProfile {
  resourceTypes: [Category!]
  typicalCapacity: String
  preferredCompensation: [CompensationType!]
  insuranceVerified: Boolean
  enhancementTypes: [Category!]
  skills: [String!]
  hourlyRate: Decimal
  certifications: [Certification!]
}

```

```

type PartyProfile {
  description: String
  website: String
  logoUrl: String
  bannerImageUrl: String
  languages: [String!]
  timezone: String
  certifications: [Certification!]
}

```

```
portfolio: [PortfolioItem!]
operatingHours: OperatingHours
}
```

```
type Certification {
  name: String!
  issuer: String!
  issuedDate: Date
  expiryDate: Date
  documentUrl: String
}
```

```
type PartyGroup {
  groupId: ID!
  groupName: String!
  groupType: GroupType!
  governanceModel: String
  memberCount: Int!
  isOpenToNewMembers: Boolean!
  minimumMemberApproval: Int!
  sharedBankAccount: Boolean!
  members: [GroupMember!]!
}
```

```
type GroupMember {
  membershipId: ID!
  memberParty: Party!
  roleInGroup: GroupRole!
  joinedAt: DateTime!
  contributionPercentage: Decimal
  isActive: Boolean!
}
```

6.5 Resource, Need, Enhancement Types

```
type Resource {
  resourceId: ID!
  supplierParty: Party!
  resourceName: String!
  resourceType: Category!
  description: String
  quantity: Decimal!
  quantityUnit: String!
  condition: Condition!
  location: Location
  availabilityStart: Date
  availabilityEnd: Date
  documentUrls: [String!]!
  verifiedByPlatform: Boolean!
  metadata: JSON
  isActive: Boolean!
  dealCount: Int!
  createdAt: DateTime!
```

```

    updatedAt: DateTime!
}

type Need {
  needId: ID!
  consumerParty: Party!
  needDescription: String!
  needCategory: Category!
  requiredQuantity: Decimal!
  quantityUnit: String!
  qualityRequirements: String
  requiredByDate: Date
  maxBudget: Decimal
  budgetCurrency: String
  acceptableVariants: String
  priority: Priority!
  location: Location
  deliveryPreferences: DeliveryPreferences
  isActive: Boolean!
  dealCount: Int!
  createdAt: DateTime!
  updatedAt: DateTime!
}

```

```

type Enhancement {
  enhancementId: ID!
  enhancerParty: Party!
  enhancementName: String!
  enhancementType: Category!
  description: String
  inputQuantity: Decimal!
  quantityUnit: String!
  serviceDurationHours: Decimal
  estimatedCompletionDays: Int
  deliverables: String
  prerequisites: String
  skills: [String!]!
  certifications: [Certification!]!
  equipment: [String!]!
  pricing: PricingInfo
  availability: AvailabilityInfo
  serviceArea: ServiceArea
  isActive: Boolean!
  dealCount: Int!
  createdAt: DateTime!
  updatedAt: DateTime!
}

```

6.6 Matching Types

```

type Match {
  matchId: ID!
  matchStatus: MatchStatus!
}

```

```
matchScore: Decimal!
matchScoreBreakdown: MatchScoreBreakdown!
matchReason: String
suggestedDealValue: Decimal
suggestedValueCurrency: String
parties: MatchParties!
expiresAt: DateTime
createdAt: DateTime!
}

type MatchScoreBreakdown {
  resourceNeedMatch: ScoreDimension!
  valueAlignment: ScoreDimension!
  trustScore: ScoreDimension!
  geographicFit: ScoreDimension!
  temporalAlignment: ScoreDimension!
  historicalSuccess: ScoreDimension!
  riskProfileMatch: ScoreDimension!
}

type ScoreDimension {
  score: Decimal!
  weight: Decimal!
  weightedScore: Decimal!
  details: JSON
}

type MatchParties {
  supplier: MatchPartyDetail!
  consumer: MatchPartyDetail!
  enhancer: MatchPartyDetail!
}

type MatchPartyDetail {
  party: Party!
  resource: Resource
  need: Need
  enhancement: Enhancement
}

type CompatibilityScore {
  overallScore: Decimal!
  scoreOutOf100: Int!
  breakdown: MatchScoreBreakdown!
  recommendation: CompatibilityRecommendation!
  recommendationText: String!
  suggestedImprovements: [String!]!
}
```

6.7 Trust & Notification Types

```
type TrustScore {  
  trustScoreId: ID!  
  party: Party!  
  overallScore: Decimal!  
  roleScores: RoleScores!  
  detailedMetrics: TrustMetrics!  
  scoreHistory: [ScoreHistoryPoint!]!  
  trustLevel: TrustLevel!  
  trustLevelDescription: String!  
  badges: [TrustBadge!]!  
  lastCalculatedAt: DateTime!  
  nextCalculationAt: DateTime!  
}
```

```
type RoleScores {  
  asSupplier: RoleScore  
  asConsumer: RoleScore  
  asEnhancer: RoleScore  
}
```

```
type RoleScore {  
  score: Decimal  
  dealsCompleted: Int  
  dealsCancelled: Int  
  averageRating: Decimal  
  topStrengths: [String!]!  
}
```

```
type TrustMetrics {  
  dealsCompletedCount: Int!  
  dealsCancelledCount: Int!  
  dealsDisputedCount: Int!  
  completionRate: Decimal!  
  averageResponseHours: Decimal  
  profileCompleteness: Decimal!  
  verificationLevel: Int!  
  longevityDays: Int!  
  totalReviews: Int!  
  averageRating: Decimal!  
  onTimeDeliveryRate: Decimal  
  budgetAccuracy: Decimal  
}
```

```
type ScoreHistoryPoint {  
  date: Date!  
  score: Decimal!  
}
```

```
type TrustBadge {  
  badgeId: ID!  
  name: String!
```

```

description: String!
earnedAt: DateTime!
}

type Review {
  reviewId: ID!
  deal: Deal!
  reviewer: Party!
  reviewed: Party!
  reviewedRole: Role!
  overallRating: Int!
  communicationRating: Int
  reliabilityRating: Int
  qualityRating: Int
  timelinessRating: Int
  reviewText: String
  isVerified: Boolean!
  isPublic: Boolean!
  createdAt: DateTime!
}

type Notification {
  notificationId: ID!
  notificationType: NotificationType!
  title: String!
  body: String!
  relatedEntityType: EntityType
  relatedEntityId: ID
  isRead: Boolean!
  isActioned: Boolean!
  actionUrl: String
  actions: [NotificationAction!]!
  createdAt: DateTime!
  expiresAt: DateTime
}

type NotificationAction {
  label: String!
  action: ActionType!
  url: String
  method: String
}

```

6.8 Dashboard Types

```

type Dashboard {
  party: Party!
  period: DashboardPeriod!
  summary: DashboardSummary!
  upcomingMilestones: [DashboardMilestone!]!
  pendingActions: [PendingAction!]!
  recentActivity: [ActivityItem!]!
  dealTrends: [DealTrendPoint!]!
}

```

```
financialSummary: FinancialSummary!  
}
```

```
type DashboardSummary {  
  totalDeals: Int!  
  byStatus: StatusCounts!  
  byRole: RoleCounts!  
  totalValue: Decimal!  
  valueReceived: Decimal!  
  valuePending: Decimal!  
  dealsInitiated: Int!  
  dealsJoined: Int!  
  averageDealValue: Decimal!  
  completionRate: Decimal!  
  averageNegotiationDays: Decimal  
}
```

```
type DashboardMilestone {  
  milestoneId: ID!  
  deal: Deal!  
  milestoneName: String!  
  dueDate: Date!  
  daysRemaining: Int!  
  status: MilestoneStatus!  
}
```

```
type PendingAction {  
  deal: Deal!  
  action: String!  
  description: String!  
  deadline: DateTime  
  priority: Priority!  
}
```

```
type ActivityItem {  
  eventType: String!  
  deal: Deal  
  description: String!  
  timestamp: DateTime!  
}
```

```
type DealTrendPoint {  
  date: Date!  
  dealsCreated: Int!  
  dealsCompleted: Int!  
  totalValue: Decimal!  
}
```

```
type FinancialSummary {  
  walletBalance: Decimal!  
  escrowBalance: Decimal!  
  pendingBalance: Decimal!
```

```
    totalEarned: Decimal!
    totalSpent: Decimal!
    upcomingPayments: [UpcomingPayment!]!
}
```

```
type UpcomingPayment {
  milestoneId: ID!
  deal: Deal!
  milestoneName: String!
  expectedAmount: Decimal!
  expectedDate: Date!
  probability: Decimal!
}
```

6.9 Input Types

```
input DealFilterInput {
  status: [DealStatus!]
  partyId: ID
  myRole: Role
  domainCategoryId: ID
  initiatorRole: Role
  fromDate: Date
  toDate: Date
  minValue: Decimal
  maxValue: Decimal
  currency: String
  location: GeoFilterInput
  isPublic: Boolean
  q: String
}
```

```
input GeoFilterInput {
  latitude: Decimal!
  longitude: Decimal!
  radiusKm: Decimal!
}
```

```
input DealSortInput {
  field: DealSortField!
  direction: SortDirection!
}
```

```
enum DealSortField {
  CREATED_AT
  UPDATED_AT
  TOTAL_DEAL_VALUE
  EXPECTED_START_DATE
  EXPECTED_END_DATE
  DEAL_STATUS
  RELEVANCE
}
```

```
input PartyFilterInput {  
  role: [Role!]  
  domainCategoryId: ID  
  verificationStatus: VerificationStatus  
  minTrustScore: Decimal  
  location: GeoFilterInput  
  available: Boolean  
  q: String  
}
```

```
input PaginationInput {  
  limit: Int = 20  
  cursor: String  
}
```

```
input MatchFilterInput {  
  domainCategoryId: ID  
  location: GeoFilterInput  
  minScore: Decimal  
  dealValueMin: Decimal  
  dealValueMax: Decimal  
}
```

```
input ReviewFilterInput {  
  role: Role  
  rating: [Int!]  
  verifiedOnly: Boolean  
}
```

```
input TermFilterInput {  
  termType: [TermType!]  
  status: [NegotiationStatus!]  
  proposedBy: ID  
}
```

```
input MilestoneFilterInput {  
  status: [MilestoneStatus!]  
  assignedTo: ID  
  overdue: Boolean  
}
```

```
input NotificationFilterInput {  
  type: [NotificationType!]  
  isRead: Boolean  
  isActioned: Boolean  
}
```

```
input SearchFilterInput {  
  type: [SearchResultType!]  
  domainCategoryId: ID  
  location: GeoFilterInput  
}
```

```

    minValue: Decimal
    maxValue: Decimal
    status: [String!]
}

input SearchSortInput {
  field: SearchSortField!
  direction: SortDirection!
}

enum SearchSortField {
  RELEVANCE
  CREATED_AT
  VALUE
  TRUST_SCORE
  DISTANCE
}

input AnalyticsFilterInput {
  partyId: ID
  domainCategoryId: ID
  dateRange: DateRangeInput!
}

input DateRangeInput {
  from: Date!
  to: Date!
}

```

6.10 Connection Types

```

type DealConnection {
  edges: [DealEdge!]!
  pageInfo: PageInfo!
  facets: DealFacets
}

type DealEdge {
  node: Deal!
  cursor: String!
}

type PageInfo {
  hasNextPage: Boolean!
  hasPreviousPage: Boolean!
  startCursor: String
  endCursor: String
}

type DealFacets {
  status: [StatusFacet!]!
  domain: [DomainFacet!]!
}

```

```
    role: [RoleFacet!]!
  }

  type PartyConnection {
    edges: [PartyEdge!]!
    pageInfo: PageInfo!
  }

  type PartyEdge {
    node: Party!
    cursor: String!
  }

  type ReviewConnection {
    edges: [ReviewEdge!]!
    pageInfo: PageInfo!
    summary: ReviewSummary!
  }

  type ReviewEdge {
    node: Review!
    cursor: String!
  }

  type ReviewSummary {
    totalReviews: Int!
    averageRating: Decimal!
    distribution: RatingDistribution!
  }

  type RatingDistribution {
    "5": Int
    "4": Int
    "3": Int
    "2": Int
    "1": Int
  }

  type MatchConnection {
    edges: [MatchEdge!]!
    pageInfo: PageInfo!
  }

  type MatchEdge {
    node: Match!
    cursor: String!
  }

  type SearchResultConnection {
    edges: [SearchResultEdge!]!
    pageInfo: PageInfo!
  }
```

```

type SearchResultEdge {
  node: SearchResult!
  cursor: String!
}

type SearchResult {
  id: ID!
  type: SearchResultType!
  title: String!
  description: String
  relevanceScore: Decimal!
  entity: Deal | Party | Resource | Need | Enhancement
}

type NotificationConnection {
  edges: [NotificationEdge!]!
  pageInfo: PageInfo!
  unreadCount: Int!
  totalCount: Int!
}

type NotificationEdge {
  node: Notification!
  cursor: String!
}

```

6.11 Enum Types

```

enum DealStatus {
  DRAFT
  SUGGESTED
  PENDING_REVIEW
  NEGOTIATING
  AWAITING_PARTY
  TERMS_LOCKED
  COMMITTED
  EXECUTING
  ON_HOLD
  COMPLETED
  DISPUTED
  CANCELLED
  EXPIRED
}

enum Role {
  SUPPLIER
  CONSUMER
  ENHANCER
}

enum TermType {

```



```
PRICE
TIMELINE
QUALITY
PAYMENT_SCHEDULE
LIABILITY
EXCLUSIVITY
TERMINATION
CUSTOM
}

enum NegotiationStatus {
    PROPOSED
    UNDER_REVIEW
    ACCEPTED
    REJECTED
    COUNTERED
    LOCKED
}

enum MilestoneStatus {
    PENDING
    IN_PROGRESS
    COMPLETED
    OVERDUE
    BLOCKED
}

enum AgreementStatus {
    DRAFT
    PENDING_SIGNATURES
    FULLY_SIGNED
    ACTIVE
    COMPLETED
    TERMINATED
    DISPUTED
    VOIDED
}

enum SignatureStatus {
    PENDING
    SIGNED
    DECLINED
}

enum TransactionType {
    PAYMENT
    ESCROW_DEPOSIT
    ESCROW_RELEASE
    MILESTONE
    FEE
    REFUND
    ADJUSTMENT
}
```

```
    DISPUTE_HOLD
}

enum TransactionStatus {
    PENDING
    COMPLETED
    FAILED
    REVERSED
    HELD
}

enum PaymentMethod {
    BANK_TRANSFER
    CARD
    PLATFORM_WALLET
    ESCROW
    CRYPTO
    IN_KIND
}

enum DistributionModel {
    FIXED_PRICE
    REVENUE_SHARE
    COST_PLUS
    HYBRID
}

enum MatchStatus {
    PENDING
    ACCEPTED_BY_ALL
    REJECTED
    EXPIRED
    CONVERTED_TO_DEAL
}

enum Priority {
    LOW
    MEDIUM
    HIGH
    URGENT
}

enum TrustLevel {
    NEW
    EMERGING
    ESTABLISHED
    HIGH
    ELITE
}

enum NotificationType {
    MATCH_SUGGESTION
```

```
    DEAL_INVITE
    TERM_UPDATE
    PAYMENT_DUE
    MILESTONE_DUE
    REVIEW_REQUEST
    DISPUTE_ALERT
    TRUST_UPDATE
    SYSTEM
}

enum SearchResultType {
    DEAL
    PARTY
    RESOURCE
    NEED
    ENHANCEMENT
}

enum SortDirection {
    ASC
    DESC
}

enum CompatibilityRecommendation {
    NOT_RECOMMENDED
    WEAK_MATCH
    MODERATE_MATCH
    STRONG_MATCH
    EXCELLENT_MATCH
}

enum DashboardPeriod {
    SEVEN_DAYS
    THIRTY_DAYS
    NINETY_DAYS
    ONE_YEAR
    ALL_TIME
}

enum AnalyticsType {
    DEAL_VOLUME
    DEAL_VALUE
    PARTY_ACTIVITY
    DOMAIN_PERFORMANCE
    TRUST_DISTRIBUTION
    REVENUE
}

enum Granularity {
    DAILY
    WEEKLY
    MONTHLY
}
```

```
    QUARTERLY
    YEARLY
}
```

6.12 Subscription Types

```
type Subscription {
  # Real-time deal updates
  dealUpdates(dealId: ID!): DealUpdate!

  # Real-time negotiation updates
  negotiationUpdates(dealId: ID!): NegotiationUpdate!

  # Real-time milestone updates
  milestoneUpdates(dealId: ID!): MilestoneUpdate!

  # Notification stream
  notificationStream: Notification!

  # Payment updates
  paymentUpdates(partyId: ID): PaymentUpdate!

  # Match updates
  matchUpdates(partyId: ID): MatchUpdate!
}
```

```
type DealUpdate {
  dealId: ID!
  eventType: DealEventType!
  deal: Deal
  previousStatus: DealStatus
  newStatus: DealStatus
  timestamp: DateTime!
}
```

```
type NegotiationUpdate {
  dealId: ID!
  eventType: NegotiationEventType!
  term: Term
  message: Message
  timestamp: DateTime!
}
```

```
type MilestoneUpdate {
  dealId: ID!
  milestoneId: ID!
  eventType: MilestoneEventType!
  milestone: Milestone
  payment: Transaction
  timestamp: DateTime!
}
```

```
type PaymentUpdate {  
  transactionId: ID!  
  eventType: PaymentEventType!  
  transaction: Transaction  
  wallet: Wallet  
  timestamp: DateTime!  
}
```

```
type MatchUpdate {  
  matchId: ID!  
  eventType: MatchEventType!  
  match: Match  
  timestamp: DateTime!  
}
```

```
enum DealEventType {  
  CREATED  
  STATUS_CHANGED  
  PARTY_JOINED  
  PARTY_LEFT  
  UPDATED  
}
```

```
enum NegotiationEventType {  
  TERM_PROPOSED  
  TERM_ACCEPTED  
  TERM_REJECTED  
  TERM_COUNTERED  
  TERMS_LOCKED  
  MESSAGE_SENT  
  INFO_REQUESTED  
  INFO_RESPONDED  
}
```

```
enum MilestoneEventType {  
  CREATED  
  UPDATED  
  PROGRESS_UPDATED  
  COMPLETED  
  VERIFIED  
  REJECTED  
  PAYMENT_RELEASED  
}
```

```
enum PaymentEventType {  
  ESCROW_CREATED  
  ESCROW_FUNDED  
  PAYMENT_RELEASED  
  REFUND_ISSUED  
  FEE_DEDUCTED  
  ROLLBACK  
}
```

```
enum MatchEventType {  
  SUGGESTED  
  ACCEPTED  
  DECLINED  
  COUNTERED  
  EXPIRED  
  CONVERTED  
}
```

6.13 Mutation Types

```
type Mutation {  
  # Deal mutations  
  createDeal(input: CreateDealInput!): Deal!  
  updateDeal(dealId: ID!, input: UpdateDealInput!): Deal!  
  transitionDeal(dealId: ID!, input: DealTransitionInput!): Deal!  
  cloneDeal(dealId: ID!, input: CloneDealInput!): Deal!  
  
  # Party mutations  
  createParty(input: CreatePartyInput!): Party!  
  updateParty(partyId: ID!, input: UpdatePartyInput!): Party!  
  addRole(partyId: ID!, input: AddRoleInput!): PartyRole!  
  removeRole(partyId: ID!, role: Role!): Boolean!  
  
  # Negotiation mutations  
  proposeTerm(dealId: ID!, input: ProposeTermInput!): Term!  
  acceptTerm(dealId: ID!, termId: ID!, input: AcceptTermInput!): Term!  
  counterTerm(dealId: ID!, termId: ID!, input: CounterTermInput!): Term!  
  rejectTerm(dealId: ID!, termId: ID!, input: RejectTermInput!): Term!  
  lockTerms(dealId: ID!): Deal!  
  
  # Agreement mutations  
  generateAgreement(dealId: ID!, input: GenerateAgreementInput): Agreement!  
  signAgreement(dealId: ID!, input: SignAgreementInput!): Signature!  
  voidAgreement(dealId: ID!, input: VoidAgreementInput!): Agreement!  
  
  # Milestone mutations  
  createMilestone(dealId: ID!, input: CreateMilestoneInput!): Milestone!  
  updateMilestone(dealId: ID!, milestoneId: ID!, input: UpdateMilestoneInput!): Milestone!  
  completeMilestone(dealId: ID!, milestoneId: ID!, input: CompleteMilestoneInput!):  
  Milestone!  
  verifyMilestone(dealId: ID!, milestoneId: ID!, input: VerifyMilestoneInput!): Milestone!  
  
  # Payment mutations  
  createEscrow(input: CreateEscrowInput!): Escrow!  
  fundEscrow(escrowId: ID!, input: FundEscrowInput!): Escrow!  
  
  # Review mutations  
  submitReview(input: SubmitReviewInput!): Review!  
  
  # Notification mutations
```

```

markNotificationRead(notificationId: ID!): Notification!
markAllNotificationsRead(filter: NotificationFilterInput): BatchResult!
}

type BatchResult {
  success: Boolean!
  affectedCount: Int!
}

```

6.14 Example Queries

Get deal with all nested data:

```

query GetDeal($dealId: ID!) {
  deal(dealId: $dealId) {
    dealId
    dealTitle
    dealStatus
    totalDealValue
    currency
    progressPercentage

    participations {
      party {
        partyId
        displayName
        trustScore
      }
      role
      participationStatus
    }

    valueDistribution {
      totalValue
      parties {
        party { displayName }
        sharePercent
        shareAmount
      }
      winWinWinScore
    }

    milestones {
      milestoneId
      milestoneName
      milestoneStatus
      completionPercentage
      dueDate
      assignedToParty { displayName }
      paymentTriggerAmount
    }
  }
}

```

```

    agreement {
      agreementStatus
      signatures {
        party { displayName }
        signedAt
      }
    }
  }

  availableActions {
    action
    allowed
    reason
  }
}
}

```

Search deals with facets:

```

query SearchDeals($filter: DealFilterInput, $pagination: PaginationInput) {
  deals(filter: $filter, pagination: $pagination) {
    edges {
      node {
        dealId
        dealTitle
        dealStatus
        totalDealValue
        participations {
          party { displayName }
          role
        }
      }
    }
    cursor
  }
  pageInfo {
    hasNextPage
    endCursor
  }
  facets {
    status { value count }
    domain { value count }
  }
}

```

Get party dashboard:

```

query GetDashboard($partyId: ID!, $period: DashboardPeriod) {
  dashboard(partyId: $partyId, period: $period) {
    summary {
      totalDeals
      byStatus { DRAFT SUGGESTED NEGOTIATING EXECUTING COMPLETED }
      completionRate
      averageDealValue
    }
  }
}

```



```

    }
    upcomingMilestones {
      deal { dealTitle }
      milestoneName
      daysRemaining
    }
    pendingActions {
      deal { dealTitle }
      action
      deadline
      priority
    }
  }
}

```

Subscription for real-time deal updates:

```

subscription OnDealUpdate($dealId: ID!) {
  dealUpdates(dealId: $dealId) {
    eventType
    previousStatus
    newStatus
    timestamp
  }
}

```

6.15 Custom Scalar Types

```

scalar Date      # ISO 8601 date (YYYY-MM-DD)
scalar DateTime  # ISO 8601 datetime (YYYY-MM-DDTHH:mm:ssZ)
scalar Decimal   # High-precision decimal number
scalar JSON      # Arbitrary JSON object
scalar GeoJSON   # GeoJSON geometry object

```

6. API Dependency Flows

This section contains Mermaid diagrams illustrating the API interactions and dependencies between services for key platform workflows.

6.1 Deal Creation Flow

```

sequenceDiagram
    autonumber
    participant C as Client
    participant GW as API Gateway
    participant DA as deal-mgmt-svc
    participant PA as party-profile-svc
    participant MA as matching-discovery-svc
    participant NT as notification-svc
    participant EB as Event Bus
    participant SE as search-svc

    C->>GW: POST /api/v1/deals
    Note over C,GW: JWT + X-Party-ID + Deal payload

```

```

GW->>GW: Validate auth & rate limit

GW->>PA: GET /internal/v1/parties/{partyId}/validate
Note over GW,PA: Check party exists, has role, KYC OK
PA-->>GW: Party validation result

GW->>PA: GET /internal/v1/parties/{consumerId}/validate
PA-->>GW: Consumer validation result

GW->>PA: GET /internal/v1/parties/{enhancerId}/validate
PA-->>GW: Enhancer validation result

GW->>DA: Create deal record
Note over GW,DA: Validate Win-Win-Win, persist deal
DA->>DA: Write deal to DB
DA->>DA: Create 3 DealParticipation records
DA-->>GW: Deal created (201)
GW-->>C: Deal response with links

DA->>EB: Publish deal.created event

par Async processing
    EB->>NT: Consume deal.created
    NT->>NT: Send notifications to invited parties

    EB->>SE: Consume deal.created
    SE->>SE: Index deal for search

    EB->>MA: Consume deal.created
    MA->>MA: Update match scoring models
end

```

6.2 Deal Lifecycle State Machine Flow

```

sequenceDiagram
    autonumber
    participant C as Client
    participant GW as API Gateway
    participant DA as deal-mgmt-svc
    participant NA as negotiation-svc
    participant AA as agreement-contract-svc
    participant EA as execution-milestone-svc
    participant PA as payment-settlement-svc
    participant NT as notification-svc
    participant EB as Event Bus

    Note over C,EB: Deal Lifecycle Transitions

    rect rgb(230, 245, 255)
        Note over C,EB: DRAFT → SUGGESTED
        C->>GW: POST /deals/{id}/transitions {transition: SUBMIT}
        GW->>DA: Validate & execute transition
    end

```

```

DA->>DA: Validate all parties have accounts
DA->>DA: Create state transition record
DA-->>GW: Transition result (200)
GW-->>C: Updated deal + notifications sent
DA->>EB: Publish deal.state_changed event
EB->>NT: Send invites to C & E
end

rect rgb(230, 255, 230)
  Note over C,EB: SUGGESTED → NEGOTIATING
  C->>GW: PATCH /deals/{id}/participations/{pid} {status: ACCEPTED}
  GW->>DA: Update participation status
  DA->>DA: Check if all parties accepted
  DA->>DA: Auto-transition to NEGOTIATING
  DA-->>GW: Participation updated
  GW->>C: Response + deal now NEGOTIATING
  DA->>EB: Publish deal.state_changed + deal.party_responded
end

rect rgb(255, 248, 230)
  Note over C,EB: NEGOTIATING → TERMS_LOCKED
  C->>GW: POST /deals/{id}/terms {term payload}
  GW->>NA: Propose term
  NA-->>GW: Term created
  GW-->>C: Term with action links

  C->>GW: POST /terms/{id}/actions/accept
  GW->>NA: Accept term
  NA->>NA: Check all parties accepted
  NA-->>GW: Term accepted
  GW-->>C: Acceptance confirmed

  C->>GW: POST /deals/{id}/actions/lock-terms
  GW->>DA: Validate all terms accepted
  DA->>DA: Execute lock-terms transition
  DA->>NA: Mark all terms locked
  DA-->>GW: Terms locked (TERMS_LOCKED)
  GW-->>C: Transition result + agreement preview
  DA->>EB: Publish negotiation.terms_locked
  EB->>AA: Trigger agreement generation
end

rect rgb(255, 230, 245)
  Note over C,EB: TERMS_LOCKED → COMMITTED
  C->>GW: POST /agreement/sign
  GW->>AA: Sign agreement
  AA->>AA: Record signature
  AA->>AA: Check all signatures received
  AA-->>GW: Signature recorded
  GW-->>C: Signature confirmed

  AA->>DA: Transition to COMMITTED

```

```

    DA-->>AA: Transitioned
    AA->>EB: Publish agreement.signed
    EB->>PA: Create escrow for deal
    EB->>NT: Notify all parties
end

rect rgb(240, 230, 255)
    Note over C,EB: COMMITTED → EXECUTING
    Note right of DA: Auto-transition after 3 days
    DA->>DA: Scheduled job triggers transition
    DA->>EA: Initialize milestone tracking
    EA-->>DA: Milestones initialized
    DA->>EB: Publish execution.started
    EB->>NT: Notify all parties execution begins
end

rect rgb(230, 255, 248)
    Note over C,EB: EXECUTING → COMPLETED (via milestones)
    C->>GW: POST /milestones/{id}/actions/complete
    GW->>EA: Complete milestone
    EA-->>GW: Milestone completed
    GW-->>C: Completion recorded

    C->>GW: POST /milestones/{id}/actions/verify
    GW->>EA: Verify milestone
    EA->>PA: Trigger payment release
    PA-->>EA: Payment released
    EA-->>GW: Verified + payment released
    GW-->>C: Verification + payment confirmation

    alt All milestones completed
        EA->>DA: Check all milestones done
        DA->>DA: Transition to COMPLETED
        DA->>EB: Publish deal.completed
        EB->>NT: Request reviews from all parties
    end
end

```

6.3 Negotiation Thread Flow

sequenceDiagram

```

autonumber
participant C1 as Client (Supplier)
participant C2 as Client (Consumer)
participant C3 as Client (Enhancer)
participant GW as API Gateway
participant NA as negotiation-svc
participant DA as deal-mgmt-svc
participant EB as Event Bus
participant NT as notification-svc
participant WS as WebSocket

```

Note over C1,WS: NEGOTIATING state - Term negotiation

C1->>GW: POST /deals/{id}/terms (propose payment schedule)

GW->>NA: Create term proposal

NA->>NA: Version term, set status PROPOSED

NA-->>GW: Term created

GW-->>C1: Term + action links

NA->>EB: Publish term.proposed

par Async notifications

EB->>NT: Notify C2 & C3 of new term

NT->>C2: Push notification + email

NT->>C3: Push notification + email

EB->>WS: Broadcast to deal channel

end

C2->>GW: POST /terms/{id}/actions/accept

GW->>NA: Accept term

NA->>NA: Record C2 acceptance

NA-->>GW: Accepted (pending C3)

GW-->>C2: Acceptance confirmed

NA->>EB: Publish term.accepted

C3->>GW: POST /terms/{id}/actions/counter

GW->>NA: Counter term

NA->>NA: Create new term (parent = original)

NA->>NA: Set original to COUNTERED

NA-->>GW: Counter-term created

GW-->>C3: Counter submitted

NA->>EB: Publish term.countered

par Async notifications

EB->>NT: Notify C1 & C2 of counter

NT->>C1: Push notification

NT->>C2: Push notification

EB->>WS: Broadcast counter to deal channel

end

C1->>GW: POST /counter-term/actions/accept

GW->>NA: Accept counter

NA->>NA: Record C1 acceptance

C2->>GW: POST /counter-term/actions/accept

GW->>NA: Accept counter

NA->>NA: Record C2 acceptance

NA->>NA: All accepted - mark LOCKED

NA-->>GW: Term locked

GW-->>C2: Counter accepted & locked

NA->>EB: Publish term.accepted + term.locked

alt All dimensions locked

NA->>DA: Notify terms ready for locking
end

6.4 Matching & Discovery Flow

sequenceDiagram

autonumber
participant C as Client
participant GW as API Gateway
participant MA as matching-discovery-svc
participant PA as party-profile-svc
participant TA as trust-reputation-svc
participant RC as resource-catalog-svc
participant SE as search-svc
participant EB as Event Bus

C->>GW: GET /api/v1/matches
Note over C,GW: X-Party-ID: supplier-party-uuid
GW->>GW: Extract party ID from header

GW->>PA: GET /internal/v1/parties/{id}/profile
PA-->>GW: Party profile with roles, domain, location

GW->>PA: GET /internal/v1/parties/{id}/resources
PA-->>GW: Party's resources

GW->>TA: GET /internal/v1/trust/{id}/score
TA-->>GW: Trust score breakdown

GW->>SE: Search for compatible consumers
Note over GW,SE: domain match, geo proximity, need alignment
SE-->>GW: Consumer candidates (ranked)

GW->>SE: Search for compatible enhancers
SE-->>GW: Enhancer candidates (ranked)

GW->>MA: Score triad combinations
Note over GW,MA: 7-dimension compatibility scoring
MA->>MA: Assemble triads from candidate pairs
MA->>MA: Calculate compatibility scores
MA->>MA: Rank by score, apply diversity
MA-->>GW: Ranked match suggestions

GW-->>C: Match list with scores & breakdowns

alt User accepts match
C->>GW: POST /matches/{id}/actions/accept
GW->>MA: Record match acceptance
MA->>MA: Check if all parties accepted

alt All parties accepted
MA->>GW: Trigger deal creation

```

    GW->>DA: POST /internal/v1/deals (from match)
    Note over GW,DA: Creates deal in DRAFT status
    DA-->>GW: Deal created
    GW-->>C: Match converted to deal
    MA->>EB: Publish match.converted_to_deal
  else Pending other parties
    MA-->>GW: Acceptance recorded
    GW-->>C: Awaiting other parties
    MA->>EB: Publish match.accepted
  end
end
end

```

6.5 Payment & Escrow Flow

sequenceDiagram

```

autonumber
participant C as Client (Consumer)
participant GW as API Gateway
participant PA as payment-settlement-svc
participant DA as deal-mgmt-svc
participant EA as execution-milestone-svc
participant PS as Payment Provider (Stripe/Adyen)
participant EB as Event Bus
participant NT as notification-svc

```

Note over C,NT: Agreement signed → Fund escrow

```

C->>GW: POST /payments/escrow/{id}/fund
GW->>PA: Initiate escrow funding
PA->>PA: Validate escrow exists, not expired
PA->>PS: Create payment intent ($45,000)
PS-->>PA: Payment intent created
PA-->>GW: Funding initiated (202)
GW-->>C: Payment URL for completion

```

alt Bank Transfer

```

C->>PS: Complete bank transfer
PS->>PA: Webhook: payment_intent.succeeded
PA->>PA: Mark escrow as FULLY_FUNDED
PA->>EB: Publish escrow.funded

```

else Card Payment

```

C->>PS: Enter card details
PS-->>PS: Process payment
PS->>PA: Webhook: charge.succeeded
PA->>PA: Mark escrow as FULLY_FUNDED
PA->>EB: Publish escrow.funded

```

end

par Async processing

```

EB->>DA: Consume escrow.funded
DA->>DA: Enable execution start

```

EB->>NT: Notify all parties

EB->>EA: Consume escrow.funded

EA->>EA: Activate milestone tracking

end

Note over C,NT: Milestone verified → Release payment

EA->>EA: Milestone verified by counter-party

EA->>PA: Request payment release

PA->>PA: Validate milestone, check escrow balance

PA->>PA: Transfer from escrow to beneficiary wallet

PA->>PA: Create transaction record

PA->>EA: Payment released

PA->>EB: Publish payment.released

par Async processing

EB->>NT: Notify beneficiary of payment

EB->>DA: Update deal financials

end

6.6 Service Dependency Map

graph TB

subgraph "External Layer"

CLIENT[Web/Mobile Client]

WEBHOOK[External Webhooks]

end

subgraph "Gateway Layer"

GW[API Gateway
Kong/AWS API GW]

WSS[WebSocket Server]

end

subgraph "Core Domain Services"

DEAL[deal-mgmt-svc]

PARTY[party-profile-svc]

NEG[negotiation-svc]

AGR[agreement-contract-svc]

EXE[execution-milestone-svc]

PAY[payment-settlement-svc]

MAT[matching-discovery-svc]

RC[resource-catalog-svc]

end

subgraph "Supporting Services"

TRUST[trust-reputation-svc]

NOTIF[notification-svc]

SEARCH[search-svc]

ANALYTICS[analytics-svc]

MEDIA[media-document-svc]

ADMIN[admin-svc]

end

subgraph "Infrastructure"

 AUTH[user-identity-svc]

 EB[(Event Bus
Kafka/RabbitMQ)]

 CACHE[(Redis Cache)]

 DB[(PostgreSQL)]

end

CLIENT --> GW

CLIENT --> WSS

WEBHOOK --> GW

GW --> DEAL

GW --> PARTY

GW --> NEG

GW --> AGR

GW --> EXE

GW --> PAY

GW --> MAT

GW --> RC

GW --> TRUST

GW --> NOTIF

GW --> SEARCH

GW --> ADMIN

DEAL --> EB

NEG --> EB

AGR --> EB

EXE --> EB

PAY --> EB

MAT --> EB

PARTY --> EB

RC --> EB

EB --> NOTIF

EB --> SEARCH

EB --> ANALYTICS

EB --> TRUST

EB --> PAY

EB --> EXE

DEAL --> AUTH

PARTY --> AUTH

PAY --> AUTH

DEAL --> DB

PARTY --> DB

NEG --> DB

AGR --> DB

EXE --> DB

PAY --> DB

MAT --> DB
RC --> DB
TRUST --> DB

MAT --> CACHE
SEARCH --> CACHE
TRUST --> CACHE

DEAL -. -> PARTY
DEAL -. -> RC
NEG -. -> DEAL
AGR -. -> DEAL
AGR -. -> NEG
EXE -. -> DEAL
EXE -. -> PAY
PAY -. -> DEAL
MAT -. -> PARTY
MAT -. -> TRUST
MAT -. -> RC
NOTIF -. -> PARTY

style DEAL fill:#e1f5fe
style PARTY fill:#e1f5fe
style NEG fill:#e1f5fe
style PAY fill:#e1f5fe
style MAT fill:#e1f5fe
style EXE fill:#e1f5fe
style AGR fill:#e1f5fe
style RC fill:#e1f5fe
style EB fill:#fff3e0
style DB fill:#fff3e0

6.7 Event Flow Architecture

graph LR

```
subgraph "Event Producers"
    DEAL_P[deal-mgmt-svc]
    NEG_P[negotiation-svc]
    AGR_P[agreement-contract-svc]
    EXE_P[execution-milestone-svc]
    PAY_P[payment-settlement-svc]
    MAT_P[matching-discovery-svc]
    PARTY_P[party-profile-svc]
    TRUST_P[trust-reputation-svc]
end
```

end

```
subgraph "Event Bus"
    KAFKA[(Kafka Cluster)]
end
```

```
subgraph "Topic Routing"
    T_DEALS[deals.events]
end
```

```
T_NEG[negotiation.events]
T_AGR[agreement.events]
T_EXEC[execution.events]
T_PAY[payments.events]
T_MATCH[matching.events]
T_PARTY[party.events]
T_TRUST[trust.events]
end

subgraph "Event Consumers"
  NOTIF_C[notification-svc]
  SEARCH_C[search-svc]
  ANALYTICS_C[analytics-svc]
  TRUST_C[trust-reputation-svc]
  MAT_C[matching-discovery-svc]
  PAY_C[payment-settlement-svc]
  ADMIN_C[admin-svc]
end

DEAL_P --> KAFKA
NEG_P --> KAFKA
AGR_P --> KAFKA
EXE_P --> KAFKA
PAY_P --> KAFKA
MAT_P --> KAFKA
PARTY_P --> KAFKA
TRUST_P --> KAFKA

KAFKA --> T_DEALS
KAFKA --> T_NEG
KAFKA --> T_AGR
KAFKA --> T_EXEC
KAFKA --> T_PAY
KAFKA --> T_MATCH
KAFKA --> T_PARTY
KAFKA --> T_TRUST

T_DEALS --> NOTIF_C
T_DEALS --> SEARCH_C
T_DEALS --> ANALYTICS_C
T_DEALS --> ADMIN_C

T_NEG --> NOTIF_C
T_NEG --> ANALYTICS_C

T_AGR --> NOTIF_C
T_AGR --> PAY_C
T_AGR --> ANALYTICS_C

T_EXEC --> NOTIF_C
T_EXEC --> PAY_C
T_EXEC --> ANALYTICS_C
```

T_EXEC --> TRUST_C

T_PAY --> NOTIF_C

T_PAY --> ANALYTICS_C

T_MATCH --> NOTIF_C

T_MATCH --> ANALYTICS_C

T_PARTY --> MAT_C

T_PARTY --> SEARCH_C

T_TRUST --> MAT_C

T_TRUST --> ANALYTICS_C

style KAFKA fill:#fff3e0

style T_DEALS fill:#e8f5e9

style T_NEG fill:#e8f5e9

style T_AGR fill:#e8f5e9

style T_EXEC fill:#e8f5e9

style T_PAY fill:#e8f5e9

style T_MATCH fill:#e8f5e9

style T_PARTY fill:#e8f5e9

style T_TRUST fill:#e8f5e9

6.8 Authentication Flow

sequenceDiagram

autonumber

participant C as Client

participant GW as API Gateway

participant AUTH as user-identity-svc

participant IDP as Identity Provider
(OAuth2/OIDC)

participant PA as party-profile-svc

C->>IDP: Authenticate (login)

IDP-->>C: Authorization code

C->>AUTH: POST /auth/token {code}

AUTH->>IDP: Exchange code for tokens

IDP-->>AUTH: ID Token + Access Token

AUTH->>AUTH: Create/validate user identity

AUTH->>AUTH: Issue platform JWT

AUTH-->>C: Platform JWT + Refresh Token

Note over C,PA: Subsequent API calls

C->>GW: GET /api/v1/deals

Note over C,GW: Authorization: Bearer <JWT>
X-Party-ID: <party-uuid>

GW->>AUTH: Validate JWT (RS256)

AUTH-->>GW: Token valid + claims

GW->>GW: Extract scopes & party ID

GW->>GW: Check rate limits

GW->>PA: Verify party belongs to user

```
PA-->>GW: Party verified
GW-->>GW: Route to appropriate service
GW-->>C: API response
```

6.9 CQRS Read/Write Flow

```
graph LR
    subgraph "Write Path (Commands)"
        CLIENT[Client]
        GW[API Gateway]
        SVC[Domain Service]
        DB[(Primary DB)]
        EB[Event Bus]
    end

    subgraph "Read Path (Queries)"
        Q_CLIENT[Client]
        Q_GW[API Gateway]
        Q_SEARCH[Search Service]
        Q_GQL[GraphQL API]
        ES[(ElasticSearch)]
    end

    subgraph "Projection Sync"
        PROJ[Projection Workers]
    end

    CLIENT --> GW
    GW --> SVC
    SVC --> DB
    SVC --> EB

    Q_CLIENT --> Q_GW
    Q_GW --> Q_SEARCH
    Q_GW --> Q_GQL
    Q_SEARCH --> ES
    Q_GQL --> ES

    EB --> PROJ
    PROJ --> ES

    style DB fill:#e1f5fe
    style ES fill:#fff3e0
    style EB fill:#fff3e0
```

Appendix A: OpenAPI Specification

The complete API is documented using OpenAPI 3.0.3 specification. The specification file is available at: - Development: <https://api-staging.dealplatform.com/openapi.json> - Production: <https://api.dealplatform.com/openapi.json>

Key OpenAPI extensions used: - **x-service-id:** Microservice responsible for the endpoint - **x-event-triggered:** Whether the endpoint publishes events - **x-idempotency-key:** Whether the endpoint supports idempotency - **x-cache-ttl:** Cache TTL for GET endpoints - **x-rate-limit-tier:** Rate limiting tier

Appendix B: Changelog

Version	Date	Changes
1.0.0	2024-01	Initial API specification

Appendix C: Glossary

Term	Definition
Deal	A potential or actualized 3-party arrangement connecting Supplier, Consumer, and Enhancer
Party	An individual, organization, or group participating on the platform
Escrow	A financial arrangement where a third party holds funds until contractual conditions are met
Milestone	A trackable deliverable or checkpoint within an active deal
Win-Win-Win	The validation that all three parties receive positive net value from a deal
ZOPA	Zone of Possible Agreement - the overlap between party expectations
Gini Coefficient	A measure of statistical dispersion used to evaluate value distribution fairness

End of API Specification Document

Document Version: 1.0.0 Last Updated: 2024 Status: Draft

Document Information

- **Version:** 1.0
- **Status:** Draft
- **Author:** UX Architecture Team
- **Scope:** Complete UX flows for 3-party deal making platform connecting Suppliers, Consumers, and Enhancers

7. User Onboarding Flows

7.1 Registration Flow Overview

The registration system supports three account types with progressive disclosure to minimize cognitive load. Users can register as individuals, organizations, or multi-entity groups.

flowchart TD

A[Entry Point: Landing Page / App Download / Referral] --> B{Account Type Selection}
B -->|Individual| C[Individual Registration]
B -->|Organization| D[Organization Registration]

B -->|Group/Cooperative| E[Group Registration]

C --> F[Auth Setup: Email/Phone + Password or SSO]
D --> F
E --> F

F --> G[Identity Verification: Email/SMS OTP]
G --> H[Role Selection: Choose 1-3 Roles]

H -->|Supplier| I[Supplier Profile]
H -->|Consumer| J[Consumer Profile]
H -->|Enhancer| K[Enhancer Profile]

I --> L[Onboarding Complete: Personalized Dashboard]
J --> L
K --> L
L --> M[Progressive Profiling Prompts]

7.2 Account Type Selection Screen

Screen: Registration / Account Type

Element	Description
Header	“How will you participate?” with subtext “Select your account type — you can always add more later”
Card 1: Individual	Icon: single person silhouette. Title: “As an Individual”. Body: “Personal projects, freelancing, sole proprietorship.” CTA: “Get Started”
Card 2: Organization	Icon: building. Title: “As an Organization”. Body: “Companies, enterprises, institutions with official registration.” CTA: “Get Started”
Card 3: Group/Cooperative	Icon: linked nodes. Title: “As a Group or Cooperative”. Body: “Multiple parties pooling resources with shared governance.” CTA: “Get Started”
Secondary Action	“Already have an account? Sign In”
Trust Signal	“256-bit encryption

Interaction Pattern: Cards expand slightly on hover/focus. Selection animates to a full-screen transition. For Group selection, additional governance questions appear in a follow-up modal.

7.3 Individual Registration

Screen: Registration / Individual Form

Step 1: Basic Information | Field | Type | Required | Validation | |———|———|———|———|
| Full Legal Name | Text input | Yes | Min 2 words, no special chars | | Email Address | Email | Yes | Valid email format, uniqueness check | | Phone Number | Tel | Yes | E.164 format, country selector | | Password | Password | Yes | Min 12 chars, mixed case, number, symbol | | Confirm Password | Password | Yes | Must match | | Country of Residence

| Select | Yes | Dropdown with search | | Timezone | Auto-detect + Select | Auto | Detected from browser, editable |

Step 2: Identity Verification - Automated verification via email OTP (6 digits, 10-min expiry) or SMS - Progress bar: “Step 2 of 4” - Option to “Verify later” with a 7-day grace period indicator

7.4 Organization Registration

Screen: Registration / Organization Form

Additional fields beyond individual:

Field	Type	Required	Validation
Organization Legal Name	Text	Yes	Min 3 chars
DBA (Doing Business As)	Text	No	Optional trading name
Registration Number	Text	Yes	Format varies by country
Legal Structure	Select	Yes	LLC, Corp, Partnership, etc.
Year Established	Number	Yes	Current year - 200
Website URL	URL	No	Valid URL format
Primary Industry	Multi-select	Yes	Predefined taxonomy
Annual Turnover Range	Select	No	<\$100K to >\$100M ranges
Number of Employees	Select	No	1-10, 11-50, etc.

Verification flow: Business email domain check + document upload (business license/certificate) reviewed within 24 hours.

7.5 Group/Cooperative Registration

Screen: Registration / Group Form

Multi-entity specific fields:

Field	Type	Required	Notes
Group Name	Text	Yes	Public-facing name
Group Type	Select	Yes	Cooperative, Consortium, Joint Venture, Informal Pool
Member Count	Number	Yes	Min 2, max 1000

Field	Type	Required	Notes
Governance Model	Select	Yes	Majority Vote, Unanimous, Delegated Representative, Weighted by Contribution
Representative Name	Text	Yes	Primary contact (self-filled from registration)
Decision Threshold	Slider	Yes	For non-unanimous models: 50%-90%
Member Onboarding	Invite system	Yes	Email invites sent during or after registration

Governance Wizard: Interactive wizard that explains each governance model with animated decision scenarios: - **Majority Vote:** “3 of 5 members must approve” - **Unanimous:** “All 5 members must approve” - **Delegated:** “1 designated representative decides for all” - **Weighted:** “Votes weighted by resource contribution percentage”

7.6 Role Selection

Screen: Registration / Role Selection

This is the critical branching point. Users can select 1-3 roles. Each role selection reveals a preview of what that role entails.

flowchart TD

```

A[Role Selection Screen] --> B[Select Supplier]
A --> C[Select Consumer]
A --> D[Select Enhancer]

B --> B1[Preview: "I have resources/assets to offer"]
C --> C1[Preview: "I need products/services/output"]
D --> D1[Preview: "I provide services that enable deals"]

B1 --> E{At least 1 role selected?}
C1 --> E
D1 --> E

E -->|Yes| F[Continue to Role-Specific Profiles]
E -->|No| G[Show tooltip: "Select at least one role to continue"]
G --> A

```

Role Cards (toggle-style, can select multiple): - **Supplier Card:** Icon: warehouse/box. “I have resources” — land, materials, IP, data, labor, equipment. - **Consumer Card:** Icon: shopping cart/target. “I want outputs” — finished products, processed materials, services. - **Enhancer Card:** Icon: spark/tools. “I enable deals” — logistics, financing, expertise, processing, certification.

Each selected role card gets a colored border: - Supplier: green accent (#2ECC71) - Consumer: blue accent (#3498DB) - Enhancer: purple accent (#9B59B6)

7.7 Profile Creation Per Role

Supplier Profile

Screen: Profile / Supplier Setup

Section	Fields	Required
Resource Categories	Multi-select from taxonomy (30+ categories)	Yes (min 1)
Specific Resources	Free-form + guided tags for each category	Yes (min 1)
Quantity/Scale	Unit type + available quantity/volume	Yes
Location	Full address or region with map pin	Conditionally
Availability	Start date, duration, seasonal toggle	Yes
Past Transactions	“Have you supplied before?” Yes/No + details	No
Pricing Model	Fixed, negotiable, revenue-share, barter	No
Certifications	Organic, Fair Trade, ISO, etc. with upload	No

Resource Taxonomy Examples: - Physical: Raw Materials, Land, Equipment, Facilities, Inventory - Digital: Data Sets, Software, APIs, Digital Content, IP - Human: Labor, Expertise, Management, Creative Services - Financial: Capital, Credit, Insurance, Guarantees

Consumer Profile

Screen: Profile / Consumer Setup

Section	Fields	Required
Need Categories	Multi-select from taxonomy	Yes (min 1)
Specific Needs	Detailed description with requirement tags	Yes (min 1)
Quantity Needed	Unit type + quantity/volume	Yes
Quality Specifications	Standards, tolerances, certifications required	No
Delivery Location	Full address or region	Yes
Timeline	Desired start date,	Yes

Section	Fields	Required
	acceptable delivery window	
Budget Range	Min-max per unit or total (can be “open”)	No
Previous Suppliers	History toggle + details	No

Enhancer Profile

Screen: Profile / Enhancer Setup

Section	Fields	Required
Service Categories	Multi-select: Logistics, Financing, Processing, Certification, Technical Advisory, Legal, Marketing	Yes (min 1)
Specific Services	Detailed capability description	Yes
Service Area	Geographic coverage or virtual	Yes
Pricing Structure	Per-deal, hourly, retainer, success-fee, revenue-share	Yes
Capacity	Maximum concurrent engagements	Yes
Certifications/Licenses	Professional credentials with upload	No
Equipment/Tools	Key assets that enable service delivery	No
Integration Requirements	APIs, physical access, documentation needs	No

7.8 Verification & KYC Flow

flowchart TD

```

A[Account Created] --> B{Account Type?}
B -->|Individual| C[Basic KYC]
B -->|Organization| D[Business KYC]
B -->|Group| E[Group KYC]

C --> C1[Phone Verification]
C1 --> C2[Email Verification]
C2 --> C3[Government ID Upload]
C2 --> C4[Skip for Later]
C3 --> C5[AI-powered ID Verification]
C5 --> C6[Selfie Liveness Check]
C6 --> F[KYC Complete: Bronze Badge]
C4 --> G[Limited Access Mode]

```

D --> D1[All Individual Steps]
 D1 --> D2[Business Registration Doc]
 D2 --> D3[Bank Account Verification]
 D3 --> D4[Beneficial Ownership Declaration]
 D4 --> H[KYC Complete: Silver Badge]

 E --> E1[All Organization Steps]
 E1 --> E2[Governance Document Upload]
 E2 --> E3[Member Identity Verification
30% of members minimum]
 E3 --> I[KYC Complete: Gold Badge]

 G -->|Complete within 7 days| F

Verification Levels & Access Matrix:

Level	Requirements	Account Badge	Deal Limit	Features
Unverified	Email only	Gray dot	View only	Browse, save searches
Bronze	Phone + ID verified	Bronze shield	3 concurrent deals	Initiate deals up to \$10K
Silver	Business verified	Silver shield	10 concurrent deals	Unlimited value, org features
Gold	Full group KYC	Gold shield	Unlimited	Group governance, priority matching

KYC Progress Widget: Persistent header widget showing “Verification progress: 3/5 complete” with expandable checklist. Color changes from red (incomplete) to amber (partial) to green (complete).

7.9 Progressive Profiling Strategy

Rather than overwhelming users upfront, the platform collects information progressively:

Phase 1: Registration (required) - Identity, contact, account type, role selection - Minimal profile: 1 resource/need/service

Phase 2: First Week (gentle prompts) - Daily micro-surveys: “What else do you supply?” (1 question per prompt) - Contextual tips in empty states - “Complete your profile to get better matches” progress indicator

Phase 3: Active Usage (contextual) - Before first deal: prompt for detailed terms templates - After first match: request feedback on match quality - Before negotiation: auto-suggest based on deal type

Phase 4: Ongoing (maintenance) - Quarterly profile refresh prompts - Resource availability seasonal updates - Auto-detect stale listings with “Still available?” nudges

Progressive Profiling UI Pattern:

```

+-----+
| Complete your profile for 3x better matches |
| [=====> ] 4 of 7 steps done |
| |
| Next: Add a certification [Add Now] [Remind me] |
+-----+

```

7.10 Onboarding Completion & First Experience

Upon completing core onboarding, users see:

Screen: Onboarding / Welcome Dashboard

1. **Confetti animation** with “Welcome to TriadDeal, [Name]!”
2. **Quick Start Guide:** 3-5 swipeable cards:
 - Card 1: “Discover matches” (arrow to discovery tab)
 - Card 2: “Post your first [resource/need/service]” (guided CTA)
 - Card 3: “How 3-party deals work” (60-second explainer video)
3. **Suggested Next Steps:** Contextual based on role(s)
4. **Preview of Matches:** “We found 12 potential matches already!” (blurred preview encouraging profile completion)

7. Deal Initiation Flows

7.1 Deal Lifecycle State Machine

stateDiagram-v2

```

[*] --> DRAFT: Create Deal
DRAFT --> SUGGESTED: Send Invitations
SUGGESTED --> NEGOTIATING: All Parties Accept
SUGGESTED --> DRAFT: Invitation Declined / Expired
NEGOTIATING --> TERMS_LOCKED: All Parties Lock Terms
TERMS_LOCKED --> NEGOTIATING: Unlock Request (unanimous)
TERMS_LOCKED --> COMMITTED: Commitment Deposits
COMMITTED --> EXECUTING: Work Begins
EXECUTING --> COMPLETED: Milestones Complete
EXECUTING --> DISPUTED: Conflict Raised
DISPUTED --> EXECUTING: Resolution
DISPUTED --> TERMINATED: Arbitration
EXECUTING --> TERMINATED: Breach / Force Majeure
COMPLETED --> [*]: Final Settlement
TERMINATED --> [*]: Post-Termination Settlement

```

7.2 Supplier-Initiated Flow

Primary Actor: Supplier who has a resource and wants to find a consumer + enhancer to create value.

flowchart TD

```

A[Supplier Dashboard] --> B[Click "Create Deal"]
B --> C[Select Resource from Portfolio]
C --> D[Define Desired Output/Outcome]

D --> E{Knows Consumer?}

```

```

E -->|Yes| F[Search & Select Consumer]
E -->|No| G[Let System Suggest Consumers]

D --> H{Knows Enhancer?}
H -->|Yes| I[Search & Select Enhancer]
H -->|No| J[Let System Suggest Enhancers]

F --> K[Review Triad Compatibility]
G --> K
I --> K
J --> K

K --> L[Configure Deal Parameters]
L --> M[Draft Terms Template]
M --> N[Send Invitations]

N --> O[Consumer Receives Invitation]
N --> P[Enhancer Receives Invitation]

O --> Q{Consumer Response}
P --> R{Enhancer Response}

Q -->|Accept| S[Check if Both Accepted]
Q -->|Decline| T[Deal Paused / Re-match]
Q -->|Counter| U[Return to Negotiation]

R -->|Accept| S
R -->|Decline| T
R -->|Counter| U

S -->|Both Yes| V[Deal → NEGOTIATING]
S -->|Waiting| W[Status: Awaiting Response]

```

Screen: Deal Creation / Supplier Flow

Step 1: Resource Selection - List of supplier's registered resources with search - Each resource card shows: name, type, location, availability status - Option to "+ Add New Resource" if desired resource isn't listed - Selected resource highlights with green border

Step 2: Output Definition - Header: "What do you want to create with this resource?" - Form fields: - Output category (multi-select taxonomy) - Output description (textarea with AI-assisted suggestion) - Expected quantity/volume - Target timeline - Geographic preference for consumer - **Smart Suggestion:** Based on resource type, AI suggests common outputs (e.g., "Raw Cotton" suggests "Yarn," "Fabric," "Garments")

Step 3: Match Selection - Split screen: Left = Consumer suggestions, Right = Enhancer suggestions - Each suggestion card shows: - Party name + trust score badge - Compatibility percentage (large, prominent) - Key matching factors (3-4 icon+label pairs) - Distance/location - Estimated deal value range - Sort options: Compatibility, Distance, Deal Value, Trust Score - Filter sidebar: Industry, Location, Experience Level, Verification Level

Step 4: Compatibility Review - Full-screen triad visualization:

+-----+			
Triad Compatibility Analysis			
[Supplier: You]			
[Enhancer] -- [Consumer]			
Overall Match Score: 87%			
+-----+ +-----+			
Supplier<>Consumer: 92% Supplier<>Enhancer: 85%			
+-----+ +-----+			
Enhancer<>Consumer: 88%			
+-----+			
Match Factors:			
[x] Location compatible			
[x] Resource/Need alignment			
[x] Enhancer fills capability gap			
[~] Timeline overlap (minor adjustment needed)			
[Review Detailed Analysis] [Proceed] [Find Better Match]			
+-----+			

Step 5: Deal Parameters & Invitation - Suggested deal name (editable) - Proposed timeline - Value distribution starting point (equal thirds by default, editable) - Message to other parties (personalized template) - Invitation expiry: 7 days default (configurable 1-14 days) - Review & Send button with “Preview invitation” option

7.3 Consumer-Initiated Flow

Primary Actor: Consumer who has a need and wants to find a supplier + enhancer to fulfill it.

flowchart TD

```

A[Consumer Dashboard] --> B[Click "Find Supply"]
B --> C[Define Need]
C --> D[Specify Requirements]
D --> E{Has Preferred Supplier?}
E -->|Yes| F[Select Supplier from Directory]
E -->|No| G[System Suggests Suppliers]

D --> H{Knows Enhancer?}
H -->|Yes| I[Select Specific Enhancer]
H -->|No| J[System Suggests Enhancers for This Supply Chain]

F --> K[Triad Compatibility Check]
G --> K
I --> K
J --> K

K --> L[Set Budget & Terms]
L --> M[Send Invitations to Supplier + Enhancer]

```

M --> N[Supplier Receives Invitation]
 M --> O[Enhancer Receives Invitation]

 N --> P{Supplier Accepts?}
 O --> Q{Enhancer Accepts?}

 P -->|Yes| R{Both Accepted?}
 P -->|No| S[Alternative Match Suggested]
 Q -->|Yes| R
 Q -->|No| S

 R -->|Yes| T[Deal → NEGOTIATING]
 R -->|No| U[Pending: Awaiting Other Party]

Screen: Deal Creation / Consumer Flow

Step 1: Need Definition - Form structure similar to supplier flow but reversed: - “What do you need?” with category selection - Need description with AI-assisted refinement - Quality specifications (can attach detailed specs document) - Quantity + unit - Delivery location with map selector - Required delivery date with flexibility toggle (“Flexible +/- 30 days”)

Step 2: Supplier Discovery - Map view (default for location-sensitive needs) - List view toggle - Supplier cards show: - Resource match percentage - Distance from delivery location - Past deal completion rate - Average review score - Resource photo/certification badges - “Request Sample” button on supplier cards (where applicable)

Step 3: Enhancer Selection - System automatically suggests enhancers that can bridge the gap between supplier’s raw output and consumer’s need - Example: Consumer needs “processed organic cotton fabric,” Supplier has “raw cotton” -> System suggests “Textile Processor” + “Organic Certifier” as enhancer options - Enhancer cards show: service match, estimated cost, timeline impact

Step 4: Budget & Terms - Budget input with range indicator showing market rates - “Open to negotiation” toggle - Preferred payment terms (advance, milestone, on-delivery) - Insurance/escrow preference - Custom requirements checklist

7.4 Enhancer-Initiated Flow

Primary Actor: Enhancer who identifies a supplier-consumer pair that would benefit from their service.

flowchart TD

```

    graph TD
      A[Enhancer Dashboard] --> B[Click "Propose Deal"]
      B --> C[Select Service/Capability to Offer]
      C --> D[System Identifies Compatible Pairs]
      D --> E[Browse Suggested Pairs]
      E --> F[Select Supplier + Consumer Pair]
      F --> G[Configure Proposed Deal]
      G --> H[Set Your Fee Structure]
      H --> I[Draft Proposal for Both Parties]
      I --> J[Send Dual Proposal]
  
```



```

J --> K[Supplier Receives Proposal]
J --> L[Consumer Receives Proposal]

K --> M{Supplier Response}
L --> N{Consumer Response}

M -->|Accept| O{Both Accepted?}
M -->|Decline| P[Proposal Declined]
M -->|Modify| Q[Counter Proposal]

N -->|Accept| O
N -->|Decline| P
N -->|Modify| Q

O -->|Yes| R[Deal → NEGOTIATING]
O -->|No| S[Pending: Awaiting Other Party]

Q --> T[Updated Proposal Sent to All]
T --> K

P --> U[Archive with Feedback]

```

Screen: Deal Creation / Enhancer Flow

Step 1: Service Selection - List of enhancer's registered services - Each service shows: current demand, active engagements, capacity remaining - Select service to "propose with this capability"

Step 2: Pair Discovery (Unique to Enhancer flow) - System shows supplier-consumer pairs where the gap between supply and need matches the enhancer's capability - Pair cards display:

```

+-----+
| Supplier: "Organic Cotton Farm, Texas" |
| Has: Raw Organic Cotton (500 tons) |
| + |
| Enhancer: [YOU - Textile Processing] |
| + |
| Consumer: "Sustainable Apparel Co." |
| Needs: Finished Organic Cotton Fabric |
| |
| Gap Analysis: |
| [Processing] + [Quality Cert] needed |
| Match to Your Service: 95% |
| |
| Estimated Deal Value: $450,000 |
| Your Potential Fee: $45,000 - $67,500 |
| |
| [View Details] [Propose Deal] |
+-----+

```

Step 3: Dual Proposal Configuration - The enhancer crafts a single proposal sent to both parties simultaneously - Proposal includes: - Value proposition for the supplier ("Get

premium pricing through processing”) - Value proposition for the consumer (“Guaranteed quality, shorter lead time”) - Enhancer’s fee structure (fixed, percentage, hybrid) - Proposed timeline - Expected outcomes for all three parties - **Smart Drafting**: AI assists in writing compelling proposals based on successful past deals

7.5 Invitation & Response Patterns

Invitation Card Design (received by any party):

```
+-----+
| [NEW DEAL INVITATION] |
| From: [Initiator Name] - [Role] |
| Deal: "Organic Cotton to Fabric Partnership" |
| Expires: 5 days remaining |
| Your Role: Supplier |
|
| Triad Preview: |
| You (Supplier) + [Enhancer Name] + [Consumer Name]|
|
| Deal Value Estimate: $450,000 |
| Your Estimated Share: $180,000 (40%) |
|
| [View Full Details] [Accept] [Decline] [Modify] |
+-----+
```

Response Options: - **Accept**: Moves deal to NEGOTIATING for this party (still waiting for others) - **Decline**: Opens feedback form (reasons: capacity, terms, trust, not interested) + alternative suggestion - **Modify**: Opens mini-negotiation with proposed changes before entering full NEGOTIATING - **Ask Question**: Threaded Q&A with initiator before responding

7.6 Deal Formation Summary Flow

flowchart LR

```
A[Draft] -->|Invitations Sent| B[Suggested]
B -->|Party 1 Accepts| C[Awaiting Others]
B -->|Party 1 Declines| D[Re-match or Close]
C -->|Party 2 Accepts| E[Awaiting Party 3]
C -->|Party 2 Declines| D
E -->|Party 3 Accepts| F[NEGOTIATING]
E -->|Party 3 Declines| D
```

7. Discovery & Matching UX

7.1 Discovery Hub Architecture

flowchart TD

```
A[Discovery Hub] --> B[Browse All]
A --> C[My Matches]
A --> D[Search]
A --> E[Map View]
A --> F[Watchlist]

B --> B1[Resources]
B --> B2[Needs]
```

B --> B3[Services]
 B --> B4[Complete Triads]

C --> C1[AI-Suggested Matches]
 C --> C2[Compatibility Scored List]

D --> D1[Text Search]
 D --> D2[Filters]
 D --> D3[Saved Searches]

E --> E1[Heat Map of Activity]
 E --> E2[Proximity Search]

7.2 Search & Browse Interface

Screen: Discovery / Browse

Layout: Left sidebar filters + main content grid

Search Bar: - Prominent centered search with placeholder: “Search resources, needs, services, or deals...” - Auto-suggest dropdown with categories - Voice search icon (mobile) - Recent searches below

Filter Sidebar:

Filter Category	Options
Party Type	Supplier, Consumer, Enhancer (toggle chips)
Category	Expandable taxonomy tree
Location	Map radius slider + region multi-select
Deal Value	Min-max range with histogram
Timeline	Available now, Within 30 days, Flexible
Verification Level	Bronze+, Silver+, Gold only
Trust Score	Slider: 0-100
Experience	New, 1-5 deals, 5-20 deals, 20+ deals
Compatibility	Show only >70% match

Sort Options: Relevance, Compatibility Score, Newest, Distance, Deal Value, Trust Score

View Toggles: Grid (default), List, Map

7.3 Match Suggestion Cards

Card Design - Resource (Supplier perspective):

```

+-----+
| [PHOTO/ICON] Resource Name      92% Match |
| Location: Austin, TX             [badge]   |
| Category: Raw Materials > Organic Cotton    |
|                                     |
| Available: 500 tons | Price: Negotiable      |
  
```

Quality: USDA Organic Certified
Supplier: GreenFields Farm
Trust: 4.8/5 12 deals completed
Why this matches you:
[x] Need alignment: 95%
[x] Location: 45 miles
[x] Timeline compatible
[Quick View] [Save] [Start Deal]

Card Design - Complete Triad Match (special card):

COMPLETE TRIAD MATCH: 89% Overall
[S: Farm Co.] + [E: Process Inc.] + [C: Apparel Ltd]
Supply: Raw Cotton -> Processing -> Fabric
Est. Value: \$500,000
Your Role: Enhancer (Processing)
Your Est. Fee: \$50,000-\$75,000
[View Deal Path] [Propose to Both] [Save]

7.4 Compatibility Score Display

Compatibility Visualization:

Compatibility Score: 87%
[=====>] Strong Match
Breakdown:
Resource-Need Location
Alignment: 95% Compatibility: 80%
[=====>] [=====>]
Timeline Trust/Experience
Overlap: 85% Score: 90%
[=====>] [=====>]
Price/Value
Compatibility: 85%
[=====>]

| [View Detailed Analysis] |
+-----+

Score Colors: - 90-100%: Deep Green (#27AE60) — “Excellent Match” - 75-89%: Light Green (#2ECC71) — “Strong Match” - 60-74%: Amber (#F39C12) — “Good Match with Notes” - 40-59%: Orange (#E67E22) — “Possible with Adjustments” - 0-39%: Red (#E74C3C) — “Low Compatibility”

7.5 Map View

Screen: Discovery / Map View

Features: - Heat map overlay showing deal density - Clustered pins that expand on zoom - Color-coded pins: Green (Supplier), Blue (Consumer), Purple (Enhancer) - Triad match pins: Star-shaped with gradient colors - Radius search tool: Click map point, drag to set radius - “Find matches along route” for logistics planning - Filter bar persists at top in compact mode

Map Pin Popup:

+-----+
| 3 Suppliers | 2 Consumers | 1 Enhancer |
| in this area |
| [List View] [Create Deal with These] |
+-----+

7.6 Save/Watchlist Functionality

Watchlist Features: - One-click save (bookmark icon on any card) - Organize into named lists: “Cotton Suppliers,” “Q3 Opportunities” - Alert settings per watchlist item: - Price changes - Availability changes - New matching parties join - Trust score changes - Compare mode: Side-by-side comparison of up to 3 saved items - Notes field on each saved item for personal annotations

7.7 Match Confidence Indicators

Confidence Levels: - **AI Recommended:** Based on algorithmic matching — shows “Sparkle” icon - **Verified Match:** Past successful deals between these parties — shows “Handshake” icon - **Community Match:** Recommended by other platform users — shows “People” icon - **Location Match:** Geographically convenient — shows “Pin” icon - **Price Match:** Within budget/value expectations — shows “Tag” icon

Match Notification:

+-----+
| [Sparkle] New Match Found! |
| We found a new consumer for your organic cotton |
| based on your updated availability. |
| Compatibility: 94% | Est. Value: \$200,000 |
| [View Match] [Dismiss] |
+-----+

7. Negotiation UX

7.1 Negotiation Interface Overview

flowchart TD

A[Deal → NEGOTIATING] --> B[Terms Dashboard]

B --> C[Value Distribution Tab]

B --> D[Terms & Conditions Tab]

B --> E[Communication Tab]

B --> F[Milestones Tab]

B --> G[Documents Tab]

C --> H[Adjust Sliders/Pie Chart]

D --> I[Edit Terms with Versioning]

E --> J[Threaded Messages]

F --> K[Define/Review Milestones]

G --> L[Upload/Review Documents]

H --> M{All Parties Agree?}

I --> M

K --> M

M -->|Yes| N[Lock Terms → TERMS_LOCKED]

M -->|No| O[Continue Negotiation]

7.2 Terms Dashboard Layout

Screen: Negotiation / Main Dashboard

Overall Layout:

```
+-----+-----+
| Deal: "Organic Cotton Partnership" | Party Status | | | | | |
| Status: NEGOTIATING | Day 5 of 30 | |
| [Value] [Terms] [Milestones] [Chat] [Docs] | [S] You: ACTIVE |
| | | | | | | [E] Process: OK |
| +-----+-----+ [C] Apparel: PENDING |
| | VALUE DISTRIBUTION | | | | | |
| | | | | | | |
| | [PIE CHART / SANKEY] | | Actions: |
| | | | | | | [Propose Change] |
| | Supplier: $180,000 (40%) [====> ] [Edit] || [Lock My Terms] |
| | Enhancer: $67,500 (15%) [==> ] [Edit] || [Ask Question] |
| | Consumer: $202,500 (45%) [=====> ] [Edit] || [Escalate] |
| | | | | | | |
| | Win-Win Score: 78/100 [====> ] || Recent Activity: |
| | "Balanced distribution with room to improve" || - E adjusted fee |
| | | | | | | - C commented |
| | [Visual Feedback Indicators] | | |
| +-----+-----+ | |
| | | | | | |
+-----+-----+
```

7.3 Value Distribution Visualizer

Primary Visualization: Interactive Pie Chart + Sankey Diagram

Pie Chart Mode: - Each party's slice is color-coded (Supplier: Green, Consumer: Blue, Enhancer: Purple) - Hover shows: party name, amount, percentage - Drag slice boundary to adjust allocation (within constraints) - Real-time recalculation of all values

Sankey Mode (toggle): - Shows flow: Resource (Supplier) -> Enhancement (Enhancer) -> Output (Consumer) - Width of flows represents value - Shows value-add at each step

Adjustment Controls: - Slider per party (0-100%, auto-balancing so total = 100%) - Absolute amount input with currency selector - "Market Rate" reference line showing typical distribution for this deal type - "Equal Split" quick-set button - "My Minimum" threshold marker that each party can set privately

7.4 Win-Win Validation Feedback

Real-Time Indicators:

```
+-----+
| Win-Win Analysis: 78/100 |
| [=====> ] |
| Factors: |
| [x] All parties above minimum threshold |
| [!] Distribution within 15% of market rate |
| [x] Value proportional to contribution |
| [ ] Risk/reward balanced across parties |
| [x] Payment terms aligned with cash flow needs |
| Suggestions to improve: |
| 1. Consider milestone-based payments for C |
| 2. E's fee could be performance-linked |
| [Apply Suggestion 1] [Apply Suggestion 2] |
+-----+
```

Feedback States: - **Green (80-100):** "Excellent win-win balance. All parties well-positioned."
- **Amber (60-79):** "Fair distribution. Review suggestions for improvement." - **Red (0-59):** "Imbalanced distribution. Risk of deal failure high."

Private vs Public: - Win-win score is visible to all parties - Individual "minimum acceptable" thresholds are PRIVATE - System alerts (privately) when a party's allocation drops below their minimum

7.5 Terms & Conditions Editor

Interface: - Structured term cards, each covering a specific topic: - Payment Terms (amount, schedule, method, currency) - Delivery Terms (schedule, location, method, acceptance criteria) - Quality Standards (specifications, inspection, rejection policy) - Intellectual Property (ownership, licensing, confidentiality) - Liability & Insurance (coverage, limits,

indemnification) - Termination (conditions, notice period, consequences) - Force Majeure (definition, notification, mitigation) - Dispute Resolution (escalation, mediation, arbitration)

Each Term Card:

```
+-----+
| Payment Terms           [Expand] |
| Status: Proposed by You | 2 versions |
|                               |
| Payment Schedule:         |
| 30% advance | 40% on delivery | 30% on acceptance|
|                               |
| [History] [Edit] [Propose Change] |
+-----+
```

Version History: - Side-by-side diff view showing changes between versions - Each version tagged with: author, timestamp, change summary - Restore previous version option - “Accepted” stamp when all parties agree to a version

7.6 In-App Messaging

Messaging Interface: - Threaded conversation organized by topic (not one long stream) - Auto-suggested topics: “Value Distribution,” “Timeline,” “Quality,” “General” - Message types: - **Text:** Standard messaging - **Proposal:** Structured change proposals (auto-formatted) - **File:** Document attachments (PDF, DOC, XLS, images) - **System:** Auto-generated notifications about changes - **Vote:** Poll/voting for group decisions

Proposal Message Format:

```
+-----+
| [Proposal] from [Supplier] |
| Change: Payment Terms     |
|                               |
| FROM: 30/40/30 split      |
| TO: 40/30/30 split (more advance) |
|                               |
| Reason: "Need more upfront for input costs" |
|                               |
| [Accept] [Counter] [Discuss] |
| Auto-expires: 48 hours      |
+-----+
```

7.7 Multi-Entity Party Voting UI

When a party is a group, the internal voting process is surfaced to other parties at a high level:

```
+-----+
| Consumer Party: GreenCo Cooperative (5 members) |
| Governance: Majority Vote (3/5 needed) |
|                               |
| Internal Voting Status:         |
| [Yes] Member A (Jane)         |
| [Yes] Member B (John)         |
| [Yes] Member C (Maria)  [== 3/5 ==] |
+-----+
```



```

| [Waiting] Member D (Alex) |
| [Waiting] Member E (Sam) |
|
| Progress: 3 of 5 approved (MAJORITY REACHED) |
| Updated: 10 minutes ago |
|
| [Internal] Your vote: [Approve] [Reject] [Abstain]|
+-----+

```

Visibility Rules: - Internal voting progress is **VISIBLE** to all parties (builds trust) - Individual votes are **ANONYMOUS** to other parties (privacy) - Only the party's representative name is shown - Threshold indicator changes color when reached

7.8 Deadline & Timeout Indicators

Negotiation Progress Bar: - 30-day default negotiation window (configurable by initiator) - Visual countdown: Green (>50% time left), Amber (10-50%), Red (<10%) - Milestone markers: "Day 7: Initial proposals due," "Day 21: Terms should be locked" - Escalation warnings at 75%, 90%, and 95% of time elapsed

Timeout Actions: - At 90%: System prompts all parties to extend or finalize - At 100%: Auto-pause with 48-hour grace period - Post-grace: Deal auto-archived with "Timeout" status, feedback collected

7.9 Proposal Summary & Lock

Screen: Negotiation / Lock Terms

Before locking, a comprehensive summary is presented:

```

+-----+
| TERMS LOCK CONFIRMATION |
|
| Please review all terms before locking. |
| Once locked, changes require unanimous approval. |
|
| +-----+
| | VALUE DISTRIBUTION || |
| | S: 40% ($180,000) | E: 15% ($67,500) ||
| | C: 45% ($202,500) | Total: $450,000 ||
| +-----+||
|
| +-----+
| | KEY TERMS ||
| | Payment: 30/40/30 schedule ||
| | Delivery: 60 days from commitment ||
| | Quality: USDA Organic certified ||
| | Milestones: 4 defined ||
| +-----+||
|
| +-----+
| | RISKS & CONSIDERATIONS ||
| | [!] Consumer has no past deals on platform ||
| | [!] Weather risk on agricultural timeline ||

```

```

| +-----+||
|                                     |
| By locking, you agree these terms are acceptable.|
| This action cannot be undone without unanimous |
| consent from all parties.                  |
|                                     |
| [Cancel and Return] [LOCK MY TERMS]          |
| +-----+

```

Lock Mechanics: - Each party locks independently - Deal advances to TERMS_LOCKED only when ALL parties have locked - Visual progress: “2 of 3 parties have locked terms” - Post-lock: Terms become read-only with “Unlock Request” option (requires unanimous approval)

7. Dashboard Designs

7.1 Dashboard Framework

All dashboards share a common shell with role-specific content zones.

```

+-----+
| [Logo] Home Discovery Deals Messages [Bell] [Profile] |
+-----+
|                                     |
| WELCOME HEADER                               |
| "Good morning, [Name]" + quick actions row    |
|                                     |
+-----+
| |                                     |
| SIDEBAR | MAIN CONTENT AREA (role-specific) |
| |                                     |
| - Home |                               |
| - My |                               |
| Profile|                               |
| - My |                               |
| [Items]|                              |
| - Deals |                             |
| - Fin. |                             |
| - Trust |                             |
| - Set. |                             |
| |                                     |
+-----+

```

7.2 Supplier Dashboard

Screen: Dashboard / Supplier

Module 1: Resource Portfolio Overview

```

+-----+
| MY RESOURCES                               [Manage] |
| +-----+ +-----+ |
| | Active: 3 | | Total Value: $2.4M| |

```

Watching: 8	Views this week: 142
[Resource Card 1] [Resource Card 2] [Resource Card 3]	
Cotton: Active	Soybeans: Pending
Land: Active	2 match requests
0 match requests	1 match req

Module 2: Active Deals Pipeline

MY DEALS				[Filter]
[Kanban View]		[List View]		
NEGOTI-	COMMITTED	EXECUTING	DONE	
ATING	Deal B	Deal C	Deal D	
Deal A				
(2)				
(2) = number of deals in this stage				

Module 3: Revenue & Performance

PERFORMANCE		[Details]
Revenue This Quarter: \$145,000		[Chart]
vs Last Quarter: +23%		(mini sparkline)
Active Deal Value: \$890,000		
Completed Deal Value: \$2.1M (lifetime)		
Top Enhancer Partners:		
1. Process Inc. (4 deals)		
2. Quality Cert Ltd (3 deals)		

Module 4: Enhancement Requests

ENHANCEMENT REQUESTS	
Parties interested in enhancing your resources:	
[Request Card]: Process Inc. wants to process your cotton. Est. value add: +40%. [View]	
[Request Card]: Certify Organic wants to certify your farm. Est. value add: +25% premium. [View]	

7.3 Consumer Dashboard

Screen: Dashboard / Consumer

Module 1: Needs Overview

+-----+	
MY NEEDS	[Post New]
Active: 2 Fulfilled: 5 Draft: 1	
[Need 1: Organic Cotton Fabric]	
Status: 3 supplier matches found	
Est. Cost: \$200,000 - \$280,000	
[View Matches] [Edit Need]	
[Need 2: Sustainable Packaging]	
Status: No matches yet (posted 2 days ago)	
[Boost Visibility] [Edit Need]	
+-----+	

Module 2: Active Deals & Output Tracking

+-----+	
ACTIVE DEALS & OUTPUTS	
Deal: "Cotton to Fabric Q3"	
[=====>] 45% Complete	
Next Milestone: Quality Inspection (Due Aug 15)	
Output Received: 200 of 500 units	
Quality Score: 98% (exceeds spec)	
[Track Delivery] [View Documents] [Message]	
+-----+	

Module 3: Cost Tracking

+-----+	
COSTS & BUDGET	
Total Committed: \$850,000	
Paid to Date: \$340,000	
Outstanding: \$510,000	
Budget vs Actual:	
[=====>] On track (2% under budget)	
Upcoming Payments:	
Aug 15: \$170,000 (Milestone 3)	
Sep 30: \$340,000 (Final delivery)	
+-----+	

7.4 Enhancer Dashboard

Screen: Dashboard / Enhancer

Module 1: Services & Demand

+-----+				
	MY SERVICES		[Manage]	
	+-----+			
	Textile Processing			
	Active Deals: 3 / Capacity: 5			
	Demand Trend: [===== high =====>]			
	Queue: 2 parties waiting			
	Avg. Deal Value: \$45,000			
	+-----+			
	+-----+			
	Organic Certification			
	Active Deals: 1 / Capacity: 10			
	Demand Trend: [==== moderate ====>]			
	+-----+			
	+-----+			

Module 2: Active Engagements

+-----+				
	ACTIVE ENGAGEMENTS			
	Revenue this month: \$135,000			
	[Engagement 1] Farm A -> Apparel B			
	Role: Processing Stage: EXECUTING			
	Fee: \$45,000 Received: \$22,500			
	[Progress Bar] [Details]			
	[Engagement 2] Farm C -> Retail D			
	Role: Quality Cert Stage: NEGOTIATING			
	Proposed Fee: \$15,000 (pending agreement)			
	[Details]			
	+-----+			

Module 3: Opportunities (Unique to Enhancers)

+-----+				
	OPPORTUNITIES: PAIRS NEEDING YOUR SERVICE			
	Based on your Textile Processing service:			
	+-----+			
	Farm X (Cotton) needs processing for			
	Apparel Y (Fabric). No processor matched.			
	Est. Deal: \$300,000 Your Est. Fee: \$30K			
	Match Score: 94%			
	[Propose Deal to Both] [Save for Later]			
	+-----+			
	+-----+			
	Co-op Z (Wool) needs processing for			
	Textile W (Yarn). One processor pending.			
	Est. Deal: \$180,000 Your Est. Fee: \$18K			

Competition: Low (1 other)	
[Propose Deal to Both]	
+-----+	
+-----+	

7.5 Deal Status Summary Widget (All Dashboards)

QUICK STATUS				[View All Deals]
+-----+ +-----+ +-----+ +-----+				
3	2	4	12	
Action	Active	Pend-	Total	
Needed	Deals	ing	Deals	
+-----+ +-----+ +-----+ +-----+				
Action Items:				
[!] 1 invitation awaiting your response				
[!] 2 proposals have new counter-offers				
[Review Now]				
+-----+				

7. Deal Execution Tracking

7.1 Execution Dashboard

flowchart TD

```

A[EXECUTING Status] --> B[Execution Dashboard]
B --> C[Milestone Timeline]
B --> D[Progress Metrics]
B --> E[Document Center]
B --> F[Party Activity]
B --> G[Payment Tracking]

C --> H[View/Update Milestones]
D --> I[Completion Percentage]
E --> J[Upload/Review Documents]
F --> K[Activity Feed]
G --> L[Payment Schedule & Status]

```

7.2 Milestone Timeline View

Screen: Execution / Milestone Timeline

MILESTONE TIMELINE: "Cotton to Fabric Q3"	
Overall: [=====>	45% Complete
TIMELINE:	
[Done] Jun 1: Deal Committed	
[Done] Jun 15: Cotton Harvested (S)	
[Done] Jul 1: Cotton Delivered to Processor (S)	

[====>]	Jul 15: Processing Complete (E)	DUE SOON
[]	Aug 1: Quality Inspection (E)	
[]	Aug 15: Fabric Delivered (E)	
[]	Sep 1: Final Acceptance (C)	
[]	Sep 15: Final Payment (C)	
	Current Phase: Processing (Day 8 of 15)	
[====>]	53% of phase complete	
+-----+		

Milestone States: - **Done:** Green checkmark, date completed - **In Progress:** Animated progress bar, due date shown - **Upcoming:** Gray, scheduled date - **Blocked:** Red warning icon, reason shown, escalation CTA - **Overdue:** Red with alert badge, days overdue shown

7.3 Progress Indicators

Overall Progress Calculation: - Weighted by milestone value/importance - Visual: Large circular progress indicator on dashboard - Breakdown by party contribution: - Supplier progress: 80% complete - Enhancer progress: 45% complete - Consumer progress: 20% complete (awaiting delivery)

Health Score:

+-----+		
Deal Health: 82/100		
[=====>]		
On Track Minor Risks		
Schedule: 2 days behind (buffer: 5 days)		
Budget: On track		
Quality: Exceeding expectations		
Communication: Active		
+-----+		

7.4 Document Upload & Verification

Document Center: - Organized by milestone and document type - Upload with drag-and-drop - Document types per role: - **Supplier:** Resource certificates, quality reports, delivery receipts - **Enhancer:** Processing reports, certification documents, completion proofs - **Consumer:** Acceptance certificates, payment confirmations, inspection reports - Verification status: Pending, Verified, Rejected (with reason) - Version control for evolving documents - E-signature integration for formal approvals

7.5 Payment Status

Payment Tracker:

+-----+	
PAYMENT SCHEDULE	

Total Deal Value: \$450,000		
+-----+ +-----+ +-----+		
Milestone	Milestone	Milestone
1: Adv.	2: Proc.	3: Final
\$135,000	\$180,000	\$135,000
PAID	PAID	PENDING
Jun 1	Jul 1	Sep 15
[Receipt]	[Receipt]	[Invoice]
+-----+ +-----+ +-----+		
Paid: \$315,000 (70%)		
Pending: \$135,000 (30%)		
Next Payment: \$135,000 due in 67 days		
+-----+		

7.6 Party Activity Feed

Activity Feed Design:

+-----+	
ACTIVITY FEED	
[All] [Supplier] [Enhancer] [Consumer] [System]	
[C] 2h ago: Consumer approved inspection report	
[E] 5h ago: Enhancer uploaded processing photos	
[S] 1d ago: Supplier confirmed delivery	
[\$] 1d ago: Milestone 2 payment released	
[E] 2d ago: Processing 60% complete update	
[Sys] 3d ago: Automated quality check passed	
[View All Activity]	
+-----+	

7.7 Completion Checklist

Completion Screen:

+-----+	
DEAL COMPLETION CHECKLIST	
All milestones complete. Review before finalizing.	
[x] All deliverables received and accepted	
[x] All payments completed	
[x] All documents uploaded	
[x] Quality inspection passed	
[x] Dispute period elapsed (if applicable)	
[] Reviews submitted by all parties	
[Submit Review] required before archive	
+-----+	

7. Notification Design

7.1 Notification Architecture

flowchart TD

```
A[Event Trigger] --> B{Notification Router}
B --> C[In-App]
B --> D[Email]
B --> E[SMS]
B --> F[Push]
B --> G[Webhook/API]

C --> H[Notification Center]
D --> I[Email Templates]
E --> J[SMS Short Form]
F --> K[Mobile Push]

H --> L{Read?}
L -->|No| M[Badge Count Increments]
L -->|Yes| N[Archive]
```

7.2 Notification Types

Category	Type	Urgency	Channels	Trigger
Deal Lifecycle	Invitation Received	High	In-app, Email, Push	Invited to deal
	Invitation Expiring	High	In-app, Email, Push	48h before expiry
	Terms Proposed	High	In-app, Email	New proposal posted
	Terms Locked	Medium	In-app, Email	All parties locked
	Milestone Due	High	In-app, Email, SMS, Push	7 days, 1 day before
	Milestone Complete	Medium	In-app, Email	Milestone marked done
	Deal Completed	Medium	In-app, Email	All milestones done
Financial	Payment Received	High	In-app, Email	Escrow released
	Payment Due	High	In-app, Email, SMS	Payment deadline approaching
	Deposit Required	High	In-app, Email, Push	Commitment phase reached
Matching	New Match Found	Medium	In-app, Email	AI match >80%
	Saved	Low	In-app	Watchlist item

Category	Type	Urgency	Channels	Trigger
Trust & Safety	Item Changed			updated
	Recommended Deal	Medium	In-app, Email	Curated opportunity
	Review Requested	Medium	In-app, Email	Post-deal review
	Trust Score Changed	Low	In-app	Score updated
	Dispute Filed	Critical	In-app, Email, SMS, Push	Dispute initiated
System	Verification Needed	Medium	In-app, Email	KYC renewal
	Account Security	Critical	All channels	Login from new device
	Terms of Service	Low	Email	Policy updates
	Platform Update	Low	In-app	New features

7.3 In-App Notification Center

Screen: Notifications / Center

+-----+	
NOTIFICATIONS	[Settings]
[All] [Unread: 12] [Mentions] [System]	
Today	
[Deal] New invitation: "Cotton Processing"	
From: Process Inc. 2 hours ago [View]	

[Payment] Milestone 2 payment of \$180,000	
received. 5 hours ago [View Receipt]	

[Milestone] Quality Inspection due in 3 days	
8 hours ago [View Milestone]	
Yesterday	
[Match] 2 new supplier matches for your fabric	
need. 1 day ago [View Matches]	

[System] Platform maintenance scheduled	
Sunday 2-4 AM UTC. 1 day ago	
[Mark All Read]	
+-----+	

Notification Card States: - **Unread:** Blue left border, bold title, timestamp emphasized - **Read:** No border, normal weight, muted timestamp - **Action Required:** Orange left border, CTA button highlighted - **Critical:** Red left border, icon indicator, bold throughout

7.4 Email Notification Templates

Email Structure (consistent across all types):

Header: TriadDeal Logo + "[Notification Type]"
Greeting: "Hi [First Name],"
Body: Clear, concise description of the event
Context Block: Deal name, your role, key details
Primary CTA: Large button linking to relevant screen
Secondary Actions: "View in Browser," "Update Preferences"
Footer: Unsubscribe, Notification settings, Support

Example: Deal Invitation Email

Subject: [Action Required] Invitation to "Organic Cotton Partnership"
as Supplier — expires in 7 days

Hi [Name],

Process Inc. has invited you to participate in a new deal as the Supplier.

DEAL: Organic Cotton Partnership
YOUR ROLE: Supplier (Resource Provider)
ESTIMATED VALUE: \$180,000 (40% of deal)
OTHER PARTIES:
- Enhancer: Process Inc. (Textile Processing)
- Consumer: Sustainable Apparel Co. (Fabric Need)
COMPATIBILITY: 87% match

[REVIEW INVITATION] [DECLINE]

This invitation expires on [Date].

Manage your notification preferences: [Settings]

7.5 SMS Notifications

SMS Rules: - Only for Critical and High urgency notifications - Character limit: 160 characters - Always include short link - No marketing content via SMS (opt-in required)

Example SMS:

TriadDeal: Payment of \$180,000 received for "Cotton Partnership". View: tdl.io/p/abc123

7.6 Notification Preferences UI

Screen: Settings / Notifications

+-----+				
	NOTIFICATION PREFERENCES			
	+-----+			
	DEAL-RELATED			
	Invitations	[In-app]	[Email]	[SMS]
	Terms Changes	[In-app]	[Email]	[]
	Milestones	[In-app]	[Email]	[SMS]
	Payments	[In-app]	[Email]	[SMS]
	Deal Complete	[In-app]	[Email]	[]
	+-----+			
	+-----+			
	MATCHING & RECOMMENDATIONS			
	New Matches	[In-app]	[Email]	[]
	Saved Item Alerts	[In-app]	[]	[]
	Weekly Digest	[In-app]	[Email]	[]
	+-----+			
	+-----+			
	ACCOUNT & SECURITY			
	Login Alerts	[In-app]	[Email]	[SMS]
	Security Events	[In-app]	[Email]	[SMS]
	+-----+			
	Quiet Hours: [9 PM] to [8 AM] [Timezone]			
	[Save Changes]			
	+-----+			

7.7 Urgency Indicators

Urgency Level Visual System:

Level	Color	Icon	Badge	Channels	Example
Critical	Red	Exclamation triangle	“URGENT”	All + repeated	Dispute filed, payment failed
High	Orange	Exclamation circle	“ACTION REQUIRED”	In-app, Email, Push	Invitation expiring, milestone due
Medium	Blue	Info circle	None	In-app, Email	Terms proposed, match found
Low	Gray	Info dot	None	In-app	Weekly digest, tips

Persistent Urgent Banner:

```

+-----+
| [URGENT] 1 invitation expires in 24 hours. |
| [Review Now] [Dismiss] |
+-----+

```

Shown as sticky banner at top of all pages until resolved.

7. Trust & Safety UX

7.1 Trust Score Display

Trust Score Architecture:

```

+-----+
| TRUST PROFILE: [Party Name] |
| |
| OVERALL TRUST SCORE: 4.7 / 5.0 |
| [=====> ] 94th percentile |
| Based on 23 verified transactions |
| |
| SCORE BREAKDOWN: |
| +-----+ +-----+ |
| | As Supplier: 4.8 | | As Consumer: 4.5 | |
| | [=====>] | | [=====> ] | |
| | 12 deals | | 8 deals | |
| +-----+ +-----+ |
| +-----+ |
| | As Enhancer: 4.9 | |
| | [=====>] | |
| | 3 deals | |
| +-----+ |
| |
| DIMENSION BREAKDOWN: |
| Reliability: [=====>] 4.8 |
| Communication: [=====> ] 4.6 |
| Quality: [=====>] 4.9 |
| Timeliness: [=====> ] 4.5 |
| Fairness: [=====>] 4.7 |
| |
+-----+

```

Score Components: - **Overall:** Weighted average across all role-specific scores - **Role-specific:** Separate scores for each role the party plays - **Dimensions:** Reliability, Communication, Quality, Timeliness, Fairness - **Calculation:** Verified transaction reviews only, weighted by deal value and recency (more recent = more weight)

Trust Badge System: - 4.5-5.0: Gold “Trusted Partner” badge - 4.0-4.4: Silver “Reliable” badge - 3.5-3.9: Bronze “Established” badge - 3.0-3.4: “Developing” (no badge) - Below 3.0: “Needs Improvement” warning indicator

7.2 Review Submission Flow

flowchart TD

```

A[Deal → COMPLETED] --> B[Review Request Notification]

```

```

B --> C[Review Form]
C --> D[Rate Each Party]
D --> E[Score Each Dimension]
E --> F[Written Review]
F --> G[Submit]
G --> H{All Reviews In?}
H -->|Yes| I[Reviews Published]
H -->|No| J[Pending Other Reviews]
I --> K[Score Recalculated]

```

Review Form Screen:

```

+-----+
| REVIEW: "Organic Cotton Partnership" |
| Deal completed on [Date] |
|
| Your Role: Supplier |
|
| Rate Enhancer: Process Inc. |
| +-----+ +-----+ +-----+ +-----+ +-----+ |
| | 1 | | 2 | | 3 | | 4 | | 5 | | |
| | | | | [x] | | | | |
| +-----+ +-----+ +-----+ +-----+ +-----+ |
|
| Dimensions (auto-filled from overall, editable): |
| Reliability: [====> ] 3/5 |
| Communication: [=====> ] 4/5 |
| Quality: [=====> ] 4/5 |
| Timeliness: [====> ] 2/5 |
| Fairness: [=====> ] 4/5 |
|
| Written Review: |
| [Textarea: What went well? What could improve?] |
| Min 50 characters. |
|
| [Submit Review] |
| Reviews are published only after all parties |
| submit, to prevent bias. |
+-----+

```

Review Rules: - Reviews are **blind**: Not visible until ALL parties submit - **Double-blind**: Parties cannot see each other's reviews before submitting their own - Minimum character requirement for written reviews (50 chars) - Reviews are tied to verified completed deals only - Disputed deals have a separate "Resolution Review" flow

7.3 Review Display

Review Card:

```

+-----+
| REVIEW FROM: Farm Cooperative (Supplier) |
| Deal: "Organic Cotton Partnership" | $450,000 |
| Date: Sep 2024 |
|

```

Overall: 4.5/5 [Verified Transaction]
"Process Inc. delivered excellent quality processing on time. Communication was proactive and transparent throughout. Highly recommend."
Reliability: 5 Communication: 5 Quality: 5
Timeliness: 3 Fairness: 5
Helpful? [Yes: 12] [No]

+-----+

7.4 Dispute Filing Flow

flowchart TD

```

A[Issue Identified] --> B[Dispute Button on Deal]
B --> C[Dispute Category Selection]
C --> D[Dispute Details Form]
D --> E[Evidence Upload]
E --> F{Attempt Resolution?}
F -->|Yes| G[Direct Communication]
F -->|No| H[Escalate to Mediation]
G --> I{Resolved?}
I -->|Yes| J[Dispute Withdrawn]
I -->|No| H
H --> K[Mediator Assigned]
K --> L[Mediation Process]
L --> M{Resolved?}
M -->|Yes| N[Resolution Implemented]
M -->|No| O[Arbitration Option]

```

Dispute Form Screen:

FILE A DISPUTE	
Deal: "Organic Cotton Partnership"	
What is the issue about?	
☐ Quality not as agreed	
☐ Delivery delayed	
☐ Payment not received	
☐ Terms not honored	
☐ Communication breakdown	
☐ Other: [Specify]	
Describe the issue:	
[Textarea with guided prompts]	
"What was agreed?"	
"What actually happened?"	
"What resolution are you seeking?"	
Upload Evidence:	
[Drag & Drop Area]	
Documents, photos, screenshots accepted	

Preferred Resolution:	
☐ Refund	
☐ Re-performance	
☐ Price adjustment	
☐ Timeline extension	
☐ Other: [Specify]	
[Submit Dispute] [Save Draft]	
The other parties will be notified immediately.	

Dispute Escalation Levels: 1. **Level 1 - Direct:** Parties communicate to resolve (first 7 days) 2. **Level 2 - Mediation:** Platform mediator joins (7-14 days) 3. **Level 3 - Arbitration:** Binding arbitration (14-30 days)

7.5 Safety Badges & Verifications

Badge Display on Profiles:

[Party Name]	[Photo]
Badges:	
[ID Verified]	[Business Verified]
[10+ Deals]	[Top Rated]
[Fast Responder]	
Hover/Click badge for details:	
ID Verified: Government ID verified on	[Date]
Business Verified: Registration #	12345
Gold Badge: Trust score	4.7/5.0

Badge Types: - **Verification:** ID Verified, Business Verified, Group Verified - **Experience:** 1 Deal, 5 Deals, 10 Deals, 25 Deals, 50+ Deals - **Performance:** Top Rated (4.5+), Fast Responder (< 2h avg), Reliable (98% on-time) - **Community:** Helpful Reviewer, Mentor (helped others)

7.6 Report & Block Functionality

Report User Flow:

REPORT: [Party Name]	
Reason for reporting:	
☐ Spam or unsolicited offers	
☐ Fraudulent behavior	
☐ Harassment or inappropriate conduct	
☐ Fake profile / Misrepresentation	
☐ Violation of terms	
☐ Other	
Details: [Textarea]	

Evidence: [Upload optional]	
[Also block this user from contacting me]	
[Submit Report]	
Reports are confidential. False reporting may result in account suspension.	

+-----+

Block Consequences: - Blocked user cannot send invitations or messages - Blocked user does not appear in match suggestions - Existing deals are not affected (but no new deals possible) - Block is reversible from Settings

7. Mobile Considerations

7.1 Responsive Priority Matrix

Feature	Desktop Priority	Mobile Priority	Notes
Dashboard	High	High	Simplified cards, stacked layout
Discovery/Search	High	High	Full-screen map, bottom sheet filters
Deal Creation	High	High	Step-by-step wizard, minimal per screen
Negotiation	High	High	Tabbed interface, focused views
Execution Tracking	High	Medium	Milestone list, detail on tap
Messaging	Medium	High	Native-feeling chat interface
Notifications	Medium	High	Badge-driven, swipe actions
Profile/	Medium	Medium	Standard

Feature	Desktop Priority	Mobile Priority	Notes
Settings			forms
Admin/ Reporting	High	Low	Desktop- first, mobile summary only

7.2 Mobile-Specific Interactions

Bottom Navigation (5 tabs):



Center FAB (Floating Action Button): - Primary CTA for deal creation - Tapped: Expands to 3 options: - “New Deal” (expand from FAB) - “Post Resource” (if Supplier) - “Post Need” (if Consumer) - “Offer Service” (if Enhancer)

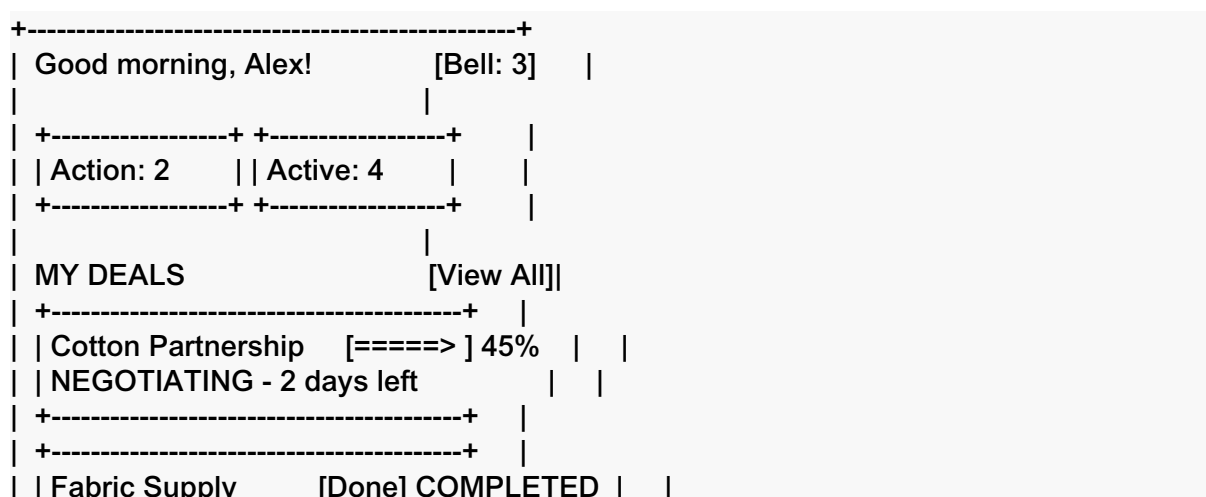
Swipe Gestures: - **Discovery cards:** Swipe right to save, swipe left to dismiss - **Notification:** Swipe to dismiss, long-swipe to mark read - **Deal list:** Swipe for quick actions (message, view progress)

Pull-to-Refresh: All list views support pull-to-refresh with haptic feedback

Native Features Integration: - **Camera:** Direct photo upload for documents, resources - **GPS:** Auto-detect location for map features - **Push Notifications:** Rich notifications with action buttons - **Biometric Auth:** Fingerprint/Face ID for sensitive actions (payments, term locking) - **Share Sheet:** Share deal profiles, resource listings

7.3 Mobile-Optimized Screens

Mobile Dashboard:



		Final review pending		
	+	-----	+	
		QUICK ACTIONS		
	[+ New Deal]	[Discovery]	[Messages: 5]	
		RECENT ACTIVITY		
	[E]	Process Inc. uploaded document...		
	[\$]	Payment received: \$45,000		
	+	-----	+	

Mobile Negotiation: - Sticky tab bar at top: Value | Terms | Chat | Files - Value tab: Vertical stacked bars instead of pie chart - Terms: Accordion expandable cards - Chat: Full-screen chat interface - Files: List with preview thumbnails

Mobile Deal Creation: - Multi-step wizard (3-5 steps max per screen) - Step indicator at top: [1] [2] [3] [4] - Keyboard-aware inputs (scroll on focus) - “Save Draft” always available - Confirmation modal before final submission

7.4 Offline Support

- View cached deal data offline
- Draft messages saved and sent when online
- Offline indicator banner when connection lost
- Queue actions for sync: “Your review will be submitted when you’re back online”

7.5 Performance Considerations

- Skeleton screens for all loading states
- Image lazy loading and progressive loading
- Virtual scrolling for long lists (deals, messages, notifications)
- Compressed assets for mobile networks
- “Tap to load heavy content” option for data-intensive visualizations

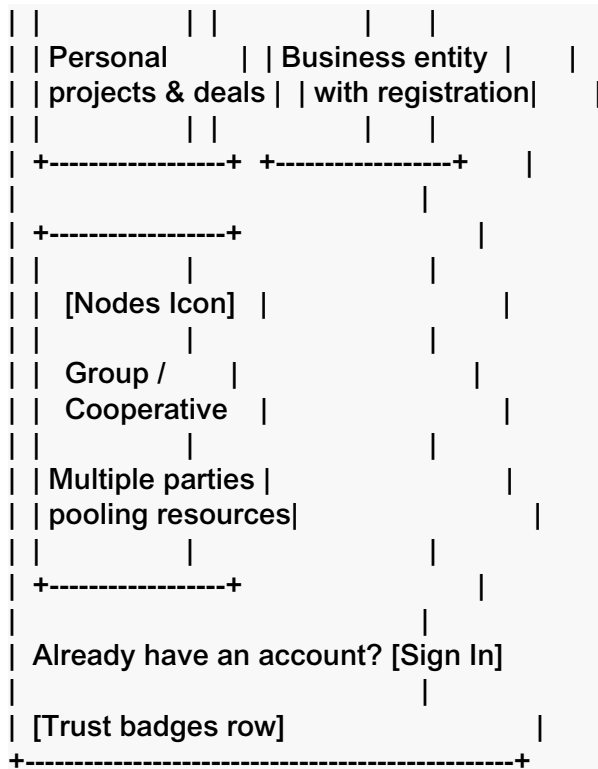
7. Wireframe Descriptions

7.1 Wireframe 1: Registration & Role Selection

Screen: Registration / Role Selection **Primary Purpose:** Guide new users through account type and role selection **Viewport:** Desktop 1440px, Mobile 375px

Layout Structure:

+	-----	+		
	[TriadDeal Logo - centered]			
	How will you participate?			
	Select your account type to get started.			
	+	-----	+	
		[Person Icon]		[Building Icon]
		Individual		Organization



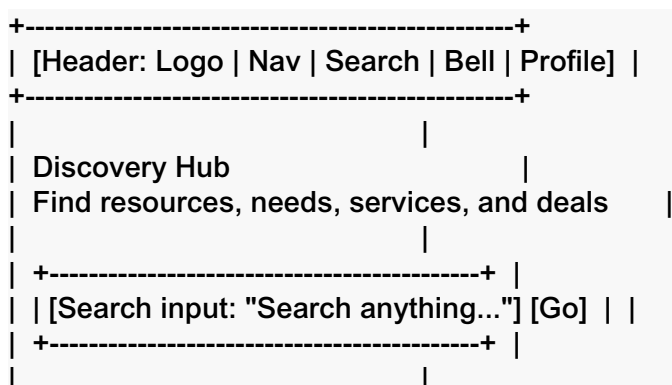
Key Elements & Positions: - **Logo:** Top-center, 120px width, subtle drop shadow - **Headline:** Centered, H1, 32px, font-weight 600 - **Subtitle:** Centered below headline, H3, 16px, color #666 - **Account Type Cards:** 2+1 layout (2 on top row, 1 centered below), equal width ~320px each, 24px gap - **Card Content:** Centered icon (64px), title (H2, 20px), description (14px, #666) - **Card Interaction:** Border 1px #E0E0E0, hover: border-color #3498DB, shadow elevation 2, transition 200ms ease - **Sign In Link:** Bottom center, 14px - **Trust Badges:** Bottom, horizontal row of shield icons + text, 12px, #999

Primary Actions: Select account type card (triggers next step) **Secondary Actions:** Sign In link **Data Displayed:** None (pure navigation) **Interaction Patterns:** Card selection expands to form, no page reload

7.2 Wireframe 2: Discovery Hub

Screen: Discovery / Browse **Primary Purpose:** Enable users to find and filter resources, needs, services, and triads **Viewport:** Desktop 1440px

Layout Structure:



```

| [Resources] [Needs] [Services] [Triads] |
| (tab filter chips) |
| |
| +-----+ +-----+ +-----+ |
| | | | | | | | | | |
| | FILTER | | [Sort: Relevance v] [Grid v] | |
| | SIDEBAR | | | | |
| | | | +-----+ +-----+ +-----+ | |
| | Category | | | Card 1 | | Card 2 | | Card | |
| | Location | | +-----+ +-----+ +-----+ | |
| | Price | | | | |
| | Timeline | | +-----+ +-----+ +-----+ | |
| | Trust | | | Card 4 | | Card 5 | | Card | |
| | Verif. | | +-----+ +-----+ +-----+ | |
| | | | | | |
| | [Apply] | | [Load More] | |
| +-----+ +-----+ +-----+ |
| |
+-----+

```

Key Elements & Positions: - **Search Bar:** Full-width container below header, centered, 640px max width - **Type Tabs:** Horizontal pill-shaped tabs, below search, 16px gap - **Filter Sidebar:** Left column, 280px fixed width, collapsible - **Filter Sections:** Stacked expandable accordions (Category, Location, Price, etc.) - **Sort Controls:** Top-right of content area, dropdown + view toggle - **Card Grid:** CSS Grid, 3 columns desktop, 2 columns tablet, 1 column mobile - **Card Size:** ~360px wide, ~400px tall, 24px gap

Primary Actions: Click card for detail view, Apply filters **Secondary Actions:** Sort, change view mode, Load more **Data Displayed:** Match cards with image, title, compatibility %, location, price, trust badge **Interaction Patterns:** Infinite scroll pagination, filter sidebar slides in on mobile, cards lift on hover

7.3 Wireframe 3: Match Detail & Triad Compatibility View

Screen: Discovery / Match Detail **Primary Purpose:** Show detailed compatibility analysis for a potential triad match **Viewport:** Desktop 1440px

Layout Structure:

```

+-----+
| [Header] |
+-----+
| Breadcrumb: Discovery > Matches > Detail |
| |
| TRIAD MATCH: 87% Compatibility |
| |
| +-----+ +-----+ +-----+ |
| | SUPPLIER | | ENHANCER | | CONSUMER | |
| | | | | | |
| | [Photo] | | [Photo] | | [Photo] | |
| | | | | | |
| | GreenFields | | Process Inc. | | Sustainable | |
| | Farm | | | | Apparel Co. | |

```

Trust: 4.8	Trust: 4.5	Trust: 4.2
12 deals	8 deals	5 deals

COMPATIBILITY BREAKDOWN

Resource-Need Alignment: 95% [=====] ||

Location Compatibility: 80% [=====>] ||

Timeline Overlap: 85% [=====>] ||

Trust/Experience Match: 90% [=====] ||

Price-Value Compatibility: 85% [=====>] ||

DEAL PATH

[Raw Cotton] --> [Processing] --> [Fabric]

\$180K \$67.5K \$202.5K

ESTIMATED TIMELINE: 60 days

TOTAL DEAL VALUE: \$450,000

[Save Match] [Share] [Start Deal with This Triad]

Key Elements & Positions: - **Triad Header:** H1 with large compatibility percentage in colored circle - **Party Cards:** 3-column equal width, each with photo, name, trust score, deal count - **Compatibility Section:** Full-width card with horizontal bar charts - **Deal Path:** Visual flow diagram showing transformation + value at each step - **Summary Stats:** 2-3 key metrics below deal path - **CTA Row:** 3 buttons — Save (secondary), Share (secondary), Start Deal (primary)

Primary Actions: Start Deal (initiates deal creation with pre-selected triad) **Secondary Actions:** Save match, Share match profile **Data Displayed:** Party profiles, compatibility breakdown, deal path, financial estimates **Interaction Patterns:** Party cards link to full profiles, compatibility bars animate on load, Deal Path nodes are clickable for detail

7.4 Wireframe 4: Negotiation Dashboard

Screen: Negotiation / Main Dashboard **Primary Purpose:** Enable all 3 parties to negotiate terms, value distribution, and milestones **Viewport:** Desktop 1440px

Layout Structure:

Deal: "Cotton Partnership" Day 12/30 NEGOTIATING [Help]		
[Value Distribution] [Terms] [Milestones] [Chat] [Documents]		
VALUE DISTRIBUTION PARTY STATUS		
[PIE CHART - interactive] [Green] S:		
You - ACTIVE		

```

| | Supplier (You): 40% $180K [=====> ] [Edit] | [Purple] E: | |
| | Enhancer: 15% $67K [==> ] [Edit] | Process Inc. | |
| | Consumer: 45% $202K [=====> ] [Edit] | Status: OK | |
| | | [Blue] C: | |
| | Market Reference: [equal split indicator] | Sustainable | |
| | | Status: PENDING |
| | Win-Win Score: 78/100 [=====> ] | | |
| | "Balanced with room to improve" | ACTIONS: | |
| | | [Propose | |
| | SUGGESTIONS: | Changes] | |
| | 1. Performance-link E's fee [Apply] | [Lock My | |
| | 2. Milestone payment for C [Apply] | Terms] | |
| | | [Ask Q] | |
| | [View Full Analysis] | +-----+ |
| +-----+ |
|
| RECENT ACTIVITY |
| [E] 1h: Enhancer reduced fee to $67,500 |
| [C] 3h: Consumer requested 30-day delivery |
| [S] 5h: You proposed 40/15/45 split |
|
| +-----+ +-----+

```

Key Elements & Positions: - **Header Bar:** Deal name + negotiation timer (countdown badge) + status badge - **Tab Navigation:** Horizontal tabs below header, sticky on scroll - **Main Content Area:** 70% width, contains active tab content - **Right Sidebar:** 30% width, party status + actions - **Pie Chart:** Center of value tab, 300px diameter, interactive - **Sliders:** Below chart, one per party, with percentage and amount - **Win-Win Score:** Bottom of value section with feedback text

Primary Actions: Edit distribution (opens adjustment panel), Lock Terms, Propose Changes
Secondary Actions: View analysis, Apply suggestions, Ask Question **Data Displayed:** Value distribution, compatibility scores, party status, activity feed **Interaction Patterns:** Pie chart slices draggable, sliders sync with chart, real-time recalculation, score updates on each adjustment

7.5 Wireframe 5: Supplier Dashboard

Screen: Dashboard / Supplier **Primary Purpose:** Provide suppliers with overview of resources, deals, revenue, and opportunities **Viewport:** Desktop 1440px

Layout Structure:

```

+-----+
| [Header with navigation] |
+-----+
| Welcome back, [Name]! Today: [Date] |
| [+ Post Resource] [Discovery] [Messages: 3] |
|
| +-----+ +-----+ |
| | Resources: 3 | | Revenue Q3: $145K | |
| | Active Deals: 4 | | Avg. Deal: $112K | |
| +-----+ +-----+ |

```

```

|
| MY RESOURCES
| +-----+ +-----+ +-----+
| | [Cotton| | [Soy- | | [Land |
| | Img] | | bean] | | Img] |
| | Cotton | | Soybeans| | Farmland|
| | Active | | Pending| | Active |
| | 2 req | | 0 req | | 1 req |
| +-----+ +-----+ +-----+
|
| DEAL PIPELINE [View All]
| +-----+ +-----+ +-----+ +-----+
| | NEGOTIAT | | COMMITTED| | EXECUTING| | COMPLE| |
| | | | | | | | | |
| | Deal A | | Deal B | | Deal C | | Deal D | |
| | $120K | | $200K | | $450K | | $85K | |
| | 2 parties| | Waiting | | Day 12 | Done | |
| +-----+ +-----+ +-----+ +-----+
|
| REVENUE TREND
| [Line chart: Jan - Sep, revenue trend]
|
| ENHANCEMENT REQUESTS
| Process Inc. wants to process your cotton...
| [View Request]
|
+-----+

```

Key Elements & Positions: - **Welcome Header:** Left-aligned H1 with date, action buttons on right - **Stats Row:** 2-4 summary cards, flex layout, equal width - **Resources Section:** Horizontal scrollable row of resource cards - **Pipeline:** Kanban-style columns, each ~220px wide, cards stacked vertically - **Revenue Chart:** Full-width, 250px height, area chart - **Enhancement Requests:** Compact list below chart

Primary Actions: Post Resource, View Deal, Start Deal from request **Secondary Actions:** View All links, resource management **Data Displayed:** Resource inventory, deal pipeline stages, revenue metrics, enhancement requests **Interaction Patterns:** Kanban cards drag between columns (if role permits), charts have tooltips on hover, resource cards expand to detail view

7.6 Wireframe 6: Deal Execution Timeline

Screen: Execution / Milestone Timeline **Primary Purpose:** Track deal progress through milestones with visual timeline **Viewport:** Desktop 1440px

Layout Structure:

```

+-----+
| [Header]
+-----+
| Cotton Partnership      Status: EXECUTING |
| [Overview] [Timeline] [Documents] [Payments] |
|

```


OVERALL PROGRESS
[=====>] 45% Complete
Deal Health: 82/100 [On Track]
PARTY PROGRESS
+-----+ +-----+ +-----+
Supplier Enhancer Consumer
80% 45% 20%
[====>] [==>] [>]
+-----+ +-----+ +-----+
MILESTONE TIMELINE
[Green Dot] Jun 1: Deal Committed (Supplier)
[Done badge]
[Green Dot] Jun 15: Cotton Harvested (Supplier)
[Done badge]
[Green Dot] Jul 1: Delivered to Processor (S)
[Done badge]
[Orange Dot] Jul 15: Processing Complete (E)
[In Progress] Due in 3 days
[====>] 53% of phase
[Update Progress] [Upload Proof]
[Gray Dot] Aug 1: Quality Inspection (E)
[Gray Dot] Aug 15: Fabric Delivery (E)
[Gray Dot] Sep 1: Final Acceptance (C)
[Gray Dot] Sep 15: Final Payment (C)
NEXT PAYMENT: \$180,000 due Jul 15
[View Payment Schedule]
RECENT ACTIVITY
[E] Uploaded processing progress photos
[S] Confirmed delivery receipt
[\$] Milestone 2 payment released
+-----+

Key Elements & Positions: - **Progress Header:** Full-width, large progress bar with percentage - **Party Progress:** 3-column breakdown showing each party’s contribution - **Timeline:** Vertical line with dot markers, left-aligned content - **Dot Colors:** Green (done), Orange (in progress), Gray (upcoming), Red (blocked/overdue) - **Active Milestone:** Expanded to show sub-progress and actions - **Activity Feed:** Bottom section, reverse chronological

Primary Actions: Update Progress, Upload Proof, View Payment Schedule **Secondary Actions:** View documents, message party responsible for milestone **Data Displayed:** Milestone list with dates, completion status, party assignments, payment schedule **Interaction Patterns:** Timeline scrollable vertically, milestone dots clickable for detail modal, progress bars animate on load

7.7 Wireframe 7: Group Voting & Consensus Interface

Screen: Negotiation / Group Voting **Primary Purpose:** Enable multi-member parties to vote on deal terms internally **Viewport:** Desktop 1440px

Layout Structure:

```
+-----+
| [Header] |
+-----+
| GROUP VOTING: Term Approval |
| Deal: Cotton Partnership | Term: Value Distribution |
| |
| YOUR PARTY: GreenCo Cooperative |
| Members: 5 | Governance: Majority (3/5) |
| |
| PROPOSED TERM: |
| +-----+ |
| | Value Split: Supplier 40%, Enhancer 15%, | |
| | Consumer 45%. Total: $450,000. | |
| | Your share as Supplier: $180,000 | |
| +-----+ |
| |
| VOTING STATUS |
| +-----+ +-----+ +-----+ |
| | Jane | | John | | Maria | |
| | [APPROVED] | | [APPROVED] | | [APPROVED] | |
| | Green] | | Green] | | Green] | |
| | 2d ago | | 1d ago | | 5h ago | |
| +-----+ +-----+ +-----+ |
| +-----+ +-----+ |
| | Alex | | Sam | | |
| | [WAITING | | [WAITING | |
| | Gray] | | Gray] | |
| | | | | |
| +-----+ +-----+ |
| |
| Progress: 3 of 5 (MAJORITY REACHED) |
| [=====> ] 60% |
| |
| YOUR VOTE |
| You haven't voted yet. Cast your vote: |
| [APPROVE] [REJECT] [ABSTAIN] |
| |
| (Rejecting will not block — majority reached) |
| |
| Other parties can see: 3/5 approved (anonymous) |
| They cannot see: Individual votes or who voted |
| |
+-----+
```

Key Elements & Positions: - **Party Info:** Header showing group name, size, governance model - **Term Summary:** Highlighted card showing what's being voted on - **Voting Grid:**

Member cards in grid layout, each showing name and vote status - **Progress Bar**: Large indicator below grid - **Your Vote**: Prominent section at bottom if user hasn't voted - **Visibility Note**: Small text explaining visibility rules

Primary Actions: Cast Vote (Approve/Reject/Abstain) **Secondary Actions**: View term details, message other members **Data Displayed**: Member list, vote status, progress toward threshold, governance rules **Interaction Patterns**: Vote buttons show confirmation modal, progress bar animates on vote, member cards update in real-time (via WebSocket)

7.8 Wireframe 8: Notification Center

Screen: Notifications / Center **Primary Purpose**: Central hub for all user notifications with filtering and actions **Viewport**: Desktop 1440px, Mobile 375px

Layout Structure:

```
+-----+
| [Header]                [Settings] |
+-----+
| NOTIFICATIONS           |
| [All: 24] [Unread: 5] [Action Needed: 2] |
|                               |
| TODAY                   |
| +-----+ |
| |[Red border] URGENT    || |
| | Invitation to "Cotton Processing" expires ||
| | in 24 hours.          ||
| | From: Process Inc. | 2h ago      ||
| | [View Invitation] [Decline]      ||
| +-----+ |
| +-----+ |
| |[Blue border] NEW      || |
| | Payment of $180,000 received for Milestone 2 ||
| | Deal: Cotton Partnership | 5h ago      ||
| | [View Receipt]          ||
| +-----+ |
| +-----+ |
| |[Orange border] ACTION || |
| | Enhancer proposed new terms: fee reduced ||
| | to $67,500. Review and respond.          ||
| | Deal: Cotton Partnership | 8h ago      ||
| | [Review Proposal]          ||
| +-----+ |
|                               |
| YESTERDAY               |
| +-----+ |
| | Match found: New consumer for your organic || |
| | cotton. Compatibility: 92%.              ||
| | 1 day ago | [View Match]                ||
| +-----+ |
| +-----+ |
| | Weekly digest: 3 new matches, 2 deals updated ||
| | 1 day ago | [View Digest]                ||
```

+-----+		
[Mark All as Read]		
+-----+		

Key Elements & Positions: - **Filter Tabs:** Horizontal, sticky below header - **Date Dividers:** “Today,” “Yesterday,” “Earlier This Week” - **Notification Cards:** Full-width, left border colored by urgency - **Card Content:** Icon, title, description, timestamp, CTA buttons - **Unread:** Bold text, blue left border, small “NEW” badge - **Read:** Normal weight, no border

Primary Actions: Click CTA buttons (View, Review, Respond), Mark as Read **Secondary Actions:** Filter by type, Mark All Read, adjust notification settings **Data Displayed:** Notification list with metadata (source, time, urgency, actions) **Interaction Patterns:** Cards swipe-dismiss on mobile, hover reveals secondary actions on desktop, real-time badge count updates

7.9 Wireframe 9: Trust Score & Review Profile

Screen: Trust / Profile Review **Primary Purpose:** Display comprehensive trust profile with scores, badges, and reviews **Viewport:** Desktop 1440px

Layout Structure:

+-----+				
[Header]				
+-----+				
+-----+				
Photo	GreenFields Farm			
	@greenfields	Member since Jan 2023		
	[Gold Shield]	[50+ Deals]	[Top Rated]	
+-----+				
TRUST SCORE: 4.7 / 5.0				
[=====>]		94th percentile		
Based on 23 verified transactions				
ROLE-SPECIFIC SCORES				
+-----+		+-----+		+-----+
Supplier	Consumer	Enhancer		
4.8/5	4.5/5	4.9/5		
12 deals	8 deals	3 deals		
[=====>]	[=====>]	[=====>]		
+-----+		+-----+		+-----+
DIMENSION BREAKDOWN				
+-----+				
Reliability	[=====>] 4.8			
Communication	[=====>] 4.6			
Quality	[=====>] 4.9			
Timeliness	[=====>] 4.5			
Fairness	[=====>] 4.7			
+-----+				

REVIEWS (23)	[Filter]
Show: [All] [As Supplier] [As Consumer]	
+-----+	
FROM: Process Inc. (Enhancer)	
Deal: Cotton Partnership \$450K Sep 2024	
Verified [Checkmark]	
"Excellent supplier. Quality consistently exceeds specifications. Reliable delivery."	
Overall: 5/5 Rel: 5 Com: 5 Qual: 5	
+-----+	
+-----+	
FROM: Sustainable Apparel (Consumer)	
Deal: Fabric Supply \$200K Jul 2024	
"Great communication, minor delay on one delivery but proactively communicated."	
Overall: 4/5 Rel: 4 Com: 5 Tim: 3	
+-----+	
[Load More Reviews]	
+-----+	

Key Elements & Positions: - **Profile Header:** Photo + name + handle + join date + badges - **Overall Score:** Large display with percentile ranking - **Role Scores:** 3-column cards showing per-role breakdown - **Dimension Chart:** Horizontal bar chart, 5 dimensions - **Reviews Section:** Filterable list, verified badge on each - **Review Cards:** Compact with reviewer info, deal context, text, dimension scores

Primary Actions: Filter reviews, Load more **Secondary Actions:** Report review (if suspicious), View deal referenced in review **Data Displayed:** Trust scores, role breakdowns, review history with context **Interaction Patterns:** Score bars animate on load, review cards expand for full text, badge hover reveals tooltip explanation

7.10 Wireframe 10: Mobile Quick Actions & Deal Creation Wizard

Screen: Mobile / **Deal Creation Wizard** **Primary Purpose:** Streamlined deal creation on mobile with step-by-step wizard **Viewport:** Mobile 375px

Layout Structure:

```

+-----+
| [< Back] Create Deal           [Save] |
+-----+
| Step 2 of 5: Select Partners          |
| [==2==> ]                            |
|                                     |
| WHO DO YOU WANT TO WORK WITH?         |
|                                     |
| CONSUMER (required for your Supplier role) |
| +-----+ |

```

[Search: Find consumers...]		
+-----+		
Suggested Consumers:		
+-----+		
[Logo]	Sustainable Apparel Co.	
Match:	92% Needs: Cotton Fabric	
Location:	45 miles Trust: 4.2	
[Select]		
+-----+		
+-----+		
[Logo]	GreenTextile Inc.	
Match:	88% Needs: Organic Cotton	
Location:	120 miles Trust: 4.5	
[Select]		
+-----+		
ENHANCER (optional)		
+-----+		
[Search or let us suggest...]		
+-----+		
Auto-suggest an enhancer? [Yes, suggest]		
SELECTED PARTIES:		
Consumer:	Sustainable Apparel Co. [x]	
Enhancer:	(auto-suggest) [Change]	
[CONTINUE (Step 3)]		
+-----+ +-----+ +-----+ +-----+		
Home	Discover + Deals Profile	
+-----+ +-----+ +-----+ +-----+		
+-----+		

Key Elements & Positions: - **Header:** Back button + title + Save (draft) action - **Step Indicator:** Progress bar with “Step X of Y” text - **Section Headers:** Role-specific partner type - **Search Input:** Full-width with icon - **Suggestion Cards:** Full-width, stacked, tap to select - **Selected Parties:** Compact chips with remove button - **Continue Button:** Full-width, fixed above bottom nav - **Bottom Nav:** Persistent, 5 tabs with centered FAB

Primary Actions: Select party, Continue to next step **Secondary Actions:** Search, Save draft, Back to previous step **Data Displayed:** Party suggestions with match scores, selected parties summary **Interaction Patterns:** Card selection shows checkmark animation, selected party chips appear above CTA, step transitions slide horizontally, keyboard pushes content up

Wizard Steps (Mobile): 1. **Select Resource** (from portfolio or create new) 2. **Select Partners** (consumer + enhancer suggestions) 3. **Define Deal Basics** (name, timeline, output) 4. **Set Initial Terms** (value split, key terms) 5. **Review & Send** (summary with send button)

Mobile-Optimized Interactions: - **Swipe:** Between wizard steps (right to go back, left to proceed) - **Haptic feedback:** On selection, step completion - **Smart defaults:** Pre-filled based

on profile - **Inline validation:** Immediate feedback on input - **Step summary:** Each step shows a “what you’ve selected so far” recap

Appendix A: Design System Notes

Color Palette

Role/Element	Primary	Light	Dark
Supplier	#2ECC71 (Green)	#D5F5E3	#27AE60
Consumer	#3498DB (Blue)	#D6EAF8	#2980B9
Enhancer	#9B59B6 (Purple)	#E8DAEF	#8E44AD
Neutral/Action	#2C3E50	#ECF0F1	#1A252F
Success	#27AE60	#D5F5E3	#1E8449
Warning	#F39C12	#FDEbd0	#D68910
Danger	#E74C3C	#FADBD8	#C0392B
Background	#F8F9FA	#FFFFFF	#E9ECEF

Typography Scale

Element	Size	Weight	Line Height
H1 (Page Title)	32px	600	40px
H2 (Section)	24px	600	32px
H3 (Card Title)	18px	600	24px
Body	16px	400	24px
Body Small	14px	400	20px
Caption	12px	400	16px
Button	14px	600	20px

Spacing Scale

Token	Value	Usage
xs	4px	Icon gaps
sm	8px	Tight spacing
md	16px	Default padding
lg	24px	Section gaps
xl	32px	Major sections
2xl	48px	Page-level spacing

Elevation (Shadows)

Level	Shadow	Usage
0	none	Flat elements
1	0 1px 3px rgba(0,0,0,0.08)	Cards at rest
2	0 4px 12px rgba(0,0,0,0.1)	Hover states

Level	Shadow	Usage
3	0 8px 24px rgba(0,0,0,0.12)	Modals, dropdowns
4	0 16px 48px rgba(0,0,0,0.15)	Full-screen overlays

Appendix B: Accessibility Considerations

WCAG 2.1 AA Compliance

- All interactive elements have minimum 44px tap target (48px on mobile)
- Color contrast ratios: 4.5:1 for body text, 3:1 for large text
- Focus indicators visible on all interactive elements
- Screen reader labels for all icons and visual elements
- Keyboard navigation support throughout
- Reduced motion preferences respected for animations
- Form validation errors announced to screen readers
- Notification alerts use ARIA live regions

Inclusive Design

- Multi-language support from day one
- Right-to-left (RTL) layout support
- Timezone-aware date/time display
- Currency formatting per locale
- Clear error messages in plain language
- Progressive disclosure for complex features
- Contextual help available on every screen

Appendix C: Edge Cases & Error States

Empty States

State	Message	CTA
No resources posted	“Start by listing your first resource”	“Post Resource”
No matches found	“We’ll notify you when matches appear”	“Refine Search”
No active deals	“Your deal pipeline is empty”	“Create First Deal”
No notifications	“You’re all caught up!”	“Explore Deals”
No reviews	“Complete your first deal to earn reviews”	“View Active Deals”

Error States

Error	Handling
Invitation expired	Show expired state with “Re-invite” or “Find alternative” options
Negotiation timeout	Grace period warning, then auto-archive with feedback collection
Payment failed	Retry flow with alternative payment method, notify

Error	Handling
	all parties
Document rejected	Reason display + re-upload flow
Party withdraws mid-deal	Automatic pause, suggest replacement, or dissolution flow
Group member leaves	Update governance threshold, notify remaining members

Loading States

- Skeleton screens for all content areas
- Inline spinners for buttons during async actions
- Progress indicators for multi-step operations
- Optimistic UI for common actions (vote, save, mark read)
- “Still loading?” timeout with refresh option after 10 seconds

Document End — TriadDeal Platform UX Design Specification v1.0

Three-Party Deal Platform: Economic Engine Specification

8. Win-Win-Win Validation Framework

8.1 Value Definition & Measurement

“Value” in the three-party platform is multi-dimensional and must be normalized to a common unit for validation purposes. The platform supports five value types:

Value Type	Notation	Description	Example
Monetary	V_m	Direct currency payment	\$10,000 cash
Barter	V_b	Non-monetary exchange	500 kg of crop in exchange
Revenue Share	V_r	Percentage of future revenue	20% of sales proceeds
Future Obligation	V_f	Deferred payment/commitment	Pay \$5,000 in 6 months
Equity	V_e	Ownership stake	15% of resulting business

Universal Value Normalization: All value types are converted to a **Present Value Equivalent (PVE)** in the platform’s base currency using domain-specific discount rates.

$$PVE = V_m + PV(V_b) + PV(V_r) + PV(V_f) + PV(V_e)$$

Where:

$$PV(V_b) = \text{MarketPrice}(\text{BarterItem}) \times \text{Quantity}$$

$$PV(V_r) = \sum [\text{ExpectedRevenue}_t \times \text{Share\%} / (1 + d)^t] \text{ for } t=1 \text{ to } n$$

$$PV(V_f) = \text{FutureAmount} / (1 + d)^t$$

$$PV(V_e) = \text{EnterpriseValue} \times \text{Equity\%}$$

d = domain_discount_rate (typically 8-15%)
n = projection_period_years (typically 3-5)

8.2 Fairness Algorithm

The fairness algorithm validates that each party receives equitable value relative to their contribution, market alternatives, and opportunity costs.

8.2.1 Total Deal Value Composition

$TDV = \text{Consumer_PVE} + \text{Supplier_PVE} + \text{Enhancer_PVE} + \text{Platform_Fee}$

Where each party's PVE represents the net value they receive from the deal.

8.2.2 Fairness Dimensions

The algorithm evaluates fairness across four dimensions:

A. Absolute Gain Check (Party must benefit)

For each party P in {Supplier, Consumer, Enhancer}:

$\text{Gain_P} = \text{Value_Received_P} - \text{Value_Contributed_P}$

Win Condition: $\text{Gain_P} > 0$ for ALL parties

Violation: If $\text{Gain_P} \leq 0 \rightarrow$ Deal BLOCKED with reason:

"[Party] would receive no net benefit from this deal.

Consider: [specific suggestions]"

B. Proportional Fairness Check (Anti-exploitation)

$\text{Share_P} = \text{Value_Received_P} / TDV$

Constraint: $\text{Share_P} \leq \text{MAX_SHARE}$ for all parties

$\text{MAX_SHARE} = 0.70$ (70% ceiling)

Constraint: $\text{Share_P} \geq \text{MIN_SHARE}$ for all parties

$\text{MIN_SHARE} = 0.10$ (10% floor)

If $\text{Share_P} > \text{MAX_SHARE}$:

$\rightarrow$ Deal FLAGGED: "[Party] receives >70% of value.

Platform recommends: [rebalancing suggestions]"

If $\text{Share_P} < \text{MIN_SHARE}$:

$\rightarrow$ Deal FLAGGED: "[Party] receives <10% of value.

Risk of deal abandonment. Consider: [suggestions]"

C. Market Rate Benchmarking

For each party P:

$\text{Market_Rate_P} = \text{DomainBenchmark}(\text{P.role}, \text{P.location}, \text{P.resource_type})$

$\text{Premium_P} = (\text{Value_Received_P} - \text{Market_Rate_P}) / \text{Market_Rate_P}$

Benchmarking Rules:

- Supplier: Compare resource rental/usage rates to market
- Enhancer: Compare service fees to industry standards

- Consumer: Compare total cost to independent sourcing

Alert Triggers:

- $\text{Premium}_P > +0.50$ (50% above market) → "Above market rate - good deal"
- $\text{Premium}_P < -0.30$ (30% below market) → "Below market rate - consider renegotiation"
- $\text{Consumer_Cost} > \text{Independent_Sourcing_Cost} \times 0.95$ →
"Consumer could source independently for similar cost.
Value proposition unclear."

D. Opportunity Cost Check

For each party P:

$\text{OC}_P = \text{Best_Alternative}_P$ (next best option outside this deal)

$\text{Net_Benefit}_P = \text{Gain}_P - \text{OC}_P$

Win-Win-Win Condition: $\text{Net_Benefit}_P > 0$ for ALL parties

This ensures each party is better off than their next best alternative.

8.2.3 Composite Fairness Score

$\text{Fairness_Score} = w1 \times \text{Absolute_Gain_Score} + w2 \times \text{Proportional_Score} + w3 \times \text{Benchmark_Score} + w4 \times \text{Opportunity_Cost_Score}$

Where:

$w1 = 0.25$ (absolute gain weight)

$w2 = 0.30$ (proportional fairness weight)

$w3 = 0.25$ (market benchmark weight)

$w4 = 0.20$ (opportunity cost weight)

Fairness_Tiers:

Score ≥ 0.90 : "Excellent" (green) - All parties strongly benefit

Score ≥ 0.70 : "Good" (light green) - All parties benefit adequately

Score ≥ 0.50 : "Fair" (yellow) - Marginal benefit for some parties

Score < 0.50 : "Poor" (red) - Deal requires renegotiation

8.3 Validation Rules Engine

RULE SET: Win-Win-Win Validation

RULE 1: Positive Gain [CRITICAL]

FORALL parties P: $\text{Gain}_P > 0$

VIOLATION_ACTION: BLOCK deal

MESSAGE: "Every party must receive positive net value"

RULE 2: Maximum Share [CRITICAL]

FORALL parties P: $\text{Share}_P \leq 0.70$

VIOLATION_ACTION: BLOCK deal

MESSAGE: "No party may receive more than 70% of total deal value"

RULE 3: Minimum Share [WARNING]

FORALL parties P: $\text{Share}_P \geq 0.10$

VIOLATION_ACTION: FLAG + require acknowledgment

MESSAGE: "Party receiving $<10\%$ may abandon deal"

RULE 4: Enhancer Proportionality [CRITICAL]

Enhancer_Value / Enhancer_Cost_Added $\leq$ 1.5

AND Enhancer_Value / Enhancer_Cost_Added $\geq$ 0.5

VIOLATION_ACTION: FLAG + suggest adjustment

MESSAGE: "Enhancer fee should be 50-150% of their cost/value added"

RULE 5: Consumer Value Proposition [CRITICAL]

Consumer_Cost < Independent_Sourcing_Cost $\times$ 0.95

OR Consumer_Cost $\leq$ Independent_Sourcing_Cost $\times$ 1.05

AND Consumer_Additional_Value > 0

(quality, convenience, reliability premium)

VIOLATION_ACTION: FLAG

MESSAGE: "Consumer must see clear savings or added value vs alternatives"

RULE 6: Deal Size Minimum [CRITICAL]

TDV $\geq$ MIN_DEAL_SIZE (domain-specific, e.g., \$500)

VIOLATION_ACTION: BLOCK

MESSAGE: "Deal value below minimum viable threshold"

RULE 7: Risk Balance [WARNING]

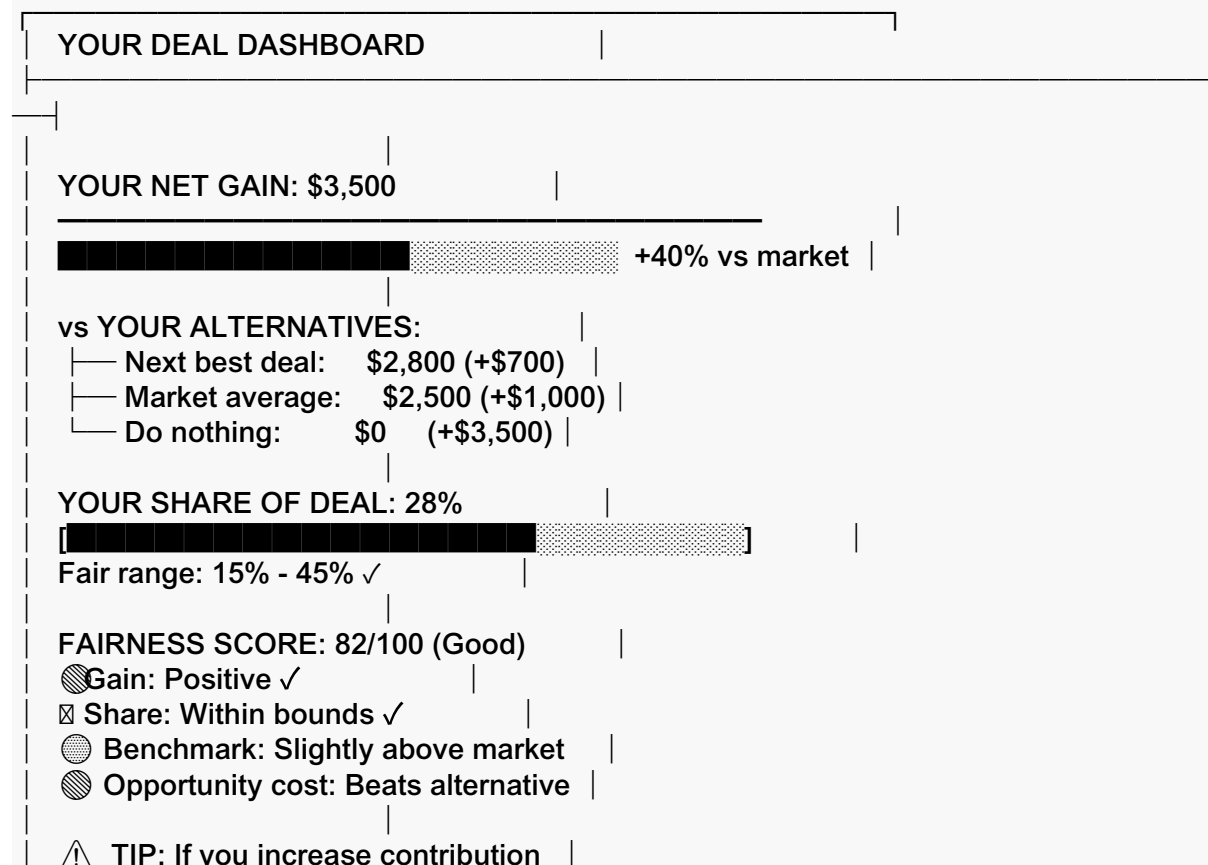
Risk_Party_Max / Risk_Party_Min $\leq$ 3.0

VIOLATION_ACTION: FLAG + suggest risk mitigation

MESSAGE: "Risk distribution too unbalanced"

8.4 Real-Time Negotiation Feedback

During deal negotiation, each party sees a **Personal Deal Dashboard**:



by 10%, you could negotiate 15% more value.

8.5 Validation Decision Tree

graph TD

```
A[Deal Proposal Submitted] --> B{Rule 1:<br/>All Gains > 0?}
B -->|No| C[BLOCK: Critical<br/>Party would lose]
B -->|Yes| D{Rule 2:<br/>All Shares ≤ 70%?}
D -->|No| E[BLOCK: Critical<br/>Exploitation risk]
D -->|Yes| F{Rule 5:<br/>Consumer sees value?}
F -->|No| G[FLAG: Consumer value<br/>proposition weak]
F -->|Yes| H{Rule 4:<br/>Enhancer fee fair?}
H -->|No| I[FLAG: Adjust enhancer<br/>compensation]
H -->|Yes| J{Rule 3:<br/>All Shares ≥ 10%?}
J -->|No| K[FLAG: Low share risk]<br/>L --> M[Proceed to<br/>Fairness Scoring]
K --> L[Require Acknowledgment]
G --> N[Suggest Adjustments]
I --> N
L --> M
N --> O[Party revises offer]
O --> A

M --> P{Fairness Score?}
P -->|≥ 0.90| Q[Status: EXCELLENT]
P -->|≥ 0.70| R[Status: GOOD]
P -->|≥ 0.50| S[Status: FAIR<br/>Recommend improvement]
P -->|< 0.50| T[Status: POOR<br/>Requires renegotiation]

Q --> U[Available for Signing]
R --> U
S --> V[Available with Warning]
T --> W[Not Available - Revise]
W --> O
```

8.6 Pseudocode: Win-Win-Win Validation Engine

class WinWinWinValidator:

```
def __init__(self, domain_config):
    self.max_share = 0.70
    self.min_share = 0.10
    self.min_deal_size = domain_config.min_deal_size
    self.discount_rate = domain_config.discount_rate

def validate_deal(self, deal) -> ValidationResult:
    """
    Main validation entry point.
    Returns: ValidationResult with status, score, and detailed feedback.
    """

    # Step 1: Normalize all values to PVE
    pve_result = self._normalize_to_pve(deal)
```

```

# Step 2: Check critical rules (hard blocks)
critical_checks = self._check_critical_rules(pve_result)
if not critical_checks.all_passed:
    return ValidationResult(
        status=DealStatus.BLOCKED,
        fairness_score=0.0,
        violations=critical_checks.violations
    )

# Step 3: Check warning rules (flags)
warning_checks = self._check_warning_rules(pve_result)

# Step 4: Calculate fairness score
fairness_score = self._calculate_fairness_score(pve_result)

# Step 5: Generate party-specific feedback
feedback = self._generate_feedback(pve_result, fairness_score)

# Step 6: Determine final status
status = self._determine_status(fairness_score, warning_checks)

return ValidationResult(
    status=status,
    fairness_score=fairness_score,
    pve_breakdown=pve_result,
    violations=critical_checks.violations,
    warnings=warning_checks.warnings,
    party_feedback=feedback
)

def _normalize_to_pve(self, deal) -> PVEResult:
    """Convert all value types to Present Value Equivalent."""
    consumer_pve = self._calculate_party_pve(deal.consumer_value)
    supplier_pve = self._calculate_party_pve(deal.supplier_value)
    enhancer_pve = self._calculate_party_pve(deal.enhancer_value)
    platform_fee = self._calculate_platform_fee(deal)

    tdv = consumer_pve + supplier_pve + enhancer_pve + platform_fee

    return PVEResult(
        consumer_pve=consumer_pve,
        supplier_pve=supplier_pve,
        enhancer_pve=enhancer_pve,
        platform_fee=platform_fee,
        total_deal_value=tdv
    )

def _check_critical_rules(self, pve) -> CriticalCheckResult:
    violations = []

# Rule 1: Positive gain for all

```

```

for party in ['consumer', 'supplier', 'enhancer']:
    gain = pve.get_gain(party)
    if gain <= 0:
        violations.append(Violation(
            rule="R1_POSITIVE_GAIN",
            party=party,
            message=f"{party} has non-positive gain: {gain}"
        ))

# Rule 2: Max share ≤ 70%
for party in ['consumer', 'supplier', 'enhancer']:
    share = pve.get_share(party)
    if share > self.max_share:
        violations.append(Violation(
            rule="R2_MAX_SHARE",
            party=party,
            message=f"{party} share {share:.1%} exceeds 70% limit"
        ))

# Rule 5: Consumer value proposition
consumer_cost = pve.consumer_contribution
alt_cost = pve.consumer_alternative_cost
if consumer_cost >= alt_cost * 0.95:
    violations.append(Violation(
        rule="R5_CONSUMER_VALUE",
        party="consumer",
        message="Consumer cost 接近 or exceeds independent sourcing"
    ))

# Rule 6: Minimum deal size
if pve.total_deal_value < self.min_deal_size:
    violations.append(Violation(
        rule="R6_MIN_DEAL_SIZE",
        party="all",
        message=f"Deal value ${pve.total_deal_value} below minimum"
    ))

return CriticalCheckResult(
    all_passed=len(violations) == 0,
    violations=violations
)

def _calculate_fairness_score(self, pve) -> float:
    """Calculate composite fairness score."""
    # Dimension 1: Absolute gain (0-1)
    gains = [pve.get_gain(p) for p in ['consumer', 'supplier', 'enhancer']]
    min_gain = min(gains)
    max_gain = max(gains)

    if max_gain <= 0:
        absolute_score = 0.0
    else:

```

```

    # Normalized: all positive = 1.0, one at zero = 0.0
    absolute_score = max(0.0, min(gains)) / (max_gain * 0.5)
    absolute_score = min(1.0, absolute_score)

    # Dimension 2: Proportional fairness (Gini-inspired)
    shares = [pve.get_share(p) for p in ['consumer', 'supplier', 'enhancer']]
    mean_share = sum(shares) / 3
    variance = sum((s - mean_share)**2 for s in shares) / 3
    proportional_score = max(0.0, 1.0 - variance * 10)

    # Dimension 3: Market benchmark
    benchmark_scores = []
    for party in ['consumer', 'supplier', 'enhancer']:
        premium = pve.get_benchmark_premium(party)
        # Score = 1.0 at 0% premium, decreases as deviates
        benchmark_scores.append(max(0.0, 1.0 - abs(premium) * 2))
    benchmark_score = sum(benchmark_scores) / 3

    # Dimension 4: Opportunity cost
    oc_scores = []
    for party in ['consumer', 'supplier', 'enhancer']:
        net_benefit = pve.get_net_benefit(party)
        oc_scores.append(1.0 if net_benefit > 0 else max(0.0, 0.5 + net_benefit /
pve.get_oc(party)))
    oc_score = sum(oc_scores) / 3

    # Weighted composite
    return (0.25 * absolute_score +
        0.30 * proportional_score +
        0.25 * benchmark_score +
        0.20 * oc_score)

def generate_party_dashboard(self, party_id, deal) -> Dashboard:
    """Generate real-time negotiation dashboard for a party."""
    pve = self._normalize_to_pve(deal)
    fairness_score = self._calculate_fairness_score(pve)

    gain = pve.get_gain(party_id)
    share = pve.get_share(party_id)
    market_premium = pve.get_benchmark_premium(party_id)
    next_best = pve.get_next_best_alternative(party_id)

    return Dashboard(
        net_gain=gain,
        share_percent=share * 100,
        vs_market=market_premium,
        vs_next_best=gain - next_best,
        vs_do_nothing=gain,
        fairness_score=fairness_score,
        checks={
            'gain_positive': gain > 0,
            'share_in_bounds': self.min_share <= share <= self.max_share,

```



```

    'beats_market': market_premium > 0,
    'beats_alternative': gain > next_best
},
tips=self._generate_tips(party_id, deal, pve)
)

```

8. Value Distribution Models

8.1 Model Comparison Overview

Dimension	Fixed Price	Revenue Share	Cost-Plus	Barter/Exchange	Hybrid
Payment Timing	Upfront	Deferred	Milestone	Immediate	Mixed
Risk Allocation	Consumer high	Shared	Balanced	Low	Flexible
Complexity	Low	Medium	Medium	High	High
Trust Required	Medium	High	Medium	High	Medium-High
Platform Fee	% of total	% of revenue	% of cost	Valuation svc	% of total
Best For	Standard goods	Joint ventures	Custom projects	Complementary resources	Complex deals
Cash Requirement	High (consumer)	Low upfront	Phased	None	Flexible
Price Certainty	High	Low	Medium	Medium	Medium

8.2 Model A: Fixed Price Model

Description: Consumer pays a predetermined fixed amount. Proceeds are split between Supplier and Enhancer.

graph LR

```

C[Consumer<br/>Pays $10,000] --> PF[Platform Fee<br/>10% = $1,000]
PF --> SP[Supplier Pool<br/>$9,000]
SP --> S[Supplier<br/>60% = $5,400]
SP --> E[Enhancer<br/>30% = $2,700]
SP --> P[Platform<br/>Retention<br/>10% = $900]

```

```

style C fill:#e1f5fe
style S fill:#e8f5e9
style E fill:#fff3e0
style P fill:#fce4ec
style PF fill:#f3e5f5
style SP fill:#f3e5f5

```

Value Flow:

Consumer: Pays \$10,000 → Receives output (crops worth \$12,000 market)
Net Gain = \$2,000 (20% savings vs market)

Supplier: Provides land (opportunity cost: \$4,000)
Receives: \$5,400
Net Gain = \$1,400

Enhancer: Provides seeds, fertilizer, labor (cost: \$2,000)
Receives: \$2,700
Net Gain = \$700

Platform: Facilitates match + escrow
Receives: \$1,900 total (\$1,000 from consumer + \$900 from pool)

TDV = \$2,000 + \$1,400 + \$700 = \$4,100 (total value created)

Platform Fee Structure:

Base Fee: 8% of deal value (minimum \$100)
Escrow Fee: 2% (waived for Platinum members)
Payment Processing: 1.5% (passed through)

Total Platform Revenue = \$1,000 (10% effective rate)

Risk Allocation: | Risk | Allocation | Mitigation | |——|———|———| | Output quality |
Supplier/Enhancer | Escrow holdback (20% until verified) | | Non-payment | Consumer | Full
escrow upfront | | Non-delivery | Supplier/Enhancer | Reputation penalty + collateral | |
Market price change | Consumer | Fixed price = locked in |

When to Use: Standardized products, clear specifications, predictable costs, parties with moderate trust.

8.3 Model B: Revenue Share Model

Description: Output is sold jointly and proceeds are distributed according to agreed percentages. Creates ongoing partnerships.

graph TD

```
subgraph "Revenue Generation"
  O[Output Produced] --> SALES[Sales Channel]
  SALES --> REV[Total Revenue<br/>$15,000]
end
```

```
subgraph "Distribution Waterfall"
  REV --> PF[Platform Fee<br/>8% = $1,200]
  PF --> ESC[Escrow Reserve<br/>5% = $750]
  ESC --> DIST[Distributable<br/>$13,050]
end
```

```
subgraph "Party Shares"
  DIST --> S[Supplier<br/>50% = $6,525]
  DIST --> E[Enhancer<br/>30% = $3,915]
  DIST --> C[Consumer<br/>20% = $2,610]
end
```

end

subgraph "Reserve Release"

ESC -. Final accounting .-> R[Released proportionally
after 90 days]

end

style REV fill:#e8f5e9

style S fill:#e8f5e9

style E fill:#fff3e0

style C fill:#e1f5fe

Revenue Share Formula:

$\text{Distributable_Revenue} = \text{Gross_Revenue} \times (1 - \text{Platform_Fee\%} - \text{Reserve\%})$

$\text{Party_Payout_P} = \text{Distributable_Revenue} \times \text{Share_P}$

Standard Share Templates:

"Equal Partners": 33.3% / 33.3% / 33.4%

"Supplier Heavy": 50% / 30% / 20% (Supplier/Enhancer/Consumer)

"Enhancer Heavy": 30% / 50% / 20%

"Consumer Heavy": 30% / 30% / 40%

"Custom": Negotiated split

Dynamic Share Adjustment (for multi-period deals):

$\text{Share_P}(t) = \text{Base_Share_P} + \text{Performance_Adjustment_P}(t)$

Where **Performance_Adjustment** is based on:

- Supplier: Output quality, timeliness, yield
- Enhancer: Input quality, support, innovation
- Consumer: Sales performance, marketing effort

Adjustment bounded: $\pm 10\%$ of base share per period

Platform Fee Structure:

Revenue Share Fee: 8% of gross revenue (collected at sale)

Management Fee: \$50/month for ongoing deals

Accounting/Reporting: Included

$\text{Platform Revenue} = \$1,200 \text{ (one-time)} + \$50/\text{month} \times \text{duration}$

Risk Allocation: | Risk | Allocation | Mitigation | |——|———|———| | Sales risk |
Shared (all parties lose if no sales) | Consumer incentivized to sell | | Production risk |
Supplier/Enhancer | Lower share if targets missed | | Price volatility | All parties | Reserve
fund buffer | | Deadlock | N/A | Pre-defined exit clauses |

When to Use: Joint ventures, ongoing relationships, uncertain market prices, aligned long-term incentives.

8.4 Model C: Cost-Plus Model

Description: Transparent cost build-up with agreed margin. Consumer sees full cost breakdown.

```
graph TD
    subgraph "Cost Stack"
        SC["Supplier Cost<br/>Resource Value<br/>$4,000"] --> TC["Total Cost Base"]
        EC["Enhancer Cost<br/>Inputs + Labor<br/>$3,500"] --> TC
        PC["Platform Cost<br/>Facilitation<br/>$500"] --> TC
        TC --> TC_TOT["Total Cost<br/>$8,000"]
    end

    subgraph "Margin Application"
        TC_TOT --> SM["Supplier Margin<br/>15% = $600"]
        TC_TOT --> EM["Enhancer Margin<br/>20% = $700"]
        TC_TOT --> PM["Platform Margin<br/>10% = $800"]
    end

    subgraph "Consumer Price"
        SM --> FP["Final Price<br/>$10,100"]
        EM --> FP
        PM --> FP
        TC_TOT --> FP
    end

    style TC_TOT fill:#fff9c4
    style FP fill:#e8f5e9
```

Cost-Plus Formula:

$$\text{Final_Price} = (\text{Supplier_Cost} + \text{Enhancer_Cost} + \text{Platform_Cost}) \times (1 + \text{Total_Margin}\%)$$

Where:

$$\text{Total_Margin}\% = \frac{(\text{Supplier_Margin} + \text{Enhancer_Margin} + \text{Platform_Margin})}{\text{Total_Cost}} \times 100$$

$$\text{Party_Payout_P} = \text{Cost_P} \times (1 + \text{Margin_P}\%)$$

Platform Fee is included in the cost stack AND takes margin.

Transparent Cost Breakdown (visible to Consumer):

COST BREAKDOWN	
SUPPLIER COSTS	
— Land opportunity cost	\$3,000
— Infrastructure use	\$500
— Labor (supervision)	\$500
— Subtotal	\$4,000
+ Margin (15%)	\$600

ENHANCER COSTS		
— Seeds & fertilizer	\$1,500	
— Equipment rental	\$800	
— Technical labor	\$1,000	
— Transport	\$200	
— Subtotal	\$3,500	
+ Margin (20%)	\$700	
PLATFORM COSTS		
— Matching & escrow	\$400	
— Dispute insurance	\$100	
— Subtotal	\$500	
+ Margin (10%)	\$500	
TOTAL PRICE	\$10,100	
vs Market Price:	\$12,000	
YOUR SAVINGS:	\$1,900	
(15.8% below market)		

Platform Fee Structure:

Included in cost stack: \$500 facilitation cost
Plus margin share: \$500 (10% on \$500 cost, or ~5% of total)
Total Platform Revenue: ~\$1,000 (9.9% effective)

Risk Allocation: | Risk | Allocation | Mitigation | | | | | Cost overrun |
Supplier/Enhancer | Pre-approved cost ceiling | | Scope creep | Consumer | Change order
process | | Inflation | All | Cost adjustment clause | | Audit | Platform | Random cost
verification |

When to Use: Custom projects, cost-sensitive consumers, high-trust relationships,
government/corporate procurement.

8.5 Model D: Barter/Exchange Model

Description: Non-monetary exchange of resources, services, and outputs. Platform provides valuation guidance.

graph LR

subgraph "What Each Party Gives"

S[Supplier] --> SG[Idle Land
6 months]

E[Enhancer] --> EG[Seeds + Expertise
Value: \$3,000]

C[Consumer] --> CG[Processing Equipment
Access + Storage]

end

subgraph "What Each Party Receives"

S --> SR[40% of Crop Output
+ Processing Service]

E --> ER[35% of Crop Output
+ Land Access for R&D]

C --> CR[25% of Crop Output
+ Premium Quality Assurance]

```

end

subgraph "Platform Role"
  P[Platform] --> PV[Valuation Service]
  P --> PE[Escrow of Non-Monetary Assets]
  P --> PD[Dispute Resolution]
end

style S fill:#e8f5e9
style E fill:#fff3e0
style C fill:#e1f5fe

```

Barter Valuation Framework:

For each barter item I from party P:

Fair_Market_Value_I = Platform_Appraised_Value(I)

Validation:

|Party_P contributions| ≈ |Party_P receipts| ± 20%

If imbalance > 20%:

→ Suggest cash adjustment or additional barter items

Monetary_Equivalent = $\sum$ Fair_Market_Value_I for all items

Platform Fee Structure:

Valuation Service: \$100 per item appraised

Escrow (non-monetary): 5% of appraised value (minimum \$200)

Matching Fee: \$250 flat

Total Platform Revenue: ~\$550-800 per deal

When to Use: Cash-constrained parties, complementary resources, mutual benefit scenarios, developing markets.

8.6 Model E: Hybrid Model

Description: Combines multiple payment structures for complex deals. Most flexible model.

```

graph TD
  subgraph "Hybrid Payment Structure"
    subgraph "Phase 1: Upfront"
      C1[Consumer] --> UP[Upfront Payment<br/>$5,000]
      UP --> S1[Supplier<br/>$3,000]
      UP --> E1[Enhancer<br/>$1,500]
      UP --> P1[Platform<br/>$500]
    end

    subgraph "Phase 2: Milestone"
      C2[Consumer] --> M1[Milestone 1<br/>Planting Complete<br/>$2,000]
      C2 --> M2[Milestone 2<br/>Harvest Complete<br/>$3,000]
      M1 --> SPLIT1[Split 50/30/20<br/>S/E/Platform]
    end
  end

```

```

    M2 --> SPLIT2[Split 50/30/20<br/>S/E/Platform]
end

subgraph "Phase 3: Revenue Share"
    SALES[Sales] --> REV[Revenue<br/>$15,000]
    REV --> PF[Platform 8%]
    PF --> RS[Revenue Split<br/>40% S / 35% E / 25% C]
end
end

style C1 fill:#e1f5fe
style C2 fill:#e1f5fe
style S1 fill:#e8f5e9
style E1 fill:#fff3e0
style RS fill:#f3e5f5

```

Hybrid Structure Formula:

$\text{Total_Deal} = \text{Upfront_Component} + \text{Milestone_Component} + \text{RevenueShare_Component}$

Where:

$\text{Upfront_Component} = \text{Fixed amount for immediate costs}$

$\text{Milestone_Component} = \sum (\text{Milestone_Payment}_i \times \text{Achievement}_i)$

$\text{RevenueShare_Component} = (\text{Gross_Revenue} - \text{Threshold}) \times \text{Share\%}$

Each component can use different distribution models.

Platform fee varies by component:

- Upfront: 10%
- Milestone: 8%
- Revenue share: 6%

Milestone Definition:

```

Milestone = {
  id: string,
  description: string,
  trigger_condition: "objective_verifiable_event",
  verification_method: "oracle" | "party_consensus" | "third_party",
  payment_amount: number,
  payment_split: {supplier: %, enhancer: %, platform: %},
  deadline: date,
  penalty_for_miss: number # % reduction for late completion
}

```

When to Use: Complex multi-phase projects, high-value deals, parties with different risk profiles, innovation partnerships.

8.7 Model Selection Decision Tree

graph TD

```

A[Deal Negotiation Begins] --> Q1{Is cash<br/>available?}
Q1 -->|No| B[Barter/Exchange Model]
Q1 -->|Yes| Q2{Ongoing relationship<br/>or one-time?}

```

Q2 -->|One-time| Q3{Need cost
transparency?}

Q2 -->|Ongoing| Q4{Revenue easily
measurable?}

Q3 -->|Yes| C[Cost-Plus Model]

Q3 -->|No| D[Fixed Price Model]

Q4 -->|Yes| E[Revenue Share Model]

Q4 -->|No| Q5{Multiple phases
or milestones?}

Q5 -->|Yes| F[Hybrid Model]

Q5 -->|No| G[Default to Fixed Price
with performance bonus]

B --> H[Platform assists
with valuation]

C --> I[Consumer approves
cost breakdown]

D --> J[Standard terms
negotiation]

E --> K[Define revenue
measurement]

F --> L[Define milestones
and triggers]

G --> J

8. Pricing & Fee Mechanism

8.1 Platform Fee Architecture

The platform uses a **tiered, dynamic fee structure** that balances accessibility with sustainability.

PLATFORM FEE STRUCTURE	
BASE FEE (applied to all deals)	
— Matching Fee: 2% of deal value	
— Escrow Fee: 1.5% of amount held	
— Payment Processing: 1% (pass-through to processor)	
Base Total: 4.5%	
SUCCESS FEE (only on completed deals)	
— Standard: 4% of deal value	
— Complex deals: 6%	
— Repeat deals: 2.5%	
EFFECTIVE RATES	
— First-time simple deal: 8.5%	
— First-time complex deal: 10.5%	
— Repeat simple deal: 7%	
MINIMUMS	
— Absolute minimum: \$50	
— Minimum by domain: varies (\$100-\$500)	

8.2 Dynamic Pricing Formula

$$\text{Platform_Fee\%} = \text{Base_Fee\%} \times \text{Trust_Multiplier} \times \text{Complexity_Multiplier} \times \text{Volume_Discount} \times \text{Domain_Adjustment}$$

Where:

Base_Fee% = 8.5% (standard rate)

Trust_Multiplier = f(min_trust_score_among_parties)

Trust >= 90 (Platinum): 0.80 (20% discount)

Trust >= 75 (Gold): 0.90 (10% discount)

Trust >= 60 (Silver): 1.00 (standard)

Trust >= 40 (Bronze): 1.15 (15% premium)

Trust < 40: 1.35 (35% premium + escrow required)

Complexity_Multiplier = f(deal_complexity_score)

Simple (fixed price, standard terms): 0.90

Medium (milestones, custom terms): 1.00

Complex (hybrid, multi-party, long-term): 1.25

Volume_Discount = f(monthly_deal_volume_for_party)

1-2 deals/month: 1.00

3-5 deals/month: 0.95 (5% discount)

6-10 deals/month: 0.90 (10% discount)

11+ deals/month: 0.85 (15% discount)

Domain_Adjustment = domain_specific_rate

Agriculture: 1.00

Manufacturing: 1.10

Technology: 0.95

Services: 1.05

Real Estate: 1.20

Example Calculation:

Deal: Agricultural produce, \$10,000

Parties: Supplier (Trust 82/Gold), Consumer (Trust 65/Silver), Enhancer (Trust 70/Silver)

Complexity: Medium (milestone-based)

Consumer volume: 4 deals/month

Trust_Multiplier = min(0.90, 1.00, 1.00) = 0.90

Complexity_Multiplier = 1.00

Volume_Discount = 0.95 (Consumer's volume)

Domain_Adjustment = 1.00

Platform_Fee% = 8.5% × 0.90 × 1.00 × 0.95 × 1.00 = 7.27%

Platform_Fee = \$10,000 × 7.27% = \$727

Minimum check: \$727 > \$100 minimum → Valid

8.3 Fee Allocation

graph LR

subgraph "Fee Payment Options"

A[Option 1:
Consumer Pays All]

B[Option 2:
Split Proportionally]

C[Option 3:
Initiator Pays]

D[Option 4:
Absorbed in Distribution]

end

subgraph "Default by Model"

A --> |Fixed Price| F1[Default: Option 1
Consumer pays all fees]

B --> |Revenue Share| F2[Default: Option 2
Deducted from revenue]

D --> |Cost-Plus| F3[Default: Option 4
Included in cost stack]

A --> |Barter| F4[Default: Option 1
Initiator pays cash fee]

B --> |Hybrid| F5[Negotiable per component]

end

style A fill:#e1f5fe

style B fill:#e8f5e9

style C fill:#fff3e0

style D fill:#fce4ec

Fee Transparency During Negotiation:

PLATFORM FEE BREAKDOWN	
Deal Value:	\$10,000
Fee Components:	
— Matching service	\$200 (2%)
— Escrow & security	\$150 (1.5%)
— Payment processing	\$100 (1%)
— Success fee	\$400 (4%)
— Trust discount	-\$123
YOUR EFFECTIVE FEE:	\$727 (7.27%)
vs Standard Rate:	\$850 (8.5%)
YOUR SAVINGS:	\$123
Paid by: Consumer (split available)	
💡 Increase your trust score to 90+ to save an additional \$60 on future deals.	

8.4 Premium Features & Pricing

Feature	Description	Price	Target Segment
Priority Matching	Your deals shown first to matching parties	\$49/month or \$15/deal	High-volume users
Trust Badge Plus	Enhanced verification badge, visible to all	\$99 one-time + \$29/month	All users
Deal Analytics	Market benchmarks, pricing suggestions, trends	\$79/month	Suppliers, Enhancers
Dispute Insurance	Platform covers up to \$5,000 in disputes	1.5% of deal value	Risk-averse users
Contract Templates	Pre-vetted legal templates for your domain	\$29/template	All users
Dedicated Support	Priority response, phone support	\$199/month	Enterprise
API Access	Integrate platform into your workflow	\$499/month	Enterprise
White-Label	Run your own branded marketplace	Custom pricing	Large organizations
Escrow Fast-Release	24-hour escrow release (vs 3-day standard)	\$25/deal	Time-sensitive
Multi-Currency	Deals in 50+ currencies with hedging	0.5% FX spread	International

8. Trust & Reputation System

8.1 Trust Score Computation

8.1.1 Score Components & Weights

Trust_Score = $\sum (\text{Component}_i \times \text{Weight}_i)$ for $i = 1$ to 8

Score range: 0-100

Tiers: Bronze (0-39), Silver (40-59), Gold (60-74), Platinum (75-100)

#	Component	Symbol	Weight	Data Source	Update Frequency
1	Transaction History	W_tx	0.25	Completed deals	Per transaction
2	Review Ratings	W_rev	0.20	Party reviews	Per review
3	Profile Completeness	W_prof	0.10	Profile data	On change
4	Verification Level	W_ver	0.15	Verification docs	On verification
5	Response	W_resp	0.10	Message/deal	Daily

#	Component	Symbol	Weight	Data Source	Update Frequency
	Rate			responses	
6	Dispute History	W_disp	0.10	Dispute outcomes	Per dispute
7	Platform Longevity	W_age	0.05	Account age	Daily
8	Community Contribution	W_comm	0.05	Referrals, help	Per action

8.1.2 Component Formulas

1. Transaction History (W_tx = 0.25)

$TX_Score = \min(100, Base_TX + Volume_Bonus + Value_Bonus + Consistency_Bonus)$

$Base_TX = \min(50, Completed_Deals \times 10)$ # 5 deals = 50 points

$Volume_Bonus = \min(20, (Completed_Deals - 5) \times 2)$ # +2 per deal after 5

$Value_Bonus = \min(15, Total_Value / \$100,000 \times 15)$ # Max at \$100K total

$Consistency_Bonus = \min(15, Monthly_Avg \times 3)$ # Regular activity bonus

Penalty:

- Deal cancellation (after acceptance): -5 per cancellation
- No-Show (accepted but didn't participate): -10 per incident
- Deal timeout (no response): -2 per timeout

2. Review Ratings (W_rev = 0.20)

$REV_Score = Weighted_Avg_Rating \times 20$ # Convert 1-5 scale to 0-100

$Weighted_Avg_Rating = \sum (Rating_j \times Reviewer_Trust_j) / \sum Reviewer_Trust_j$

Where:

$Rating_j$ = average of multi-dimensional ratings for review j

$Reviewer_Trust_j$ = trust score of the party who left the review (normalized 0-1)

Multi-dimensional ratings (each 1-5):

- Communication: Was responsive and clear
- Reliability: Delivered as promised
- Value Delivered: Met or exceeded expectations
- Fairness: Acted fairly in negotiations and execution

$Review\ Score = (Communication + Reliability + Value + Fairness) / 4$

Minimum Reviews: Score starts at 50 (neutral) until 3 reviews received

3. Profile Completeness (W_prof = 0.10)

$PROF_Score = \text{sum of completed sections:}$

Basic info (name, photo): 20 points

Bio/Description: 15 points

Location verified: 10 points

Business details: 15 points

Portfolio/Past work:	15 points
Skills/Specializations:	10 points
Availability/Preferences:	10 points
External links:	5 points

Maximum:	100 points
----------	------------

4. Verification Level ($W_{\text{ver}} = 0.15$)

VER_Score = sum of verification levels:

Email verified:	10 points
Phone verified:	15 points
Government ID verified:	30 points
Business registration:	25 points
Bank account linked:	10 points
Professional certification:	10 points

Maximum:	100 points (or 110 with bonus)
----------	--------------------------------

Bonus: +10 for video verification interview

5. Response Rate ($W_{\text{resp}} = 0.10$)

RESP_Score = Response_Rate $\times$ 80 + Timeliness_Score $\times$ 20

Response_Rate = Responses_Sent / Messages_Received (last 90 days)
Capped at 1.0 (100%)

Timeliness_Score = based on average response time:

< 1 hour:	100 points
< 4 hours:	80 points
< 24 hours:	60 points
< 48 hours:	40 points
< 72 hours:	20 points
> 72 hours:	0 points

6. Dispute History ($W_{\text{disp}} = 0.10$)

DISP_Score = 100 - Dispute_Penalty

Dispute_Penalty = Disputes_Filed_Against $\times$ Severity_Factor +
Disputes_Lost $\times$ 15 +
Pattern_Penalty

Severity_Factor:

Resolved amicably:	5 points
Mediated resolution:	10 points
Lost arbitration:	20 points

Pattern_Penalty:

3+ disputes in 6 months:	additional 10 points
5+ disputes in 6 months:	additional 25 points

Floor: 0 points (can't go negative)

7. Platform Longevity ($W_{age} = 0.05$)

AGE_Score = $\min(100, \text{Account_Age_Days} / 10)$
1000 days (~2.7 years) = maximum score

Decay: -1 point per 30 days of inactivity after 90 days

8. Community Contribution ($W_{comm} = 0.05$)

COMM_Score = $\min(100,$
 Successful_Referrals $\times 10 +$
 Helpful_Answers $\times 5 +$
 Mentor_Sessions $\times 15 +$
 Community_Recognition $\times 20$
)

8.1.3 Final Trust Score Formula

TRUST_SCORE = $(0.25 \times \text{TX_Score} +$
 $0.20 \times \text{REV_Score} +$
 $0.10 \times \text{PROF_Score} +$
 $0.15 \times \text{VER_Score} +$
 $0.10 \times \text{RESP_Score} +$
 $0.10 \times \text{DISP_Score} +$
 $0.05 \times \text{AGE_Score} +$
 $0.05 \times \text{COMM_Score})$

Bounds: $0 \leq \text{TRUST_SCORE} \leq 100$

Tier Assignment:

Platinum: 75-100

Gold: 60-74

Silver: 40-59

Bronze: 0-39

Decay: If no activity for 180 days, score decays by 2 points/month

8.2 Trust Score Visualization

graph TD

 subgraph "Trust Score Display"

 T[Overall: 72/100]

 🏆 Gold Tier]

 T --> R1[Supplier Score: 78

 15 deals • \$45K value]

 T --> R2[Consumer Score: 65

 8 deals • \$32K value]

 T --> R3[Enhancer Score: 71

 12 deals • \$28K value]

 end

 subgraph "Component Breakdown"

 T --> C1[📊 Transactions: 80/100]

 T --> C2[☆ Reviews: 68/100]

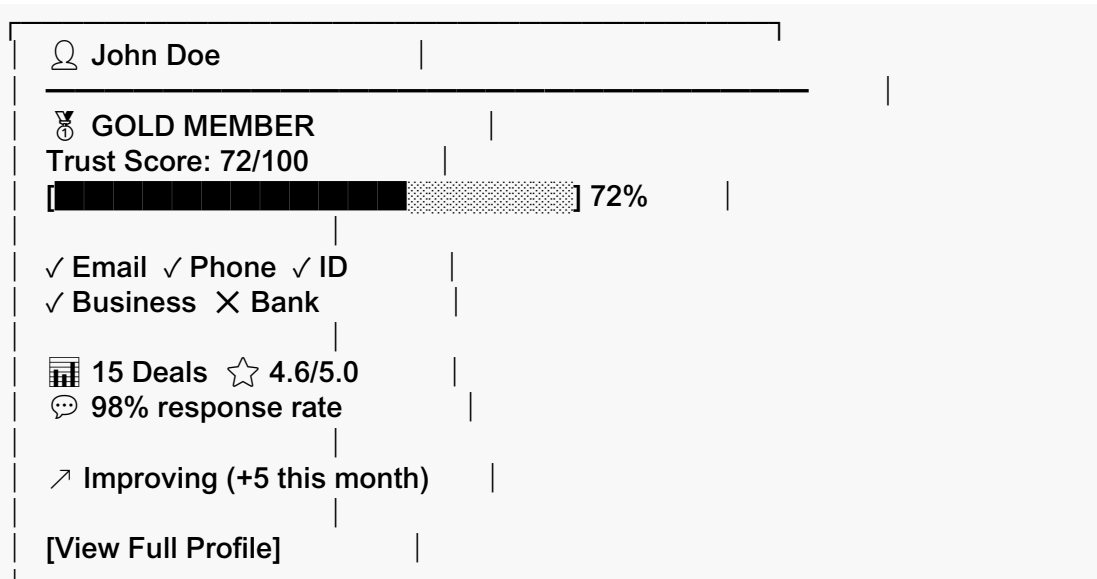
```

T --> C3[📄 Profile: 90/100]
T --> C4[✓ Verified: 75/100]
T --> C5[💬 Response: 82/100]
T --> C6[⚖️ Disputes: 95/100]
T --> C7[📅 Longevity: 50/100]
T --> C8[🤝 Community: 30/100]
end

subgraph "Trend"
  T --> TR[Trend: 📈 Improving
    +5 points in 30 days
    +12 points in 90 days]
end

```

Trust Badge Display:



8.3 Review Mechanism

8.3.1 Review Rules

ELIGIBILITY:

- Only parties who completed a deal together can review
- Review window: 7 days after deal completion
- Extension: Up to 14 days if dispute filed
- Minimum interaction: Deal must have progressed past escrow release

ANONYMITY:

- Reviews are attributed (party name visible)
- Individual dimension ratings are anonymous
- Written feedback is attributed

VISIBILITY:

- Reviews revealed simultaneously to ALL parties
- Timing: 72 hours after first review submitted OR
7 days after deal completion (whichever is later)
- Prevents retaliatory reviews

EDITING:

- Reviews can be edited within 48 hours of submission
- After reveal: reviews are permanent
- Follow-up reviews allowed for ongoing deals (monthly)

8.3.2 Review Challenge Process

graph TD

```
A[Review Published] --> B{Party disputes<br/>review?}
B -->|No| C[Review remains public]
B -->|Yes| D[Submit challenge<br/>with evidence]

D --> E[Platform moderates<br/>within 48 hours]
E --> F{Challenge valid?}

F -->|Yes| G[Options for challenger:]
G --> H1[Remove review]
G --> H2[Edit review text]
G --> H3[Add response note]

F -->|No| I[Review stands]
I --> J[Challenger gets<br/>strike on record]
J --> K{3 strikes?}
K -->|Yes| L[Review privilege<br/>suspended 30 days]
K -->|No| C

H1 --> C
H2 --> C
H3 --> C
```

8.3.3 Review Weighting

$\text{Effective_Rating} = \text{Raw_Rating} \times \text{Reviewer_Weight} \times \text{Recency_Weight} \times \text{Deal_Value_Weight}$

$\text{Reviewer_Weight} = \text{Reviewer_Trust_Score} / 100$

Reviews from highly-trusted parties count more

Recency_Weight:

Last 90 days: 1.0
91-180 days: 0.85
181-365 days: 0.70
1-2 years: 0.50
> 2 years: 0.30

$\text{Deal_Value_Weight} = \min(1.5, \sqrt{\text{Deal_Value} / \$10,000})$

Larger deals have more weight, capped at 1.5x

8.4 Trust Computation Flow

graph TD

```
A[Event Occurs] --> B{Event Type}

B -->|Deal Completed| C[Update TX_Score
+deal count]
```



```

+deal value
trigger review request]
B -->|Review Submitted| D[Update REV_Score
recalculate weighted average
update dimensional scores]
B -->|Profile Updated| E[Recompute PROF_Score]
B -->|Verification Added| F[Recompute VER_Score]
B -->|Message Sent| G[Update RESP_Score
recalculate response rate]
B -->|Dispute Resolved| H[Update DISP_Score
apply penalties/rewards]
B -->|Referral Converted| I[Update COMM_Score]

C --> J[Recalculate TRUST_SCORE]
D --> J
E --> J
F --> J
G --> J
H --> J
I --> J

J --> K{Tier Change?}
K -->|Yes| L[Notify user
Update badges
Apply new fee tier]
K -->|No| M[Silent update]

L --> N[Log to audit trail]
M --> N

```

8. Incentive Alignment

8.1 Why Use the Platform (vs Going Direct)

Benefit	Platform Advantage	Direct Deal
Trust	Verified parties, escrow, dispute resolution	Self-managed risk
Discovery	AI matching across thousands of parties	Limited personal network
Pricing	Market benchmarks, fairness validation	Information asymmetry
Security	Escrow, insurance, smart contracts	No protection
Efficiency	Standardized contracts, automated execution	Custom legal review
Reputation	Build cross-deal trust score	One-shot interactions
Financing	Revenue share, milestone payments	Cash only
Analytics	Market trends, pricing optimization	No data

Quantified Value Proposition:

For Suppliers:

Platform average: 23% higher revenue vs direct deals

Reason: Access to more consumers, competitive bidding, value validation

For Consumers:

Platform average: 18% lower cost vs independent sourcing

Reason: Transparent pricing, multiple supplier options, quality assurance

For Enhancers:

Platform average: 35% more deal flow vs self-sourced

Reason: Matched to ready deals, reputation amplification, standardized terms

8.2 Network Effects Design

graph TD

subgraph "Cross-Side Network Effects"

S[More Suppliers] -->|Better selection| C[Consumers join]

E[More Enhancers] -->|Better deals| C

C[More Consumers] -->|More demand| S

S -->|Complementary services| E

E -->|Better outputs| C

C -->|Revenue| S

end

subgraph "Same-Side Network Effects"

S -->|Competition| SB[Better prices for consumers]

S -->|Learning| SK[Knowledge sharing]

E -->|Specialization| EB[Higher quality services]

end

subgraph "Trust Network Effects"

T[More trusted users] --> TA[More deals completed]

TA --> TM[More reviews]

TM --> T

end

8.3 Referral Incentive Structure

Referral Program: "Triple Win Referrals"

Structure:

Referrer (existing user) invites New User

Rewards (cumulative):

Milestone	Referrer	New User	Platform
Signup	\$10 credit	\$25 credit	Cost: \$35
1st deal	\$25 cash	Fee waived	Cost: \$50
3rd deal	\$50 cash	-	Cost: \$50
5th deal	1yr Premium	6mo Premium	Cost: \$300

Total potential per successful referral: ~\$435

Referral tiers for referrers:

Advocate: 1-3 referrals → Standard rewards

Ambassador: 4-10 referrals → 1.5x multiplier + exclusive badge

Champion: 11+ referrals → 2x multiplier + revenue share (1% of referee fees for 1yr)

Viral Coefficient Target: $K > 0.5$ (each user refers >0.5 new active users)

8.4 Loyalty & Repeat Deal Benefits

Loyalty Program: "Trusted Partner Tiers"

Benefits by Cumulative Deal Count:

Benefit	Newbie 1-2	Regular 3-9	Partner 10-24	Champion 25+
Platform fee	8.5%	7.5%	6.5%	5.5%
Priority match	No	Standard	High	Highest
Escrow hold	100%	75%	50%	25%
Dispute credit	\$0	\$200/yr	\$500/yr	\$1,000/yr
Support	Email	Chat	Phone	Dedicated
Badges	Basic	Silver	Gold	Platinum
Analytics	Basic	Standard	Advanced	Custom
Deal size limit	\$10K	\$50K	\$250K	Unlimited
Review weight	1.0x	1.1x	1.2x	1.5x
Referral bonus	1.0x	1.2x	1.5x	2.0x

Partner-lock benefits (deal with same party 3+ times):

- Additional 1% fee reduction
- Fast-track dispute resolution
- Co-marketing opportunities
- Custom contract templates

8.5 Quality Incentives

Positive Feedback Loop:

High Trust Score → Better Matches → More Deals → More Reviews → Higher Trust

↑

Concrete Benefits of Higher Trust:

Trust 90+ (Platinum):

- Matched first in search results
- Eligible for largest deals
- Lowest platform fees (-35%)
- Instant escrow release option
- Featured partner status

Trust 75+ (Gold):

- Priority matching
- Large deal eligibility
- Reduced fees (-20%)
- Fast escrow release (24hr)

Trust 60+ (Silver):

- Standard matching
- Medium deal eligibility
- Standard fees
- Standard escrow (3-day)

Trust 40-59:

- Lower priority matching
- Small deal limit (\$50K)
- Standard fees
- Extended escrow (7-day)

Trust 0-39:

- Last in matching
- \$10K deal limit
- +35% fee premium
- 100% escrow required
- Enhanced monitoring

8. Risk Management

8.1 Escrow Design

8.1.1 Escrow Architecture

graph TD

subgraph "Escrow Flow"

C[Consumer] -->|Deposit 100%| E[Escrow Account]

E -->|Hold| H[Held Funds]

H -->|Milestone 1| S1[Supplier: 30%]

H -->|Milestone 1| E1[Enhancer: 20%]

H -->|Platform fee| P1[Platform: 10%]

H -->|Milestone 2| S2[Supplier: 20%]

H -->|Milestone 2| E2[Enhancer: 15%]

H -->|Final delivery| S3[Supplier: Final]

H -->|Final delivery| E3[Enhancer: Final]

end

```

subgraph "Escrow Release Conditions"
  RC1[Condition Types]
  RC1 --> RC2[Party consensus
  All 3 agree]
  RC1 --> RC3[Oracle verification
  External data]
  RC1 --> RC4[Milestone deadline
  Auto if no dispute]
  RC1 --> RC5[Third-party inspection
  Professional verifier]
end

```

8.1.2 Escrow Amount & Holdback

$\text{Escrow_Deposit} = \text{Deal_Value} \times \text{Escrow_Percentage}$

Escrow_Percentage by Trust Score:

Platinum (90+): 25% (75% released on proof of work)
 Gold (75-89): 50% (50% released on proof of work)
 Silver (60-74): 75% (25% released on proof of work)
 Bronze (40-59): 100% (full escrow required)
 Below 40: 100% + 20% Collateral

$\text{Holdback_Reserve} = \text{Deal_Value} \times 5\%$ (held until final confirmation)

Release Triggers:

1. Party Consensus: All 3 parties confirm → immediate release
2. Timeout: If no dispute filed within dispute_window (7 days) → auto-release
3. Oracle: External verification condition met → auto-release
4. Mediator: Approved mediator signs off → release

Partial Release (milestone deals):

At each milestone M: $\text{Release_P}(M) = \text{Total_Payout_P} \times \text{Milestone_Weight_M}$

8.2 Default Risk Mitigation

Risk Scenario	Probability	Impact	Mitigation	Cost Bearer
Supplier doesn't deliver resource	Medium	High	Escrow holdback + collateral + reputation penalty	Supplier (forfeits holdback)
Enhancer doesn't provide inputs	Medium	High	Milestone payments + performance bond	Enhancer (forfeits bond)
Consumer doesn't pay	Low (escrow)	High	100% upfront escrow required	Consumer (no refund if they back out)
Output quality below spec	Medium	Medium	Inspection clause + rework	Responsible party

Risk Scenario	Probability	Impact	Mitigation	Cost Bearer
Delay in delivery	High	Medium	requirement Liquidated damages clause + deadline penalties	Delaying party
Force majeure	Low	High	Insurance + deal suspension + renegotiation	Shared / Insurance

8.2.1 Insurance Options

Deal Insurance Tiers:

Basic (\$0.50 per \$100 deal value):

- Covers: Non-delivery by counterparty
- Payout: Up to 50% of escrow
- Deductible: \$500

Standard (\$1.00 per \$100 deal value):

- Covers: Non-delivery, quality defects, fraud
- Payout: Up to 100% of escrow
- Deductible: \$250
- Includes: Mediation cost coverage

Premium (\$2.50 per \$100 deal value):

- Covers: All of above + force majeure, market loss, currency risk
- Payout: Up to 150% of deal value
- Deductible: \$0
- Includes: Full dispute resolution, legal support

Platform Fund: 2% of all fees allocated to a mutual insurance pool for cases where individual insurance doesn't apply.

8.2.2 Collateral Requirements

$\text{Collateral_Required} = \max(0, \text{Risk_Score} \times \text{Deal_Value} - \text{Escrow_Coverage})$

$\text{Risk_Score} = f(\text{Party_Trust}, \text{Deal_Size}, \text{Domain_Risk}, \text{Counterparty_Trust})$

Trust < 40: Risk_Score = 0.30
 Trust 40-59: Risk_Score = 0.15
 Trust 60-74: Risk_Score = 0.08
 Trust 75+: Risk_Score = 0.03

Deal > \$100K: +0.05
 Deal > \$500K: +0.10
 New domain: +0.05

Collateral Form:

- Cash deposit (preferred)
- Bank guarantee/letter of credit
- Cryptocurrency (with 150% value requirement for volatility)
- Asset-backed security (platform-approved assets)

Collateral Return:

- Full return: Upon successful deal completion
- Partial return: If dispute resolved in party's favor
- Forfeiture: If party found at fault in dispute

8.3 Fraud Prevention

graph TD

```
subgraph "Fraud Detection Pipeline"
    A[New Account/Deal] --> B{Automated Screening}

    B --> C[Identity Verification]
    C --> C1[Document OCR + forgery check]
    C --> C2[Biometric liveness check]
    C --> C3[Database cross-reference]

    B --> D[Behavioral Analysis]
    D --> D1[Typing pattern analysis]
    D --> D2[Device fingerprinting]
    D --> D3[IP/geolocation check]

    B --> E[Deal Pattern Detection]
    E --> E1[Price anomaly detection]
    E --> E2[Circular deal detection]
    E --> E3[Velocity checks]

    C1 & C2 & C3 --> F{Risk Score > Threshold?}
    D1 & D2 & D3 --> F
    E1 & E2 & E3 --> F

    F -->|Yes| G[Enhanced Review]
    G --> G1[Manual verification]
    G --> G2[Delay deal 48 hours]
    G --> G3[Require additional collateral]

    F -->|No| H[Clear for platform]

    G1 --> I{Confirmed fraud?}
    I -->|Yes| J[Account banned  
Funds frozen  
Law enforcement notified]
    I -->|No| H
end
```

Fraud Detection Rules:

AUTO-FLAG TRIGGERS:

1. New account + deal > \$50K within 48 hours → Enhanced review
2. Price 50%+ below market benchmark → Manual review
3. Same device/IP creating multiple accounts → Block + investigate
4. Circular deal pattern (A→B→C→A) → Block all parties
5. Payment from high-risk jurisdiction → Enhanced KYC
6. Rapid-fire deal proposals (>10/day) → Rate limit + review

7. Reviews from same IP/network → Review weight = 0
8. Profile photo reverse-image search fail → Require re-verification
9. Bank account name mismatch → Block until resolved
10. Chargeback history >5% → Account restricted

RISK SCORING:

Each trigger adds points to a risk score (0-100)

0-30: Green - Normal processing

31-60: Yellow - Enhanced monitoring

61-80: Orange - Manual review required

81-100: Red - Block until cleared

8.4 Dispute Resolution

8.4.1 Three-Tier Resolution System

graph TD

A[Dispute Filed] --> B{Tier 1:
Direct Negotiation}

B -->|Resolved in 7 days| C[Agreement documented
Escrow released per terms]

B -->|Not resolved| D{Tier 2:
Platform Mediation}

D --> E[Mediator assigned
within 24 hours]

E --> F[Evidence collection
5-day window]

F --> G[Mediator proposal
within 3 days]

G -->|All parties accept| C

G -->|Rejected| H{Tier 3:
Binding Arbitration}

H --> I[Arbitrator panel
3 industry experts]

I --> J[Formal hearing
within 14 days]

J --> K[Binding decision
within 7 days]

K --> L[Decision enforced
Escrow distributed]

K --> M[Loser pays
arbitration costs]

C --> N[Dispute logged

Party scores updated]

L --> N

M --> N

8.4.2 Dispute Process Details

Tier 1: Direct Negotiation (Days 1-7)

Initiation: Any party files dispute with description and claim

Process:

- 48-hour cooling-off period
- Structured negotiation chat with templates
- Platform suggests fair splits based on deal terms
- Automatic extension available (once, +7 days)

Cost: Free

Success Rate Target: 65%

Tier 2: Platform Mediation (Days 8-18)

Initiation: Auto-escalated if Tier 1 fails

Mediator: Platform-trained mediator (industry expert)

Process:

- Mediator reviews all evidence
- Separate confidential sessions with each party
- Mediator proposes settlement (non-binding)
- If accepted: documented and executed
- If rejected: escalates to Tier 3

Evidence Accepted:

- Chat logs (automatically captured)
- Delivery confirmations
- Third-party inspection reports
- Photos/videos with timestamps
- Financial transaction records

Cost: \$200 (split equally, refunded if settlement reached)

Success Rate Target: 25% (of remaining disputes)

Tier 3: Binding Arbitration (Days 19-45)

Panel: 3 arbitrators (1 chosen by each party, 1 platform-appointed)

Process:

- Formal evidence submission (10-day window)
- Virtual hearing (each party presents case)
- Deliberation and binding decision
- Decision enforced by platform (escrow distributed)

Cost: \$1,500 + 2% of disputed amount

Loser pays all costs (split 50/50 if mixed decision)

Decision Types:

- Full refund to one party
- Proportional split
- Partial refund with damages
- Rework/replacement order
- Deal cancellation with penalty assignment

Success Rate Target: 100% (binding)

8.4.3 Penalty Structure

DISPUTE OUTCOME PENALTIES:

For party found at fault:

- Trust Score: -15 to -30 points (based on severity)
- Escrow forfeiture: 10-50% of held amount
- Platform fee premium: +10% for next 3 deals
- Review flag: "Dispute: Found at fault" on profile

Severity levels:

Minor (communication, minor delay): -15 points, 10% escrow

Moderate (quality issue, partial non-delivery): -20 points, 25% escrow

Major (fraud, complete non-delivery): -30 points, 50% escrow, account review

For party filing frivolous dispute:

- Trust Score: -10 points
- Forfeit dispute fee
- Warning on profile
- 3 frivolous disputes = suspension

For mutual fault:

- Both parties: -10 points
- Escrow split 50/50
- No additional penalties

8.5 Market Risk Management

Price Volatility Protection:

1. Price Lock: Fixed price deals lock at proposal acceptance
2. Price Band: Revenue share deals can specify min/max price
3. Indexed Pricing: Prices tied to commodity index (e.g., wheat futures)
4. Collar: Price floor and ceiling defined upfront

Force Majeure:

1. Deal automatically suspended
2. No penalties for non-delivery during force majeure
3. Insurance claims triggered
4. Renegotiation window opens after force majeure ends
5. If renegotiation fails within 30 days: deal cancelled, escrow returned pro-rata

Domain-Specific Risk Adjustments:

Agriculture: Weather insurance, crop yield guarantees
Manufacturing: Quality inspection bonds, delivery insurance
Technology: IP escrow, code escrow for SaaS deals
Real Estate: Title insurance, appraisal requirements

8.6 Counterparty Risk Limits

Deal Size Limits by Trust Tier:

Tier	Max Deal Size	Max Outstanding	Collateral Required
Bronze	\$10,000	\$20,000	100% escrow
Silver	\$50,000	\$100,000	75% escrow
Gold	\$250,000	\$500,000	50% escrow
Platinum	\$1,000,000	\$2,000,000	25% escrow
Super Platinum*	\$5,000,000	\$10,000,000	Negotiable

*Super Platinum: 100+ completed deals, 95+ trust score, 2+ years on platform

Multi-Party Risk Check:

Total_Exposure_P = $\sum$ Active_Deal_Values for party P

If Total_Exposure_P > Max_Outstanding:

- Block new deals until exposure reduced
- Suggest completing or insuring current deals

8. Economic Sustainability

8.1 Platform Monetization

Revenue Streams (Target Mix):

Revenue Stream	Target %	Description
Transaction fees	60%	% of deal value
Premium subscriptions	20%	Monthly fees
Value-added services	12%	Insurance, etc
Interest on escrow	5%	Float income
Data/analytics (B2B)	3%	Market reports

Target Unit Economics:

Average Deal Value (ADV): \$15,000

Take Rate: 8% average

Revenue per Deal: \$1,200

Gross Margin: 75% (\$900 per deal)

Customer Acquisition Cost: \$150 (blended across 3 party types)

Lifetime Value: \$4,500 (average 5 deals per active user)

LTV/CAC Ratio: 30:1 (strong)

Payback Period: 0.17 deals (first deal profitable)

Break-even: Platform breaks even at ~500 deals/month

8.2 Liquidity Bootstrapping

graph TD

```
subgraph "Phase 1: Seed (Months 1-6)"
    A1[Recruit 50 anchor suppliers
in 1 domain]
    A2[Offer 0% fees for first 3 deals]
    A3[Platform guarantees payment
to suppliers]
    A4[Recruit 100 enhancers
with service credits]

    A1 --> A5[Launch with 150 supply-side
parties ready]
    A2 --> A5
    A3 --> A5
    A4 --> A5
end

subgraph "Phase 2: Match (Months 7-12)"
    B1[Targeted consumer acquisition]
```

PPC + partnerships]
B2[First 100 deals fully subsidized
Platform takes all risk]
B3[Case studies + testimonials]
B4[Referral program launch]

B1 --> B5[Achieve 100 completed deals
Network effects begin]
B2 --> B5
B3 --> B5
B4 --> B5

end

subgraph "Phase 3: Growth (Months 13-24)"

C1[Introduce standard fees
Gradual: 4% → 6% → 8%]
C2[Premium features launch]
C3[Second domain expansion]
C4[Enterprise sales]

C1 --> C5[Self-sustaining marketplace
Organic growth >50%]
C2 --> C5
C3 --> C5
C4 --> C5

end

A5 --> B5

B5 --> C5

Bootstrapping Investment:

Phase 1 Budget: \$200,000

- Anchor supplier incentives: \$100,000
- Platform development: \$60,000
- Marketing/PR: \$40,000

Phase 2 Budget: \$300,000

- Consumer acquisition: \$150,000
- Deal subsidies (guarantees): \$100,000
- Operations: \$50,000

Phase 3 Budget: \$200,000

- Domain expansion: \$100,000
- Premium feature development: \$60,000
- Marketing: \$40,000

Total Seed Investment: \$700,000

Break-even Timeline: Month 18

8.3 Market Maker Role

Platform as Market Maker:

When a domain has insufficient supply:

1. Platform becomes temporary Supplier or Enhancer
2. Platform sources resources at wholesale, provides at retail
3. Platform takes margin risk to ensure Consumer demand is met
4. Gradually replaces platform supply with third-party supply

Example: Farm land domain

- Platform leases 500 acres directly from landowners
- Platform offers these to Consumers with Enhancer matching
- As independent Suppliers join, platform reduces its inventory
- Target: Platform <10% of supply within 12 months per domain

Economic Impact:

- Reduces chicken-egg problem
- Ensures Consumer experience (always supply available)
- Generates early revenue (platform keeps full margin)
- Provides market data for pricing benchmarks

8.4 Subsidization Strategy

Optimal Subsidization: Subsidize the HARDER side to acquire

Framework:

If Suppliers > Enhancers > Consumers (in signup ease):

- Subsidize Consumers (fee discounts, first-deal guarantee)
- Subsidize Enhancers (service credits, visibility boost)

If Consumers > Suppliers > Enhancers:

- Subsidize Enhancers (input cost coverage, guaranteed payment)
- Subsidize Suppliers (land value guarantee, no-risk trial)

Dynamic Subsidy Allocation:

Subsidy_Budget_per_Domain = \$10,000/month initially

Allocate based on supply/demand ratio:

If Supplier/Consumer ratio < 0.5: 50% to supplier acquisition

If Supplier/Consumer ratio > 2.0: 50% to consumer acquisition

Otherwise: Balanced across all parties

Subsidy Forms (in order of preference):

1. Fee credits (low cash cost, high perceived value)
2. Escrow guarantees (builds trust, contingent cost)
3. Direct payment (only for anchor partners)
4. Revenue share (performance-linked)

8.5 Growth Economics by Party Type

Metric	Supplier	Consumer	Enhancer		
CAC	\$120	\$180	\$90		
Avg Deals/Year	4	6	8		
Avg Deal Value	\$18,000	\$15,000	\$12,000		
Annual Revenue	\$5,760	\$7,200	\$7,680		

Gross Margin	75%	75%	75%
Annual Gross	\$4,320	\$5,400	\$5,760
LTV (3 years)	\$12,960	\$16,200	\$17,280
LTV/CAC	108:1	90:1	192:1
Payback (deals)	0.11	0.17	0.08

Key Insight: Enhancers have the best unit economics (highest frequency, lowest CAC). Prioritize enhancer acquisition in early stages.

8.6 Domain Expansion Economics

Domain Expansion Framework:

Phase 1: Market Research (2 weeks, \$5,000)

- TAM analysis
- Party density assessment
- Competitive landscape
- Regulatory requirements

Phase 2: Soft Launch (4 weeks, \$15,000)

- Recruit 20 anchor parties per role
- Seed 5-10 initial deals
- Calibrate pricing benchmarks
- Validate unit economics

Phase 3: Full Launch (8 weeks, \$30,000)

- Marketing campaign
- Full feature enablement
- Community building
- PR and case studies

Go/No-Go Criteria:

- GO if: 20+ parties per role signed, 5+ deals completed, positive NPS from pilot users
- NO-GO if: <10 parties per role after 4 weeks, no deals completed, regulatory barriers

Domain Priority Matrix:

Score = TAM × Accessibility × Strategic_Fit × Platform_Synergy

Priority 1 (Score >70): Launch immediately

Priority 2 (Score 50-70): Launch within 6 months

Priority 3 (Score 30-49): Evaluate further

Priority 4 (Score <30): Do not pursue

8. Smart Contract / Automated Execution

8.1 Smart Contract Architecture

graph TD

subgraph "Smart Contract Layers"

SC[Three-Party Deal Smart Contract]

```

SC --> V[Validation Layer
Win-win-win check
Terms verification]

SC --> E[Escrow Layer
Fund locking
Conditional release]

SC --> M[Milestone Layer
Trigger detection
Auto-payment]

SC --> D[Dispute Layer
Timelock escalation
Evidence anchoring]

SC --> O[Oracle Layer
External data feeds
Verification]
end

subgraph "External Integrations"
O --> OR1[Weather API
Agriculture domain]
O --> OR2[Shipping/Logistics
Delivery confirmation]
O --> OR3[Quality Labs
Inspection reports]
O --> OR4[Financial Feeds
Price indices]
O --> OR5[IoT Sensors
Real-time monitoring]
end

```

8.2 Milestone-Triggered Automatic Payments

// Pseudocode: Milestone Payment Contract

```

contract ThreePartyDeal {

    struct Milestone {
        bytes32 id;
        string description;
        uint256 paymentAmount;
        address paymentTo;    // Supplier, Enhancer, or split
        TriggerType triggerType;
        bytes32 triggerCondition;
        bool verified;
        uint256 deadline;
        uint256 completedAt;
    }

    enum TriggerType {

```

```

    TIME_BASED,      // e.g., "30 days after planting"
    ORACLE_FEED,     // e.g., "rainfall > 50mm"
    PARTY_CONSENSUS, // All 3 parties confirm
    THIRD_PARTY_VERIFY, // Inspector signs off
    EXTERNAL_EVENT   // API callback (delivery confirmed)
}

// Milestone completion triggers automatic payment
function completeMilestone(bytes32 milestoneId, bytes proof) external {
    Milestone storage m = milestones[milestoneId];

    require(!m.verified, "Already verified");
    require(isTriggerMet(m.triggerType, m.triggerCondition, proof),
        "Trigger condition not met");

    m.verified = true;
    m.completedAt = block.timestamp;

    // Auto-release payment from escrow
    releaseEscrow(milestoneId, m.paymentTo, m.paymentAmount);

    emit MilestoneCompleted(milestoneId, msg.sender, m.paymentAmount);

    // Check if all milestones complete → release holdback
    if (allMilestonesComplete()) {
        releaseHoldback();
    }
}

// Oracle callback for external verification
function oracleCallback(bytes32 milestoneId, bool conditionMet, bytes signature)
    external onlyAuthorizedOracle {

    require(conditionMet, "Oracle reports condition not met");
    completeMilestone(milestoneId, signature);
}

// Dispute timelock: If no dispute filed within window, auto-release
function checkTimelock(bytes32 milestoneId) external {
    Milestone storage m = milestones[milestoneId];

    require(m.verified, "Milestone not yet verified");
    require(block.timestamp > m.completedAt + DISPUTE_WINDOW,
        "Dispute window still open");
    require(!m.disputed, "Dispute filed");

    // Auto-release to designated party
    finalizePayment(milestoneId);
}
}

```


8.3 Conditional Execution Logic

CONDITIONAL PAYMENT TYPES:

1. IF-THEN Payments:

IF [condition] THEN release [amount] to [party]

Example: IF crop_yield > 5_tons_per_acre
THEN release bonus \$500 to Enhancer

2. GRADUATED Payments:

Payment varies based on continuous metric

Example: Quality_Score 1-10 → Payment = Base × (0.7 + 0.03 × Score)
Score 5 → 85% of base
Score 8 → 94% of base
Score 10 → 100% of base

3. PENALTY Payments:

Deduction for missed targets

Example: Delivery_delay_days × \$100/day deduction from Supplier payment
Max penalty: 20% of total payment

4. BONUS Payments:

Extra reward for exceeding targets

Example: Revenue > \$20,000 → 10% of excess as bonus to all parties

5. CONTINGENT Payments:

Payment only if future event occurs

Example: IF product wins industry award within 1 year
THEN \$5,000 bonus to Enhancer from revenue share

8.4 Oracle Integration

ORACLE TYPES AND USE CASES:

Oracle Type			
Oracle Type	Data Source	Use Case	
Weather Oracle	NOAA, Weather APIs	Crop insurance triggers	
	Force majeure events		
Price Oracle	Commodity exchanges	Revenue share valuation	
	Market data feeds	Indexed pricing	
Delivery Oracle	FedEx, UPS, IoT GPS	Milestone: delivery	
	Confirm shipment		

Quality Oracle	SGS, Bureau Veritas	Inspection results	
	Certified labs	Quality disputes	
IoT Oracle	Sensors on equipment	Usage-based billing	
	Condition monitoring		
Identity Oracle	Government databases	KYC verification	
	Business registration		

Oracle Security:

- Multi-oracle consensus: Require 2 of 3 oracles to agree
- Staked oracles: Oracles post collateral, slashed if incorrect
- Decentralized oracle networks (Chainlink integration)
- Fallback to manual verification if oracle disagreement

8.5 Smart Contract Deployment Flow

graph TD

A[Deal Terms Agreed] --> B[Platform generates
smart contract]

B --> C[Parties review
contract terms]

C --> D{All parties
approve?}

D -->|No| E[Return to negotiation]

E --> A

D -->|Yes| F[Consumer deposits
to escrow contract]

F --> G[Contract deployed
to blockchain]

G --> H[Deal ACTIVE]

H --> I{Milestone
triggered?}

I -->|Yes| J[Verify condition]

J --> K{Condition
verified?}

K -->|Yes| L[Auto-release payment]

K -->|No| M[Hold payment
flag for review]

I -->|No| N{Dispute
filed?}

N -->|Yes| O[Pause auto-execution]

O --> P[Escalate to mediation]

N -->|No| Q{All milestones
complete?}

Q -->|No| H

Q -->|Yes| R[Release final holdback]

R --> S[Deal CLOSED]

L --> H

M --> H

P --> H

Appendix A: Mathematical Formula Reference

A.1 Value Distribution Formulas

Fixed Price:

$$\begin{aligned}\text{Platform_Fee} &= \text{Deal_Value} \times \text{Fee\%} \\ \text{Net_Pool} &= \text{Deal_Value} - \text{Platform_Fee} \\ \text{Supplier_Payout} &= \text{Net_Pool} \times \text{Supplier_Share\%} \\ \text{Enhancer_Payout} &= \text{Net_Pool} \times \text{Enhancer_Share\%}\end{aligned}$$

Revenue Share:

$$\begin{aligned}\text{Distributable} &= \text{Gross_Revenue} \times (1 - \text{Platform_Fee\%} - \text{Reserve\%}) \\ \text{Party_Payout_P} &= \text{Distributable} \times \text{Share_P\%}\end{aligned}$$

Cost-Plus:

$$\begin{aligned}\text{Total_Cost} &= \text{Supplier_Cost} + \text{Enhancer_Cost} + \text{Platform_Cost} \\ \text{Final_Price} &= \text{Total_Cost} \times (1 + \text{Margin_}\%) \\ \text{Party_Payout_P} &= \text{Cost_P} \times (1 + \text{Margin_P}\%)\end{aligned}$$

Hybrid:

$$\begin{aligned}\text{Total} &= \text{Upfront} + \sum \text{Milestone_i} + \text{Revenue_Share_Component} \\ \text{Each component} &\text{ uses respective formula}\end{aligned}$$

A.2 Trust Score Formula

$$\text{TRUST_SCORE} = \sum (\text{Component_i} \times \text{Weight_i})$$

Components:

$$\begin{aligned}\text{TX_Score} &= \min(100, \text{Deals} \times 10 + \max(0, (\text{Deals} - 5) \times 2) + \text{Value}/\$100\text{K} \times 15 + \text{Monthly} \times 3) \\ \text{REV_Score} &= 20 \times \sum (\text{Rating_j} \times \text{ReviewerTrust_j}) / \sum \text{ReviewerTrust_j} \\ \text{PROF_Score} &= \text{sum of profile section completions (max 100)} \\ \text{VER_Score} &= \text{sum of verification levels (max 100 + 10 bonus)} \\ \text{RESP_Score} &= \text{ResponseRate} \times 80 + \text{Timeliness} \times 20 \\ \text{DISP_Score} &= 100 - (\text{LostDisputes} \times 15 + \text{FiledAgainst} \times \text{Severity} + \text{PatternPenalty}) \\ \text{AGE_Score} &= \min(100, \text{Days}/10) \\ \text{COMM_Score} &= \min(100, \text{Referrals} \times 10 + \text{Answers} \times 5 + \text{Mentor} \times 15)\end{aligned}$$

Weights: 0.25, 0.20, 0.10, 0.15, 0.10, 0.10, 0.05, 0.05

A.3 Dynamic Pricing Formula

$$\text{Fee\%} = 8.5\% \times \text{Trust_Mult} \times \text{Complexity_Mult} \times \text{Volume_Discount} \times \text{Domain_Adj}$$

Trust_Mult:

$$\begin{aligned}\text{Platinum (90+):} & 0.80 \\ \text{Gold (75-89):} & 0.90 \\ \text{Silver (60-74):} & 1.00 \\ \text{Bronze (40-59):} & 1.15 \\ <40: & 1.35\end{aligned}$$

Complexity_Mult: 0.90 / 1.00 / 1.25

Volume_Discount: 1.00 / 0.95 / 0.90 / 0.85

Domain_Adj: 0.95 to 1.20 depending on domain

A.4 Fairness Score

$$\text{Fairness} = 0.25 \times \text{Abs_Gain} + 0.30 \times \text{Prop_Fair} + 0.25 \times \text{Benchmark} + 0.20 \times \text{Opp_Cost}$$

$$\text{Abs_Gain} = \min(1.0, \max(0, \min(\text{Gain_P})) / (\max(\text{Gain_P}) \times 0.5))$$

$$\text{Prop_Fair} = \max(0, 1 - \text{variance}(\text{Shares}) \times 10)$$

$$\text{Benchmark} = \text{avg}(\max(0, 1 - |\text{premium_P}| \times 2))$$

$$\text{Opp_Cost} = \text{avg}(1.0 \text{ if } \text{net_benefit} > 0 \text{ else } \max(0, 0.5 + \text{net_benefit}/\text{OC}))$$

Appendix B: Decision Tree Summary

graph TD

START[Platform Interaction] --> Q{New or
Existing User?}

Q -->|New| ONBOARD[Onboarding Flow]

ONBOARD --> V1[Verify Identity
Build Profile]

V1 --> T1[Initial Trust Score: 35]

T1 --> BROWSE[Browse Deals
\$10K limit]

Q -->|Existing| CHECK{Check Trust
& Status}

CHECK --> PROPOSE[Propose Deal]

CHECK --> MATCH[Search Matches]

PROPOSE --> MODEL{Select
Distribution Model}

MODEL --> FIX[Fixed Price]

MODEL --> REV[Revenue Share]

MODEL --> COST[Cost-Plus]

MODEL --> BART[Barter]

MODEL --> HYB[Hybrid]

FIX & REV & COST & BART & HYB --> VALIDATE[Win-Win-Win Validation]

VALIDATE --> FAIR{Fairness
Score >= 0.50?}

FAIR -->|No| RENEG[Suggest Adjustments]

RENEG --> PROPOSE

FAIR -->|Yes| ESCROW[Escrow Setup]

ESCROW --> EXECUTE[Deal Execution]

EXECUTE --> MILE{Milestone
Reached?}

MILE -->|Yes| RELEASE[Auto/Payment Release]

MILE -->|No| DISPUTE{Dispute?}

DISPUTE -->|Yes| RESOLVE[Tier 1→2→3 Resolution]

DISPUTE -->|No| EXECUTE

RESOLVE --> COMPLETE[Deal Complete]

RELEASE --> COMPLETE

COMPLETE --> REVIEW[Mutual Review]

REVIEW --> UPDATE[Update Trust Scores]

UPDATE --> INCENTIVE[Loyalty/Referral Rewards]

Appendix C: Glossary

Term	Definition
Consumer	Party that wants output/product from a deal
Supplier	Party that provides resource/asset for a deal
Enhancer	Party that provides enabling input/service
TDV	Total Deal Value - sum of all party values
PVE	Present Value Equivalent - normalized value measure
Escrow	Third-party holding of funds until conditions met
Trust Score	Composite 0-100 reputation metric
Win-Win-Win	Condition where all three parties benefit
Fairness Score	0-1 measure of deal equity
Platform Fee	Commission charged by platform
Milestone	Defined achievement triggering payment
Oracle	External data feed for smart contracts
Holdback	Reserve portion of payment held until completion
CAC	Customer Acquisition Cost
LTV	Lifetime Value

Document Version 1.0 - Economic Engine Specification This specification is designed to be implemented incrementally, with the Win-Win-Win Validation Framework and Fixed Price Model as MVP features, followed by the full Trust System and Smart Contract integration in subsequent releases.

9. Cross-Cutting Concerns

This section integrates security, observability, compliance, and audit considerations that span across all architectural layers and domains described in Chapters 3 through 8. These concerns are implemented as shared infrastructure services and policies applied consistently across all microservices.

9.1 Security Architecture

Cross-Reference: For detailed authentication flows, authorization models, and threat mitigation, see Chapter 5, Section 5.8 (Security Architecture). For API-level security standards including JWT tokens, RBAC+ABAC permissions, and rate limiting, see Chapter 6, Section 6.1 (API Standards & Conventions).

9.1.1 Authentication & Authorization Summary

The platform uses **OAuth2/OIDC** for user authentication with JWT access tokens (RS256, 1-hour expiry) and refresh tokens (30-day rotation). Authorization combines **RBAC** (role-based) and **ABAC** (attribute-based) models:

Layer	Implementation	Details
Authentication	OAuth2/OIDC + PKCE	JWT tokens with RS256 signing
API Authorization	RBAC + ABAC	Claims-based with deals:read , payments:write , etc.
Multi-Party Auth	X-Party-ID header	Required when user belongs to multiple parties
Idempotency	Idempotency-Key header	UUID-based, 24-hour retention
Rate Limiting	Tiered (Free/Pro/Enterprise/Internal)	60-5000 req/min per tier

9.1.2 Data Protection

Concern	Implementation
Encryption at Rest	AES-256 for all databases, S3, backups
Encryption in Transit	TLS 1.3 for all external, mTLS for internal service mesh
PII Handling	Tokenization of sensitive fields; GDPR-compliant deletion
Secret Management	HashiCorp Vault with dynamic secrets
API Security	Input validation, parameterized queries, XSS/CSRF protection

9.1.3 Threat Mitigation

Threat	Mitigation
DDoS	Cloudflare + AWS Shield Advanced
Injection Attacks	Input validation, parameterized queries, WAF rules
Privilege Escalation	RBAC with least privilege, regular audit
Data Exfiltration	Network segmentation, DLP monitoring
Fraud	Multi-layer detection (see Chapter 8, Section 8.6.3)

9.2 Observability

Cross-Reference: For service-specific monitoring and alerting configurations, see Chapter 5, Section 5.11 (Cross-Cutting Concerns). For API-level tracing headers and correlation IDs, see Chapter 6, Section 6.1.12.

9.2.1 Three Pillars of Observability

+-----+	+-----+	+-----+
METRICS	LOGS	TRACES

```

| Prometheus/Grafana| | ELK Stack      | | Jaeger/Zipkin  |
| - Service health | | - Centralized  | | - Request flows |
| - Business KPIs  | | - Structured JSON| | - Latency breakdown
| - SLA/SLO        | | - Full-text search| | - Error context  |
| - Custom dashboards| | - Log aggregation| | - Dependency map |
+-----+ +-----+ +-----+

```

9.2.2 Alerting Tiers

Severity	Response Time	Channels	Examples
P0-Critical	5 minutes	PagerDuty + SMS + Slack	Payment system down, data corruption
P1-High	15 minutes	Slack + Email	Service degradation, >1% error rate
P2-Medium	1 hour	Slack	Elevated latency, resource exhaustion
P3-Low	4 hours	Email	Non-critical metric thresholds

9.2.3 Distributed Tracing

All API requests include correlation headers (**X-Request-ID**, **X-Correlation-ID**, **X-B3-TraceId**, **X-B3-SpanId**) that propagate through the service mesh. Traces capture: - Request latency per service hop - Database query execution times - External API call latencies - Event publishing/consuming timestamps

9.3 Compliance

Regulation	Requirements	Implementation
GDPR	Data portability, right to erasure, consent management	Automated data export/deletion pipelines, consent tracking
PCI DSS	Payment card data security	Tokenization, no raw card storage, encrypted transmission
SOC 2	Security, availability, confidentiality controls	Documented policies, access controls, audit trails
Anti-Money Laundering	Suspicious transaction monitoring	Automated pattern detection, reporting to authorities
Local Tax Laws	Transaction tax collection	Configurable tax rules per jurisdiction

9.4 Audit Trail

Cross-Reference: For the event sourcing schema used for deal lifecycle audit trails, see Chapter 5, Section 5.2 (Event Sourcing for Deal Lifecycle). For domain-level business rules and invariants, see Chapter 3, Section 3.8.

All significant actions are recorded in an immutable audit log:

Event Category	Events Captured	Retention
Authentication	Login, logout, password change, MFA events	7 years
Deal Lifecycle	Every state transition, proposal, counter-proposal	10 years
Financial	All transactions, escrow operations, settlements	10 years
Admin Actions	User suspensions, deal interventions, config changes	7 years
API Access	All API calls with request/response hashes	2 years
Data Access	PII access, bulk exports, admin queries	7 years

The audit log uses the event sourcing pattern (see Chapter 5, Section 5.2) ensuring every state change is captured as an immutable event with: - Event type and version - Timestamp (recorded_at, occurred_at) - Actor (user_id, service_id) - Correlation and causation IDs - SHA-256 checksum for integrity

10. Implementation Roadmap

10.1 Overview

The platform is built in four phases over approximately 24 months. Each phase delivers incremental value while building toward the full vision.

Phase 1 (MVP)	Months 1-6	Core deal management, matching, negotiation, user profiles
Phase 2	Months 7-12	Full negotiation engine, value distribution, escrow payments
Phase 3	Months 13-18	Advanced matching AI, reputation system, analytics
Phase 4	Months 19-24	Enterprise features, API platform, multi-domain expansion

10.2 Phase 1: MVP (Months 1-6)

Goal: Launch a functional 3-party deal platform with core deal lifecycle, basic matching, and user management.

Component	Features	Deliverables
User & Identity	Registration, login, KYC (basic), profile creation	identity-svc v1
Party & Profile	Individual/Org profiles, role selection (S/C/E), basic trust score	party-profile-svc v1
Deal Management	Deal CRUD, 13-state lifecycle (basic transitions), DRAFT through	deal-mgmt-svc v1

Component	Features	Deliverables
	NEGOTIATING	
Matching	Basic algorithmic matching (resource-need alignment, geographic fit), compatibility score	matching-svc v1
Negotiation	Single-threaded negotiation, proposal/counter-proposal, basic versioning	negotiation-svc v1
Agreement	Template-based contracts, digital signature capture (basic)	agreement-svc v1
Execution	Manual milestone tracking, basic progress updates	execution-svc v1
Notification	Email and in-app notifications, basic templates	notification-svc v1
Web Frontend	Responsive web app, dashboard, deal creation wizard, negotiation UI	Web App v1
Admin Console	User management, deal oversight, basic moderation tools	Admin v1

Phase 1 Architecture: - Core microservices (8 services) - PostgreSQL for primary data, Redis for caching - Kafka for async events (basic topics) - REST APIs (external), gRPC (internal) - Deployed on Kubernetes (single region)

Phase 1 Success Criteria: - [] 100+ registered parties across all 3 roles - [] 10+ completed deals end-to-end - [] Deal lifecycle works from DRAFT to COMPLETED - [] Basic matching produces >70% compatibility scores - [] All 13 deal states functional

10.3 Phase 2: Full Negotiation & Payments (Months 7-12)

Goal: Complete the negotiation engine, add escrow payments, Win-Win-Win validation, and value distribution.

Component	Features	Deliverables
Negotiation	Multi-threaded parallel negotiation, term dimension locking, ZOPA visualization	negotiation-svc v2
Value Distribution	Win-Win-Win validation engine, 5 payment models,	value-distribution-svc v1

Component	Features	Deliverables
Payment/Settlement	fairness scoring Escrow integration (Stripe/Flutterwave), milestone-based releases, refund handling	payment-svc v1
Resource Catalog	Full resource taxonomy, search, filtering, availability tracking	resource-catalog-svc v1
Trust & Reputation	Review system (blind/double-blind), trust score computation, badge system	trust-reputation-svc v1
Search	Full-text search, faceted filtering, Elasticsearch integration	search-svc v1
Media/Document	File upload/storage, document management, OCR	media-svc v1
Analytics	Basic dashboards, deal metrics, platform health monitoring	analytics-svc v1
Dispute Resolution	Tier 1 (direct) and Tier 2 (mediation) dispute handling	admin-svc v2

Phase 2 Architecture Additions: - Event sourcing for deal lifecycle (full implementation) - CQRS with read model projections - Saga patterns for distributed transactions - Multi-region deployment (active-passive) - GraphQL API layer

Phase 2 Success Criteria: - [] Win-Win-Win validation catches <5% bad deals pre-commitment - [] Escrow payments flow correctly with milestone releases - [] Dispute resolution resolves 90% of disputes within 14 days - [] Trust scores accurately predict deal success (correlation >0.7) - [] 500+ parties, 50+ completed deals

10.4 Phase 3: AI & Advanced Features (Months 13-18)

Goal: Add intelligent matching, advanced reputation features, comprehensive analytics, and scale the platform.

Component	Features	Deliverables
AI Matching	Machine learning-based compatibility prediction, preference learning, auto-suggestions	matching-svc v2
Dynamic Pricing	Market-based price suggestions, value	value-distribution-svc v2

Component	Features	Deliverables
Reputation	optimization, demand forecasting Cross-deal reputation analysis, skill endorsements, domain expertise scores	trust-reputation-svc v2
Analytics	Predictive analytics, market trend reports, personalized insights	analytics-svc v2
API Platform	Public API with developer portal, webhook management, SDKs	API Platform v1
Enterprise	SSO/SAML, custom branding, bulk operations, dedicated support	Enterprise Module
Mobile	Native iOS and Android apps with push notifications	Mobile Apps v1
Multi-Domain	Expanded category taxonomy, domain-specific templates and validations	Category System v2

Phase 3 Architecture Additions: - ML model serving infrastructure (TensorFlow Serving) - Advanced caching strategies (CDN, edge caching) - Multi-AZ deployment with automated failover - Chaos engineering program - Performance optimization (sub-100ms p95 API latency)

Phase 3 Success Criteria: - [] AI matching improves deal completion rate by >20% - [] Platform supports 10,000+ parties - [] 1,000+ completed deals across 5+ domains - [] API Platform has 10+ integrated third-party applications - [] 99.9% uptime SLA achieved

10.5 Phase 4: Scale & Expansion (Months 19-24)

Goal: Scale to enterprise volume, expand to multiple domains and geographies, and achieve platform sustainability.

Component	Features	Deliverables
Global Expansion	Multi-currency, multi-language, regional compliance, localized payment providers	Global Platform
Enterprise Suite	White-label capability, advanced analytics, custom integrations, SLA guarantees	Enterprise v2

Component	Features	Deliverables
Smart Contracts	Blockchain-based settlement option, automated execution via oracles	Smart Contract Module
Insurance	Integrated deal insurance, risk pools, premium pricing	Insurance Module
Market Making	Platform-as-market-maker for low-liquidity domains	Market Maker v1
Ecosystem	Partner marketplace, plugin architecture, community contributions	Ecosystem v1

Phase 4 Architecture Additions: - Multi-region active-active deployment - Blockchain integration (Ethereum/Polygon) - Advanced security (hardware security modules, confidential computing) - Self-healing infrastructure with automated remediation

Phase 4 Success Criteria: - [] 100,000+ registered parties - [] 10,000+ completed deals - [] Operations in 10+ countries - [] Platform profitability achieved - [] Top 3 marketplace in primary domain

10.6 Roadmap Timeline Summary

Month: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
 |<----- Phase 1: MVP ----->|<----- Phase 2: Core ----->|<----- Phase 3: AI ----->|<-- Phase 4 -->|

Core: [====Identity=====]

[====Party & Profile=====]

[=====Deal Mgmt=====]

[====Matching (basic)=====]

[====Negotiation (basic)=====]

[====Notification=====]

[=====Web Frontend=====]

Payment: [====Payment/Settlement=====]

Value: [====Win-Win-Win Engine=====]

Trust: [====Trust & Reputation=====]

AI: [====AI Matching=====]
 [====Analytics=====]

Scale: [====Global=====]
 [====Enterprise=====]

Appendix A: Glossary

This glossary combines all domain-specific terms, ubiquitous language definitions, and technical abbreviations used throughout this document.

A.1 Domain Terms

Term	Definition
3-Party Deal	A transaction involving Supplier, Consumer, and Enhancer roles around a specific value exchange. The fundamental unit of the platform.
Acknowledgment	A party's confirmation that they have received and reviewed a deal proposal, indicating willingness to enter negotiation.
Aggregate	A cluster of associated objects treated as a single unit for data changes. See Chapter 3, Section 3.7.
Aggregate Root	The single entity within an aggregate that serves as the entry point and enforces invariants.
Agreement	The final legally binding contract capturing all locked terms, signed by all parties.
Barter Model	A value distribution model where goods/services are exchanged directly without monetary payment. See Chapter 8, Section 8.2.4.
Commitment	A binding deposit made by each party upon locking terms, demonstrating good-faith intent to execute the deal.
Compatibility Score	A 0-100 score measuring how well three parties fit together across 7 dimensions: resource-need alignment, value alignment, trust score, geographic fit, temporal alignment, historical success, and risk profile.
Consumer	A party that expresses a need for a specific output, product, or service.
Cost-Plus Model	A value distribution model where parties transparently share costs and add agreed profit margins. See Chapter 8, Section 8.2.3.
Deal	An aggregate entity representing the entire lifecycle of a 3-party value exchange, from initial proposal through completion or cancellation.
Deal Participation	An association between a Party and a Deal, recording the party's role (Supplier/Consumer/Enhancer) and status (e.g., ACCEPTED).
Delivery Phase	A sub-state of EXECUTING where the Supplier's resource contribution is delivered and verified.
Domain Invariant	A business rule that must always be true, enforced within an aggregate boundary.
Enhancer	A party that adds value to enable the Supplier-Consumer transaction, providing expertise, services, or materials.
Entity	A domain object with a unique identity that persists over

Term	Definition
	time.
Enhancement Phase	A sub-state of EXECUTING where the Enhancer's input is applied to transform the Supplier's resource toward the Consumer's need.
Escrow	A third-party holding of funds until predefined conditions (milestones) are met.
Execution Phase	A sub-state of EXECUTING where the deal is actively being performed and milestones are tracked.
Fairness Algorithm	The set of rules ensuring equitable value distribution among parties. See Chapter 8, Section 8.1.
Fixed Price Model	A value distribution model with a single, predetermined total price. See Chapter 8, Section 8.2.1.
Gini Coefficient	A measure of inequality in value distribution (0 = perfectly equal, 1 = maximally unequal). Used in Win-Win-Win validation.
Good-Faith Deposit	A payment required to raise a dispute, discouraging frivolous claims.
Hybrid Model	A value distribution model combining elements of fixed price, revenue share, and cost-plus. See Chapter 8, Section 8.2.5.
Matching Engine	The algorithmic system that identifies and ranks compatible Supplier-Consumer-Enhancer triads.
Milestone	A defined checkpoint in deal execution with associated deliverables and payments.
Party	An individual, organization, or group participating in the platform. Each party may act in one or more roles.
PartyGroup	A composite entity allowing multiple people/organizations to act as a single Party with configurable governance.
Proposal	An offer submitted by a party during negotiation, specifying terms for one or more deal dimensions.
Proposal Version	An immutable snapshot of a proposal at a specific point in time.
Revenue Share Model	A value distribution model where parties share generated revenue according to agreed percentages. See Chapter 8, Section 8.2.2.
Role	One of three party types in a deal: SUPPLIER, CONSUMER, or ENHANCER.
Settlement	The financial reconciliation process upon deal completion, distributing escrowed funds according to the value distribution model.
Supplier	A party offering resources or assets that can be leveraged by other parties.
Term	A specific condition or parameter of a deal being negotiated (e.g., price, timeline, deliverables).

Term	Definition
Term Dimension	A category of terms being negotiated (e.g., Value Distribution, Deliverables, Timeline, Roles & Responsibilities, Risk & Termination).
Trust Score	A composite reputation metric (0-100) derived from reviews, completion history, and platform behavior. See Chapter 8, Section 8.4.
Value Distribution	The allocation of monetary or non-monetary value among the three parties in a deal.
Value Object	An immutable object without identity, defined solely by its attributes (e.g., GeoPoint, Money).
Verification Stage	A deal sub-state where deliverables are inspected against acceptance criteria before payment release.
Win-Win-Win	The core platform principle that all three parties must derive positive net value from a deal. See Chapter 8, Section 8.1.
ZOPA	Zone of Possible Agreement — the overlap between party acceptable ranges for a given term dimension.

A.2 Technical Abbreviations

Abbreviation	Full Term
ABAC	Attribute-Based Access Control
CQRS	Command Query Responsibility Segregation
DDD	Domain-Driven Design
DLQ	Dead Letter Queue
ER	Entity-Relationship
ES	Event Sourcing
GDPR	General Data Protection Regulation
gRPC	Google Remote Procedure Call
HPA	Horizontal Pod Autoscaler
HSM	Hardware Security Module
HSTS	HTTP Strict Transport Security
IoC	Inversion of Control
JWT	JSON Web Token
KPI	Key Performance Indicator
KYC	Know Your Customer
MFA	Multi-Factor Authentication
ML	Machine Learning
MVP	Minimum Viable Product
OIDC	OpenID Connect
PKCE	Proof Key for Code Exchange
PVE	Present Value Equivalent

Abbreviation	Full Term
RBAC	Role-Based Access Control
REST	Representational State Transfer
Saga	A pattern for managing distributed transactions
SLO	Service Level Objective
SSO	Single Sign-On
TDV	Total Deal Value
TLS	Transport Layer Security
WAL	Write-Ahead Log

Appendix B: Decision Log

This appendix records key architectural and design decisions made during the development of this document, along with their rationale and alternatives considered.

ID	Decision	Context	Rationale	Alternatives Rejected	Consequences
A D- 00 1	Microservices over Monolith	System architecture style (Chapter 5)	Independent deployment of services allows teams to scale development; different services have different scaling profiles; fault isolation prevents cascade failures; supports polyglot persistence for different data access patterns.	Monolith: rejected due to team scaling constraints and risk of tight coupling. Modular monolith: considered for MVP but would require costly migration later.	Increased operational complexity (service mesh, distributed tracing); requires robust DevOps practices; inter-service communication overhead.
A D- 00 2	Event Sourcing for Deal Lifecycle	Deal data persistence pattern (Chapter 5, Section 5.2)	Provides complete audit trail for regulatory compliance and dispute	CRUD with soft deletes: insufficient for audit requirements. Snapshots only: would lose intermediate state history.	Increased storage requirements; learning curve for the team; eventual consistency for

ID	Decision	Context	Rationale	Alternatives Rejected	Consequences
			resolution; enables state reconstructi on at any point in time; supports analytics on historical patterns; immutable events prevent data tampering.		read models; need for snapshot optimization.
A D- 00 3	Polymorph ic Party Model	How to represent different party types (Chapter 3, Section 3.5)	Single Party entity with subtypes (Individual, Organizatio n, PartyGroup) avoids data duplication; shared authenticati on, trust score, and wallet; groups participate identically in deals without special-case logic.	Separate tables per party type: would duplicate common fields and complicate foreign keys. Single table with nullable fields: would create sparse tables and lose type safety.	PartyGroup adds governance complexity; all party operations must go through the Party aggregate root.
A D- 00 4	13-State Deal Lifecycle	Granulari ty of deal states (Chapter 4, Section 4.1)	Sufficient granularity to track deal progress, enforce business rules, trigger notifications , and support analytics	7-state model (simpler): insufficient for tracking clarifications and holds. 20+ state model: too complex, many states would be rarely used.	Requires careful state machine implementation; needs comprehensive timeout handling for time-limited states.

ID	Decision	Context	Rationale	Alternatives Rejected	Consequences
			without being unwieldy. States like AWAITING_PARTY and ON_HANDLED provide clear semantic meaning for common scenarios.		
A D- 00 5	Escrow-Based Payments	Payment security model (Chapter 8, Section 8.6.1)	Protects all parties: Consumer ensures payment only for verified deliverables ; Supplier/Enhancer guaranteed payment upon fulfillment; platform mediates disputes with held funds. Reduces counterparty risk and fraud.	Direct payment: high risk of non-payment or non-delivery. Letter of credit: too slow and expensive for smaller deals. Blockchain escrow: considered for Phase 4 but adds complexity for MVP.	Requires payment provider integration; escrow fees increase platform cost; funds held create regulatory considerations.
A D- 00 6	CQRS with Separate Read/Write Models	Data access pattern (Chapter 5, Section 5.2)	Write models optimized for consistency and validation; read models optimized for query	Single model: would compromise both read and write performance. Direct DB reads: would couple read clients to write schema.	Eventual consistency between write and read models; need for projection infrastructure; increased system complexity.

ID	Decision	Context	Rationale	Alternatives Rejected	Consequences
			performanc e; enables near- realtime analytics; read models can be tailored per use case (search, dashboards, API responses).		
A D- 00 7	Kafka for Async Event Bus	Message broker selection (Chapter 5, Section 5.2)	High throughput for event sourcing; durable persistence for replay capability; partition- based ordering per deal; excellent ecosystem (Kafka Connect, Kafka Streams); battle-tested at scale.	RabbitMQ: excellent for simple routing but less suited for event sourcing. NATS: lightweight but fewer enterprise features. Pulsar: promising but smaller ecosystem.	Operational complexity of Kafka cluster; need for careful topic design and monitoring.
A D- 00 8	gRPC for Inter- Service Communication	Internal service protocol (Chapter 5, Section 5.3)	High performanc e with HTTP/2 multiplexin g; strong typing with Protocol Buffers; bi- directional streaming support; excellent code generation	REST for internal: higher latency, no strong typing. GraphQL for internal: overkill for service-to- service. Message queue only: insufficient for request-response patterns.	Requires proto definitions and code generation; less human-readable for debugging; load balancer considerations for HTTP/2.

ID	Decision	Context	Rationale	Alternatives Rejected	Consequences
			across languages; native service mesh integration.		
A D- 00 9	PostgreSQL as Primary Database	Main data store (Chapter 5, Section 5.4)	ACID compliance for financial data; rich data types (JSONB, arrays, spatial); excellent DDD aggregate support; strong consistency guarantees; mature tooling and ecosystem.	MongoDB: eventual consistency model unsuitable for financial transactions. MySQL: fewer advanced features. CockroachDB: interesting for global distribution but higher operational cost.	Single-writer bottleneck per aggregate; requires careful sharding strategy for scale.
A D- 01 0	Win-Win-Validation Engine	Deal fairness validation (Chapter 8, Section 8.1)	Mathematical validation ensures platform integrity; prevents exploitative deals; builds long-term trust; automated checking scales better than manual review.	Manual review: too slow, subjective, doesn't scale. Simple percentage rules: insufficient for complex value types. No validation: would lead to unfair deals and platform abandonment.	Computationally intensive; requires accurate market data; may reject borderline deals that humans would approve.
A D- 01 1	Saga Pattern for Distributed Transactions	Multi-service transaction coordination (Chapter 5, Section	Maintains data consistency across services without distributed locks; compensati	2PC (Two-Phase Commit): too slow, creates locks, not suitable for microservices. TCC (Try-Confirm-Cancel): more efficient but complex to implement.	Eventual consistency; need for compensating transaction design; requires idempotent operations.

ID	Decision	Context	Rationale	Alternatives Rejected	Consequences
		5.3)	ng transactions handle failures gracefully; fits naturally with event-driven architecture.		
A D-01 2	Kubernetes for Container Orchestration	Deployment platform (Chapter 5, Section 5.7)	Industry standard; excellent service discovery; auto-scaling; self-healing; rich ecosystem (Istio, Prometheus, Helm); cloud-agnostic.	Docker Swarm: simpler but less feature-rich. Nomad: lightweight but smaller ecosystem. ECS: AWS-only, vendor lock-in.	Steep learning curve; complex initial setup; requires dedicated platform team.
A D-01 3	RBAC + ABAC Authorization Model	Access control design (Chapter 5, Section 5.6)	RBAC provides simple, understandable role-based permissions. ABAC extensions allow fine-grained, context-sensitive access (e.g., deal participant, ownership, verification level).	Pure RBAC: insufficient for complex deal-based permissions. Pure ABAC: too complex, hard to reason about. ACL per resource: doesn't scale.	More complex authorization logic; requires careful testing of permission matrices.
A D-01 4	Tiered Trust Score System	Reputation computation	8-component weighted formula	Simple average rating: too gameable, doesn't account for deal value or reviewer trust. Elo-style: complex to	Computation overhead; requires careful weight tuning;

ID	Decision	Context	Rationale	Alternatives Rejected	Consequences
		(Chapter 8, Section 8.4)	provides comprehensive reputation assessment; recency weighting ensures scores reflect current behavior; tier assignment simplifies user understanding; trust-adjusted fees incentivize good behavior.	explain. Binary (trusted/untrusted): insufficient granularity.	may disadvantage new users initially.
A D- 01 5	5 Value Distribution Models	Flexibility in deal pricing (Chapter 8, Section 8.2)	Fixed Price (simple), Revenue Share (aligned incentives), Cost-Plus (transparent), Barter (no cash), Hybrid (complex deals). Covers the spectrum from simple to complex deal structures.	Single model: too restrictive for diverse deal types. More models: adds complexity with diminishing returns. Fully custom: impossible to automate validation.	Model selection UI complexity; need for model-specific validation logic; education required for users.

Document End — 3-Party Deal Making Platform Software Design Document

Version: 1.0

Date: June 2026

Status: Draft for Review

Classification: Confidential

This document integrates six specialist-authored design documents into a single unified software design specification. All content has been preserved with consistent terminology, renumbered sections, and added cross-references between chapters.